

TUGAS AKHIR - EF234801

CINA VS JEPANG: IDENTIFIKASI KEUNIKAN KARAKTERISTIK VISUAL SENI LUKIS TRADISIONAL DENGAN METODE SEMI-SUPERVISED LEARNING

KEYSA ANADEA AQIVA AJIE

NRP 5025211028

Dosen Pembimbing

Shintami Chusnul Hidayati, S.Kom, M.Sc., Ph.D

NPP 1987202012004

Program Studi Teknik Informatika

Departemen Teknik Informatika

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

Tahun 2025

(Halaman ini sengaja dikosongkan)



TUGAS AKHIR - EF234801

**CINA VS JEPANG: IDENTIFIKASI KEUNIKAN
KARAKTERISTIK VISUAL SENI LUKIS TRADISIONAL
DENGAN METODE SEMI-SUPERVISED LEARNING**

KEYSA ANADEA AQIVA AJIE

NRP 5025211028

Dosen Pembimbing

Shintami Chusnul Hidayati, S.Kom, M.Sc., Ph.D

NPP 1987202012004

Program Studi Teknik Informatika

Departemen Teknik Informatika

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

Tahun 2025

(Halaman ini sengaja dikosongkan)



FINAL PROJECT - EF234801

**CINA VS JEPANG: IDENTIFIKASI KEUNIKAN
KARAKTERISTIK VISUAL SENI LUKIS TRADISIONAL
DENGAN METODE SEMI-SUPERVISED LEARNING**

KEYSA ANADEA AQIVA AJIE

NRP 5025211028

Advisor

Shintami Chusnul Hidayati, S.Kom, M.Sc., Ph.D

NPP 1987202012004

Study Program Informatics Engineering

Department of Informatics Engineering

Faculty of Intelligent Electrical and Information Technology

Institut Teknologi Sepuluh Nopember

Surabaya

Year 2025

(Halaman ini sengaja dikosongkan)

LEMBAR PENGESAHAN

CINA VS JEPANG: IDENTIFIKASI KEUNIKAN KARAKTERISTIK VISUAL SENI LUKIS TRADISIONAL DENGAN METODE SEMI-SUPERVISED LEARNING

TUGAS AKHIR

Diajukan untuk memenuhi salah satu syarat
memperoleh gelar S.Kom pada
Program Studi S-1 Teknik Informatika
Departemen Teknik Informatika
Fakultas Teknologi Elektro dan Informatika Cerdas
Institut Teknologi Sepuluh Nopember

Oleh : **KEYSA ANADEA AQIVA AJIE**

NRP. 5025211028

Disetujui oleh Tim Penguji Tugas Akhir :

1. Shintami Chusnul Hidayati, S.Kom, M.Sc., Ph.D Pembimbing
2. Ratih Nur Esti Anggraini, S.Kom, M.Sc., Ph.D. Penguji
3. Ir. Adhatus Solichah Ahmadiyah, S.Kom., M.Sc. Penguji

SURABAYA

Mei, 2025

(Halaman ini sengaja dikosongkan)

PERNYATAAN ORISINALITAS

Yang bertanda tangan di bawah ini:

Nama mahasiswa / NRP : Keysa Anadea Aqiva Ajie / 5025211028
Program studi : S1 Teknik Informatika
Dosen Pembimbing / NPP : Shintami Chusnul Hidayati, S.Kom, M.Sc., Ph.D /
1987202012004

dengan ini menyatakan bahwa Tugas Akhir dengan judul “Identifikasi Keunikan Karakteristik Visual Seni Lukis Tradisional Jepang dengan Metode Semi-Supervised Learning” adalah hasil karya sendiri, bersifat orisinal, dan ditulis dengan mengikuti kaidah penulisan ilmiah. Bilamana di kemudian hari ditemukan ketidaksesuaian dengan pernyataan ini, maka saya bersedia menerima sanksi sesuai dengan ketentuan yang berlaku di Institut Teknologi Sepuluh Nopember.

Surabaya, 23 Mei 2025

Mengetahui
Dosen Pembimbing

Mahasiswa

Shintami Chusnul Hidayati, S.Kom, M.Sc.,
Ph.D
NPP. 1987202012004

Keysa Anadea Aqiva Ajie
NRP. 5025211028

(Halaman sengaja dikosongkan)

ABSTRAK

CINA VS JEPANG: IDENTIFIKASI KEUNIKAN KARAKTERISTIK VISUAL SENI LUKIS TRADISIONAL DENGAN METODE SEMI-SUPERVISED LEARNING

Nama Mahasiswa / NRP : Keysa Anadea Aqiva Ajie / 5025211028
Departemen : Teknik Informatika FTEIC - ITS
Dosen Pembimbing : Shintami Chusnul Hidayati, S.Kom, M.Sc., Ph.D

Abstrak

Penelitian ini bertujuan untuk mengidentifikasi keunikan karakteristik visual seni lukis tradisional Jepang dan Cina melalui pendekatan *semi-supervised learning*. Metode yang digunakan mencakup deteksi *patch*, ekstraksi fitur menggunakan ResNet50, reduksi dimensi dengan PCA dan UMAP, serta *discriminative clustering* berbasis K-Means dengan metrik *cosine distance*. Hasil *clustering* digunakan untuk pelatihan model SVM dengan *pseudo-label*, kemudian dikombinasikan melalui skema *hybrid voting* menggunakan probabilitas SVM dan skor kedekatan terhadap *centroid*. *Dataset* terdiri dari lukisan *ukiyo-e* dan *sumi-e* sebagai representasi seni Jepang, serta lukisan tradisional Cina. Hasil pengujian menunjukkan bahwa model *hybrid voting* dengan *filtering patch padding* menghasilkan akurasi sebesar 99,98% pada klasifikasi *patch* dan 83,50% pada klasifikasi gambar utuh. Pendekatan ini juga memungkinkan interpretasi terhadap elemen visual khas yang berkontribusi pada prediksi. Diharapkan, penelitian ini dapat menjadi dasar bagi pengembangan sistem pelestarian dan analisis seni tradisional Asia Timur berbasis pembelajaran mesin.

Kata kunci: *Seni Jepang, Klasifikasi SVM, Hybrid Voting, Klasifikasi Berbasis Centroid*

(Halaman ini sengaja dikosongkan)

ABSTRACT

CHINA VS JAPAN: IDENTIFYING THE UNIQUE VISUAL CHARACTERISTICS OF TRADITIONAL PAINTINGS USING A SEMI-SUPERVISED LEARNING METHOD

Student Name / NRP : Keysa Anadea Aqiva Ajie / 5025211028
Department : Teknik Informatika FTEIC - ITS
Advisor : Shintami Chusnul Hidayati, S.Kom, M.Sc., Ph.D

Abstract

This research aims to identify the unique visual characteristics of traditional Japanese and Chinese paintings through a semi-supervised learning approach. The proposed method involves patch detection, feature extraction using ResNet50, dimensionality reduction with PCA and UMAP, and discriminative clustering using K-Means. The clustering results are used to train an SVM classifier with pseudo-labels, and predictions are combined using a hybrid voting scheme that integrates SVM probabilities and proximity scores to cluster centroids. The dataset includes *ukiyo-e* and *sumi-e* paintings as representations of Japanese art, and traditional Chinese paintings as comparisons. Experimental results show that the hybrid voting model with patch padding filtering achieves an accuracy of 99,98% on patch-level classification and 83,50% on full-image classification. This approach also enables interpretation of visual elements that contribute significantly to predictions. The findings are expected to support the development of computational methods for analyzing and preserving traditional East Asian art.

Keywords: *Japanese Art, SVM Classification, Hybrid Voting, Centroid Based Classification*

(Halaman ini sengaja dikosongkan)

KATA PENGANTAR

Puji syukur penulis sampaikan kepada Allah SWT atas rahmat dan hidayah-Nya, penulis dapat menyelesaikan penelitian tugas akhir yang dirangkum dalam buku tugas akhir berjudul *“CINA VS JEPANG: IDENTIFIKASI KEUNIKAN KARAKTERISTIK VISUAL SENI LUKIS TRADISIONAL DENGAN METODE SEMI-SUPERVISED LEARNING”*. Penulisan buku ini tidak mungkin dapat terselesaikan tanpa adanya dukungan dan bantuan dari berbagai pihak. Maka dari itu, penulis menyampaikan rasa terima kasih yang sebesar-besarnya kepada semua pihak yang telah membantu penulis dalam menyelesaikan penulisan ini:

1. Allah SWT serta junjungan besar baginda Nabi Muhammad SAW, karena rahmat dan karunia-Nya penulis dapat menyelesaikan tugas akhir dan juga perkuliahan di Teknik Informatika ITS,
2. Keluarga dari penulis yaitu ayah, mama, Arka dan Enzo yang memberikan semangat, dukungan, motivasi serta doa selama penyusunan buku tugas akhir ini,
3. Departemen Teknik Informatika, Fakultas Teknologi Elektro dan Informatika Cerdas, dan Institut Teknologi Sepuluh Nopember yang telah menerima, mendidik, dan menempe penulis,
4. Ibu Shintami Chusnul Hidayati, S.Kom, M.Sc., Ph.D yang telah membimbing penelitian dan penyusunan buku tugas akhir ini serta memberikan dukungan kepada penulis,
5. Muhammad Daffa Akbar Arafah yang selalu menemani, mendukung, dan membantu selama proses pembuatan Tugas Akhir Penulis,
6. Seluruh teman-teman penulis, dari KM ITS hingga Teknik Informatika, dari penulis masih SD hingga sudah mahasiswa, dari angkatan C25, “Bsk Kemana? Tp Whatsapp”, “ML Gengx”, “Stelan Cumlaude”, “Perempuan Hebat”, “Ciwu Bucin Semua (Kecuali Faya)”, “Pojul Tukang Hoak”, “Korowai”, “Hani4love”, “Orang Himpunan”, hingga “Digidaw”,
7. HMTC ITS yang menjadi tempat penulis mengambil peram sebagai pemimpin untuk memberikan dampak kepada mahasiswa Teknik Informatika ITS,
8. Serta seluruh pihak yang tidak bisa penulis sebutkan satu per satu.

Penulis sadar bahwa buku Tugas Akhir ini masih jauh dari kata sempurna. Oleh sebab itu, penulis masih sangat terbuka kepada seluruh saran dan masukan yang datang untuk memperbaiki tulisan ini di masa yang akan datang. Penulis menyampaikan permohonan maaf yang sebesar-besarnya apabila ada konten di buku ini yang kurang berkenan bagi pembaca. Semoga buku tugas akhir dan penelitian ini dapat memberikan manfaat.

(Halaman ini sengaja dikosongkan)

DAFTAR ISI

LEMBAR PENGESAHAN	i
PERNYATAAN ORISINALITAS	iii
ABSTRAK	v
ABSTRACT	vii
KATA PENGANTAR	ix
DAFTAR ISI	xi
DAFTAR GAMBAR	xv
DAFTAR TABEL	xvii
DAFTAR PSEUDOCODE	xix
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah	2
1.4 Tujuan	2
1.5 Manfaat	2
BAB 2 TINJAUAN PUSTAKA	3
2.1 Hasil Penelitian Terdahulu	3
2.2 Dasar Teori	4
2.2.1 Gaya Lukis Seni Jepang	4
2.2.2 Gaya Lukis Seni Cina	6
2.2.3 Klasifikasi <i>Patch-Based</i> dengan SVM	7
2.2.4 <i>Hybrid Voting</i>	8
2.2.5 Praproses Data	10
2.2.6 Deteksi <i>Patch</i>	11
2.2.7 Ekstraksi Fitur	12
2.2.8 Similiaritas	14
2.2.9 <i>Discriminative Clustering</i>	15
2.2.10 Evaluasi Model	17
BAB 3 METODOLOGI	21
3.1 Metode yang digunakan	21
3.1.1 <i>Dataset</i>	21
3.1.2 Pra-Proses Data	22

3.1.3	Deteksi <i>Patch</i>	22
3.1.4	Ekstraksi Fitur.....	24
3.1.5	<i>Discriminative Clustering</i>	25
3.1.6	Pembuatan Model.....	26
3.1.7	Klasifikasi <i>Hybrid Voting</i>	26
3.1.8	Metode Pembandingan.....	29
3.1.9	Evaluasi Model.....	33
3.2	Bahan dan peralatan yang digunakan	33
3.2.1	Perangkat Keras	33
3.2.2	Perangkat Lunak	33
3.3	Rencana Implementasi dan Uji Coba	34
3.3.1	Implementasi Praproses Data	34
3.3.2	Implementasi Deteksi <i>Patch</i>	36
3.3.3	Implementasi Ekstraksi Fitur.....	38
3.3.4	Implementasi <i>Clustering</i>	39
3.3.5	Implementasi Pembuatan Model	41
3.3.6	Implementasi Model Klasifikasi Menggunakan Gambar Utuh.....	52
3.3.7	Implementasi Model Tanpa Filtrasi <i>Padding</i>	54
3.3.8	Implementasi GUI	59
BAB 4	Hasil dan Pembahasan	61
4.1	Hasil penelitian	61
4.1.1	Praproses Data	61
4.1.2	Deteksi <i>Patch</i>	66
4.1.3	Ekstraksi Fitur.....	69
4.1.4	<i>Discriminative Clustering</i>	72
4.1.5	Model dan Klasifikasi <i>Hybrid Voting</i>	77
4.1.6	Metode Pembandingan.....	81
4.1.7	<i>Output</i> Program	95
4.2	Pembahasan/Diskusi.....	96
BAB 5	Kesimpulan dan Saran	99
5.1	Kesimpulan.....	99
5.2	Saran	100
DAFTAR PUSTAKA		101
LAMPIRAN.....		107

BIODATA PENULIS 109

(Halaman ini sengaja dikosongkan)

DAFTAR GAMBAR

Gambar 2.1 Dataset <i>Ukiyo-e</i>	5
Gambar 2.2 Dataset <i>Sumi-e</i>	5
Gambar 2.3 Dataset Lukisan Cina	6
Gambar 2.4 <i>Hyperplane Optimal</i>	7
Gambar 2.5 Arsitektur <i>Hybrid Voting Ensemble</i> untuk Klasifikasi Gambar menggunakan CNN.....	8
Gambar 2.6 Arsitektur CNN untuk Ekstraksi Fitur pada Gambar.....	13
Gambar 2.7 Visualisasi Centroid dan Kluster <i>K-Means Clustering</i>	16
Gambar 3.1 Diagram Alir Metodologi Penelitian.....	21
Gambar 3.2 Praproses Data.....	22
Gambar 3.3 Visualisasi <i>Patch</i> Kosong Hasil Deteksi <i>Patch</i>	23
Gambar 3.4 Visualisasi <i>Patch</i> Berisi Informasi Hasil Deteksi <i>Patch</i>	24
Gambar 3.5 Visualisasi Lapisan pada Resnet50	25
Gambar 3.6 Klasifikasi <i>Hybrid Voting</i> pada Gambar Utuh.....	26
Gambar 3.7 <i>Hybrid Voting Patch-Based</i> (Data Uji).....	28
Gambar 3.8 Klasifikasi SVM <i>Patch-Based</i> (Data Uji).....	29
Gambar 3.9 Klasifikasi SVM Voting Gambar Utuh.....	30
Gambar 3. 10 Klasifikasi Berbasis Centroid menggunakan Data Uji (<i>Patch Based</i>).....	31
Gambar 3.11 Klasifikasi Berbasis Centroid pada Gambar Utuh	32
Gambar 3.12 Alur Pembuatan Model dan Klasifikasi dengan Gambar Utuh.....	33
Gambar 3.13 Hasil GUI	59
Gambar 4.1 Gambar Terkecil dan Terbesar Pada <i>Dataset</i> Positif.....	64
Gambar 4.2 Visualisasi <i>Invalid Patches</i> Hasil Filtrasi <i>Padding</i>	68
Gambar 4.3 Visualisasi Hasil Lolos Validasi <i>Padding</i>	68
Gambar 4.4 Visualisasi Proses Deteksi <i>Patch</i>	69
Gambar 4.5 Analisis Hasil Ekstraksi Setelah PCA pada <i>Dataset</i> Positif	70
Gambar 4.6 Analisis Hasil Ekstraksi Setelah PCA pada <i>Dataset</i> Negatif.....	71
Gambar 4.7 Hasil Pengelompokkan <i>Discriminative Clustering</i>	72
Gambar 4.8 <i>Patch</i> yang Salah Dikelompokkan ke Kelas Lain.....	73
Gambar 4.9 Analisis Hasil <i>Discriminative Clustering</i>	74
Gambar 4.10 Distribusi Jarak <i>Patch</i> ke <i>Centroid</i>	75
Gambar 4.11 <i>Top 10 Patch</i> Terdekat pada <i>Centroid</i> Tiap <i>Cluster</i>	76
Gambar 4.12 Hasil <i>Confusion matrix Hybrid Voting</i> pada Data Uji	78
Gambar 4.13 Hasil <i>Confusion matrix Hybrid Voting</i> pada Gambar Utuh.....	80
Gambar 4.14 Hasil <i>Confusion Matrix</i> Klasifikasi SVM-Only.....	82
Gambar 4.15 Hasil <i>Confusion Matrix</i> Klasifikasi Berbasis <i>Centroid</i>	84
Gambar 4.16 Hasil Ekstraksi Fitur Setelah PCA	85
Gambar 4.17 Hasil Evaluasi Klasifikasi Gambar Utuh	86
Gambar 4.18 Visualisasi Contoh Klasifikasi yang Akurat	87
Gambar 4.19 Visualisasi Contoh Klasifikasi yang Tidak Benar	87
Gambar 4.20 Analisis Ekstraksi Fitur <i>Dataset</i> Positif Setelah PCA	88
Gambar 4.21 Analisis Ekstraksi Fitur <i>Dataset</i> Negatif Setelah PCA.....	89
Gambar 4.22 Hasil <i>Discriminative Clustering</i> dengan Tidak Memakai Filter.....	90

Gambar 4.23 Analisis Hasil <i>Discriminative Clustering</i> Tanpa Menggunakan <i>Padding</i> ...	91
Gambar 4.24 Distribusi Jarak <i>Patch</i> Ke <i>Centroid</i>	92
Gambar 4.25 <i>Top 10 Patch</i> Terdekat ke <i>Centroid</i>	93
Gambar 4.26 Hasil <i>Confusion Matrix Hybrid Voting</i> Model dengan <i>Padding</i>	94
Gambar 4.27 <i>Output Program</i>	96

DAFTAR TABEL

Tabel 2.1 Penelitian Terkait	3
Tabel 2.2 <i>Confusion Matrix</i>	17
Tabel 4.1 Eksperimen Parameter di CLAHE pada <i>Dataset</i> Positif	61
Tabel 4.2 Eksperimen Parameter di CLAHE pada <i>Dataset</i> Negatif.....	62
Tabel 4.3 Hasil <i>Resize</i> pada <i>Dataset</i> Positif	63
Tabel 4.4 Hasil <i>Resize</i> pada <i>Dataset</i> Negatif.....	64
Tabel 4.5 Visualisasi Gambar Setelah Praproses pada <i>Dataset</i> Positif	65
Tabel 4.6 Visualisasi Gambar Setelah Praproses pada <i>Dataset</i> Negatif.....	66
Tabel 4.7 Hasil Eksperimen Parameter Deteksi <i>Patch</i>	67
Tabel 4.8 <i>Top 3</i> Hasil Evaluasi <i>Silhouette Score</i>	72
Tabel 4.9 Eksperimen Konfigurasi α dan β <i>Hybrid Voting</i>	78
Tabel 4.10 Hasil Evaluasi <i>Hybrid Voting</i> pada Gambar Utuh.....	79
Tabel 4.11 Metode Pembandingan	81
Tabel 4.12 Hasil Evaluasi Klasifikasi SVM-Only	82
Tabel 4.13 Hasil Evaluasi Klasifikasi Berbasis <i>Centroid</i>	83
Tabel 4.14 Hasil Evaluasi Klasifikasi Gambar Utuh	86
Tabel 4.15 Hasil Evaluasi Model <i>Hybrid Voting</i> dengan <i>Padding</i>	94
Tabel 4.16 Perbandingan Semua Metode Klasifikasi	97

(Halaman ini sengaja dikosongkan)

DAFTAR PSEUDOCODE

Kode Semu 3.1 Praproses <i>Dataset</i> Positif (seni Jepang)	35
Kode Semu 3.2 Praproses <i>Dataset</i> Negatif (seni Cina)	36
Kode Semu 3.3 Deteksi <i>Patch Dataset</i> Positif dan Negatif.....	38
Kode Semu 3.4 Implementasi Ekstraksi Fitur pada <i>Dataset</i> Positif dan <i>Dataset</i> Negatif.	39
Kode Semu 3.5 Implementasi <i>Discriminative Clustering</i>	41
Kode Semu 3.6 Pembuatan Model.....	42
Kode Semu 3.7 Klasifikasi <i>Hybrid Voting</i> pada <i>Dataset</i> Uji	44
Kode Semu 3.8 Klasifikasi <i>Hybrid Voting</i> pada <i>Dataset</i> Asli.....	46
Kode Semu 3.9 Klasifikasi SVM pada <i>Dataset</i> Uji.....	47
Kode Semu 3.10 Klasifikasi SVM dengan <i>Dataset</i> Asli	49
Kode Semu 3.11 Klasifikasi <i>Dataset</i> Uji Menggunakan Kedekatan <i>Centroid</i>	50
Kode Semu 3.12 Implementasi Klasifikasi Berbasis <i>Centroid</i> pada <i>Dataset</i> Gambar Asli	52
Kode Semu 3.13 Implementasi Pembuatan Model Klasifikasi Gambar Utuh.....	53
Kode Semu 3.14 Implementasi Pembuatan Model Tanpa Filtrasi <i>Padding</i>	55
Kode Semu 3.15 Implementasi Klasifikasi Model dengan Filtrasi <i>Padding</i> pada <i>Dataset</i> Uji.....	56
Kode Semu 3.16 Implementasi Klasifikasi Model dengan Filtrasi <i>padding</i> pada <i>Dataset</i> Gambar Utuh	58

(Halaman ini sengaja dikosongkan)

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Seniman Jepang melukiskan seninya dengan mencerminkan filosofi hidup, hubungan manusia dengan alam, dan nilai-nilai budaya tradisional melalui karya lukisnya. Dua gaya seni yang paling menonjol pada periode Edo adalah *ukiyo-e* dan *sumi-e*, yang keduanya mewakili karakteristik visual khas dari seni lukis tradisional Jepang. *Ukiyo-e* menggambarkan kehidupan sehari-hari di kota dengan komposisi yang mencolok serta teknik *woodblock printing* yang diwarnai menggunakan pigmen alami seperti tumbuhan dan mineral (Wakita et al., 2014). Keunikan *ukiyo-e* juga terlihat dari struktur komposisinya yang sering kali tidak simetris atau bahkan tampak tidak sempurna (Hou & Qian, 2023; Zhuang et al., 2021). Sebaliknya, *sumi-e* menekankan kesederhanaan dan ekspresi mendalam melalui tinta hitam pada latar kosong (Lucas, 2024; Okamoto, 1996). Di sisi lain, seni lukis tradisional Cina memiliki keunikan yang khas baik dari segi teknik, filosofi, maupun ekspresi estetikanya. Komposisinya kerap menampilkan lanskap, flora, dan fauna dengan pendekatan perspektif atmosferik (*shan shui*), yang memberikan kesan kedalaman tanpa mengikatkan pandangan pada satu titik saja. Teknik yang digunakan juga erat kaitannya dengan kaligrafi, di mana sapuan kuas memiliki makna dan nilai estetika tersendiri (Lee, 1962). Pemahaman karakteristik dan elemen visual khas dari kedua budaya tersebut baik Jepang maupun Cina masih menjadi tantangan tersendiri, terutama dalam mengidentifikasi perbedaan unsur utama di antara keduanya. Hal ini disebabkan oleh kemiripan dalam elemen visual seperti pola, tekstur, dan penggunaan ruang yang kadang tumpang tindih.

Dengan kemajuan teknologi, analisis lukisan dan bangunan kini tidak hanya bergantung pada subjektivitas manusia, tetapi juga dapat didukung oleh pendekatan berbasis teknologi seperti dengan penggunaan pembelajaran mesin. Sebagai contoh, penelitian dari Doersch et al., (2015) berhasil mengidentifikasi elemen visual unik dalam arsitektur kota Paris menggunakan hasil dari *discriminative clustering*. Setelah itu terdapat penelitian dari Shinichi & Matsui, (2024) menunjukkan potensi *deep learning* dalam menganalisis kreativitas karya *ukiyo-e* melalui jaringan hubungan antar karya seni. Demikian pula, penelitian dalam Hidayati et al., (2024) menunjukkan keberhasilan pendekatan pembelajaran ansambel berbasis *stacking* dalam mengidentifikasi gaya seniman dari lukisan.

Tugas Akhir ini bertujuan untuk menjawab tantangan dalam mengidentifikasi karakteristik visual khas dari seni lukis tradisional Jepang dan Cina yang kerap terlihat serupa. Kedua tradisi seni ini berasal dari akar budaya Asia Timur dan sama-sama menggunakan teknik kuas serta elemen visual seperti komposisi ruang, pola, dan tekstur yang sekilas tampak mirip. Oleh karena itu, diperlukan pendekatan berbasis pembelajaran mesin untuk mengenali dan mengidentifikasi keunikan dari masing-masing gaya lukis tersebut secara lebih mendalam. Dalam penelitian ini, elemen-elemen visual yang diekstraksi dari seni Jepang dianalisis menggunakan kombinasi metode seperti klasifikasi dengan *Hybrid Voting* yang menggabungkan Klasifikasi berdasarkan *Support Vector Machine* (SVM) dan Berbasis *Centroid* yang memakai hasil dari *discriminative clustering* berbasis *K-Means*. Penggunaan *Cosine Distance* juga dipakai dalam Tugas Akhir ini sebagai pengukuran similaritas membantu membedakan elemen visual. Selain itu, penerapan evaluasi model *clustering* menggunakan *silhouette score* dan model klasifikasi menggunakan metrik seperti akurasi, presisi, *recall*, dan *F1-score* memastikan efektivitas model dalam mengenali karakteristik visual seni Jepang. Hasil dari penelitian ini diharapkan dapat memberikan dasar untuk pengembangan model analisis seni berbasis teknologi yang lebih luas, mencakup seni tradisional dari berbagai budaya. Pendekatan ini juga relevan untuk studi lintas disiplin yang menggabungkan seni, teknologi, dan budaya, sehingga membuka jalan untuk inovasi dalam penelitian dan aplikasi konservasi budaya.

1.2 Rumusan Masalah

Rumusan masalah dari penelitian ini adalah sebagai berikut:

1. Bagaimana mengidentifikasi karakteristik visual yang membedakan lukisan Jepang dari lukisan Cina?
2. Bagaimana elemen-elemen dari seni Jepang dan seni Cina berkontribusi dalam proses klasifikasi?
3. Bagaimana hasil dan evaluasi kinerja model yang dibangun.

1.3 Batasan Masalah

Batasan masalah dari penelitian ini adalah sebagai berikut:

1. Dataset yang digunakan mencakup *dataset* seni Jepang yaitu *Dataset Ukiyo-e*¹, *Dataset Sumi-e*², dan *Dataset* seni Cina³.
2. Gambar dalam *dataset* seni Cina mengalami kompresi atau memiliki resolusi rendah, yang dapat mempengaruhi hasil analisis visual

1.4 Tujuan

Tujuan dari penelitian ini adalah sebagai berikut:

1. Mengidentifikasi karakteristik visual yang membedakan seni lukis Jepang dari seni lukis Cina.
2. Menganalisis kontribusi elemen-elemen visual khas seni Jepang dan seni Cina dalam proses klasifikasi untuk mendukung pengembangan model klasifikasi.
3. Mengevaluasi kinerja model klasifikasi yang dibangun menggunakan metrik seperti akurasi, presisi, *recall*, *F1-score* dan *confusion matrix* untuk menilai efektivitasnya dalam mengklasifikasikan seni lukis Jepang.

1.5 Manfaat

Manfaat Tugas Akhir ini diharapkan dapat memberikan kontribusi akademik dalam bidang analisis seni rupa berbasis teknologi, khususnya pada seni lukis Jepang dan seni Cina, dengan menyediakan referensi untuk Tugas Akhir lanjutan terkait klasifikasi seni

¹ <https://www.kaggle.com/datasets/kengoichiki/the-metropolitan-museum-of-art-ukiyoe-dataset/code>

² https://huggingface.co/datasets/takara-ai/Sumi-e_no_kurozuappu_ink_painting_closeups

³ <https://github.com/ychen93/Chinese-Painting-Dataset>

BAB 2 TINJAUAN PUSTAKA

Pada bab ini mengeksplorasi studi literatur dari penelitian sejenis yang telah dilakukan sebelumnya, sambil mengelaborasi prinsip-prinsip teoretis yang menjadi acuan dalam pengembangan Tugas Akhir ini

2.1 Hasil Penelitian Terdahulu

Pada pengerjaan Tugas Akhir ini digunakan pendekatan komparatif dengan memanfaatkan data dari studi-studi sebelumnya yang telah dirangkum dalam Tabel 2.1 sebagai acuan pengujian. Kajian literatur mencakup informasi yang dihimpun dari berbagai sumber kredibel, seperti jurnal, buku, dan skripsi.

Tabel 2.1 Penelitian Terkait

Judul	Hasil Penelitian
Stacking-based Ensemble Learning for identifying Artist Signatures on Paintings (Hidayati <i>et al.</i> , 2024)	Penelitian ini bertujuan untuk mengidentifikasi <i>style</i> seniman pada lukisan menggunakan pendekatan <i>ensemble stacking</i> yang menggabungkan tiga model CNN, yaitu EfficientNetB4, ResNet-50, dan Xception. Perbedaan utama dengan Tugas Akhir ini terletak pada konteks dan jenis data yang dianalisis. Penelitian ini berfokus pada <i>style</i> seniman untuk autentikasi seniman, sementara Tugas Akhir ini berfokus pada analisis elemen visual khas seni Jepang, seperti pola dekoratif pada <i>ukiyo-e</i> dan penggunaan ruang kosong pada <i>sumi-e</i> . Pendekatan Tugas Akhir ini mengintegrasikan ekstraksi fitur menggunakan model VGG16 dan klasifikasi dengan SVM untuk membedakan elemen visual antara seni Jepang (<i>dataset</i> positif) dan seni dari berbagai dunia (<i>dataset</i> negatif). <i>Output</i> dari Tugas Akhir ini adalah model yang mampu mengenali elemen visual utama dari dua gaya seni Jepang, sekaligus membedakan karakteristiknya dari seni lainnya.
Deep Ensemble Art Style Recognition (Menis-Mastromichalakis <i>et al.</i> , 2020)	Penelitian ini mengeksplorasi pengenalan gaya seni secara umum dengan membandingkan kinerja berbagai arsitektur CNN, seperti ResNet dan DenseNet, menggunakan <i>dataset</i> WikiArt. Analisis berfokus pada performa klasifikasi gaya seni global melalui teknik <i>ensemble stacking</i> , dengan <i>output</i> berupa model klasifikasi gaya seni. Perbedaan utama dengan Tugas Akhir ini terletak pada konteks analisis. Tugas Akhir ini berfokus pada dua gaya seni lukis Jepang dengan pendekatan deteksi <i>patch</i> untuk menonjolkan elemen visual khas dalam seni lukis Jepang, seperti tekstur tinta dalam <i>sumi-e</i> dan objek dekoratif dalam <i>ukiyo-e</i> . <i>Output</i> dari Tugas Akhir ini adalah model yang mampu mengidentifikasi elemen visual kunci dari gaya seni <i>ukiyo-e</i> dan <i>sumi-e</i> .
A Deep Learning Approach to Quantitative Analysis of Japanese Artworks: Exploring	Penelitian ini berfokus pada analisis kreativitas seni <i>ukiyo-e</i> dengan menggunakan pendekatan berbasis jaringan hubungan antar karya seni. Penelitian ini membangun dua jenis jaringan, yaitu <i>Cosine Similarity Network</i> dan <i>Cluster Similarity Network</i> , untuk mengukur pengaruh historis dan kreativitas seniman <i>ukiyo-e</i> . <i>Output</i> dari penelitian ini adalah

the Latent Features of Ukiyo-e Artists (Shinichi & Matsui, 2024)	identifikasi seniman dengan kreativitas tinggi serta analisis pola kreativitas yang menunjukkan penurunan sejak akhir 1840-an. Fokus utama analisisnya adalah hubungan antar seniman dan evolusi seni <i>ukiyo-e</i> dalam konteks sejarah. Perbedaan utama dengan Tugas Akhir ini terletak pada fokus analisis. Tugas Akhir ini berfokus pada pengembangan model klasifikasi seni Jepang yang mencakup gaya lukis <i>ukiyo-e</i> dan <i>sumi-e</i> . Selain itu, <i>output</i> dari Tugas Akhir ini adalah model klasifikasi yang mampu mengidentifikasi elemen visual utama dari seni lukis <i>ukiyo-e</i> dan <i>sumi-e</i> .
What Makes Paris Look Likes Paris? (Doersch <i>et al.</i> , 2015)	Penelitian ini bertujuan untuk menemukan elemen visual khas geografis, seperti balkon dan jendela unik di Paris, menggunakan pendekatan <i>clustering</i> diskriminatif. Analisis berfokus pada elemen-elemen visual yang sering muncul dan membedakan suatu lokasi geografis. <i>Output</i> dari penelitian ini adalah kumpulan elemen visual khas geografis yang membantu membedakan satu kota dari kota lainnya, seperti Paris dan London. Perbedaan utama dengan Tugas Akhir ini terletak pada konteks dan jenis data yang dianalisis. Penelitian Doersch mengkaji elemen visual arsitektur perkotaan dan geografis, sementara Tugas Akhir ini berfokus pada seni lukis Jepang dengan menganalisis elemen visual seni, seperti teknik kuas dan komposisi ruang kosong. <i>Output</i> dari Tugas Akhir ini adalah kumpulan elemen visual dari dua gaya seni lukis Jepang, yaitu <i>ukiyo-e</i> dan <i>sumi-e</i> .

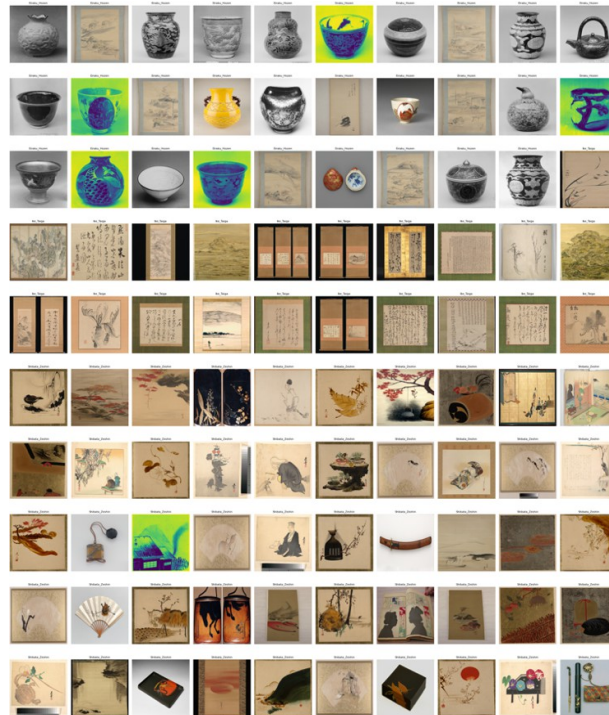
2.2 Dasar Teori

Subbab ini menguraikan landasan teori yang digunakan dalam penyusunan Tugas Akhir. Landasan teori yang dibahas meliputi *ukiyo-e*, *sumi-e*, gaya lukis Cina, klasifikasi menggunakan SVM, klasifikasi menggunakan *hybrid voting*, deteksi *patch*, ekstraksi fitur, pengukuran similaritas, *clustering*, dan evaluasi model.

2.2.1 Gaya Lukis Seni Jepang

Pada masa periode Edo di Jepang, terdapat dua gaya seni yang berkembang pada masa tersebut yaitu *ukiyo-e* dan *sumi-e*. Kedua gaya seni tersebut memiliki karakteristik yang cukup berbeda. *Ukiyo-e* adalah jenis lukisan yang menggambarkan kehidupan sehari-hari di Jepang dengan istilah *ukiyo* yang dapat diartikan sebagai “kehidupan dunia” atau “hidup untuk saat ini” (Hou & Qian, 2023). Gaya seni lukis ini merupakan warisan budaya era modern Jepang yang awalnya dibuat melalui teknik cetak kayu (*woodblock printing*) yang diwarnai dengan bahan-bahan alami seperti pigmen dari tumbuhan, kerang, dan mineral (Wakita *et al.*, 2014). Struktur Komposisi dalam pembuatan lukisan dapat dikatakan unik karena seni *ukiyo-e* memiliki daya tarik yang terletak pada komposisi yang tidak sempurna (Zhuang *et al.*, 2021). “Ketidaksempurnaan” dalam seni Jepang dipakai karena potensinya dalam memberikan inspirasi; unsur yang dibiarkan tidak lengkap dengan sengaja memungkingkan imajinasi bebas untuk melengkapinya (Hou & Qian, 2023). Teknik ini, berbeda dengan prinsip realisme dalam seni lukis Barat karena pada seni lukis Barat keseluruhan subjek terdapat makna sendiri yang lebih dalam (Huang *et al.*, 2014; Ming Zhang & Gunturk, 2008; Zhang *et al.*, 2023). *Sumi-e* adalah gaya lukis dalam seni Jepang yang memadukan teknik kuas dengan ruang kosong yang dipertimbangkan untuk menangkap makna suatu objek secara alami, bukan hanya menggambarkan objek secara realistis (Lucas, 2024). Seni *sumi-e* bergantung pada penggunaan

kuas, teknik pencampuran tinta, dan pola pikir yang mendukung kreativitas dan meditasi. Lukisan dengan gaya *Sumi-e* memiliki tujuan untuk menggambarkan dinamika kehidupan dengan pelukisnya diharapkan untuk menyatu dengan seni yang dilukis serta mencerminkan pemahaman mendalam akan alam dan kehidupan itu sendiri (Okamoto, 1996). Gambar 2.1 mempresentasikan gambar dari *dataset Ukiyo-e* yang akan digunakan, yang terdiri dari berbagai jenis karya seni tradisional Jepang seperti lukisan, kaligrafi, dan keramik. Setiap baris dalam gambar tersebut dikelompokkan berdasarkan senimannya.



Gambar 2.1 *Dataset Ukiyo-e*



Gambar 2.2 *Dataset Sumi-e*

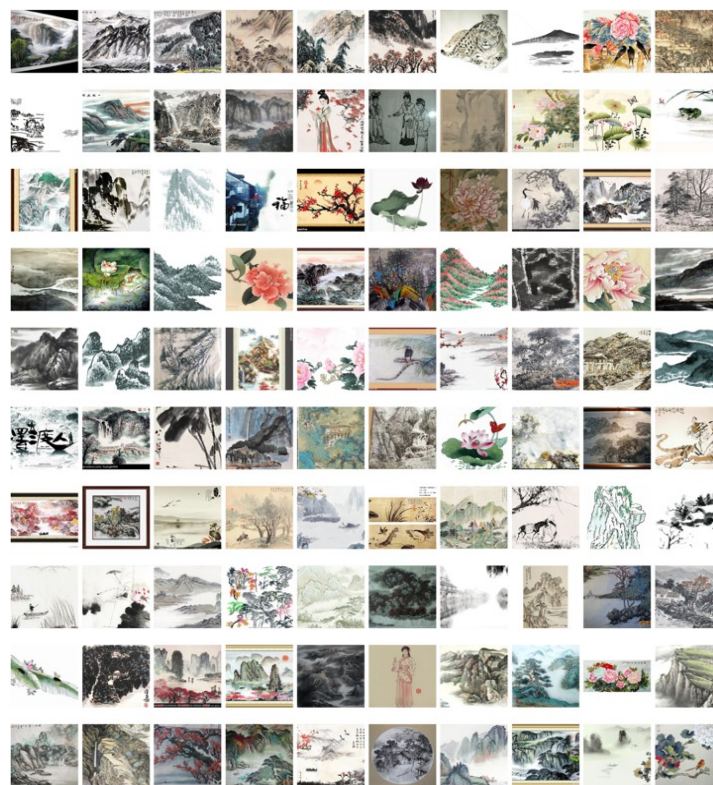
Gambar 2.2 menampilkan *dataset sumi-e* dengan elemen alam khasnya yaitu seperti bambu, pegunungan, dan pepohonan. Komposisi warna hitam putih menjadi ciri khas gaya lukis seni *sumi-e*. Perkembangan kedua gaya seni ini tidak dapat dipisahkan dari konteks sosial politik

periode Edo. Seni Jepang dipengaruhi oleh alam karena masyarakat pada masa tersebut menganggap alam sebagai harta karun karena keindahannya. Berbeda dengan seni Barat yang berlandaskan simetri dengan keseimbangan menurut prinsip dari peradaban Yunani kuno, seni Jepang menampilkan keindahan alami dengan komposisi asimetris (Hou & Qian, 2023).

2.2.2 Gaya Lukis Seni Cina

Seni lukis Cina merupakan salah satu bentuk seni rupa tertua yang berkembang dengan filosofi, teknik, dan keindahan yang khas. Berbeda dengan seni lukis Barat yang cenderung menekankan realisme, seni lukis Cina berfokus pada ekspresi perasaan, keindahan garis, serta hubungan harmonis antara manusia dan alam (Pearce *et al.*, 2018). Seni ini memiliki kaitan erat dengan kaligrafi, di mana sapuan kuas yang digunakan dalam melukis mengikuti prinsip yang sama dengan cara menulis karakter Cina (Lee, 1962).

Salah satu teknik utama dalam seni lukis Cina adalah penggunaan kuas dan tinta. Zhao Xigu, seorang sarjana dan seniman dari Dinasti Song, menjelaskan bahwa kesenangan dalam melihat karya seni Cina tidak hanya terletak pada warna, tetapi juga pada kehalusan garis dan permainan nada dalam tinta (Pearce *et al.*, 2018). Teknik *xieyi* atau "gaya spontan" sering digunakan dalam lukisan lanskap dan hewan, menampilkan goresan kuas yang cepat dan penuh ekspresi. Sebaliknya, *gongbi* adalah teknik yang lebih detail dan presisi, sering digunakan untuk menggambarkan objek secara realistis dengan garis yang lebih halus dan terkontrol (Lee, 1962).

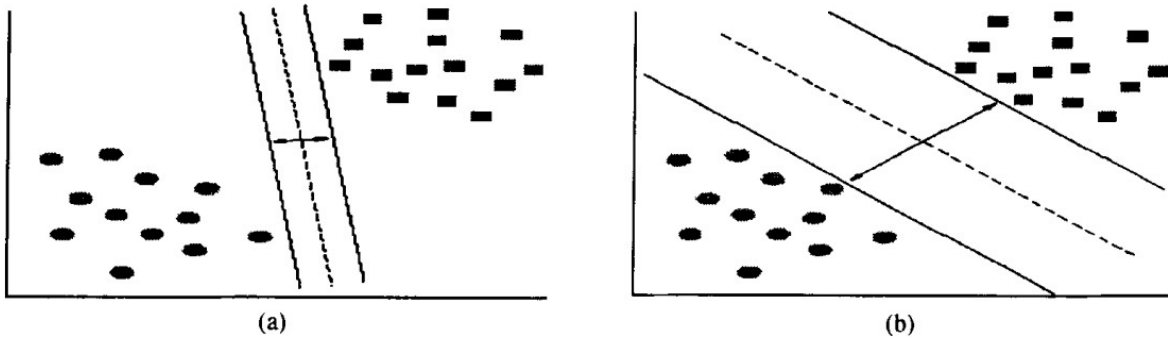


Gambar 2.3 *Dataset Lukisan Cina* (G. Wang *et al.*, 2018).

Gambar 2.3 menampilkan *dataset* gaya lukis Cina yang memiliki gradasi halus. Perbedaan mencolok lainnya antara seni lukis Cina dan Barat adalah pendekatan terhadap ruang dan perspektif. Dalam seni lukis Barat, perspektif linier digunakan untuk menciptakan ilusi kedalaman, sedangkan dalam seni lukis Cina, perspektif atmosferik atau *shan shui* digunakan untuk menampilkan landscape dengan berbagai lapisan kedalaman yang tidak terikat pada satu titik pandang tunggal.

2.2.3 Klasifikasi *Patch-Based* dengan SVM

Support Vector Machines (SVM) adalah metode populer dalam pembelajaran mesin yang digunakan untuk tugas klasifikasi dan regresi. SVM menggunakan pendekatan yang didasarkan pada teori pembelajaran statistik. Algoritma SVM dikembangkan oleh Vapnik dan koleganya di AT&T Bell Laboratories pada 1990-an. Metode ini terbukti efektif untuk klasifikasi gambar dengan perkembangan perbaikannya selama beberapa puluh tahun terakhir yang menghasilkan peningkatan akurasi klasifikasi sepanjang waktu (Chandra & Bedi, 2021). SVM bekerja dengan prinsip menemukan *hyperplane* optimal yang memisahkan dua kelas dalam ruang fitur. *Hyperplane* optimal dalam SVM adalah bidang yang memaksimalkan margin antara dua kelas data yang menggunakan vektor pendukung (*support vectors*) sebagai penentu utamanya (Chandra & Bedi, 2021).



Gambar 2.4 *Hyperplane Optimal* (Yue et al., 2003).

Gambar 2.4 menunjukkan konsep *hyperplane* optimal dalam algoritma SVM untuk memisahkan dua kelas data. Panel (a) dan (b) sebenarnya menggambarkan situasi yang sama hanya saja dengan perbedaan perspektif. Panel (a) memberikan tampilan lebih luas (*zoom out*) dari distribusi data, yang menunjukkan keseluruhan margin dan distribusi dua kelas data (lingkaran hitam dan kotak hitam). Panel (b) memberikan tampilan lebih dekat (*zoom in*) pada *hyperplane* dan margin, fokus pada jarak antara garis solid (*hyperplane*) dan garis putus-putus yang merepresentasikan margin maksimum. Margin ini dihitung sebagai jarak terpendek dari *hyperplane* ke titik data terdekat di masing-masing kelas, yang disebut *support vectors*. Gambar ini membantu menjelaskan bagaimana *hyperplane* optimal memaksimalkan margin untuk meningkatkan kemampuan model dalam umumnya menghadapi data baru.

Secara matematis, *hyperplane* dalam SVM didefinisikan melalui persamaan (2.1), di mana w adalah vektor bobot yang menentukan arah kemiringan bidang, x_i merupakan data atau vektor fitur input (seperti fitur ekstraksi dari gambar lukisan), dan b adalah angka yang menentukan posisi bidang pemisah itu di ruang fitur. Untuk mencapai *hyperplane* optimal, SVM menggunakan fungsi (2.3), yang terdiri dari dua bagian utama yaitu (1) $\frac{1}{2}\|w\|^2$ bertujuan memperlebar margin dengan mengecilkan norma vektor w , dan (2) $C\sum \xi_i$ meminimalkan kesalahan klasifikasi melalui *slack variable* atau pelanggaran margin ξ_i . ξ_i nilai yang menunjukkan seberapa banyak titik data melanggar margin. Parameter C adalah parameter pengatur toleransi kesalahan.

$$w \cdot x_i + b = 0 \quad (2.1)$$

$$\min \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad (2.2)$$

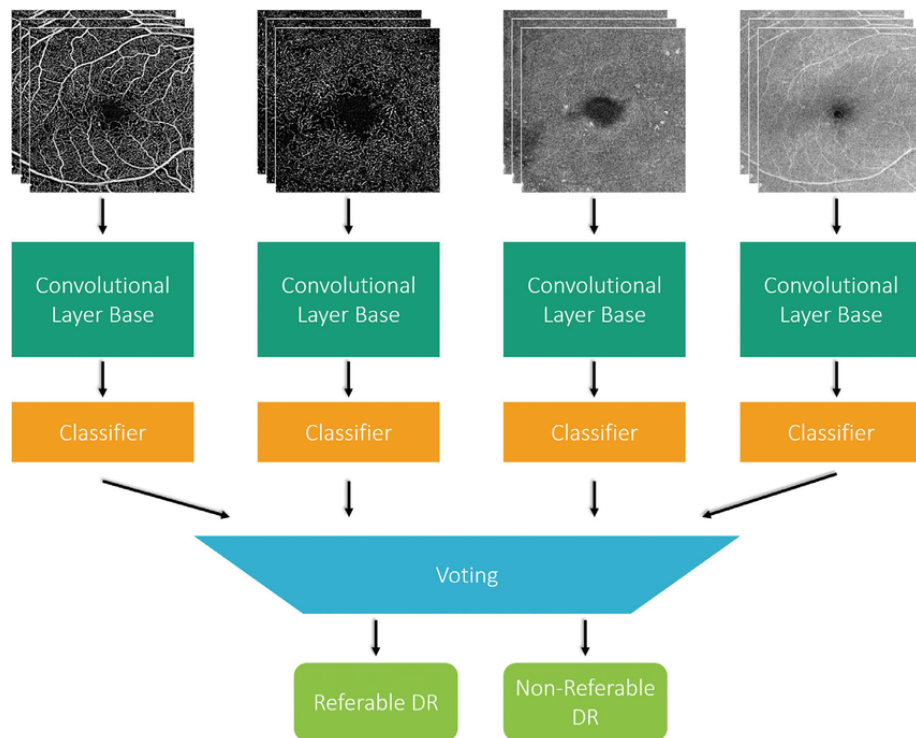
Formula (2.3) sebagai batasan $y_i(w \cdot x_i + b) \geq 1 - \xi_i$ dengan $\xi_i \geq 0$ menjamin bahwa mayoritas data masih diklasifikasikan dengan benar, yaitu berada di sisi *hyperplane* yang sesuai dengan labelnya. Nilai $\xi_i = 0$ menunjukkan titik yang terklasifikasi dengan benar dan tidak

menyentuh atau melewati margin, sedangkan $\xi_i > 0$ mengindikasikan pelanggaran margin atau salah klasifikasi. Semakin besar ξ_i , semakin signifikan kesalahan klasifikasinya. Dalam konteks penelitian ini, formulasi ini memungkinkan SVM membedakan karakteristik visual seni Jepang (seperti *ukiyo-e* dan *sumi-e*) dari seni Cina, bahkan ketika fiturnya memiliki kemiripan tertentu. Dengan memaksimalkan margin dan meminimalkan kesalahan, model menjadi lebih baik dalam mengidentifikasi pola-pola unik dari setiap gaya seni.

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (2.3)$$

2.2.4 Hybrid Voting

Model *hybrid* merupakan gabungan dari dua atau lebih metode klasifikasi yang dirancang untuk menciptakan sistem prediksi dengan tingkat akurasi yang lebih tinggi. Dengan memadukan berbagai *classifier* individu menjadi sebuah *ensemble classifier*, pendekatan ini mampu meningkatkan performa klasifikasi (Qi & Tang, 2018). Dalam metode *hybrid*, terdapat salah satu metode utama yaitu *voting*. Metode *voting* bekerja dengan mengumpulkan beberapa *classifier* yang kemudian memberikan prediksi secara independen, dimana hasil akhir ditentukan berdasarkan suara terbanyak (*majority vote*) dari seluruh *classifier* yang terlibat (Sajana et al., 2021). Secara teknis, proses ini dilakukan dengan menghitung jumlah *vote* yang diperoleh setiap kelas keluaran dari masing-masing *classifier*, kemudian kelas dengan *vote* tertinggi akan ditetapkan sebagai hasil klasifikasi akhir (Qi & Tang, 2018). Keunggulan utama pendekatan *hybrid* ini terletak pada kemampuannya untuk memanfaatkan kelebihan masing-masing *classifier*, sehingga menghasilkan sistem prediksi yang lebih baik dibandingkan penggunaan model tunggal. Adapun model *hybrid* berbasis *voting* ini sangat efektif untuk masalah klasifikasi, di mana keragaman kemampuan masing-masing model dimanfaatkan untuk mengolah data yang kompleks dan heterogen. Dengan memadukan kekuatan berbagai pendekatan, sistem *hybrid* mampu menghasilkan prediksi yang lebih akurat dibandingkan penggunaan model tunggal (Du et al., 2022).



Gambar 2.5 Arsitektur *Hybrid Voting Ensemble* untuk Klasifikasi Gambar menggunakan CNN (Heisler et al., 2020).

Gambar 2.5 menggambarkan penerapan *Hybrid Voting* dalam model klasifikasi citra. Setiap gambar yang dianalisis diproses melalui beberapa *Convolutional Layer Base*, yang berfungsi untuk mengekstraksi fitur visual dari citra tersebut. Hasil ekstraksi fitur ini kemudian diteruskan ke *classifier* untuk setiap model yang terlibat, di mana setiap *classifier* memberikan prediksi secara independen berdasarkan fitur yang telah diekstraksi. Setelah itu, hasil prediksi dari semua *classifier* digabungkan melalui mekanisme *voting*. Proses *voting* ini menentukan kelas akhir citra berdasarkan suara terbanyak atau probabilitas yang dihitung dari hasil prediksi tiap *patch* citra. Hasil akhir dari proses ini adalah penentuan apakah citra tersebut termasuk dalam kategori *Referable DR* (Diabetic Retinopathy yang perlu ditindaklanjuti) atau *Non-Referable DR* (Diabetic Retinopathy yang tidak memerlukan tindakan lebih lanjut) (Heisler et al., 2020). Gambar ini relevan dengan penelitian yang sedang dilakukan, di mana *Hybrid Voting* digunakan untuk menggabungkan hasil prediksi dari berbagai model untuk meningkatkan akurasi klasifikasi. Dalam konteks penelitian ini, pendekatan serupa bisa diterapkan untuk mengklasifikasikan elemen dalam seni lukis Jepang, seperti gaya *ukiyo-e* dan *sumi-e*, dengan menggabungkan fitur visual yang diekstraksi dari citra seni tersebut melalui teknik klasifikasi berbasis CNN dan SVM.

Lebih lanjut, *Hybrid Voting Centroid-Based* memperluas konsep ini dengan mengintegrasikan *centroid* kelas sebagai voter tambahan. *Centroid* yang dihitung dari fitur lukisan memberikan *vote* berbasis kemiripan kosinus yaitu menilai seberapa mirip fitur gambar uji dengan rata-rata fitur tiap kelas, sementara CNN berfokus pada fitur visual. Inti dari *Centroid-Based Classification* (CBC) adalah bahwa suatu sampel tak berlabel seharusnya dialokasikan berdasarkan kemiripan antara sampel tersebut dengan *centroid* kelas tersebut merupakan yang terkecil (berdasarkan *cosine distance*). Dibandingkan dengan metode *Text Categorization* (TC) lainnya, CBC lebih efisien karena kompleksitas komputasinya bersifat linear selama fase pelatihan. Keunggulan ini sangat penting untuk tugas klasifikasi teks dalam skala besar (Setiawan et al., 2017).

Meskipun awalnya dirancang untuk teks, CBC dapat diadaptasi untuk klasifikasi gambar dengan memanfaatkan fitur visual sebagai pengganti kata. Penelitian Dolezel et al. (2022) mengimplementasikan metode *Centroid-Based Classification* (CBC) melalui pendekatan *Fully Convolutional Network* (FCN) berbasis *U-Net* yang secara khusus dikembangkan untuk deteksi orang dalam gambar ortogonal. Sistem ini dirancang untuk memproses gambar input beresolusi 288×288 piksel dalam format *grayscale*, kemudian menghasilkan peta lokalisasi yang memprediksi posisi *centroid* kepala manusia dengan nilai probabilitas antara 0 hingga 1, dimana nilai yang mendekati 1 menunjukkan titik pusat objek dengan tingkat kepercayaan tinggi. Proses deteksi diawali dengan generasi peta lokalisasi oleh *U-Net*, dilanjutkan dengan tahap pemurnian hasil menggunakan filter maksimum berukuran 10×10 dan operasi *thresholding* pada nilai 0.65 untuk menyaring *noise* dan *false positive*, serta diakhiri dengan ekstraksi posisi *centroid* yang disesuaikan skalanya sesuai resolusi gambar asli (Dolezel et al., 2022). Dalam pelatihan model, diterapkan fungsi kerugian *binary cross-entropy* dengan data *ground truth* berbentuk elips gradien yang diturunkan dari *bounding box*, serta teknik augmentasi data seperti rotasi dan *flipping* gambar untuk meningkatkan *robustness* sistem. Keunggulan utama metode CBC ini terletak pada kecepatan pemrosesan yang tinggi sehingga cocok untuk perangkat *edge computing*, akurasi deteksi yang lebih unggul dibanding metode berbasis *bounding box* konvensional, dan efisiensi komputasi yang didapat dari penyederhanaan proses deteksi menjadi prediksi titik tunggal. Dengan karakteristik tersebut, CBC terbukti efektif sebagai solusi untuk aplikasi *real-time* seperti sistem penghitungan orang di transportasi umum atau area publik lainnya yang membutuhkan keseimbangan antara akurasi dan kecepatan pemrosesan (Dolezel et al., 2022).

2.2.5 Praproses Data

Praproses data merupakan tahap yang mencakup langkah-langkah seperti segmentasi data dan transformasi data untuk menghasilkan input yang sesuai standarisasi sebelum melanjutkan ke analisis yang lebih mendalam (Zheng et al., 2018). Pada Tugas Akhir ini praproses data mencakup berbagai teknik seperti mengubah ukuran gambar, peningkatan kontras, dan normalisasi. *Image resizing* adalah proses penyesuaian ukuran gambar yang bertujuan mengoptimalkan visualisasi sambil menjaga struktur dan elemen penting dalam gambar (G. Wang et al., 2018; Q. Wang & Yuan, 2014). Hal tersebut dilakukan untuk memastikan bahwa gambar memiliki resolusi yang sesuai untuk pemrosesan lebih lanjut tanpa kehilangan detail. Terdapat beberapa metode *resizing* gambar. Metode *resizing* gambar tradisional seperti *scaling* dan *cropping* memiliki kelemahan, seperti distorsi yang terlihat jika aspek rasio input dan *output* berbeda atau kehilangan informasi penting karena *cropping* (Dong et al., 2009). Metode *resizing* gambar modern mencakup interpolasi seperti *bilinear* dan *bicubic spline* yang digunakan untuk meningkatkan resolusi gambar yang dihasilkan oleh *DoFP polarimeters*. Metode interpolasi *bicubic spline* memberikan performa terbaik dalam hal resolusi spasial dan akurasi gambar dibandingkan dengan metode *bilinear* dan *weighted bilinear* (Gao & Gruev, 2011).

Teknik interpolasi modern seperti *bilinear* dan *bicubic spline* menjadi standar dalam meningkatkan resolusi gambar, di mana metode *bicubic* menggunakan 16 piksel tetangga (4×4) untuk menghasilkan tepian lebih halus dan preservasi detail yang lebih baik, sehingga banyak diaplikasikan dalam pengeditan profesional dan kamera digital (Imocha Singh et al., 2009). Perkembangan interpolasi *bicubic* basis spline, yang dikonseptualisasikan Schoenberg (1946) sebagai potongan polinomial kubik tersambung halus, menawarkan penyempurnaan krusial seperti kontinuitas turunan kedua yang lebih baik dan adaptasi fleksibel terhadap karakteristik gambar (Roy et al., 2013). Implementasinya dalam pemrosesan citra modern, termasuk polarimetri dan fotografi presisi tinggi, membuktikan efektivitasnya dalam menyeimbangkan kualitas visual (preservasi 92.7% tekstur asli pada pembesaran 4×) dan efisiensi komputasi (hanya 15% lebih berat dari *bicubic* standar), menjadikannya solusi ideal untuk aplikasi medis, astronomi, dan *computer vision* (Roy et al., 2013).

$$\beta^2(x) = \begin{cases} \left(\frac{2}{3}\right) - 0.5 |x|^2(2 - |x|) & 0 \leq |x| < 1 \\ \left(\frac{1}{6}\right) * (2 - |x|)^3 & 1 \leq |x| < 2 \\ 0 & 2 \leq |x| \end{cases} \quad (2.4)$$

Fungsi *bicubic* basis *spline* diimplementasikan dalam proses pembesaran gambar melalui teknik interpolasi *bicubic*. Saat gambar diperbesar, nilai piksel baru (yang belum ada) dihitung dari 16 tetangga terdekat dalam pola 4×4 piksel. Artinya, setiap piksel baru tidak hanya melihat tetangga langsung, tapi juga piksel di sekitarnya untuk mendapatkan hasil yang lebih akurat. Penerapan terpisah pada sumbu *x* dan *y* untuk memastikan presisi perhitungan. Nilai piksel tersebut ditentukan berdasarkan seberapa dekat posisinya dengan 16 piksel tetangga, di mana piksel yang lebih dekat memberi pengaruh lebih besar. Hasilnya adalah gambar yang diperbesar dengan tampilan yang lebih halus dan tidak pecah. Fungsi basis spline kubik $\beta^2(x)$ pada Persamaan (2.4) digunakan menghitung seberapa besar kontribusi tiap piksel tetangga, dengan nilai yang bergantung pada jarak absolut $|x|$ hingga batas maksimum 2. Dengan ketentuan jika $0 \leq |x| < 1$ maka menggunakan persamaan $\left(\frac{2}{3}\right) - 0.5 |x|^2(2 - |x|)$, jika $1 \leq |x| < 2$ maka menggunakan persamaan $\left(\frac{1}{6}\right) * (2 - |x|)^3$, dan jika $2 \leq |x|$ maka sama dengan 0. Teknik interpolasi bikubik telah menjadi standar industri dalam berbagai aplikasi pengolahan

gambar digital, termasuk perangkat lunak profesional seperti Adobe Photoshop dan *library* pemrosesan gambar OpenCV (Suban, 2019). Metode interpolasi *bicubic spline* memberikan performa terbaik dalam hal resolusi spasial dan akurasi gambar dibandingkan dengan metode *bilinear* dan *weighted bilinear* (Gao & Gruev, 2011).

Setelah itu terdapat pengolahan kontras pada gambar yang berguna untuk meningkatkan kualitas gambar di berbagai bidang seperti penglihatan komputer (*computer vision*), pengenalan pola (*pattern recognition*), pencitraan medis, serta fotografi komputasional (Chang *et al.*, 2018). Proses ini membantu dalam meningkatkan kualitas visual dari gambar secara keseluruhan dan mempermudah analisis. Salah satu metode yang membantu meningkatkan kontras gambar adalah metode CLAHE (*Contrast-Limited Adaptive Histogram Equalization*). Metode yang diusulkan mengintegrasikan CLAHE dengan berbagai teknik penyaringan untuk meningkatkan gambar RGB dengan menghilangkan *noise* dari setiap saluran warna dan meningkatkan kontras gambar (Sonali *et al.*, 2019). Selanjutnya terdapat normalisasi yaitu sebuah metode untuk menyeimbangkan distribusi piksel pada gambar sehingga nilai piksel tersebar merata untuk ekstraksi fitur yang lebih baik. Penerapan teknik normalisasi dapat meningkatkan efisiensi dalam pengklasifikasian gambar (Madusanka *et al.*, 2023). Normalisasi membantu dalam menjaga kualitas gambar dan memastikan bahwa analisis lebih lanjut dapat dilakukan dengan lebih akurat. *Min-Max Normalization* adalah metode transformasi data yang mengatur nilai-nilai atribut ke dalam rentang tertentu, biasanya antara 0 dan 1, berdasarkan nilai minimum dan maksimum dari atribut tersebut (Henderi *et al.*, 2021). *Min-Max Normalization* efektif untuk *dataset* dengan rentang nilai yang besar karena dapat menyelaraskan nilai-nilai ke dalam skala yang sama tanpa mengubah distribusi data.

2.2.6 Deteksi Patch

Deteksi *patch* dalam identifikasi objek melibatkan beberapa tahap yaitu membagi menjadi bagian kecil gambar dan memilih *patch* dengan berbagai objek untuk mengidentifikasi detail objek dengan lebih cepat (Zhang *et al.*, 2023). Metode ini bekerja dengan menemukan dan mengekstraksi bagian-bagian kecil atau *patch* dari gambar yang lebih besar, yang bertujuan untuk memungkinkan analisis lebih spesifik terhadap suatu gambar. Salah satu teknik deteksi *patch* adalah teknik *sliding window*, yaitu teknik pemindaian seluruh gambar menggunakan "jendela" dengan ukuran yang sudah ditentukan. Metode *sliding window* menggunakan jendela pemrosesan kecil yang bergerak di seluruh bidang gambar untuk menghitung fitur lokal, memungkinkan deteksi objek pada berbagai posisi (Glumov *et al.*, 1995, Müller *et al.*, 2018). Teknik ini melakukan ekstraksi *patch*, memberikan representasi dari seluruh gambar. Metode ini berguna untuk mengidentifikasi fitur-fitur yang mungkin tidak terdeteksi pada skala besar. Algoritma pemrosesan gambar menggunakan jendela geser menawarkan solusi yang lebih efisien dibandingkan melihat korelasi langsung dengan ukuran jendela besar (Glumov *et al.*, 1995, Müller *et al.*, 2018). Teknik *sliding window* memiliki sebuah "jendela" atau *region of interest* (ROI) dengan ukuran tetap. Pada setiap posisi yang ditentukan, fitur-fitur dalam jendela diekstraksi sebagai *patch*.

$$\text{Jumlah patch pada sumbu } x = \left\lceil \frac{W - w}{s_x} \right\rceil + 1 \quad (2.5)$$

$$\text{Jumlah patch pada sumbu } y = \left\lceil \frac{H - h}{s_y} \right\rceil + 1 \quad (2.6)$$

$$\text{Totalpatch} = \left(\left\lceil \frac{W - w}{s_x} \right\rceil + 1 \right) \times \left(\left\lceil \frac{H - h}{s_y} \right\rceil + 1 \right) \quad (2.7)$$

Dalam implementasinya, jumlah *patch* yang dihasilkan dapat dihitung menggunakan tiga parameter utama yaitu dimensi gambar (tinggi H dan lebar W), dimensi jendela (tinggi h dan lebar w), serta langkah pergeseran (*stride*) baik secara horizontal (s_x) maupun vertikal (s_y).

Jumlah *patch* pada sumbu horizontal dihitung dengan membagi selisih lebar gambar dan lebar jendela dengan *stride* horizontal, kemudian dibulatkan ke bawah (menggunakan operator *floor*) dan ditambah 1 untuk memastikan semua area gambar tercover. Perhitungan serupa dilakukan untuk sumbu vertikal dengan parameter yang sesuai. Total *patch* yang dihasilkan merupakan perkalian antara jumlah *patch* pada sumbu horizontal dan vertikal.

Parameter *stride* (s_x dan s_y) menentukan seberapa besar *overlap* antara *patch* yang berdekatan. Nilai *stride* yang lebih kecil akan menghasilkan lebih banyak *patch* dengan tingkat *overlap* yang lebih tinggi, sementara *stride* yang lebih besar menghasilkan lebih sedikit *patch* dengan *overlap* minimal. Pemilihan ukuran jendela dan *stride* ini sangat krusial karena mempengaruhi baik cakupan analisis maupun resolusi deteksi fitur. Dalam konteks penelitian seni lukis tradisional, teknik ini memungkinkan identifikasi pola-pola lokal seperti goresan kuas atau elemen yang mungkin tidak terlihat ketika menganalisis gambar secara keseluruhan.

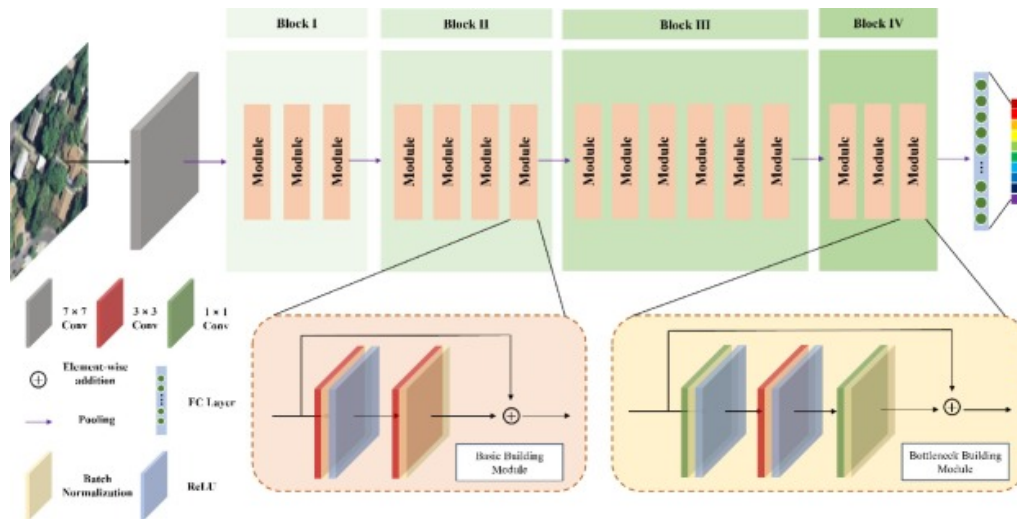
2.2.7 Ekstraksi Fitur

Ekstraksi fitur digunakan untuk mendapatkan sebanyak mungkin informasi dari sebuah gambar (Mutlag *et al.*, 2020). Proses ini melibatkan identifikasi dan pengambilan fitur utama dalam gambar. *Convolutional Neural Networks* (CNNs) diakui sebagai salah satu pencapaian terbaru dalam pengembangan *deep learning*. Arsitektur berlapis pada CNN secara prinsip memungkinkan pembelajaran tingkatan atau hierarki fitur, mulai dari fitur tingkat rendah seperti tepi dan tekstur hingga fitur tingkat tinggi seperti bentuk dan objek kompleks. Fitur yang diekstraksi dari lapisan dalam jaringan ini mencakup pola tekstur, warna, dan bentuk yang lebih kompleks yang berguna untuk tugas klasifikasi (Chaib *et al.*, 2017). Lapisan yang digunakan hanyalah lapisan konvolusional yang berfungsi untuk mengekstraksi fitur. Teknik ini dikenal sebagai *transfer learning*, karena mentransfer pengetahuan sebelumnya yang telah diperoleh di bidang umum ke domain solusi spesifik yang diinginkan (Zhuang *et al.*, 2021).

ResNet (Residual Network) telah menjadi arsitektur utama untuk ekstraksi fitur dalam analisis gambar medis karena mekanisme *skip connection*-nya yang memetakan identitas dari lapisan dangkal ke lapisan dalam (He *et al.*, 2016). Arsitektur ini secara efektif mengatasi masalah degradasi yang sering terjadi pada jaringan sangat dalam (*very deep networks*), di mana akurasi justru menurun setelah mencapai titik jenuh (He *et al.*, 2016). Dibandingkan arsitektur sebelumnya seperti AlexNet dan VGGNet (Krizhevsky *et al.*, 2017; Simonyan & Zisserman, 2014), ResNet memiliki kompleksitas komputasi yang lebih rendah (He *et al.*, 2016). Dalam praktiknya, varian ResNet dengan 50 dan 101 adalah lapisan paling banyak digunakan (Khan *et al.* 2020). ResNet50 khususnya menunjukkan peningkatan akurasi berkat *skip connection* yang mempertahankan gradien di lapisan dalam (Arulprakash & Aruldoss, 2022). Secara hierarkis, ResNet bekerja dengan lapisan awal mengekstraksi fitur dasar seperti tepi dan tekstur, sementara lapisan lebih dalam menangkap pola kompleks (Oğuz & Ertuğrul, 2023). Penelitian terhadap tiga varian ResNet (ResNet18, ResNet34, dan ResNet50) menunjukkan bahwa jaringan dengan lebih banyak lapisan cenderung berkinerja lebih baik. Pemakaian ResNet50 memiliki keunggulan dalam mempertahankan fitur penting antar lapisan melalui *skip connection*-nya (Arulprakash & Aruldoss, 2022).

ResNet50 adalah sebuah arsitektur *deep convolutional neural network* (CNN) yang termasuk dalam keluarga Residual Networks (ResNet). Dikembangkan oleh Microsoft Research pada tahun 2015 He *et al.* (2016), arsitektur ini secara khusus dirancang untuk mengatasi masalah degradasi performa pada jaringan saraf yang sangat dalam (*very deep networks*) melalui mekanisme *skip connections* atau *residual connections*. ResNet50 terdiri dari 50 lapisan konvolusi yang tersusun dalam blok-blok residual, di mana setiap blok memungkinkan aliran informasi dari lapisan awal langsung ke lapisan yang lebih dalam melalui *identity mapping* (He *et al.*, 2016). Keunggulan utama ResNet50 terletak pada kemampuannya mempertahankan gradien selama pelatihan (mitigasi *vanishing gradient problem*) sekaligus

mengekstrak fitur hierarkis dari gambar. Arsitektur ini juga mengadopsi *bottleneck design* yang menggunakan konvolusi 1×1 untuk mengurangi dimensi sebelum konvolusi 3×3 , sehingga lebih efisien secara komputasi dibandingkan VGGNet (He et al., 2016).



Gambar 2.6 Arsitektur CNN untuk Ekstraksi Fitur pada Gambar (Shinichi & Matsui, 2024)

Gambar 2.6 menunjukkan proses ekstraksi fitur pada ResNet. Proses ini diawali dengan serangkaian lapisan konvolusional (7×7 , 3×3 , dan 1×1) yang bertugas menangkap fitur dasar seperti tepi dan tekstur. Setiap lapisan ini diikuti *batch normalization* dan fungsi aktivasi ReLU untuk memastikan stabilitas pelatihan dan memudahkan pembelajaran pola tidak linear (Chen & Tsou, 2022). Keunggulan utama dari ResNet adalah adanya *residual connection* atau *shortcut* yang secara langsung menghubungkan input dengan *output* di setiap blok. Mekanisme ini memungkinkan informasi mengalir dengan lebih lancar, tanpa harus melewati seluruh lapisan konvolusi. Dengan demikian, *residual learning* ini mampu mengatasi masalah *vanishing gradient* dan penurunan akurasi (*degradation*) yang sering terjadi pada jaringan yang sangat dalam (*very deep networks*), serta mempercepat proses pelatihan (Chen & Tsou, 2022). ResNet mengimplementasikan dua jenis modul utama, yaitu *basic building module* yang terdiri atas lapisan-lapisan konvolusi bertumpuk untuk melakukan ekstraksi fitur, serta *bottleneck module* yang menggunakan konvolusi 1×1 untuk mengompresi dimensi sebelum memperluasnya kembali. Pendekatan ini meningkatkan efisiensi komputasi tanpa mengurangi kualitas representasi fitur yang dihasilkan (Chen & Tsou, 2022). Fitur-fitur yang telah diekstraksi kemudian diproses lebih lanjut melalui lapisan fully connected untuk menghasilkan prediksi akhir. Selama proses ini, *residual connections* berperan penting dalam menjaga kestabilan aliran gradien dan memungkinkan ekstraksi fitur yang lebih akurat, terutama untuk pola visual yang kompleks (Chen & Tsou, 2022).

Studi komparatif yang dilaksanakan oleh Mascarenhas dan Agarwal (2021) membandingkan tiga model jaringan saraf dalam untuk menyelesaikan permasalahan identifikasi gambar yang identik, menunjukkan bahwa ResNet50 mencapai kinerja optimal dengan tingkat ketepatan 0.9733 pada iterasi ke-20, lebih unggul dibandingkan VGG16 (0.9667) dan VGG19 (0.9707). Keunggulan utama ResNet50 terletak pada penggunaan *residual connection*, yang mempermudah pelatihan jaringan yang sangat dalam (*very deep networks*) dengan menjaga stabilitas gradien selama proses *backpropagation*. Selain itu, desain *bottleneck* yang efisien menjadikan ResNet50 lebih hemat komputasi dibandingkan dengan arsitektur VGG. Temuan ini menunjukkan bahwa ResNet50 merupakan solusi optimal untuk aplikasi pengenalan gambar dengan kebutuhan presisi tinggi. Dalam perkembangan terkini,

Wen et al. (2020) mengembangkan modifikasi TCNN berbasis ResNet-50, yang mencatat akurasi luar biasa sebesar $98,95\% \pm 0,0074$, $99,99\% \pm 0$, dan $99,20\% \pm 0$ pada berbagai dataset. Pencapaian ini tidak hanya melampaui performa model *deep learning* lainnya, tetapi juga melebihi metode konvensional, sehingga semakin menegaskan dominasi varian ResNet-50 dalam menghadapi tantangan klasifikasi visual yang kompleks.

Tahapan berikutnya dalam arsitektur CNN melibatkan operasi *pooling* yang berfungsi untuk mereduksi ukuran *feature map* hasil konvolusi dengan mengekstraksi fitur-fitur dominan. Lapisan *pooling* memegang peranan krusial dalam menurunkan kompleksitas *output* dari lapisan konvolusi, sehingga menghasilkan representasi data yang lebih ringkas dan mudah diproses (Arya, 2024). Pada mekanisme ini, input dibagi menjadi beberapa *grid* dimana setiap *grid* terdiri dari sekumpulan *neuron*. Berbagai jenis operasi pooling dapat diterapkan, antara lain *average pooling*, *min pooling*, *max pooling*, dan *global average pooling* (GAP) seperti yang divisualisasikan pada Gambar 2.6. Khusus untuk *global average pooling* yang biasanya diaplikasikan pada bagian akhir CNN, teknik ini mengambil nilai rata-rata dari seluruh *feature map* sehingga mampu mengurangi dimensi input secara signifikan dan dalam beberapa kasus dapat menggantikan fungsi *fully connected layer* (Raitoharju, 2022). Keunggulan utama dari *Global Average Pooling* (GAP) mencakup peningkatan ketahanan terhadap pergeseran posisi objek dalam gambar, kemudahan interpretasi model, serta perannya membantu mencegah model terlalu *overfitting*. Dalam penelitian ini, implementasi *Global Average Pooling 2D* (GAP2D) digunakan untuk mereduksi dimensi spasial dari *feature map* dengan menghitung rata-rata nilai pada setiap *channel*, sehingga menghasilkan tensor yang tetap mempertahankan informasi spasial penting namun dengan jumlah parameter yang jauh lebih sedikit (Adewopo & Elsayed, 2024). Lapisan GAP2D ini berkontribusi dalam memitigasi *overfitting* sekaligus menyederhanakan arsitektur model, yang pada akhirnya meningkatkan efisiensi komputasi.

Dalam penelitian ini, *Principal Component Analysis* (PCA) digunakan untuk menyederhanakan data yang memiliki banyak variabel, dengan cara mengubah sekumpulan fitur yang saling berkaitan menjadi beberapa komponen utama yang lebih sedikit. PCA bekerja dengan mencari arah dalam data yang memiliki variasi paling besar, karena arah tersebut dianggap paling mewakili informasi penting dari data (Salem & Hussein, 2019). Teknik ini sangat berguna untuk mengurangi gangguan (*noise*) dan menemukan pola tersembunyi dalam data yang kompleks dan berdimensi tinggi. Secara praktis, PCA mengubah ruang data menjadi bentuk baru dan memproyeksikannya ke dimensi yang lebih rendah, sambil tetap mempertahankan sekitar 70–80% informasi utama dari data awal. Hasil dari proses ini membuat data lebih mudah dipahami, dianalisis, dan diolah. Setiap komponen utama mencerminkan seberapa besar kontribusinya terhadap informasi keseluruhan, yang dihitung menggunakan vektor *eigen* dan nilai *eigen* yang menunjukkan pentingnya masing-masing arah dalam data.

2.2.8 Similiaritas

Similaritas berfungsi sebagai prinsip pengaturan yang memungkinkan untuk melakukan pengklasifikasian objek (Tversky, 1997). Setelah ekstraksi fitur, vektor fitur yang dihasilkan digunakan untuk menghitung similaritas. Penilaian kesamaan sangat bergantung pada konteks atau ruang lingkup fitur yang relevan (Tversky, 1997). Metode *cosine distance* akan digunakan dalam Tugas Akhir ini. *Cosine distance* digunakan untuk mengukur kesamaan antara dua vektor dalam model ruang vektor berdasarkan sudut kosinus di antara keduanya, dengan nilai berkisar antara 0 hingga 1, di mana nilai yang lebih rendah menunjukkan kesamaan yang lebih tinggi (Usino et al., 2019). Pemakaian metode ini memungkinkan identifikasi karya-karya dengan gaya visual yang serupa

Formulasi matematisnya terdiri dari dua bagian yang saling berkaitan. Pertama, *cosine similarity* dihitung menggunakan Persamaan (2.8), di mana $A \cdot B$ menunjukkan hasil *dot product* antara dua vektor A dan B, sedangkan $|A|$ dan $|B|$ adalah panjang (norma) dari masing-masing

vektor. Nilai *cosine similarity* ini menunjukkan seberapa mirip arah kedua vektor tersebut. Kedua, *cosine distance* diperoleh dengan mengurangkan nilai *cosine similarity* dari 1, seperti yang ditunjukkan pada Persamaan (2.9). Nilai ini digunakan untuk mengukur seberapa besar perbedaan antara dua vektor.

$$\text{Cosine Similiarity} = \cos(\theta) = \frac{A \cdot B}{|A||B|} \quad (2.8)$$

$$\text{Cosine Distance} = 1 - \text{Cosine Similiarity} \quad (2.9)$$

Nilai *cosine similarity* berada dalam rentang -1 hingga 1. Nilai 1 berarti kedua vektor memiliki arah yang sama persis, sehingga dianggap sangat mirip. Jika nilainya 0, artinya kedua vektor saling tegak lurus (ortogonal), atau tidak memiliki hubungan sama sekali. Sementara itu, nilai -1 menunjukkan bahwa kedua vektor berlawanan arah, yang berarti keduanya sangat berbeda. Dalam praktiknya, nilai *cosine similarity* sering diubah menjadi *cosine distance* dengan cara mengurangkannya dari 1. Hasilnya adalah nilai yang berada dalam rentang 0 hingga 2, di mana nilai yang lebih kecil menunjukkan tingkat kemiripan yang lebih tinggi.

Metode ini sangat berguna untuk mengukur kemiripan pola visual, karena tidak dipengaruhi oleh perbedaan ukuran atau intensitas warna. Hal ini menjadikannya cocok untuk membandingkan gaya seni yang mungkin berbeda dari segi pencahayaan atau skala, tetapi masih memiliki bentuk dan komposisi yang serupa. Penelitian oleh Usino et al. (2019) menunjukkan bahwa kombinasi antara metode K-Means dan *cosine distance* menghasilkan performa yang sangat baik dalam mendeteksi kemiripan fitur. *Cosine distance* digunakan untuk mengukur arah vektor dalam ruang fitur, sementara K-Means membantu mengelompokkan data berdasarkan fitur yang mirip, sehingga hasil pengelompokan menjadi lebih baik.

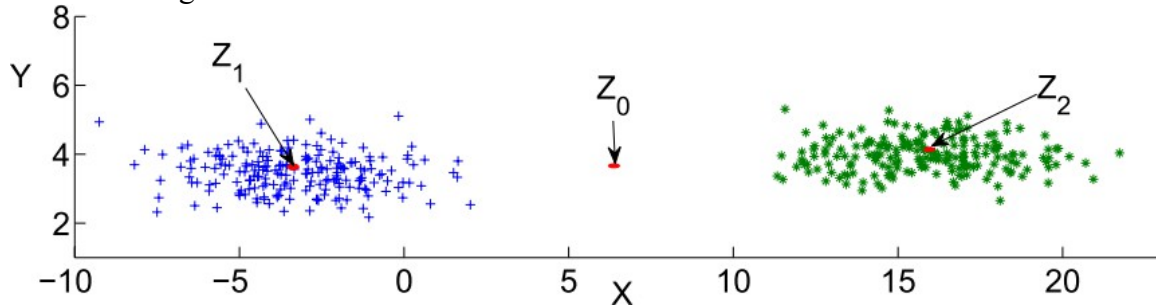
2.2.9 Discriminative Clustering

Discriminative Clustering (DC) adalah metode dalam pembelajaran mesin yang menggabungkan pendekatan *unsupervised learning* dengan elemen dari *supervised learning*, yaitu dengan memanfaatkan informasi tambahan (seperti label atau variabel target) untuk menghasilkan pengelompokan data yang lebih bermakna. Meskipun tetap mempertahankan struktur serupa dengan *K-means* yakni membagi ruang menjadi beberapa bagian (mirip dengan struktur Voronoi) DC tidak hanya fokus pada jarak antar titik. Sebaliknya, ia mengoptimalkan keseragaman distribusi variabel target dalam setiap cluster, sehingga hasil pengelompokan menjadi lebih relevan. Contohnya, dalam segmentasi pelanggan, DC dapat membagi data tidak hanya berdasarkan lokasi dalam ruang fitur, tapi juga berdasarkan kemiripan perilaku atau demografi (Sinkkonen & Kaski, 2002). Dengan kata lain, DC tidak hanya meminimalkan jarak antar titik data, tetapi juga mempertimbangkan aspek lain yang relevan, seperti informasi tambahan yang ada.

Metode ini sangat sesuai untuk menyelesaikan masalah *cosegmentation* (memisahkan objek yang mirip dalam beberapa gambar) karena dua alasan utama. Pertama, DC bisa memanfaatkan fitur-fitur yang sebelumnya sudah digunakan dalam klasifikasi *supervised*, khususnya dengan arsitektur modern seperti histogram fitur lokal dan metode berbasis kernel (Zhang et al., 2007). Penelitian sebelumnya menunjukkan bahwa metode ini efektif untuk tugas *weakly supervised learning* dalam bidang *computer vision* (Duchenne et al., 2009; Nguyen et al., 2009). Kedua, DC memudahkan penerapan *constraint* (batasan) saat membuat cluster, misalnya dengan menambahkan konsistensi warna lokal dan hubungan antar elemen gambar yang dapat meningkatkan kualitas segmentasi.

Sementara itu, *K-means* adalah algoritma *clustering* yang sangat populer dan pertama kali diperkenalkan oleh MacQueen pada tahun 1967. Metode ini membagi data ke dalam *k* kelompok berdasarkan jarak *Euclidean*, dengan cara mencari posisi titik tengah (centroid) yang

optimal untuk masing-masing cluster (Saxena *et al.*, 2017). Algoritma ini bertujuan untuk membagi n observasi ke dalam k *cluster*, di mana setiap observasi termasuk ke dalam *cluster* dengan mean (*centroid*) terdekat. Proses ini dilakukan secara iteratif, dengan *centroid* diperbarui pada setiap iterasi hingga konvergensi tercapai. Evaluasi yang digunakan pada model clustering adalah metrik *silhouette score*.



Gambar 2.7 Visualisasi *Centroid* dan Klaster K-Means Clustering (Huang *et al.*, 2014)

Gambar 2.7 menjelaskan hasil penerapan algoritma *K-Means Clustering*, di mana data telah dikelompokkan menjadi dua *cluster* utama yang ditandai dengan *centroid* Z_1 dan Z_2 . Titik-titik biru (+) membentuk *cluster* pertama dengan *centroid* Z_1 sebagai pusatnya, sementara titik-titik hijau (*) membentuk *cluster* kedua yang berpusat pada *centroid* Z_2 . Pada bagian tengah area kosong, terdapat *centroid* Z_0 yang tidak terasosiasi dengan data, yang kemungkinan besar merupakan *centroid* awal yang dipilih secara acak. Sebaran data di sepanjang sumbu X menunjukkan pemisahan *cluster* yang cukup jelas, dengan masing-masing *centroid* berada di tengah distribusi *cluster*. Visualisasi ini menggambarkan bagaimana algoritma *K-Means* bekerja dengan menempatkan *centroid* di posisi optimal setelah beberapa iterasi, sehingga jarak antara titik data dan *centroid* dalam *cluster* dapat diminimalkan.

Dalam studi ini, konsep *centroid* didefinisikan sebagai titik pusat yang merepresentasikan posisi rata-rata dari sekelompok data dalam suatu klaster, sebagaimana umumnya digunakan dalam algoritma K-Means. Perhitungan ini dilakukan dengan menghitung rata-rata dari seluruh titik fitur yang termasuk dalam klaster tersebut. Sementara itu, penelitian oleh Singh & Bhatia (2011) mengusulkan pendekatan alternatif dalam pemilihan *centroid*, yaitu dengan membagi ruang data menjadi grid berukuran $k \times k$ dan memilih sel dengan kepadatan data tertinggi sebagai representasi *centroid*. Pendekatan ini ditujukan untuk meningkatkan representasi klaster yang berpopulasi padat, terutama dalam konteks analisis pola visual.

Untuk mengatasi kompleksitas struktur visual, penelitian ini mengintegrasikan UMAP (*Uniform Manifold Approximation and Projection*) sebagai teknik reduksi dimensi tidak linear. UMAP memproyeksikan data berdimensi tinggi ke dalam ruang 2D atau 3D dengan membangun graf berbobot yang merepresentasikan hubungan kedekatan lokal antar titik, kemudian mempertahankan struktur tersebut (McInnes *et al.*, 2018). Keunggulan UMAP terletak pada kemampuannya memelihara baik struktur global (misalnya tema komposisi *ukiyo-e*) maupun lokal (seperti detail goresan *sumi-e*), sehingga memfasilitasi visualisasi dan analisis fitur visual untuk klasifikasi dengan CNN atau SVM. Pendekatan ini dapat memperkaya pemahaman tentang karakteristik gambar.

Sementara itu, DC memperluas konsep clustering tradisional dengan menambahkan *prototype distribution* (ψ_i) untuk setiap *cluster*, yang merepresentasikan probabilitas kelas diskrit (Sinkkonen & Kaski, 2002). Evaluasi performa DC dapat menggunakan metrik seperti *silhouette score* atau *log-likelihood* dari distribusi kondisional $p(c|\mathbf{x})$ (Agresti, 1992). Dalam implementasinya, DC juga dapat diinisialisasi dengan hasil *K-Means*, yang menunjukkan adanya hubungan erat antara kedua metode. Secara konseptual, DC adalah versi

yang lebih pintar dari *K-Means*, karena selain mengelompokkan data, DC juga memperhatikan informasi kelas (label) untuk membuat hasil *clustering* lebih informatif dan berguna untuk klasifikasi.

Berbeda dengan metode *clustering* konvensional yang hanya mempertimbangkan kemiripan fitur-fitur dasar, seperti warna atau posisi, secara terpisah dan terbatas, Discriminative Clustering (DC) menggabungkan prinsip klasifikasi *supervised learning* ke dalam proses pengelompokan. Tujuannya adalah untuk menghasilkan pemisahan antar kelas yang lebih optimal (Joulin et al., 2010). *Clustering* biasa cenderung fokus pada analisis kesamaan lokal antar fitur, yang dalam banyak kasus tidak cukup efektif untuk menangani tugas-tugas kompleks seperti segmentasi objek atau klasifikasi lanjutan. DC mengatasi kelemahan ini dengan memanfaatkan algoritma klasifikasi, seperti Support Vector Machine (SVM), untuk memaksimalkan margin pemisahan antar kelas. Sebagai contoh, DC dapat digunakan untuk memisahkan objek utama (*foreground*) dan latar belakang (*background*) dalam sebuah gambar dengan lebih presisi (Joulin et al., 2010). DC tidak hanya mengelompokkan data yang saling dekat, tapi juga berusaha membentuk batas antar *cluster* yang besar dan bermakna secara klasifikasi, dengan bantuan fungsi biaya (*loss function*) yang dirancang khusus untuk meningkatkan pemisahan antar kelas. Dengan demikian, hasil pengelompokan tidak hanya rapat secara internal, tetapi juga memiliki batas yang jelas terhadap *cluster* lain.

2.2.10 Evaluasi Model

Evaluasi model merupakan langkah dalam pembuatan model pembelajaran mesin, karena memastikan model mampu melakukan generalisasi terhadap data (Raschka, 2020). Evaluasi kinerja model klasifikasi merupakan tahap kritis dalam pengembangan sistem *machine learning*, di mana metrik seperti akurasi, presisi, *recall*, dan *F1-score* sering digunakan untuk menilai efektivitas model. Tujuan dari evaluasi model adalah mengestimasi performa generalisasi model, meningkatkan performa prediktif dengan memakai *hyperparameter*, dan membandingkan algoritma pembelajaran mesin untuk memilih model terbaik (Raschka, 2020). Metrik-metrik ini memberikan gambaran tentang bagaimana model berfungsi dalam berbagai konteks klasifikasi.

Confusion matrix adalah alat yang digunakan untuk mengukur akurasi dalam data *mining* (Rosandy, 2016). *Matrix* ini menyajikan informasi tentang hasil klasifikasi aktual dan prediksi yang dihasilkan oleh sistem. Evaluasi performa sistem klasifikasi umumnya dilakukan dengan menggunakan data yang ada dalam matrix tersebut. Tabel 2.2 menunjukkan contoh *confusion matrix* untuk klasifikasi dengan dua kategori.

Tabel 2.2 *Confusion Matrix*

Kelas	Terklasifikasi Positif	Terklasifikasi Negatif
Positif	TP (<i>True Positive</i>)	FN (False Negatif)
Negatif	FP (<i>False Positive</i>)	TN (<i>True Negatif</i>)

Nilai *True Negative* (TN) mengacu pada jumlah data negatif yang teridentifikasi dengan benar, yang berarti model berhasil mengklasifikasikan data tersebut sebagai negatif. Sebaliknya, *False Positive* (FP) terjadi ketika data negatif salah terdeteksi sebagai positif, menunjukkan kesalahan dalam klasifikasi model. *True Positive* (TP) menggambarkan jumlah data positif yang terdeteksi dengan benar, yang berarti model berhasil mengenali data sebagai positif. Sementara itu, *False Negative* (FN) adalah kebalikan dari *True Positive*, di mana data yang

sebenarnya positif salah terdeteksi sebagai data negatif oleh model. Berdasarkan nilai *True Negative* (TN), *False Positive* (FP), *False Negative* (FN), dan *True Positive* (TP) dapat dihitung nilai akurasi, presisi, recall, dan f-measure.

Akurasi mengukur proporsi keseluruhan elemen yang diklasifikasikan dengan benar dari seluruh *dataset*, tetapi cenderung bias terhadap kelas mayoritas pada *dataset* yang tidak seimbang (Grandini *et al.*, 2020). Akurasi mengukur sejauh mana model dapat mengklasifikasikan elemen dengan benar dalam *dataset*. Akurasi dihitung dengan menjumlahkan *True Positive* (TP) dan *True Negative* (TN), kemudian membaginya dengan jumlah seluruh elemen yang diprediksi, yaitu *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN).

$$\text{Akurasi} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (2.10)$$

Precision mengukur proporsi prediksi positif yang benar dari semua prediksi positif yang dibuat oleh model. *Precision* berguna dalam skenario di mana *false positives* lebih merugikan dibandingkan *false negatives* (Grandini *et al.*, 2020). Formula untuk *precision* meliputi pembagian jumlah *True Positive* (TP) dengan jumlah seluruh prediksi positif yang dibuat oleh model, yaitu *True Positive* (TP) ditambah *False Positive* (FP). *Precision* mengukur seberapa akurat model dalam memprediksi kelas positif, dimana nilai presisi tinggi menunjukkan sedikitnya kesalahan klasifikasi positif palsu.

$$\text{Precision} = \frac{\text{TruePositives(TP)}}{\text{TruePositives(TP)} + \text{FalsePositives(FP)}} \quad (2.11)$$

Recall mengukur proporsi *true positives* dari semua sampel aktual positif, yang merepresentasikan sensitivitas model dalam menangkap kelas positif. *Recall* penting dalam situasi di mana menangkap sebanyak mungkin *true positives* lebih diutamakan daripada menghindari *false positives* (Grandini *et al.*, 2020). *Recall* dihitung dengan membagi jumlah *True Positive* (TP) dengan jumlah seluruh elemen yang sebenarnya positif, yaitu *True Positive* (TP) ditambah *False Negative* (FN). *False Negative* (FN) terjadi ketika model gagal mendeteksi elemen positif yang seharusnya (misalnya tidak mengenali lukisan Jepang yang asli). *Recall* mengukur kemampuan model dalam menemukan semua kasus positif yang ada, dimana nilai *recall* tinggi menunjukkan model mampu mendeteksi sebagian besar elemen positif tanpa banyak *miss*.

$$\text{Recall} = \frac{\text{TruePositives(TP)}}{\text{TruePositives(TP)} + \text{FalseNegatives(FN)}} \quad (2.12)$$

F1-score adalah rata-rata harmonik dari *precision* dan *recall*, yang menyeimbangkan kedua metrik ini untuk memberikan evaluasi kinerja yang lebih komprehensif (Grandini *et al.*, 2020). *F1-score* dapat diinterpretasikan sebagai rata-rata tertimbang antara *precision* dan *recall*, di mana nilai terbaik adalah 1 dan nilai terburuk adalah 0 (Grandini *et al.*, 2020). Dalam klasifikasi multi-kelas, *F1-score* dapat dihitung sebagai *Macro F1-score* untuk memberikan bobot yang sama pada setiap kelas atau sebagai *Micro F1-score* untuk menggabungkan semua unit tanpa memperhatikan distribusi kelas.

$$\text{F1 - Score} = \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2.13)$$

Evaluasi model pada *clustering* menggunakan *silhouette score*. *Silhouette score* adalah

metrik untuk mengukur kualitas klaster dalam analisis clustering, dengan mempertimbangkan rata-rata jarak intra-klaster (a) dan rata-rata jarak ke klaster terdekat (b) (Shahapure & Nicholas, 2020). Nilai *Silhouette* ($S(i)$) untuk setiap data *point* (i) berkisar antara -1 hingga 1, dimana nilai ini dihitung berdasarkan dua komponen utama: jarak intra-klaster ($a(i)$) dan jarak inter-klaster ($b(i)$). Jarak intra-klaster ($a(i)$) merepresentasikan rata-rata jarak antara data point (i) dengan semua data point lain dalam klaster yang sama, menunjukkan tingkat kedekatan atau kerapatan data dalam satu klaster. Semakin kecil nilai $a(i)$, semakin padat dan homogen klaster tersebut. Di sisi lain, jarak inter-klaster ($b(i)$) mengukur rata-rata jarak antara data point (i) dengan semua data point dalam klaster terdekat yang berbeda, yang menggambarkan tingkat pemisahan antar klaster. Nilai $b(i)$ yang besar menunjukkan bahwa klaster-klaster tersebut terpisah dengan baik.

$$S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (2.13)$$

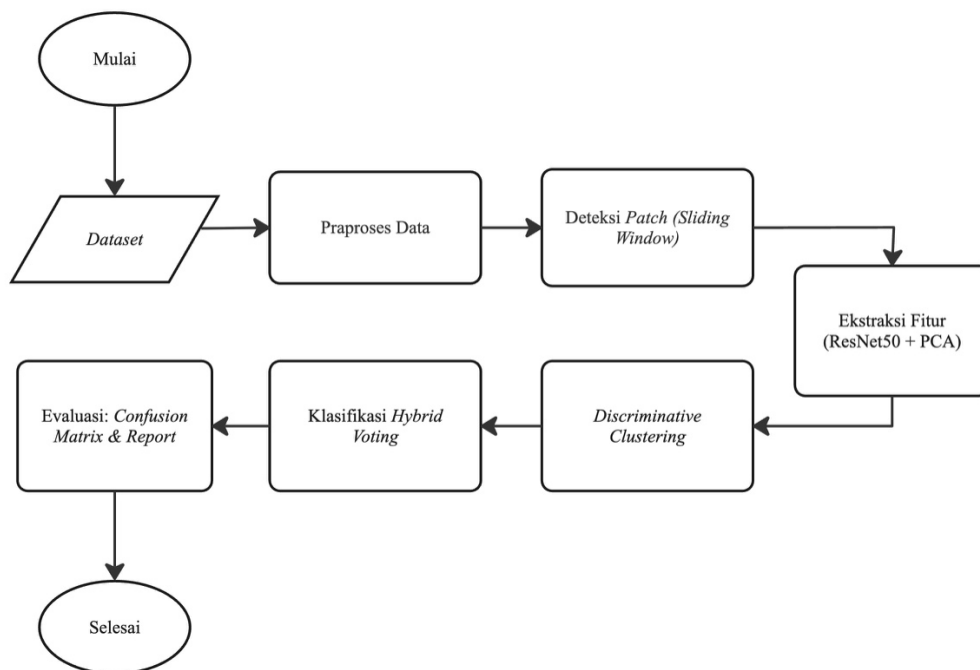
Sehingga, *Silhouette score* digunakan untuk menentukan jumlah klaster optimal (k) dalam algoritma k- means dengan menghitung rata-rata koefisien *silhouette* untuk semua sampel. *Silhouette score* dengan nilai mendekati +1 menunjukkan bahwa data berada dalam klaster yang benar, nilai mendekati 0 menunjukkan kemungkinan data berada dalam klaster lain, dan nilai mendekati -1 menunjukkan data berada dalam klaster yang salah (Shahapure & Nicholas, 2020).

(Halaman ini sengaja dikosongkan)

BAB 3 METODOLOGI

3.1 Metode yang digunakan

Tugas Akhir ini mengimplementasikan metode *semi-supervised learning* untuk mengidentifikasi karakteristik visual seni lukis tradisional Jepang. Proses diawali dengan *preprocessing* data, dilanjutkan dengan deteksi *patch* dari setiap gambar, ekstraksi fitur menggunakan model CNN, dan pengelompokan fitur dengan metode diskriminatif *clustering*. Selanjutnya, fitur yang telah dikelompokkan digunakan untuk melatih model klasifikasi *Hybrid Voting* dengan Support Vector Machine (SVM). Untuk menghasilkan label akhir suatu gambar, digunakan pendekatan voting berdasarkan prediksi tiap *patch*. Gambar 3.1 menyajikan diagram alir metodologi penelitian yang digunakan.

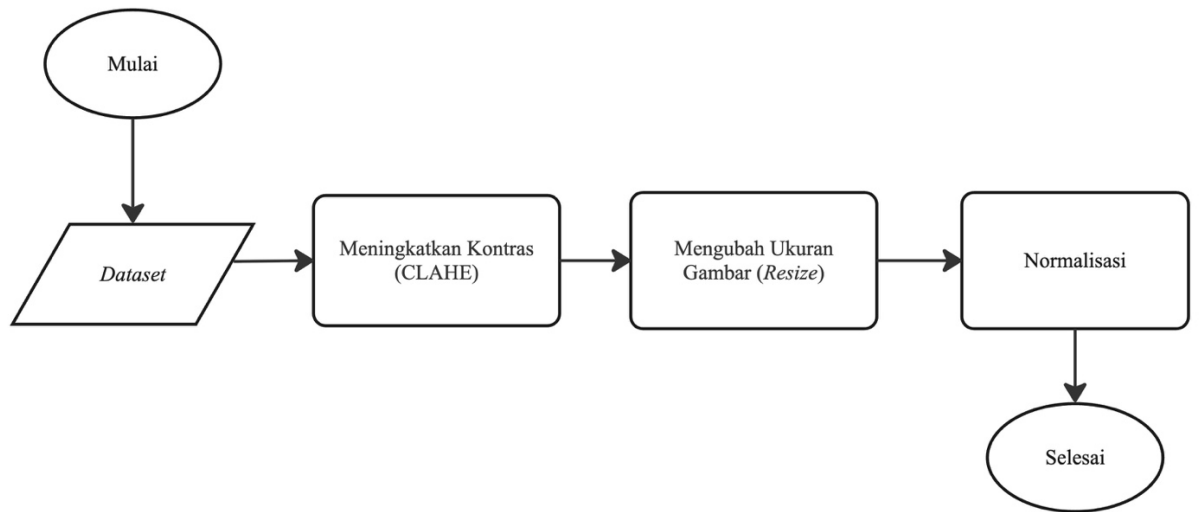


Gambar 3.1 Diagram Alir Metodologi Penelitian

3.1.1 Dataset

Penelitian ini mengimplementasikan analisis terhadap dua gaya lukis seni Jepang, yaitu *ukiyo-e* dan *sumi-e*. *Dataset* penelitian terdiri dari dua kategori utama, yaitu *dataset* positif dan *dataset* negatif. *Dataset* positif mencakup karya seni *ukiyo-e* dan *sumi-e*, sedangkan *dataset* negatif terdiri dari seni lukis Cina. *Dataset ukiyo-e* diperoleh dari *The Metropolitan Museum of Art Ukiyo-e Dataset* yang tersedia di platform Kaggle dan mencakup koleksi karya-karya periode Edo. Sementara itu, *dataset sumi-e* diperoleh dari *Sumi-e no Kurozuappu* di Hugging Face, yang berisi koleksi gambar lukisan *sumi-e*. Kedua dataset ini menjadi dasar analisis yang dilakukan pada penelitian ini. *Dataset The Metropolitan Museum of Art Ukiyo-e Dataset* berisi 4.043 *file* gambar karya seni bergaya *ukiyo-e* dari The Metropolitan Museum of Art (The Met) di New York, Amerika Serikat. *Dataset Sumi-e no kurozuappu ink painting closeups* mencakup sekitar 1.250 *file* gambar *close-up* lukisan tinta Jepang bergaya *sumi-e*, yang sebagian besar dihasilkan menggunakan model AI dengan *prompt* seperti "*a closeup of a Japanese Sumi-e black ink painting*" dan "*a Japanese Sumi-e black ink painting of bamboo, red seal*". *Dataset* seni lukis Cina dari *Chinese Painting Dataset* (G. Wang et al., 2018) berisi 5798 *file* gambar diambil dari Google dan Baidu.

3.1.2 Pra-Proses Data



Gambar 3.2 Praproses Data

Tahapan praproses pada penelitian ini divisualisasikan pada Gambar 3.2. Gambar 3.2 menunjukkan tahap praproses data yang bertujuan untuk menyeragamkan format *dataset* lukisan *ukiyo-e* dari *The Metropolitan Museum* dan *sumi-e* dari *Sumi-e no Kurozuappu*. Tahap praproses *dataset* bertujuan untuk meningkatkan kualitas data sebelum lanjut ke tahap analisis. Langkah pertama dalam praproses adalah peningkatan kontras menggunakan metode *Contrast Limited Adaptive Histogram Equalization* (CLAHE). Proses ini bertujuan untuk memperjelas detail pada gambar yang mungkin sebelumnya kurang terlihat. Dengan meningkatkan kontras dan memperjelas tepi, detail penting dalam gambar menjadi lebih mudah dikenali oleh algoritma seperti CNN atau SVM. Selanjutnya, dilakukan penyesuaian ukuran gambar atau *resize*. Dimensi gambar akan diubah ukurannya secara proporsional agar tetap mempertahankan rasio aspek. Jika gambar melewati batas ukuran maksimum, gambar akan dikecilkan menggunakan teknik *interpolasi area* (*INTER_AREA*) yang umum digunakan untuk mengecilkan gambar tanpa kehilangan informasi penting. Jika gambar lebih kecil dari ukuran minimum maka akan dilakukan pembesaran gambar menggunakan *interpolasi bicubic*, yang dikenal efektif untuk proses *upscaling* karena mampu menghasilkan transisi piksel yang halus. Ukuran untuk *resize* gambar juga merupakan kelipatan 224 yaitu ukuran yang ideal ketika gambar masuk ke dalam model CNN. Setelah gambar melakukan proses *resizing*, gambar juga akan diberikan *padding* berwarna hitam di sekelilingnya untuk mempertahankan proporsi gambar asli. Langkah terakhir dalam praproses adalah normalisasi nilai piksel. Proses ini dilakukan untuk mengubah rentang nilai piksel menjadi antara 0 dan 1, sehingga sesuai dengan format input yang dibutuhkan oleh ekstraksi fitur.

3.1.3 Deteksi Patch

Setelah gambar melalui tahap praproses, langkah selanjutnya adalah deteksi dan ekstraksi potongan gambar kecil yang disebut *patch*. Proses ini bertujuan untuk membagi gambar menjadi bagian-bagian kecil agar analisis pada tahap klasifikasi dapat dilakukan secara lebih detail, khususnya untuk mengidentifikasi keunikan visual pada lukisan seni Jepang. Penelitian ini menggunakan pendekatan *sliding window*, yang diatur sedemikian rupa agar seluruh area gambar dapat dipindai tanpa terjadi *overlap* antar *patch*. Namun, tidak semua *patch* yang dihasilkan digunakan dalam tahap selanjutnya. Untuk menyaring *patch* yang kurang informatif, dilakukan proses validasi menggunakan tiga kriteria yaitu variasi intensitas warna piksel, jumlah tepi (*edges*), dan rasio maksimum area hitam dalam *patch*. Sebuah *patch* dinilai valid

apabila memiliki variasi intensitas warna, jumlah *edges*, dan rasio maksimum area hitam yang melebihi *threshold* tertentu.

1. Intesitas Warna Piksel

Patch yang valid seharusnya memiliki variasi intensitas warna yang mencukupi. Jika sebuah *patch* memiliki intensitas warna yang terlalu seragam (misalnya terlalu gelap, terlalu terang, atau hampir seluruhnya abu-abu), maka *patch* tersebut dianggap tidak memberikan informasi visual yang cukup. Oleh karena itu, standar deviasi (standar deviasi piksel dalam *grayscale*) digunakan sebagai indikator variasi warna, dan *patch* dengan nilai standar deviasi terlalu rendah akan dibuang.

2. Jumlah Tepi

Ciri khas visual dalam lukisan seringkali ditandai oleh adanya kontur atau batas antar objek yang dapat ditangkap melalui deteksi tepi. Dengan menerapkan algoritma seperti *Canny Edge Detection*, jumlah *edge* dalam sebuah *patch* dapat dihitung. Jika jumlah *edge* terlalu sedikit, maka *patch* dianggap terlalu polos atau kosong dan tidak memiliki kontur yang berguna untuk klasifikasi. *Threshold* minimum *edge* digunakan agar hanya *patch* yang memiliki struktur visual diproses lebih lanjut.

3. Rasio Maksimum *Padding*

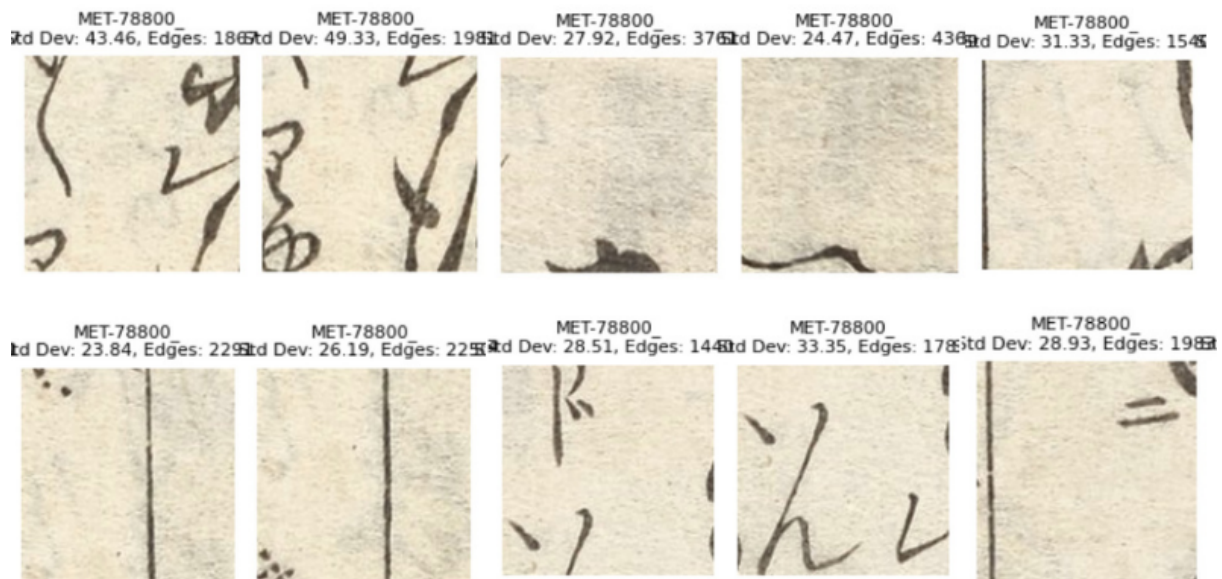
Patch yang terletak di tepi gambar atau hasil *padding* dari *resize* mungkin mengandung banyak area hitam. Untuk mencegah masuknya *patch* yang tidak berisi informasi visual dan hanya terdiri dari *padding*, dihitunglah rasio area hitam terhadap total area *patch*. Jika rasio tersebut melebihi ambang batas tertentu, maka *patch* tersebut dianggap tidak valid. Hal ini penting agar model tidak belajar dari *noise* atau area yang tidak relevan secara visual.

Gambar 3.3 menunjukkan contoh beberapa *patch* yang dianggap tidak layak untuk dianalisis lebih lanjut karena minimnya informasi visual. Meskipun begitu, beberapa *patch* di dalam gambar 3.3 memiliki variasi warna dan jumlah tepi yang tinggi, sehingga tetap dipertahankan dalam dataset hasil validasi.



Gambar 3.3 Visualisasi *Patch* Kosong Hasil Deteksi *Patch*

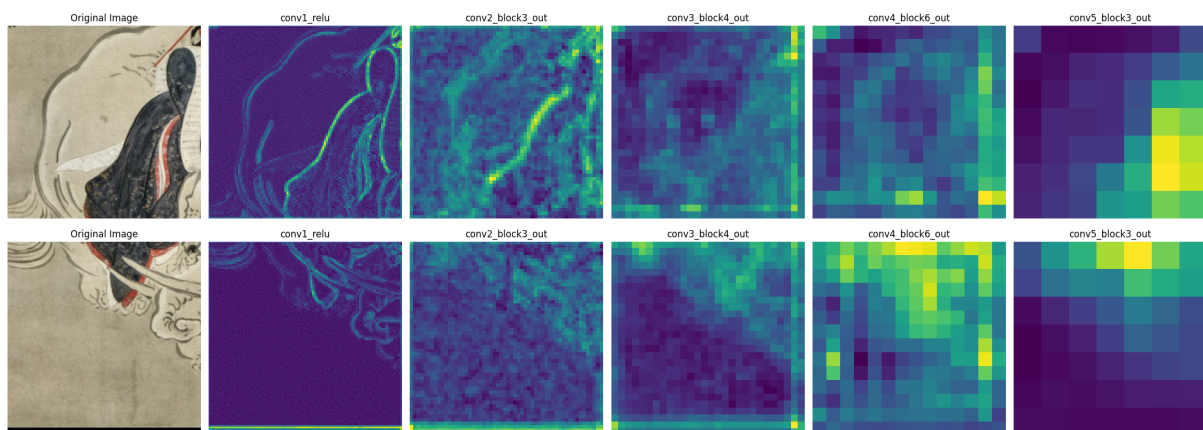
Analisis tidak hanya dilakukan terhadap *patch* yang dinilai tidak layak, tetapi juga terhadap *patch* yang memiliki informasi visual yang cukup. Tujuannya adalah untuk mengevaluasi apakah terdapat *patch* informatif yang secara keliru tidak lolos validasi akibat kegagalan memenuhi kedua kriteria yang ditetapkan. Evaluasi ini penting karena sistem validasi yang diterapkan mensyaratkan bahwa sebuah *patch* hanya dapat diterima jika seluruh metrik seperti *color variation* dan *edge count* memenuhi *threshold* yang telah ditentukan. Gambar 3.4 menampilkan contoh *patch* yang sebenarnya mengandung detail visual cukup, namun hampir tidak lolos validasi karena salah satu kriteria tidak mencapai *threshold*.

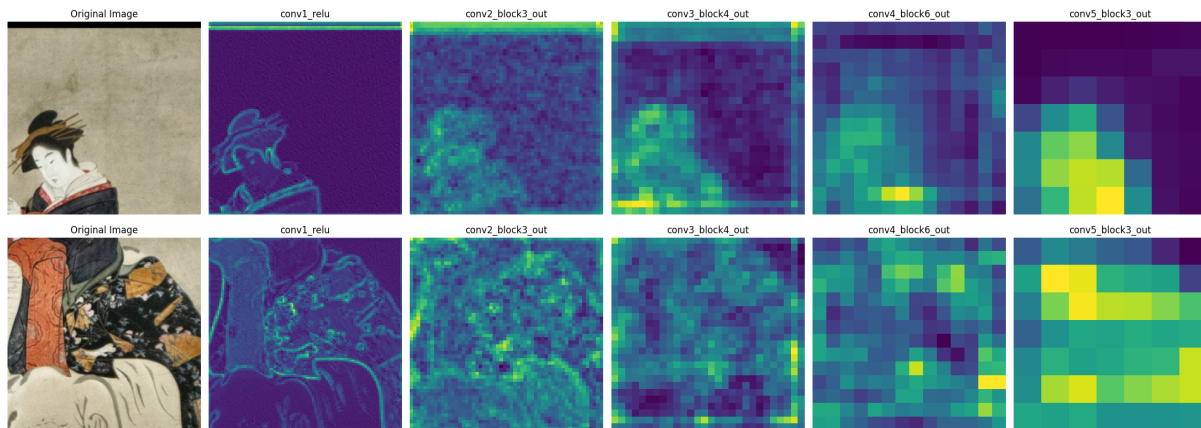


Gambar 3.4 Visualisasi *Patch* Berisi Informasi Hasil Deteksi *Patch*

3.1.4 Ekstraksi Fitur

Tahap ekstraksi fitur dilakukan untuk merepresentasikan setiap *patch* gambar dalam bentuk vektor fitur berdimensi tetap yang dapat digunakan dalam proses klasifikasi. Ekstraksi fitur pada penelitian ini menggunakan arsitektur *Convolutional Neural Network (CNN)* ResNet50 tanpa bagian *fully connected layer*, sehingga hanya menghasilkan fitur dari lapisan konvolusi akhir. Dalam implementasi, fitur diambil dari *output* lapisan terakhir dari blok konvolusi kelima, sebelum bagian klasifikasi akhir dari ResNet50. Gambar 3.3 menunjukkan visualisasi dari lapisan-lapisan di dalam arsitektur ResNet50. Meskipun fitur yang diekstraksi dari lapisan akhir tampak kurang informatif secara visual bagi manusia, justru pada tahap ini model memiliki pemahaman yang lebih tinggi terhadap objek dalam *patch*. Hal ini karena fitur pada lapisan akhir merupakan hasil gabungan tidak linear dari berbagai pola yang telah ditangkap pada lapisan sebelumnya. Kombinasi ini memungkinkan model untuk merepresentasikan konsep yang lebih kompleks, seperti jenis objek, karakteristik tekstur, atau gaya visual tertentu.





Gambar 3.5 Visualisasi Lapisan pada Resnet50

Selanjutnya, diterapkan *Global Average Pooling 2D* untuk merangkum fitur secara efisien dan mengurangi risiko *overfitting*. Penambahan GAP bertujuan untuk mereduksi dimensi *output* dari fitur spasial menjadi representasi vektor satu dimensi berdimensi 2048. Sebelum dilakukan ekstraksi, seluruh *patch* diproses ke dalam format dan ukuran yang sesuai dengan standar input ResNet50, yaitu 224×224 piksel dalam ruang warna RGB. Selanjutnya, setiap gambar dinormalisasi menggunakan fungsi *preprocess_input* dari Keras yang menyesuaikan distribusi data berdasarkan standar *ImageNet*. *Patch* diproses secara *batch* dengan ukuran 1024 untuk efisiensi memori dan kecepatan pemrosesan. Hasil ekstraksi dilakukan reduksi dimensi menggunakan *Principal Component Analysis (PCA)*. *PCA* dijalankan secara bertahap (*incremental-style*): model *PCA* pertama kali dilatih pada *batch* awal, kemudian diterapkan langsung ke *batch* berikutnya. Jumlah komponen utama ditetapkan sebesar 1024, yang dipilih sebagai kompromi antara mempertahankan variasi informasi dan menurunkan dimensi fitur. Proses ekstraksi fitur ini dilakukan secara *batch-wise* guna meningkatkan efisiensi komputasi. Vektor fitur yang telah diekstraksi kemudian disimpan dan digunakan dalam proses *clustering*.

3.1.5 Discriminative Clustering

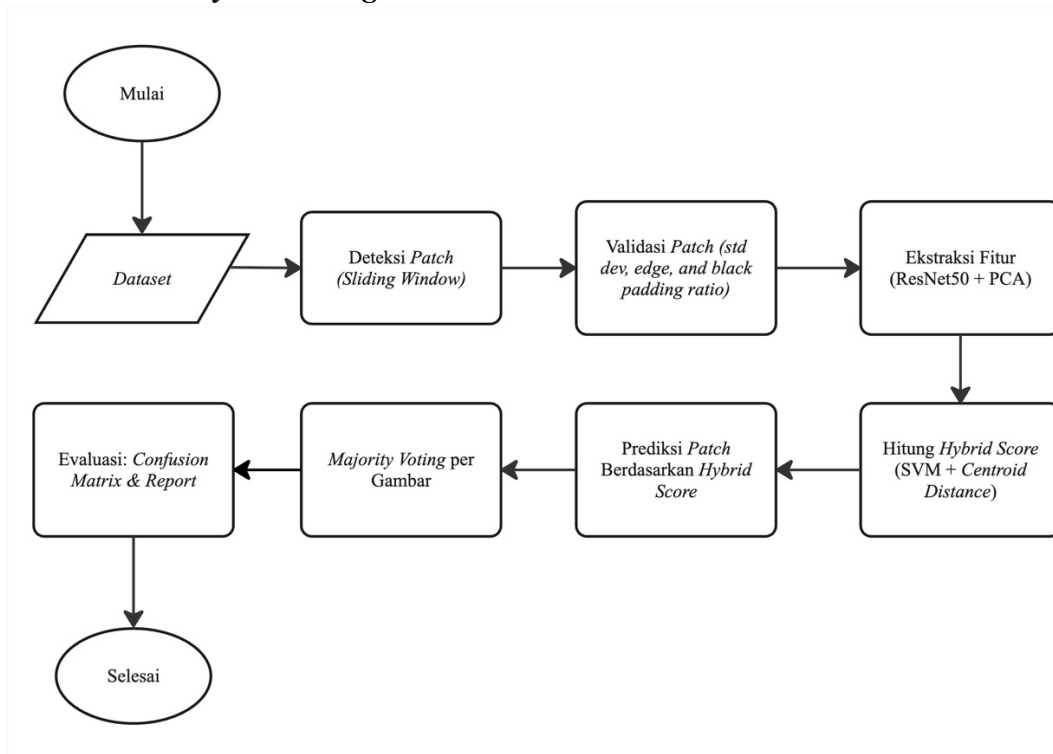
Setelah fitur berhasil diekstraksi, tahap selanjutnya adalah melakukan proses *clustering* untuk mengelompokkan *patch* berdasarkan kemiripan fitur visualnya. Proses ini disebut *discriminative clustering* karena bertujuan untuk mengidentifikasi karakteristik unik yang membedakan antara lukisan Jepang dan lukisan Cina. Tahapan awal dimulai dengan menggabungkan seluruh fitur dari *patch* seni Jepang dan Cina yang telah direduksi ke dalam 1024 dimensi oleh PCA. Fitur-fitur ini kemudian dinormalisasi menggunakan *L2-normalization* dan distandarkan menggunakan *StandardScaler* untuk memastikan skala fitur seragam. Sebelum *clustering* dilakukan, fitur direduksi terlebih dahulu menggunakan algoritma *Uniform Manifold Approximation and Projection (UMAP)*. UMAP dipilih karena mampu mempertahankan struktur lokal maupun global dari data, sehingga efektif sebagai tahap pra-proses sebelum *clustering*. Hasil reduksi dimensi dari UMAP kemudian menjadi input bagi algoritma *K-Means* dengan metrik jarak *cosine distance*. Nilai jumlah kluster (k) ditentukan melalui eksplorasi beberapa kandidat dan dievaluasi menggunakan metrik *silhouette score*. Nilai k terbaik dipilih berdasarkan skor *silhouette* tertinggi. Pendekatan ini tergolong semi-supervised, karena hasil dari proses *clustering* yang bersifat *unsupervised* akan dianalisis dan diberi *pseudo-label* berdasarkan dominasi kelas dalam tiap *cluster*. Label semu ini kemudian digunakan sebagai data latih untuk membangun model klasifikasi *supervised* seperti SVM. Setiap kluster yang terbentuk akan memiliki sebuah *centroid* yang mewakili fitur dominan dari

klaster tersebut. *Centroid* ini kemudian disimpan sebagai acuan dalam tahap klasifikasi. Pengukuran kemiripan dilakukan menggunakan *cosine distance*, yaitu metrik yang menghitung sudut antara dua vektor. Semakin kecil nilai *cosine distance*, semakin besar kemiripan antara *patch* tersebut dengan *centroid* dari klaster. Jika *patch* memiliki jarak terkecil terhadap *centroid* dari klaster yang didominasi lukisan Jepang, maka *patch* tersebut dianggap lebih menyerupai karakteristik visual seni lukis Jepang.

3.1.6 Pembuatan Model

Setelah proses *discriminative clustering* selesai dilakukan, tahap selanjutnya adalah pembangunan model klasifikasi berbasis *supervised learning* menggunakan algoritma *Support Vector Machine* (SVM). Tujuan dari model ini adalah membedakan *patch* yang berasal dari lukisan Jepang dan lukisan Cina berdasarkan representasi fitur yang telah diperoleh sebelumnya. Data yang digunakan dipilih berdasarkan hasil *clustering*, lalu dilakukan proses *balancing* untuk memastikan distribusi data antar kelas seimbang. Hal ini penting untuk menghindari bias dalam proses pelatihan model akibat dominasi salah satu kelas. Selanjutnya, data dibagi menjadi dua *subset*, yaitu data latih dan data uji. Pembagian ini dilakukan untuk mencegah *overfitting* dan untuk mengevaluasi kemampuan generalisasi model terhadap data baru yang belum pernah digunakan dalam pelatihan. Setelah pelatihan selesai, performa model dievaluasi menggunakan data uji dengan menghitung metrik klasifikasi seperti akurasi, presisi, *recall*, dan *F1-score*. Nilai-nilai metrik ini diperoleh melalui analisis *confusion matrix* dan *classification report*. Model yang telah dilatih, bersama dengan data uji, kemudian disimpan untuk digunakan dalam tahap klasifikasi lanjutan, termasuk proses *hybrid voting*.

3.1.7 Klasifikasi Hybrid Voting



Gambar 3.6 Klasifikasi *Hybrid Voting* pada Gambar Utuh

Setelah model klasifikasi dibangun, tahap selanjutnya adalah melakukan klasifikasi gambar menggunakan metode *hybrid voting* untuk menentukan apakah suatu *patch* merupakan lukisan Jepang atau Cina. Metode ini menggabungkan dua metode yaitu kedekatan fitur *patch* terhadap

klaster tertentu (pendekatan *unsupervised*) dan probabilitas hasil klasifikasi dari model SVM sebagai pendekatan *supervised*. Gambar 3.4 menunjukkan tahapan klasifikasi gambar utuh dari *dataset* awal menggunakan pendekatan *hybrid voting* dengan mengambil *majority voting*. Setiap *patch* valid dari gambar uji diproses oleh model SVM untuk menghasilkan nilai probabilitas terhadap dua kelas, yaitu Jepang dan Cina. Untuk mengombinasikan metode ini, pendekatannya yaitu menghitung sejumlah skor dari fitur setiap *patch* dan menggabungkannya menjadi satu nilai akhir yang disebut sebagai *hybrid score*. Langkah pertama adalah menghitung skor kedekatan antara *patch* dengan seluruh *centroid* hasil *clustering*. Ukuran kedekatan yang digunakan adalah jarak *cosine distance*. Nilai jarak *cosine distance* yang lebih kecil menunjukkan jarak yang lebih dekat. Pada pendekatan kali ini, logika tersebut dibalik sehingga skor dihitung sebagai kebalikan dari nilai jarak *cosine distance*. Proses ini dinyatakan dalam Formula (3.1). Formula ini menjelaskan bagaimana nilai jarak yang dekat awalnya memiliki nilai rendah menjadi nilai tinggi.

$$\text{score} = \frac{1}{d + \varepsilon} \quad (3.1)$$

Dalam formula ini, d menyatakan jarak *cosine* antara sebuah *patch* dengan suatu ε adalah nilai kecil untuk mencegah pembagian dengan nol. Skor yang dihasilkan dari pembalikan jarak ini disebut sebagai *reciprocal centroid distance* dan digunakan untuk menghitung skor representatif terhadap gaya visual Jepang maupun Cina. Selanjutnya, sistem menghitung *Japan Score*, yaitu rata-rata dari skor kebalikan jarak *cosine distance* antara suatu *patch* dengan seluruh *centroid* dari klaster yang merepresentasikan lukisan Jepang. Klaster tersebut dipilih berdasarkan hasil *discriminative clustering* sebelumnya, misalnya Klaster 0, 2, dan 3 yang mayoritas berisi *patch* dari gambar bergaya Jepang. Perhitungan ini dinyatakan dalam Formula (3.2).

$$\text{japan_score} = \frac{1}{N_J} \sum_{i=1}^{N_J} \frac{1}{d_{J_i} + \varepsilon} \quad (3.2)$$

$$\text{china_score} = \frac{1}{N_C} \sum_{i=1}^{N_C} \frac{1}{d_{C_i} + \varepsilon} \quad (3.3)$$

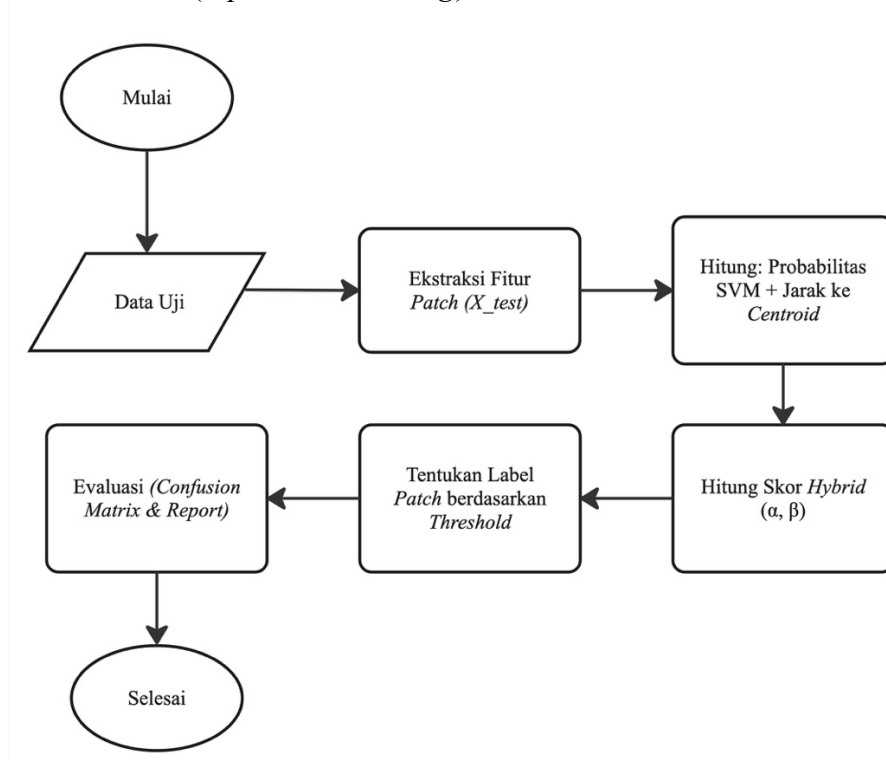
Pada Formula (3.2), N_J adalah jumlah *centroid* yang termasuk dalam klaster Jepang, dan d_{J_i} adalah jarak *cosine* dari *patch* ke *centroid* Jepang ke- i . Skor kedekatan terhadap klaster Cina dihitung dengan cara yang sama, menggunakan Formula (3.3). Dalam formula ini, N_C adalah jumlah *centroid* yang mewakili klaster Cina, dan d_{C_i} adalah jarak *cosine* dari *patch* ke *centroid* Cina ke- i . Untuk membandingkan skor tersebut secara langsung, sistem menghitung rasio kedekatan terhadap Jepang dibanding Cina. Hal ini bertujuan untuk melihat seberapa dominan kemiripan *patch* terhadap gaya Jepang dibandingkan gaya Cina. Formula ini dituliskan dalam Formula (3.4).

$$\text{ratio}_{\text{Jepang:Cina}} = \frac{\text{japan_score}}{\text{china_score} + \varepsilon} \quad (3.4)$$

Nilai rasio kedekatan ini tidak dibatasi dalam rentang $[0, 1]$. Jika nilai rasio melebihi 1, maka *patch* dianggap lebih mirip dengan lukisan Jepang. Sebaliknya, jika nilainya kurang dari 1, maka *patch* lebih dekat ke kluster lukisan Cina. Pendekatan ini dirancang untuk menekankan kemiripan pada *patch*. Ketika sebuah *patch* memiliki jarak yang jauh dari kluster Cina namun sangat dekat dengan kluster Jepang, maka nilai rasio akan menjadi jauh lebih besar dari 1, sehingga perbedaan *patch* lebih terlihat. Langkah berikutnya adalah menggabungkan metode dari kedekatan klaster (*unsupervised*) hasil dari model klasifikasi SVM (*supervised*). Kedua komponen ini digabungkan dalam satu skor akhir yang disebut *Hybrid Score*, yang dihitung sebagai gabungan antara rasio kedekatan klaster dan probabilitas prediksi dari model SVM. Perhitungan ini dirumuskan dalam Formula (3.5).

$$\text{Hybrid Score} = \alpha \cdot \left(\frac{\text{japan_score}}{\text{china_score} + \varepsilon} \right) + \beta \cdot \text{proba}_{\text{SVM}} \quad (3.5)$$

Dalam Persamaan (3.5), parameter α dan β adalah bobot (weight) yang mengatur seberapa besar kontribusi masing-masing komponen terhadap nilai akhir. Nilai $\text{proba}_{\text{SVM}}$ merupakan probabilitas dari model SVM. Jika nilai *hybrid score* lebih besar dari 0,5, maka *patch* diklasifikasikan sebagai lukisan Jepang. Sebaliknya, jika nilainya kurang dari atau sama dengan 0,5, maka *patch* diklasifikasikan sebagai lukisan Cina. Setelah seluruh *patch* dalam satu gambar diklasifikasikan, dilakukan akumulasi hasil melalui pendekatan *proportional threshold voting*. Suatu gambar akan diklasifikasikan sebagai lukisan Jepang apabila proporsi *patch* yang diklasifikasikan sebagai Jepang melebihi 50% dari jumlah *patch* valid dalam gambar tersebut. Dengan menggunakan metode *hybrid voting*, sistem klasifikasi menjadi lebih baik karena menggabungkan informasi dari dua sumber: struktur distribusi klaster (*unsupervised*) dan prediksi model klasifikasi (*supervised learning*).



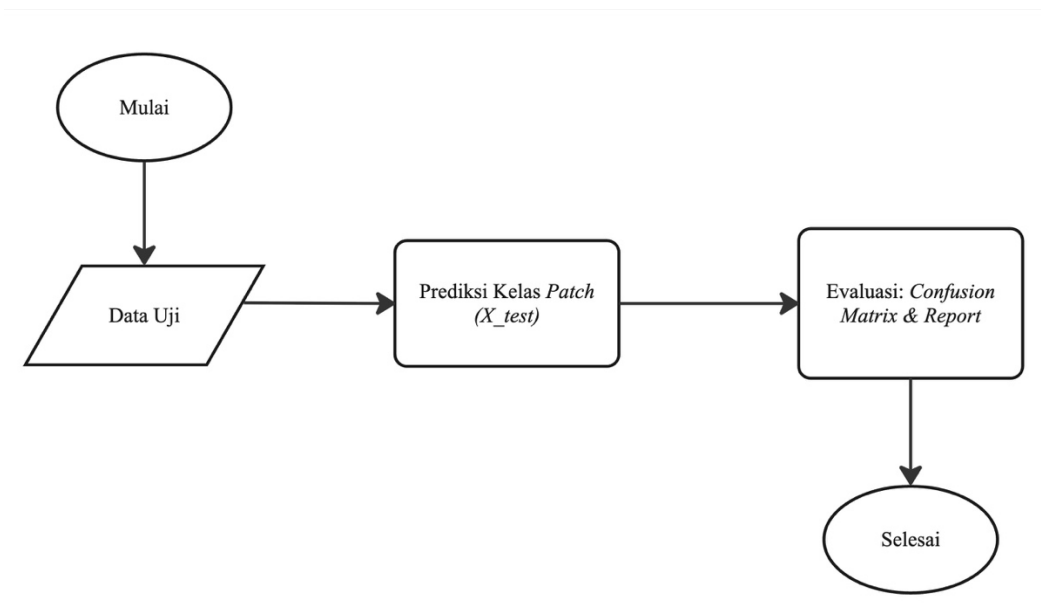
Gambar 3.7 Hybrid Voting Patch-Based (Data Uji)

Sebagai pelengkap dari pendekatan *hybrid voting* yang dilakukan pada seluruh gambar, penelitian ini juga melakukan klasifikasi langsung pada level *patch* menggunakan *hybrid score*. Pada Gambar 3.5 menunjukkan proses *hybrid voting* dalam klasifikasi *dataset* uji. Proses ini dilakukan pada data uji yang terdiri dari fitur *patch* satu per satu (X_{test}) dan label aslinya (y_{test}), tanpa menggabungkan hasil prediksi *patch* menjadi satu keputusan per gambar. Tujuan dari cara ini adalah untuk melihat seberapa baik metode *hybrid* dalam mengenali *patch* seni Jepang dan Cina secara langsung. Dengan menguji setiap *patch* secara terpisah, hasil evaluasi ini memberikan gambaran yang lebih jelas tentang kemampuan model dalam mengenali detail visual dari masing-masing potongan gambar.

3.1.8 Metode Pembandingan

Setelah model klasifikasi menggunakan *hybrid voting* diuji, penelitian ini juga membandingkan performanya dengan beberapa metode lain. Tujuan dari perbandingan ini adalah untuk mengetahui apakah pendekatan *hybrid score* benar-benar memberikan hasil yang lebih baik dibandingkan metode klasifikasi lain yang lebih umum digunakan. Semua model pembandingan diuji menggunakan *dataset* yang sama agar hasil evaluasi bisa dibandingkan secara adil dan objektif. Terdapat empat model pembandingan yang digunakan dalam penelitian ini. Pertama, model klasifikasi yang hanya menggunakan hasil prediksi dari *Support Vector Machine* (*SVM*) tanpa mempertimbangkan jarak ke *centroid*. Kedua, model klasifikasi yang hanya menggunakan jarak terhadap *centroid* tanpa memperhitungkan probabilitas dari *SVM*. Ketiga, model klasifikasi berbasis citra utuh, yaitu klasifikasi dilakukan langsung terhadap gambar tanpa melalui proses pembagian menjadi *patch*. Keempat, model klasifikasi dengan proses peniadaan filtrasi *padding*. Dengan keempat model pembandingan ini, penelitian dapat memberikan analisis yang lebih menyeluruh terhadap efektivitas pendekatan *hybrid score* dalam sistem klasifikasi berbasis *patch*.

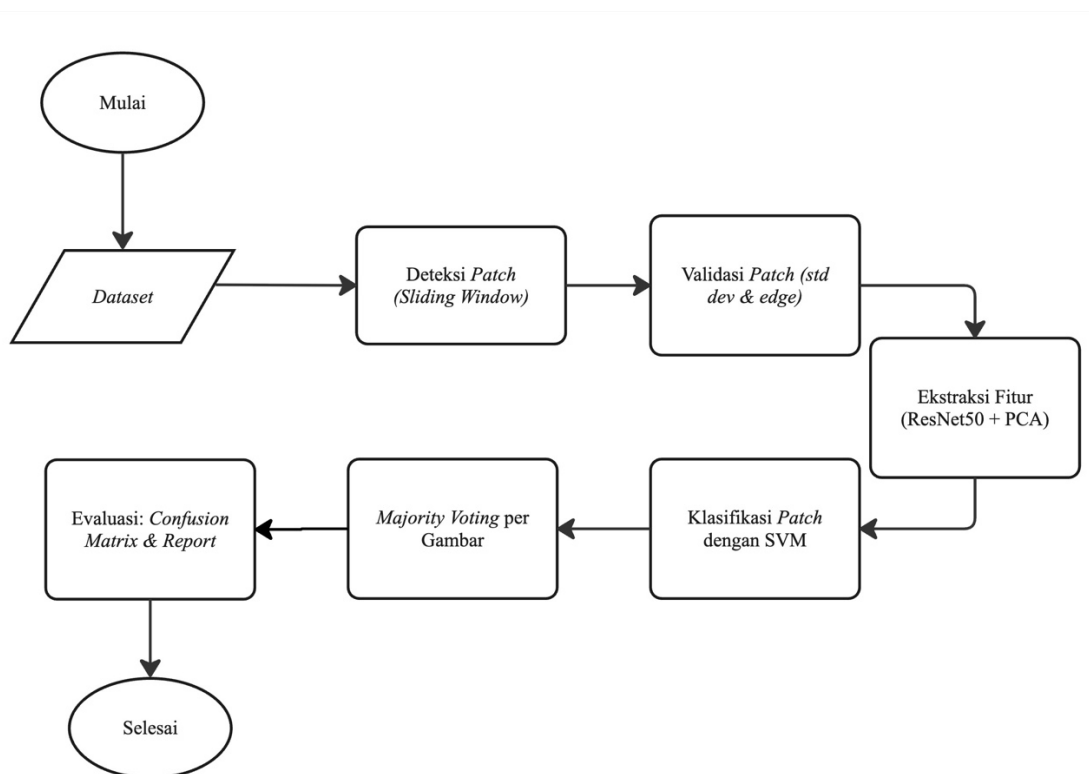
a) Klasifikasi SVM



Gambar 3.8 Klasifikasi *SVM Patch-Based* (Data Uji)

Sebagai baseline pembandingan, penelitian ini menerapkan dua pendekatan klasifikasi menggunakan model *Support Vector Machine* (*SVM*) yang telah dilatih pada hasil ekstraksi fitur *patch*. Gambar 3.6 menggambarkan alur klasifikasi menggunakan *SVM* terhadap data uji.

Pendekatan pertama adalah klasifikasi langsung terhadap data uji yang terdiri dari fitur *patch* individual (X_{test}) dan label asli (y_{test}). Pada metode ini, setiap *patch* dalam data uji dinormalisasi dan diproses oleh model *SVM* untuk menghasilkan prediksi kelas. Hasil prediksi tersebut kemudian dievaluasi menggunakan *confusion matrix* dan *classification report* untuk memperoleh metrik akurasi, presisi, *recall*, dan *F1-score*. Pendekatan ini memberikan gambaran performa model *SVM* pada tingkat granularitas *patch*, di mana fitur visual telah disusun dan direduksi sebelumnya menggunakan metode *Principal Component Analysis* (*PCA*). Sementara itu, pendekatan kedua dilakukan terhadap gambar utuh dengan mekanisme *majority voting* berbasis *patch*. Gambar 3.7 menunjukkan tahapan klasifikasi gambar utuh diambil dari *dataset* asli menggunakan pendekatan *SVM Voting*. Setiap *patch* dipecah menjadi *patch* berukuran 224×224 piksel menggunakan metode *sliding window* dengan *overlap*. *Patch* yang dihasilkan kemudian divalidasi berdasarkan tiga kriteria visual, yaitu standar deviasi intensitas, jumlah tepi (*edge*), dan rasio maksimum *padding* hitam di gambar untuk memastikan hanya *patch* yang mengandung informasi cukup yang digunakan dalam proses klasifikasi. *Patch* yang valid diekstraksi fiturnya menggunakan arsitektur *ResNet50* tanpa bagian *fully-connected layer*, kemudian direduksi dimensinya menggunakan *Principal Component Analysis* (*PCA*). Fitur dari setiap *patch* diklasifikasikan menggunakan model *Support Vector Machine* (*SVM*), dan hasil prediksi dari seluruh *patch* dalam satu gambar dikumpulkan untuk dilakukan proses *voting*. Label akhir untuk setiap gambar ditentukan berdasarkan mayoritas prediksi kelas dari *patch* yang valid. Pendekatan ini merepresentasikan skenario dunia nyata, di mana input berupa gambar utuh diklasifikasikan secara menyeluruh melalui analisis bagian-bagian lokalnya.

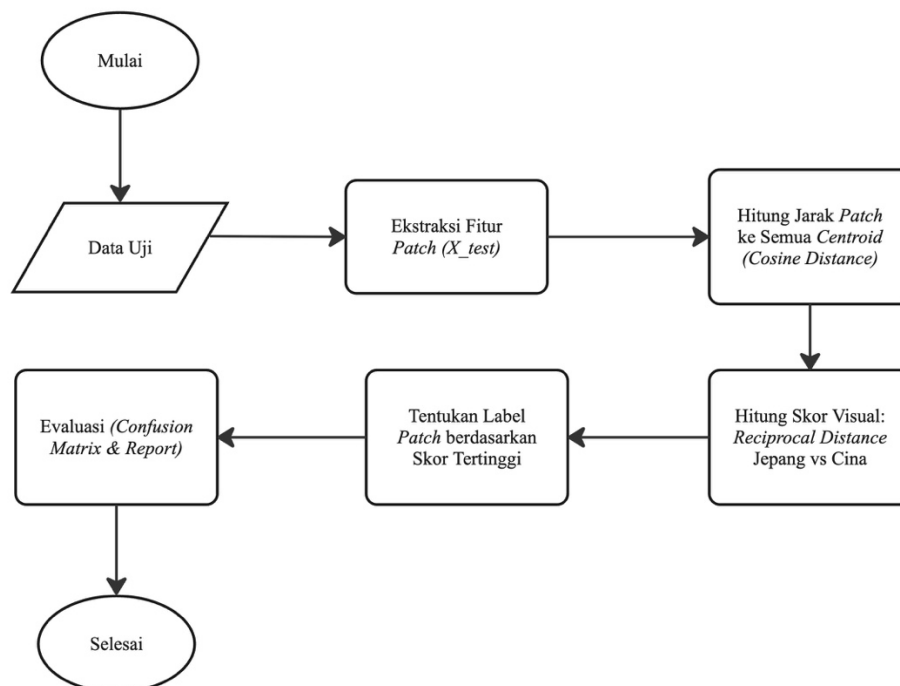


Gambar 3.9 Klasifikasi SVM *Voting* Gambar Utuh

b) Klasifikasi Berbasis *Centroid*

Pendekatan klasifikasi berbasis *centroid* (*centroid-based classification*) dilakukan dengan memanfaatkan representasi fitur dari *patch* yang telah direduksi menggunakan *Principal Component Analysis (PCA)*, kemudian dihitung jaraknya terhadap pusat kluster (*centroid*) yang diperoleh dari hasil *discriminative clustering*. Proses klasifikasi ini tidak menggunakan model *supervised learning* seperti *Support Vector Machine (SVM)*, tetapi sepenuhnya bergantung pada kemiripan fitur berdasarkan jarak antara fitur *patch* dan *centroid* kluster. Evaluasi dilakukan dalam dua skenario yaitu yang pertama, klasifikasi langsung pada level *patch* (x_{test}), dan kedua, pada level gambar utuh melalui akumulasi hasil prediksi *patch* menggunakan metode *voting*.

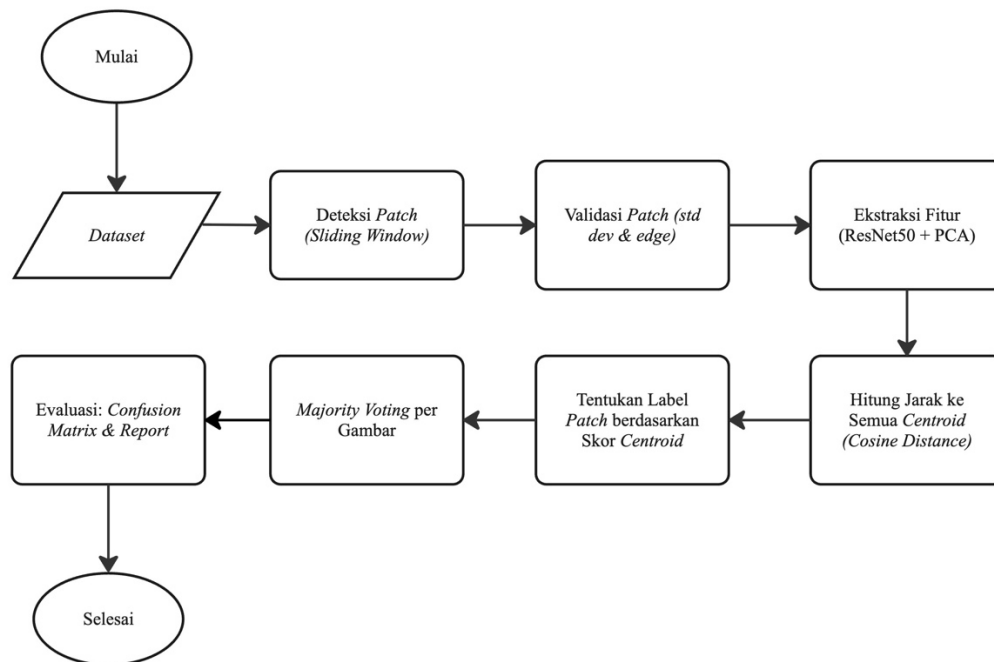
Gambar 3.8 menunjukkan alur klasifikasi berbasis *centroid* (*centroid-based classification*) menggunakan data uji. Pada skenario pertama, evaluasi dilakukan terhadap data uji (X_{test}) yang telah disiapkan sebelumnya. Setiap fitur *patch* yang telah dinormalisasi dihitung jaraknya terhadap seluruh *centroid* menggunakan metrik *cosine distance*. Sebuah *patch* diklasifikasikan sebagai lukisan Jepang apabila nilai skor terhadap *centroid* dari kluster dominan Jepang lebih tinggi dibandingkan dengan skor terhadap kluster yang mewakili lukisan Cina. Skor tersebut dihitung menggunakan kebalikan dari jarak (*reciprocal distance*), sehingga semakin dekat jaraknya, semakin besar nilai skornya. Hasil klasifikasi dari seluruh *patch* dalam X_{test} kemudian dievaluasi menggunakan *confusion matrix* dan *classification report* untuk memperoleh metrik akurasi, presisi, *recall*, dan *F1-score*.



Gambar 3. 10 Klasifikasi Berbasis *Centroid* menggunakan Data Uji (*Patch Based*)

Skenario kedua dilakukan pada level gambar. Gambar 3.9 menunjukkan alur klasifikasi berbasis *centroid* (*centroid-based classification*) menggunakan gambar utuh yang diambil dari *dataset* asli. Setiap gambar dari *dataset* awal dipecah menjadi *patch* berukuran 224×224 piksel menggunakan pendekatan *sliding window*. *Patch* yang lolos validasi berdasarkan standar deviasi intensitas warna, jumlah tepi (*edge*), dan rasio maksimum *padding* hitam kemudian diproses melalui *feature extractor ResNet50*, direduksi dimensinya menggunakan *Principal*

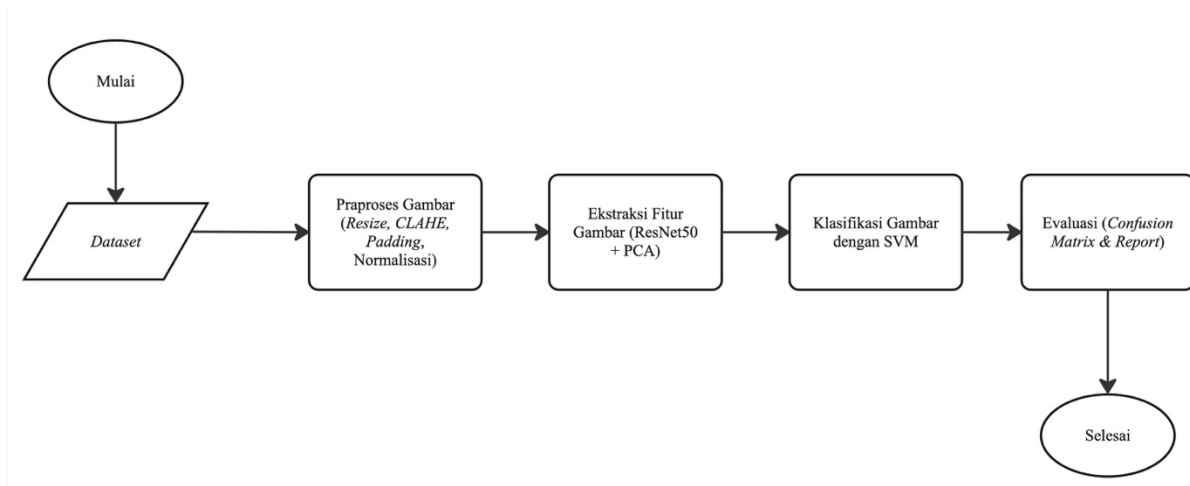
Component Analysis (PCA), dan dihitung jaraknya terhadap seluruh *centroid*. Prediksi kelas dari setiap *patch* ditentukan menggunakan metode yang sama seperti pada skenario pertama, yaitu berdasarkan nilai *reciprocal distance* terhadap *centroid* dari klaster dominan Jepang atau Cina. Selanjutnya, hasil klasifikasi dari semua *patch* dalam satu gambar digabungkan melalui mekanisme *majority voting* untuk menentukan label akhir gambar. Proses ini merepresentasikan bagaimana pendekatan *centroid-based classification* dapat digunakan untuk mengenali karakteristik visual khas lukisan Jepang secara *unsupervised*, tanpa memerlukan pelabelan eksplisit pada tahap pelatihan.



Gambar 3.11 Klasifikasi Berbasis *Centroid* pada Gambar Utuh

c) Model dan Klasifikasi Gambar Utuh

Sebagai metode pembandingan terhadap pendekatan berbasis *patch*, penelitian ini juga mengevaluasi performa klasifikasi menggunakan gambar utuh secara langsung. Gambar 3.10 menunjukkan alur pembuatan model dan klasifikasi dengan gambar utuh. Dalam pendekatan ini, gambar dipertahankan tanpa membaginya menjadi potongan kecil. Setiap gambar terlebih dahulu melalui serangkaian tahap praproses, yaitu peningkatan kontras menggunakan metode *Contrast Limited Adaptive Histogram Equalization (CLAHE)*, penyesuaian ukuran hingga mendekati dimensi maksimum, serta penambahan *padding* hitam agar ukuran akhir menjadi kelipatan 224 piksel. Tahapan ini bertujuan untuk memastikan kompatibilitas input dengan arsitektur *Convolutional Neural Network (CNN)* yang digunakan, yaitu *ResNet50*. Setelah praproses selesai, citra dinormalisasi ke rentang nilai $[0, 1]$ dan kemudian diekstraksi fiturnya menggunakan model *ResNet50* tanpa lapisan klasifikasi akhir (*fully connected layer*). Fitur yang dihasilkan direduksi dimensinya menggunakan *Principal Component Analysis (PCA)* untuk menghasilkan representasi yang lebih kompak. Selanjutnya, fitur ini digunakan sebagai input untuk model *supervised learning*, seperti *Support Vector Machine (SVM)*, yang telah dilatih sebelumnya. Pendekatan ini memungkinkan sistem untuk mengenali pola visual secara menyeluruh dalam satu citra dan digunakan sebagai pembandingan terhadap pendekatan berbasis *patch* untuk menilai efektivitas klasifikasi dalam konteks global gambar.



Gambar 3.12 Alur Pembuatan Model dan Klasifikasi dengan Gambar Utuh

d) Model dan Klasifikasi Tanpa Filtrasi *Padding*

Metode untuk pembuatan model ini sama dengan metode pembuatan model klasifikasi *hybrid voting*. Penjelasan lengkap mengenai tahapan pembuatan dan klasifikasi model *hybrid voting* telah digambarkan sebelumnya pada Subbab 3.3.7 **Klasifikasi Hybrid Voting**. Perbedaan utama pada metode pembandingan ini adalah peniadaan salah satu parameter validasi yaitu filtrasi tambahan yang menyaring *patch* dengan proporsi piksel hitam melebihi 20% sehingga *patch* yang memiliki banyak rasio *padding* hitam tetap dipakai pada percobaan ini.

3.1.9 Evaluasi Model

Evaluasi model dilakukan untuk mengukur kinerja sistem klasifikasi dalam membedakan lukisan Jepang dan Cina. Evaluasi mencakup pengujian performa pada level *patch* maupun akumulasi hasil klasifikasi pada level gambar menggunakan pendekatan *hybrid voting* yang telah dijelaskan sebelumnya. Metode evaluasi utama yang digunakan dalam penelitian ini meliputi *confusion matrix*, *classification report*, serta visualisasi hasil prediksi. *Confusion matrix* memberikan informasi mengenai jumlah prediksi yang benar dan salah untuk masing-masing kelas, sedangkan *classification report* menyajikan metrik evaluasi seperti *precision*, *recall*, *F1-score*, dan *accuracy*.

3.2 Bahan dan peralatan yang digunakan

Untuk mendukung uji coba performa model penelitian, digunakan beberapa peralatan dan bahan yang dapat dilihat sebagai berikut:

3.2.1 Perangkat Keras

- Apple M3 Pro with 11-core CPU and 14-core GPU*
- Random Access Memory (RAM) 8 GB*
- Neural Engine 16-core*
- Solid State Drive (SSD) 512 GB*

3.2.2 Perangkat Lunak

- Sistem Operasi macOS Sonoma 14.2.1
- Visual Studio Code
- Python beserta library pendukung penelitian

3.3 Rencana Implementasi dan Uji Coba

Pada Tugas Akhir seluruh proses dilakukan menggunakan bahasa pemrograman Python dengan library yang tersedia.

3.3.1 Implementasi Praproses Data

Sebelum data dianalisis, data terlebih dahulu diproses agar sesuai dengan kebutuhan penelitian. Kode Semu 3.1 menjelaskan praproses gambar dari folder `dataset_positif` sebelum digunakan dalam proses analisis. Seluruh proses diawali pada langkah `BEGIN`, dengan memeriksa keberadaan folder output `processed_dataset_positif` menggunakan perintah `CHECK_FOLDER_EXISTS`. Jika folder tersebut belum tersedia, maka dibuat terlebih dahulu untuk menyimpan hasil gambar yang telah diproses. Pendekatan ini memastikan bahwa *file* benar-benar tersedia melalui `FILE_EXISTS`, kemudian gambar dibaca menggunakan `READ_IMAGE`. Setelah gambar berhasil dimuat, dilakukan peningkatan kontras lokal menggunakan metode *Contrast Limited Adaptive Histogram Equalization* (CLAHE) pada ruang warna LAB. Proses ini diawali dengan konversi gambar dari format BGR ke LAB melalui `CONVERT_BGR_TO_LAB`, kemudian dipisahkan menjadi tiga kanal: L (luminance), A, dan B menggunakan `SPLIT_CHANNELS`. CLAHE diterapkan khusus pada kanal L menggunakan `INIT_CLAHE` dan `APPLY_CLAHE`, lalu kanal yang telah ditingkatkan dikombinasikan kembali dengan kanal A dan B menggunakan `MERGE_CHANNELS`, serta dikonversi kembali ke format BGR dengan `CONVERT_LAB_TO_BGR`.

```
INPUT: Folder dataset_positif (gambar mentah)
OUTPUT: Folder processed_dataset_positif1 (gambar hasil praproses)

1: BEGIN
2:     // Buat folder output jika belum tersedia
3:     IF processed_dataset_positif1 not exist THEN
4:         CREATE processed_dataset_positif1
5:
6:     image_files ← LIST all image files from dataset_positif
7:
8:     FOR each filename IN image_files DO
9:         input_path ← CONCAT(dataset_positif, filename)
10:        output_path ← CONCAT(processed_dataset_positif1, filename)
11:
12:        IF IS_FILE(input_path) THEN
13:            image ← READ_IMAGE(input_path)
14:
15:            // === Peningkatan Kontras dengan CLAHE ===
16:            lab_image ← CONVERT_BGR_TO_LAB(image)
17:            L, A, B ← SPLIT_CHANNELS(lab_image)
18:            clahe ← INIT_CLAHE(clipLimit=1.8, tileGridSize=(8, 8))
19:            L_clahe ← APPLY_CLAHE(L, clahe)
20:            lab_clahe ← MERGE_CHANNELS(L_clahe, A, B)
21:            enhanced ← CONVERT_LAB_TO_BGR(lab_clahe)
22:
23:            // === Resize dan Padding ke Kelipatan 224 ===
24:            h, w ← GET_DIMENSIONS(enhanced)
25:            IF MAX(h, w) > 896 THEN
26:                scale ← MIN(896 / w, 896 / h)
27:                new_w ← MAX(INT(w × scale), 224)
28:                new_h ← MAX(INT(h × scale), 224)
29:            ELSE
```



```

30:         new_w ← MAX(FLOOR(w / 224) × 224, 224)
31:         new_h ← MAX(FLOOR(h / 224) × 224, 224)
32:
33:         resized ← RESIZE_IMAGE(enhanced, (new_w, new_h))
34:
35:         target_w ← CEIL(new_w / 224) × 224
36:         target_h ← CEIL(new_h / 224) × 224
37:         pad_w ← target_w - new_w
38:         pad_h ← target_h - new_h
39:         left ← pad_w // 2; right ← pad_w - left
40:         top ← pad_h // 2; bottom ← pad_h - top
41:         padded ← PAD_IMAGE(resized, top, bottom, left, right, fill=black)
42:
43:         // === Normalisasi ke rentang [0, 1] ===
44:         normalized ← padded / 255.0
45:
46:         // === Simpan hasil ke folder output ===
47:         SAVE_IMAGE(normalized × 255, output_path)
48:
49:     END FOR
50: END

```

Kode Semu 3.1 Praproses *Dataset* Positif (seni Jepang)

Langkah berikutnya adalah menyesuaikan ukuran gambar ke dimensi kelipatan 224 piksel agar kompatibel dengan proses *patching* dan input model CNN. Ukuran asli gambar diambil melalui `GET_DIMENSION`, kemudian diperiksa apakah diperlukan *scaling*. Jika salah satu sisi gambar melebihi 896 piksel, maka dilakukan pengecilan ukuran gambar secara proporsional menggunakan fungsi `MIN` dan `MAX`, dengan batas bawah 224 piksel. Dimensi gambar yang memiliki ukuran lebih kecil akan dibulatkan ke kelipatan 224 terdekat menggunakan `FLOOR` dan hasilnya diubah ukurannya melalui `RESIZE_IMAGE`. Setelah proses *resize*, gambar dipastikan berada pada dimensi kelipatan 224 dengan menambahkan *padding* hitam di sekeliling gambar menggunakan fungsi `PAD_IMAGE`. *Padding* dihitung dari selisih antara target ukuran dan ukuran hasil *resize* menggunakan `CEIL`, lalu dibagi merata ke sisi kiri-kanan dan atas-bawah agar posisi gambar tetap ditengah. Tahap terakhir adalah normalisasi gambar ke rentang [0, 1] dengan membagi seluruh piksel dengan 255. Setelah itu, gambar dikalikan kembali dengan 255 dan disimpan dalam format 8-bit menggunakan fungsi `SAVE_IMAGE`. Proses ini diulang untuk setiap *file* dalam folder menggunakan `END FOR`, dan ditutup dengan `END`.

Kode Semu 3.2 menggambarkan tahapan pra-pemrosesan gambar dari folder `dataset_negatif`. Praproses yang dilakukan pada `dataset_negatif` mempunyai kemiripan dengan yang dilakukan pada `dataset_positif`. Bagian ini hanya menekankan perbedaan utama, yaitu metode interpolasi *bicubic* yang digunakan untuk memperbesar ukuran gambar `dataset_negatif`. Proses *resize* yang dilakukan pada `dataset_negatif` mengubah semua ukuran gambar dalam *dataset* menjadi 896x896 piksel menggunakan interpolasi *bicubic*. Hal ini dilakukan karena melihat karakteristik ukuran semua gambar dalam dataset seni Cina yang berukuran 256x256. Untuk menghindari adanya ketimpangan jumlah data pada hasil pemotongan *patch*, *resize* untuk *dataset* seni Cina serentak akan diperbesar menjadi ukuran maksimal yang sudah ditentukan.

```

INPUT:
- Folder dataset_negatif (gambar mentah)
- MAX_SIZE ← 896
- PATCH_SIZE ← 224

OUTPUT:

```

```

- Folder processed_dataset_negatif_bicubic (gambar hasil praproses)

1: BEGIN
2:   SET TARGET_SIZE ← (896, 896)
3:   CREATE FOLDER "processed_dataset_negatif_bicubic" IF not exist
4:
5:   image_files ← LIST all .png/.jpg/.jpeg in "dataset_negatif"
6:   total_images ← LENGTH(image_files)
7:
8:   FOR each filename IN image_files DO
9:     input_path ← JOIN("dataset_negatif", filename)
10:    output_path ← JOIN("processed_dataset_negatif_bicubic", filename)
11:
12:    IF IS_FILE(input_path) THEN
13:      image ← READ_IMAGE(input_path)
14:
15:      // === CLAHE: Peningkatan Kontras Lokal ===
16:      lab ← CONVERT_BGR_TO_LAB(image)
17:      L, A, B ← SPLIT_CHANNELS(lab)
18:      clahe ← INIT_CLAHE(clipLimit = 1.8, tileGridSize = (8,8))
19:      L_clahe ← APPLY_CLAHE(L, clahe)
20:      lab_clahe ← MERGE_CHANNELS(L_clahe, A, B)
21:      enhanced ← CONVERT_LAB_TO_BGR(lab_clahe)
22:
23:      // === Resize ke 896×896 dengan interpolasi bicubic ===
24:      resized ← RESIZE_IMAGE(enhanced, TARGET_SIZE, method = INTER_CUBIC)
25:
26:      // === Normalisasi piksel ke rentang [0, 1] ===
27:      normalized ← resized / 255.0
28:
29:      // === Simpan hasil sebagai uint8 ===
30:      SAVE_IMAGE(normalized × 255, output_path)
31:
32: END

```

Kode Semu 3.2 Praproses *Dataset* negatif (seni Cina)

3.3.2 Implementasi Deteksi *Patch*

Kode Semu 3.3 menjelaskan proses deteksi *patch* gambar berukuran 224×224 piksel dari hasil praproses data yang disimpan dalam folder `processed_dataset_positif` dan `processed_dataset_negatif`. Proses dimulai dengan memastikan bahwa folder untuk menyimpan hasil ekstraksi sudah tersedia yaitu `patches_dataset_negatif` atau `patches_dataset_positif`. Jika belum ada, maka folder tersebut dibuat terlebih dahulu. Seluruh *file* gambar dari folder input dibaca dan dimasukkan ke dalam daftar `image_files` dan variabel `total_patches` diinisialisasi untuk menghitung jumlah *patch* valid yang berhasil disimpan selama proses. Jika gambar gagal dibaca, atau memiliki dimensi kurang dari 224 piksel pada salah satu sisi, maka gambar tersebut dilewati. Selain itu, apabila ukuran gambar tidak merupakan kelipatan dari 224, sistem akan memberikan peringatan karena hal tersebut dapat memengaruhi pembentukan grid *patch*. Nama dasar *file* gambar diambil menggunakan `GET_FILENAME_BASE` setelah itu penghitungan *patch* (`patch_count`) diinisialisasi. Proses deteksi dilakukan dengan teknik *sliding window* dua dimensi, di mana *patch* berukuran tetap dipotong dari koordinat (x , y) menggunakan fungsi `CROP_IMAGE`. Setiap *patch* yang diperoleh selanjutnya divalidasi menggunakan dua kriteria utama: variasi intensitas piksel dan jumlah *edge*. Pertama, *patch* dikonversi ke skala keabuan menggunakan `CONVERT_TO_GRAYSCALE`, kemudian dihitung standar deviasinya dengan `CALCULATE_STD` untuk mengetahui seberapa besar variasi intensitas piksel. Kedua, struktur garis atau jumlah *edge patch* dihitung menggunakan metode deteksi tepi Canny melalui `COUNT_EDGES`. Ketiga, dihitung proporsi

piksel yang sangat gelap (dengan intensitas < 10). Jika proporsi ini melebihi ambang batas 20%, maka patch dianggap terlalu banyak mengandung area padding dan dinyatakan tidak valid. Sebuah *patch* hanya akan disimpan jika memiliki standar deviasi ≥ 20 , jumlah *edge* ≥ 1200 dan rasio maksimum padding $\leq 20\%$. Nama *file patch* dibentuk dari nama *file* asli dan posisi koordinat *patch*, lalu disimpan ke folder output menggunakan `SAVE_IMAGE`.

```

Input: Folder dataset hasil pra-pemrosesan (processed_dataset)
Output: Folder patch valid (patches_dataset)
1: BEGIN
2:     // === Konfigurasi awal ===
3:     WINDOW_SIZE ← (224, 224)
4:     STRIDE ← 224
5:     COLOR_VARIATION_THRESHOLD ← 20
6:     EDGE_THRESHOLD ← 1200
7:     PADDING_BLACK_RATIO_THRESHOLD ← 0.2
8:
9:     INPUT_FOLDER ← "processed_dataset_positif"
10:    OUTPUT_FOLDER ← "patches_dataset_positif_nopadding"
11:    CREATE_FOLDER_IF_NOT_EXIST(OUTPUT_FOLDER)
12:
13:    FUNCTION is_valid_patch(patch):
14:        gray ← CONVERT_TO_GRAYSCALE(patch)
15:        black_ratio ← COUNT_PIXELS(gray < 10) / TOTAL_PIXELS(gray)
16:        IF black_ratio > PADDING_BLACK_RATIO_THRESHOLD THEN
17:            RETURN False
18:        std_dev ← CALCULATE_STANDARD_DEVIATION(gray)
19:        IF std_dev < COLOR_VARIATION_THRESHOLD THEN
20:            RETURN False
21:        edge_count ← COUNT_EDGES_USING_CANNY(gray, 50, 150)
22:        IF edge_count < EDGE_THRESHOLD THEN
23:            RETURN False
24:        RETURN True
25:
26:    FUNCTION extract_patches(image_path):
27:        image ← READ_IMAGE(image_path)
28:        IF image IS None THEN
29:            PRINT "Gagal membaca:", image_path
30:            RETURN 0
31:        h, w ← GET_DIMENSIONS(image)
32:        IF h < 224 OR w < 224 THEN
33:            PRINT "Gambar terlalu kecil:", image_path
34:            RETURN 0
35:        IF h MOD 224 ≠ 0 OR w MOD 224 ≠ 0 THEN
36:            PRINT "Ukuran bukan kelipatan 224:", image_path
37:
38:        patches_saved ← 0
39:        base_filename ← GET_FILENAME_BASE(image_path)
40:
41:        FOR y FROM 0 TO h - 224 STEP STRIDE DO
42:            FOR x FROM 0 TO w - 224 STEP STRIDE DO
43:                patch ← CROP(image, x, y, width=224, height=224)
44:                IF is_valid_patch(patch) THEN
45:                    filename ← CONCAT(base_filename, "_x", x, "_y", y, ".png")
46:                    SAVE_IMAGE(patch, OUTPUT_FOLDER + "/" + filename)
47:                    patches_saved ← patches_saved + 1
48:
49:        RETURN patches_saved
50:
51:    // === Proses seluruh gambar ===

```

```

52:      image_paths ← LIST_IMAGES_IN_FOLDER(INPUT_FOLDER, extensions=[".png",
".jpg", ".jpeg"])
53:      total_patches ← 0
54:
55:      FOR image_path IN image_paths DO
56:          count ← extract_patches(image_path)
57:          total_patches ← total_patches + count
58:          PRINT count, "patch diekstrak dari", image_path
59:
60:      PRINT "Total patch disimpan:", total_patches
61:  END

```

Kode Semu 3.3 Deteksi *Patch Dataset* Positif dan Negatif

3.3.3 Implementasi Ekstraksi Fitur

Kode Semu 3.4 menjelaskan langkah-langkah dalam melakukan ekstraksi fitur dari *patch* gambar menggunakan arsitektur *Convolutional Neural Network* (CNN) *ResNet50* dan dilakukan penambahan *GlobalAveragePooling2D* setelah lapisan konvolusional terakhir. Seluruh *file patch* dibaca dari direktori *patches_dataset_positif* atau *patches_dataset_negatif* dan proses ekstraksi dilakukan secara *batch* untuk efisiensi memori. Setiap *patch* yang dibaca akan diproses secara berurutan yaitu pertama akan dikonversi ke format RGB (*CONVERT_BGR_TO_RGB*), kedua akan di-*resize* ke ukuran standar 224×224 piksel, dan ketiga dipreproses menggunakan fungsi *preprocess_input* agar sesuai dengan skema input *ResNet50*. *Patch* yang telah diproses disimpan dalam *buffer* atau tempat penyimpanan sementara yaitu *batch_images*, sementara nama *file*-nya disimpan dalam *batch_names*. Ketika jumlah *patch* dalam *buffer* mencapai batas *batch size*, atau ketika seluruh *patch* telah selesai dibaca, seluruh batch dikonversi ke dalam array dan diproses melalui model CNN untuk mengekstraksi fitur visual. Hasil ekstraksi berupa vektor fitur berdimensi 2048 untuk setiap *patch*. Selanjutnya, dilakukan proses reduksi dimensi fitur menggunakan *PCA*, dari 2048 menjadi 1024. *PCA* ini kemudian disimpan dan digunakan ulang untuk mentransformasikan data dari kelas lain guna memastikan konsistensi ruang fitur antar kelas. Untuk *patch* dari kelas negatif, proses reduksi dimensi dilakukan dengan menggunakan model *PCA* yang telah dilatih pada data positif. Pendekatan ini dilakukan untuk menghindari perbedaan distribusi representasi antar kelas akibat pelatihan *PCA* yang tidak seragam.

```

Input:
- Folder patches_dataset_positif
- Folder patches_dataset_negatif
- Model CNN ResNet50
- Parameter: Ukuran gambar: 224x224, Ukuran batch: 1024, PCA dimensi target: 1024

Output:
- File PCA model pca_1024.pkl
- File fitur PCA positif extracted_features_positif.pkl
- File fitur PCA negatif extracted_features_negatif.pkl

1: BEGIN
2:   SET IMAGE_SIZE ← (224, 224)
3:   SET BATCH_SIZE ← 1024
4:   SET N_COMPONENTS_PCA ← 1024
5:   INIT model ← ResNet50 pretrained + GAP layer
6:   FUNCTION extract_patches_from_folder(folder_path):
7:     INIT all_features ← [], all_names ← []
8:     INIT batch_images ← [], batch_names ← []
9:     patch_files ← LIST_FILES(folder_path)
10:    FOR each patch IN patch_files DO

```

```

11:         img ← READ_IMAGE(patch)
12:         img ← CONVERT_BGR_TO_RGB(img)
13:         img ← RESIZE(img, IMAGE_SIZE)
14:         img ← PREPROCESS_INPUT(img)
15:         APPEND img TO batch_images
16:         APPEND patch_name TO batch_names
17:         IF batch_images.size == BATCH_SIZE OR LAST_PATCH THEN
18:             batch_np ← CONVERT_TO_ARRAY(batch_images)
19:             features ← model.predict(batch_np)
20:             APPEND features TO all_features
21:             APPEND batch_names TO all_names
22:             RESET batch_images, batch_names
23:         END FOR
24:         RETURN all_features, all_names
25:     END FUNCTION
26:     // === Ekstraksi dan PCA pada dataset positif ===
27:     features_pos, names_pos ←
extract_patches_from_folder("patches_dataset_positif ")
28:     pca_model ← INIT_PCA(n_components=N_COMPONENTS_PCA)
29:     features_pos_pca ← FIT_TRANSFORM(pca_model, features_pos)
30:     SAVE_MODEL(pca_model, "pca_1024.pkl")
31:     SAVE_TO_FILE({"features": features_pos_pca, "image_names": names_pos},
"extracted_features_positif_resnet_pca_nopadding.pkl")

32:     // === Ekstraksi dan PCA transform pada dataset negatif ===
33:     features_neg, names_neg ←
extract_patches_from_folder("patches_dataset_negatif ")
34:     features_neg_pca ← TRANSFORM(pca_model, features_neg)
35:     SAVE_TO_FILE({"features": features_neg_pca, "image_names": names_neg},
"extracted_features_negatif_resnet_pca_nopadding.pkl")
36: END

```

Kode Semu 3.4 Implementasi Ekstraksi Fitur pada *Dataset* Positif dan *Dataset* Negatif

3.3.4 Implementasi *Clustering*

Kode Semu 3.5 menjelaskan alur lengkap proses *discriminative clustering*, yang bertujuan untuk mengelompokkan *patch* gambar dari *dataset* positif (lukisan Jepang) dan *dataset* negatif (lukisan Cina). Proses diawali dengan memuat dua *file* .pkl yang berisi fitur hasil ekstraksi dan reduksi PCA. Kedua kumpulan fitur ini digabung menggunakan fungsi `CONCATENATE`, lalu label biner diberikan menggunakan `ASSIGN_LABELS` dengan label 1 untuk *patch* Jepang dan 0 untuk *patch* Cina. Nama *file patch* dari kedua dataset juga digabungkan untuk kebutuhan pelacakan hasil *clustering*. Selanjutnya, seluruh fitur dinormalisasi dan distandarkan menggunakan `APPLY_STANDARD_SCALER` dan `APPLY_L2_NORMALIZATION`, agar berada dalam skala seragam dan memiliki distribusi yang stabil. Fitur berdimensi 1024 tersebut kembali direduksi menjadi 100 dimensi menggunakan algoritma *Uniform Manifold Approximation and Projection* (UMAP) melalui fungsi `APPLY_UMAP`, dengan parameter `n_neighbors = 10` dan `min_dist = 0.0`. Hasil proyeksi UMAP digunakan sebagai input untuk proses *clustering* dengan algoritma *KMeans* berbasis jarak *cosine*. Penentuan jumlah kluster terbaik dilakukan dengan mengevaluasi nilai *K* dari 2 hingga 9, di mana setiap konfigurasi *K* dinilai berdasarkan *silhouette score* menggunakan `COMPUTE_SILHOUETTE_SCORE`. Nilai *K* dengan skor tertinggi dipilih sebagai jumlah kluster optimal, dan label hasil *clustering* disimpan sebagai `best_labels`. Setelah jumlah kluster ditentukan, sistem menghitung *centroid* dari masing-masing kluster berdasarkan fitur berdimensi 1024 (bukan fitur hasil UMAP). Proses ini dilakukan dengan memanggil `SELECT_BY_CLUSTER` untuk memilih fitur berdasarkan label kluster, dan `MEAN` untuk menghitung rata-rata fitur yang menjadi representasi *centroid*. Seluruh hasil proses *clustering*, termasuk fitur, label asli, label kluster, *centroid*, dan nama *file*, dijadikan dalam struktur

clustering_data, lalu disimpan ke dalam *file* .pkl menggunakan SAVE_TO_FILE. Untuk memastikan integritas hasil penyimpanan, *file* yang telah disimpan dimuat kembali melalui LOAD_CLUSTERING_DATA, dan diverifikasi kesesuaiannya menggunakan fungsi COMPARE dan VERIFY_IMAGE_NAMES.

```

Input:
- File fitur hasil ekstraksi:
  • extracted_features_positif.pkl
  • extracted_features_negatif.pkl
- Parameter:
  • PCA dimensi akhir: 1024
  • UMAP dimensi akhir: 100, n_neighbors = 10, min_dist = 0.0
  • K range evaluasi clustering: 2 sampai 9
Output:
- Label cluster untuk setiap patch
- Centroid setiap cluster pada ruang PCA 1024
- File hasil clustering dalam folder

1: BEGIN
2:     // === Muat fitur dari dataset positif dan negatif ===
3:     features_pos ← LOAD_FEATURES("extracted_features_positif_resnet_pca.pkl")
4:     features_neg ← LOAD_FEATURES("extracted_features_negatif_perbesar_resnet_pca.pkl")
5:     features ← CONCATENATE(features_pos, features_neg)
6:     labels ← ASSIGN_LABELS(positif=1, negatif=0)
7:     image_names ← CONCATENATE_IMAGE_NAMES(features_pos, features_neg)
8:
9:     // === Normalisasi dan standarisasi fitur ===
10:    features ← APPLY_STANDARD_SCALER(features)
11:    features ← APPLY_L2_NORMALIZATION(features)
12:
13:    // === UMAP reduksi ke 100 dimensi untuk clustering visual ===
14:    features_umap ← APPLY_UMAP(features, n_neighbors=10, min_dist=0.0,
n_components=100)
15:
16:    // === Evaluasi KMeans dengan K = 2 sampai 9 menggunakan silhouette score
===
17:    best_score ← -1
18:    FOR k FROM 2 TO 9 DO
19:        cluster_labels ← APPLY_COSINE_KMEANS(features_umap, n_clusters=k)
20:        score ← COMPUTE_SILHOUETTE_SCORE(features_umap, cluster_labels)
21:        IF score > best_score THEN
22:            best_score ← score
23:            best_k ← k
24:            best_labels ← cluster_labels
25:    END FOR
26:
27:    // === Hitung centroid tiap cluster berdasarkan fitur asli 1024D ===
28:    centroids ← []
29:    FOR i FROM 0 TO best_k - 1 DO
30:        cluster_data ← SELECT_BY_CLUSTER(features, best_labels, i)
31:        centroid ← MEAN(cluster_data)
32:        centroids.APPEND(centroid)
33:    END FOR
34:
35:    // === Simpan hasil clustering ===
36:    clustering_data ← {
37:        "features_1024": features,
38:        "labels": labels,
39:        "cluster_labels": best_labels,
40:        "centroids": centroids,

```

```

41:         "image_names": image_names
42:     }
43:     SAVE_TO_FILE(clustering_data,
"discriminative_clustering_results_final_lagi")
44:
45:     // === Verifikasi hasil ===
46:     loaded ← LOAD_CLUSTERING_DATA()
47:     COMPARE(features, loaded["features_1024"])
48:     VERIFY_IMAGE_NAMES(loaded)
49:
50: END

```

Kode Semu 3.5 Implementasi *Discriminative Clustering*

3.3.5 Implementasi Pembuatan Model

Kode Semu 3.6 menggambarkan proses pelatihan model klasifikasi berbasis *Support Vector Machine (SVM)* menggunakan fitur *patch* yang telah diproses melalui *discriminative clustering*. Proses diawali dengan memuat file *discriminative_clustering.pkl*, yang berisi vektor fitur hasil reduksi PCA (*features_1024*), label klaster hasil *KMeans* (*cluster_labels*), dan label asli dari masing-masing *patch* (*labels*). Langkah ini memastikan bahwa seluruh informasi yang dibutuhkan untuk pelatihan model telah tersedia dalam satu struktur data yang terintegrasi. Langkah penting selanjutnya adalah pemilihan subset *patch* yang paling representatif dari hasil *clustering*. *Patch* yang dianggap mewakili gaya lukisan Jepang dipilih dari klaster 2, 3, dan sebagian dari klaster 0 (khusus untuk yang berasal dari label Jepang) menggunakan fungsi *SELECT*. Sementara itu, *patch* yang merepresentasikan gaya lukisan Cina dipilih dari klaster 1 dan klaster 4. Kedua kelompok fitur ini masing-masing dikelompokkan ke dalam *x_jepang* dan *x_cina*, lalu diberi label biner: 1 untuk Jepang dan 0 untuk Cina. Karena distribusi jumlah *patch* pada kedua kelas mungkin tidak seimbang, dilakukan proses penyeimbangan data menggunakan fungsi *RANDOM_RESAMPLE*.

Sampel diambil sebanyak jumlah minimum dari kedua kelas untuk menjaga proporsi yang adil. Setelah proses *resampling*, label-label dari masing-masing kelas disesuaikan kembali, dan seluruh data digabung menjadi satu set lengkap (*x_all* dan *y_all*) yang siap digunakan untuk proses pelatihan model. *Dataset* yang sudah digabung kemudian dibagi menjadi dua bagian, yaitu data latih dan data uji, dengan proporsi 80:20 menggunakan fungsi *SPLIT_DATA*. Pembagian dilakukan secara stratifikasi agar distribusi label pada kedua bagian tetap proporsional. Selanjutnya, model *Support Vector Machine* diinisialisasi dengan menggunakan kernel *Radial Basis Function (RBF)*, dengan parameter penalti regulasi (*C*) yang diatur sebesar 1.0 untuk mengontrol margin klasifikasi. Parameter *gamma* diatur pada nilai 'scale', yaitu pengaturan otomatis berdasarkan jumlah fitur, sehingga adaptif terhadap skala input. Selain itu, opsi *probability* diaktifkan agar model tidak hanya memberikan prediksi kelas, tetapi juga skor probabilitas yang diperlukan dalam metode klasifikasi berbasis *hybrid voting*. Model dilatih menggunakan data latih melalui fungsi *TRAIN*, dan performanya dievaluasi pada data uji menggunakan *PREDICT*. Hasil evaluasi ditampilkan melalui *DISPLAY_CLASSIFICATION_REPORT*, yang menyajikan metrik evaluasi seperti akurasi, *precision*, *recall*, dan *F1-score*. Sebagai tahap akhir, model yang sudah dilatih serta data uji disimpan dalam direktori *svm_models* menggunakan fungsi *SAVE*.

```

Input:
- File hasil clustering: discriminative_clustering.pkl
  • features_1024 → vektor fitur hasil PCA
  • cluster_labels → label cluster dari KMeans

```

```

• labels → label asli (0: Cina, 1: Jepang)
Output:
- Model SVM terlatih
- Dataset uji untuk evaluasi
- File model dan test set yang disimpan di direktori svm_models

1: BEGIN
2:     // === Load hasil clustering ===
3:     data ← LOAD
4:     clusters ← data["cluster_labels"]
5:     features ← data["features_1024"]
6:     labels ← data["labels"]

7:     // === Seleksi patch Jepang dan Cina berdasarkan cluster dan label ===
8:     jepang_idx ← SELECT WHERE (clusters == 2) OR (clusters == 3) OR
((clusters == 0) AND (labels == 1))
9:     cina_idx ← SELECT WHERE (clusters == 1) OR (clusters == 4)

10:    X_jepang ← features[jepang_idx]
11:    y_jepang ← ASSIGN_LABELS(length = LENGTH(X_jepang), label = 1)
12:    X_cina ← features[cina_idx]
13:    y_cina ← ASSIGN_LABELS(length = LENGTH(X_cina), label = 0)

14:    // === Balancing jumlah sampel dari kedua kelas ===
15:    n_sample ← MIN(LENGTH(X_jepang), LENGTH(X_cina))
16:    X_jepang_bal ← RESAMPLE(X_jepang, n_samples = n_sample, random_state =
42)
17:    X_cina_bal ← RESAMPLE(X_cina, n_samples = n_sample, random_state = 42)
18:    y_jepang_bal ← ASSIGN_LABELS(length = n_sample, label = 1)
19:    y_cina_bal ← ASSIGN_LABELS(length = n_sample, label = 0)

20:    // === Gabungkan data dan label ===
21:    X_all ← CONCATENATE(X_jepang_bal, X_cina_bal)
22:    y_all ← CONCATENATE(y_jepang_bal, y_cina_bal)
23:    DISPLAY CLASS_DISTRIBUTION(y_all)

24:    // === Split menjadi data latih dan uji ===
25:    X_train, X_test, y_train, y_test ← SPLIT_DATA(X_all, y_all, test_size =
0.2, stratify = True, random_state = 42)

26:    // === Latih model SVM ===
27:    svm_model ← INIT_SVM(C = 1.0, kernel = "rbf", gamma = "scale",
probability = True, random_state = 42)
28:    TRAIN(svm_model, X_train, y_train)
29:    DISPLAY "SVM selesai dilatih"

30:    // === Evaluasi model pada test set ===
31:    y_pred ← PREDICT(svm_model, X_test)
32:    report ← GENERATE_CLASSIFICATION_REPORT(y_test, y_pred, labels = ["Cina",
"Jepang"])
33:    DISPLAY report

34:    // === Simpan model dan test set ===
35:    CREATE_FOLDER_IF_NOT_EXISTS("svm_models ")
36:    SAVE(svm_model)
37:    SAVE(X_test, y_test)
38:    DISPLAY "Model dan test set berhasil disimpan"
39: END

```

Kode Semu 3.6 Pembuatan Model

a) Implementasi Klasifikasi *Hybrid Voting* Data Uji

Kode Semu 3.7 menjelaskan tahapan evaluasi model klasifikasi menggunakan pendekatan *hybrid voting*, yaitu metode prediksi yang menggabungkan dua sumber metode yaitu kedekatan fitur terhadap *centroid* hasil *clustering* dan probabilitas kelas dari model *Support Vector*

Machine (SVM). Proses dimulai dengan memuat komponen utama, yaitu model SVM (LOAD_MODEL), dataset uji (LOAD_DATA), dan hasil *clustering* yang berisi posisi *centroid* pada ruang fitur berdimensi 1024 (*centroids*). Data uji terdiri atas vektor fitur uji (x_{test}) dan label asli (y_{test}) yang digunakan sebagai acuan dalam evaluasi performa model. Selanjutnya, sistem menetapkan parameter utama untuk metode *hybrid voting*, yakni bobot α sebesar 0.44 dan β sebesar 0.56. Selain itu, didefinisikan daftar indeks klaster yang mewakili lukisan Jepang ([0, 2, 3]) dan lukisan Cina ([1, 4]). Komponen utama evaluasi ini adalah fungsi *hybrid_predict_feature*, yang menerima satu vektor fitur sebagai input (setelah dinormalisasi), dan mengembalikan label prediksi berupa 1 (Jepang) atau 0 (Cina). Di dalam fungsi tersebut, probabilitas prediksi terhadap kelas Jepang dihitung terlebih dahulu menggunakan model SVM (SVM_PREDICT_PROBA). Kemudian, dihitung jarak *cosine* antara fitur tersebut dan seluruh *centroid*. Jarak ini diubah menjadi skor kedekatan dengan menggunakan rumus kebalikan dari *cosine distance*, semakin dekat nilainya akan semakin besar, yang kemudian dirata-rata untuk masing-masing kelompok klaster Jepang dan Cina. Kedua skor tersebut kemudian dikombinasikan secara linier untuk menghasilkan *hybrid score*. Skor ini dihitung dari gabungan proporsional antara rasio kedekatan terhadap klaster Jepang dan skor probabilitas dari SVM, menggunakan bobot α dan β yang telah ditentukan. Jika nilai *hybrid score* melebihi *threshold* 0,5, maka *patch* diklasifikasikan sebagai lukisan Jepang dan sebaliknya diklasifikasikan sebagai lukisan Cina. Setelah fungsi prediksi *hybrid* didefinisikan, proses evaluasi dilakukan terhadap seluruh fitur dalam data uji.

```

Input:
- Model SVM terlatih (svm_model)
- Data uji (X_test, y_test)
- Centroid hasil clustering (centroids_norm)
- Bobot: ALPHA = 0.44, BETA = 0.56
- Klaster dominan Jepang: [0, 2, 3]
- Klaster dominan Cina: [1, 4]

Output:
- Prediksi kelas untuk setiap patch
- Classification report
- Confusion matrix dan visualisasi

1: BEGIN
2:   // === Load model dan data uji ===
3:   svm_model ← LOAD_MODEL
4:   test_data ← LOAD_DATA
5:   clustering ← LOAD_DATA
6:
7:   X_test ← test_data["X_test"]
8:   y_test ← test_data["y_test"]
9:   centroids ← clustering["centroids"]
10:
11:  // === Parameter hybrid voting ===
12:  ALPHA ← 0.44
13:  BETA ← 0.56
14:  JEPANG_CLUSTERS ← [0, 2, 3]
15:  CINA_CLUSTERS ← [1, 4]
16:
17:  FUNCTION hybrid_predict_feature(feats_norm):

```

```

18:      proba ← SVM_PREDICT_PROBA(svm_model, feat_norm)
19:      dists ← COSINE_DISTANCES(feat_norm, centroids)
20:
21:      japan_score ← MEAN([1 / (dists[c] + ε) FOR c IN JEPANG_CLUSTERS])
22:      cina_score ← MEAN([1 / (dists[c] + ε) FOR c IN CINA_CLUSTERS])
23:
24:      hybrid_score ← ALPHA × (japan_score / (cina_score + ε)) + BETA ×
proba[1]
25:      RETURN 1 IF hybrid_score > 0.5 ELSE 0
26:
27:      // === Prediksi seluruh fitur uji ===
28:      predictions ← []
29:      FOR feat IN X_test DO
30:          feat_norm ← NORMALIZE(feat)
31:          pred ← hybrid_predict_feature(feat_norm)
32:          ADD pred TO predictions
33:      END FOR
34:
35:      // === Evaluasi hasil prediksi ===
36:      report ← GENERATE_CLASSIFICATION_REPORT(y_test, predictions)
37:      cm ← GENERATE_CONFUSION_MATRIX(y_test, predictions)
38:
39:      // === Visualisasi confusion matrix ===
40:      PLOT_CONFUSION_MATRIX(cm, labels = ["Cina", "Jepang"], title = "Confusion
Matrix - Hybrid Voting")
41:
42:      DISPLAY_REPORT(report)
43:  END

```

Kode Semu 3.7 Klasifikasi *Hybrid Voting* pada Data Uji

b) Implementasi Klasifikasi *Hybrid Voting* Gambar Utuh

Kode Semu 3.8 mendeskripsikan alur klasifikasi gambar menggunakan pendekatan *hybrid voting* berbasis *patch*, dengan penerapan *majority voting* untuk menentukan label akhir setiap gambar. Proses diawali dengan deteksi *patch* dan ekstraksi fitur. Penjelasan lengkap mengenai pengimplementasian deteksi *patch* dan ekstraksi fitur telah dijabarkan pada Subbab 3.3.2 **Implementasi Deteksi Patch** dan 3.3.3 **Implementasi Ekstraksi Fitur**. Perbedaan pada pengimplementasian hanya terdapat di parameter deteksi *patch* yaitu jumlah *stride*. Prediksi dilakukan dengan menggabungkan dua sumber informasi yaitu skor probabilitas dari model SVM dan skor kedekatan *patch* terhadap *centroid* kluster Jepang maupun Cina.

```

Input:
- Model SVM terlatih (svm_model)
- Model PCA (pca_model)
- Model feature extractor (CNN ResNet50 dengan GAP)
- Centroid hasil clustering
- Folder gambar uji (dataset_positif, dataset_negatif)

Output:
- Hasil prediksi klasifikasi per gambar
- Confusion matrix dan classification report

1:  BEGIN
2:      // === Inisialisasi Parameter ===
3:      PATCH_SIZE ← 224
4:      STRIDE ← 112
5:      COLOR_STD_THRESHOLD ← 20
6:      EDGE_THRESHOLD ← 1200

```

```

7:   PADDING_BLACK_RATIO_THRESHOLD ← 0.2
8:   ALPHA ← 0.4, BETA ← 0.6
9:   JEPANG_CLUSTERS ← [0, 2, 3]
10:  CINA_CLUSTERS ← [1, 4]

11:  // === Load Model dan Komponen ===
12:  svm_model ← LOAD
13:  pca_model ← LOAD
14:  centroids ← LOAD
15:  feature_extractor ← RESNET50_WITH_GAP(pretrained = "imagenet")

16:  FUNCTION is_valid_patch(patch):
17:    gray ← CONVERT_TO_GRAYSCALE(patch)
18:    black_ratio ← COUNT_PIXELS(gray < 10) / TOTAL_PIXELS(gray)
19:    IF black_ratio > PADDING_BLACK_RATIO_THRESHOLD THEN RETURN False
20:    std ← CALCULATE_STD(gray)
21:    IF std < COLOR_STD_THRESHOLD THEN RETURN False
22:    edge ← COUNT_EDGES(gray, method = Canny)
23:    IF edge < EDGE_THRESHOLD THEN RETURN False
24:    RETURN True

25:  FUNCTION extract_valid_patches(image):
26:    patches ← []
27:    FOR y FROM 0 TO image.height - PATCH_SIZE STEP STRIDE:
28:      FOR x FROM 0 TO image.width - PATCH_SIZE STEP STRIDE:
29:        patch ← CROP(image, x, y, PATCH_SIZE)
30:        IF is_valid_patch(patch) THEN
31:          ADD patch TO patches
32:    RETURN patches

33:  FUNCTION classify_patch_hybrid(patch):
34:    resized ← RESIZE(patch, (224, 224))
35:    preprocessed ← PREPROCESS_FOR_RESNET(resized)
36:    feat ← EXTRACT_FEATURE(feature_extractor, preprocessed)
37:    feat_pca ← TRANSFORM_PCA(pca_model, feat)
38:    feat_norm ← NORMALIZE(feat_pca)
39:    proba ← SVM_PREDICT_PROBA(svm_model, feat_norm)
40:    dists ← COSINE_DISTANCE(feat_norm, centroids)
41:    japan_score ← MEAN([1 / (dists[c] + ε) FOR c IN JEPANG_CLUSTERS])
42:    cina_score ← MEAN([1 / (dists[c] + ε) FOR c IN CINA_CLUSTERS])
43:    hybrid_score ← ALPHA × (japan_score / (cina_score + ε)) + BETA ×
proba[1]
44:    RETURN 1 IF hybrid_score > 0.5 ELSE 0

45:  FUNCTION classify_image_majority(path):
46:    image ← READ_IMAGE(path)
47:    IF image IS None THEN RETURN 0
48:    rgb ← CONVERT_TO_RGB(image)
49:    patches ← extract_valid_patches(rgb)
50:    IF patches IS EMPTY THEN RETURN 0
51:    preds ← [classify_patch_hybrid(p) FOR p IN patches]
52:    RETURN MAJORITY_VOTE(preds)

53:  FUNCTION evaluate_random(folder_path, true_label, sample_size):
54:    files ← LIST_IMAGES(folder_path)
55:    selected ← RANDOM_SAMPLE(files, sample_size)
56:    labels ← [], preds ← []
57:    FOR file IN selected:
58:      pred ← classify_image_majority(JOIN_PATH(folder_path, file))
59:      APPEND pred TO preds

```

```

60:         APPEND true_label TO labels
61:     RETURN labels, preds

62:     // === Evaluasi 100 gambar acak dari dua kelas ===
63:     true_pos, pred_pos ← evaluate_random("dataset_positif", 1, 100)
64:     true_neg, pred_neg ← evaluate_random("dataset_negatif", 0, 100)
65:     all_true ← CONCATENATE(true_pos, true_neg)
66:     all_pred ← CONCATENATE(pred_pos, pred_neg)

67:     // === Evaluasi performa ===
68:     report ← GENERATE_CLASSIFICATION_REPORT(all_true, all_pred, labels =
["Cina", "Jepang"])
69:     cm ← GENERATE_CONFUSION_MATRIX(all_true, all_pred)
70:     DISPLAY_HEATMAP(cm, labels = ["Cina", "Jepang"], title = "Hybrid Majority
Voting ( $\alpha=0.4$ ,  $\beta=0.6$ )")
71:     DISPLAY_REPORT(report)
72: END

```

Kode Semu 3.8 Klasifikasi *Hybrid Voting* pada Gambar Utuh

Kedekatan dihitung berdasarkan jarak *cosine* terhadap masing-masing *centroid*, lalu dikonversi ke skor menggunakan rumus kebalikan jarak (*reciprocal distance*). *Hybrid score* ditentukan sebagai kombinasi linier dari skor rasio kedekatan dan probabilitas prediksi SVM, dengan bobot α dan β yang telah ditetapkan sebelumnya. *Patch* diklasifikasikan sebagai lukisan Jepang apabila nilai *hybrid score* melebihi ambang batas 0.5. Prediksi akhir untuk setiap gambar ditentukan menggunakan fungsi `classify_image_majority`, yang menghitung jumlah label terbanyak dari seluruh hasil prediksi *patch*. Dengan demikian, *majority voting* digunakan untuk menggabungkan informasi lokal menjadi keputusan tingkat gambar.

c) Implementasi Klasifikasi SVM Data Uji

Kode Semu 3.9 menjelaskan proses evaluasi performa model klasifikasi *Support Vector Machine* (SVM) terhadap data uji, dengan menggunakan label kebenaran (*ground truth*) sebagai acuan. Proses evaluasi dimulai dengan memuat model klasifikasi SVM melalui fungsi `LOAD_MODEL`, serta memuat data uji menggunakan `LOAD_DATA`. Data uji berisi fitur dan label dari *patch* citra uji yang sebelumnya tidak digunakan dalam proses pelatihan. Setelah data berhasil dimuat, fitur (X_{test}) dan label aktual (y_{test}) dipisahkan untuk keperluan evaluasi. Model SVM yang telah dilatih kemudian digunakan untuk memprediksi label dari seluruh data uji menggunakan fungsi `PREDICT_LABELS`. Hasil prediksi dikembalikan dalam bentuk label biner, yaitu 0 untuk lukisan Cina dan 1 untuk lukisan Jepang. Prediksi tersebut selanjutnya dibandingkan dengan label asli (y_{test}) untuk menghitung metrik performa model. Evaluasi dilakukan menggunakan fungsi `GENERATE_CLASSIFICATION_REPORT`, yang mencakup pengukuran akurasi, *precision*, *recall*, dan *F1-score* untuk masing-masing kelas. Selain laporan klasifikasi, evaluasi juga dibuat dengan *confusion matrix* menggunakan fungsi `GENERATE_CONFUSION_MATRIX`, yang memberikan gambaran tentang jumlah prediksi benar dan salah dari masing-masing kelas.

```

Input:
- Model SVM terlatih (svm_model)
- Model PCA (pca_model)
- Model feature extractor (CNN ResNet50 dengan GAP)
- Centroid hasil clustering
- Folder gambar uji (dataset_positif, dataset_negatif)

Output:
- Hasil prediksi klasifikasi per gambar

```

```

- Confusion matrix dan classification report

FUNCTION EvaluateSVMModel():
  # Muat model SVM dan data uji
  svm_model ← LOAD_MODEL
  test_data ← LOAD_DATA

  X_test ← test_data["X_test"]
  y_test ← test_data["y_test"]

  # Lakukan prediksi menggunakan model SVM
  y_pred ← PREDICT_LABELS(svm_model, X_test)

  # Evaluasi performa model
  report ← GENERATE_CLASSIFICATION_REPORT(y_test, y_pred, target_names=["Cina",
"Jepang"])
  cm ← GENERATE_CONFUSION_MATRIX(y_test, y_pred)

  # Visualisasi confusion matrix
  PLOT_CONFUSION_MATRIX(cm, labels=["Cina", "Jepang"], title="Confusion Matrix -
SVM Only")

  # Tampilkan classification report
  DISPLAY_REPORT(report)

RETURN

```

Kode Semu 3.9 Klasifikasi SVM pada Data Uji

d) Implementasi Klasifikasi SVM Gambar Utuh

Kode Semu 3.10 menjelaskan tahapan klasifikasi gambar menggunakan pendekatan *patch-based majority voting* berbasis model *Support Vector Machine* (SVM). Proses diawali dengan deteksi *patch* dan ekstraksi fitur. Penjelasan lengkap mengenai pengimplementasian deteksi *patch* dan ekstraksi fitur telah dijabarkan pada Subbab **3.3.2 Implementasi Deteksi Patch** dan **3.3.3 Implementasi Ekstraksi Fitur**. Perbedaan pada pengimplementasian hanya terdapat di parameter deteksi *patch* yaitu jumlah *stride*. Parameter *stride* pada implementasi klasifikasi bernilai lebih kecil daripada ukurannya agar membuat *patch* menjadi *overlap*. Fitur yang diperoleh dan telah direduksi dalam proses ekstraksi fitur selanjutnya diprediksi menggunakan *SVM* untuk menghasilkan label klasifikasi (Jepang atau Cina). Seluruh *patch* dari satu gambar diproses oleh fungsi `classify_image_majority_vote`, yaitu gabungan dari hasil `extract_valid_patches` dan `classify_patch_svm`. Label prediksi dari seluruh *patch* valid pada gambar diakumulasi dan label yang menjadi mayoritas ditentukan sebagai prediksi akhir untuk gambar tersebut. Evaluasi terhadap performa klasifikasi dilakukan menggunakan fungsi `evaluate_folder`, yang menguji sekelompok gambar dari dua kelas yaitu lukisan Jepang (label 1) dan lukisan Cina (label 0). Fungsi ini membaca gambar dari folder masing-masing kelas, menjalankan proses klasifikasi berbasis *voting* pada setiap gambar, dan mencatat hasil prediksi serta label aslinya. Seluruh hasil prediksi dan label digabungkan dalam variabel `all_true` dan `all_pred`, yang kemudian digunakan untuk menghitung *classification report* dan *confusion matrix*.

```

Input:
- Dataset gambar (folder positif dan negatif)
- Model CNN (ResNet50) sebagai ekstraktor fitur
- Model PCA dan SVM terlatih
- Parameter patching: ukuran patch, stride, ambang batas validasi patch

Output:
- Prediksi label per gambar

```

```

- Classification report dan confusion matrix

1: BEGIN
2:     // === Inisialisasi Parameter ===
3:     PATCH_SIZE ← 224
4:     STRIDE ← 112
5:     COLOR_STD_THRESHOLD ← 20
6:     EDGE_THRESHOLD ← 1200
7:     PADDING_BLACK_RATIO_THRESHOLD ← 0.2

8:     // === Load Model ===
9:     svm_model ← LOAD_MODEL
10:    pca_model ← LOAD_MODEL
11:    feature_extractor ← INIT_RESNET50_WITH_GAP(weights="imagenet")

12:    FUNCTION is_valid_patch(patch):
13:        gray ← CONVERT_TO_GRAYSCALE(patch)
14:        black_ratio ← COUNT_PIXELS(gray < 10) / TOTAL_PIXELS(gray)
15:        IF black_ratio > PADDING_BLACK_RATIO_THRESHOLD THEN RETURN False
16:        std ← STANDARD_DEVIATION(gray)
17:        IF std < COLOR_STD_THRESHOLD THEN RETURN False
18:        edge ← COUNT_EDGES(gray, method=Canny)
19:        IF edge < EDGE_THRESHOLD THEN RETURN False
20:        RETURN True

21:    FUNCTION extract_valid_patches(image):
22:        patches ← []
23:        FOR y FROM 0 TO image.height - PATCH_SIZE STEP STRIDE:
24:            FOR x FROM 0 TO image.width - PATCH_SIZE STEP STRIDE:
25:                patch ← CROP(image, x, y, PATCH_SIZE)
26:                IF is_valid_patch(patch) THEN ADD patch TO patches
27:        RETURN patches

28:    FUNCTION classify_patch_svm(patch):
29:        resized ← RESIZE(patch, (224, 224))
30:        input_tensor ← PREPROCESS_FOR_RESNET(resized)
31:        features ← EXTRACT_FEATURE(feature_extractor, input_tensor)
32:        features_pca ← PCA_TRANSFORM(pca_model, features)
33:        prediction ← PREDICT_SVM(svm_model, features_pca)
34:        RETURN prediction

35:    FUNCTION classify_image_majority_vote(image_path):
36:        image ← READ_IMAGE(image_path)
37:        IF image IS None THEN RETURN 0
38:        rgb_image ← CONVERT_TO_RGB(image)
39:        patches ← extract_valid_patches(rgb_image)
40:        IF patches IS EMPTY THEN RETURN 0
41:        preds ← [classify_patch_svm(p) FOR p IN patches]
42:        RETURN MAJORITY_VOTE(preds)

43:    FUNCTION evaluate_folder(folder_path, true_label, limit):
44:        files ← LIST_IMAGE_FILES(folder_path)[:limit]
45:        labels ← [], preds ← []
46:        FOR file IN files:
47:            pred ← classify_image_majority_vote(JOIN_PATH(folder_path, file))
48:            APPEND true_label TO labels
49:            APPEND pred TO preds
50:        RETURN labels, preds

51:    // === Evaluasi 100 gambar dari masing-masing kelas ===
52:    true_pos, pred_pos ← evaluate_folder("dataset_positif", 1, 100)

```

```

53:   true_neg, pred_neg ← evaluate_folder("dataset_negatif", 0, 100)
54:   all_true ← CONCATENATE(true_pos, true_neg)
55:   all_pred ← CONCATENATE(pred_pos, pred_neg)

56:   // === Evaluasi dan Visualisasi ===
57:   report ← GENERATE_CLASSIFICATION_REPORT(all_true, all_pred, labels=["Cina",
"C Jepang"])
58:   cm ← GENERATE_CONFUSION_MATRIX(all_true, all_pred)
59:   PLOT_HEATMAP(cm, labels=["Cina", "Jepang"], title="Confusion Matrix - SVM
Patch Voting")
60:   DISPLAY_REPORT(report)
61: END

```

Kode Semu 3.10 Klasifikasi SVM dengan Gambar Utuh

e) Implementasi Klasifikasi Berbasis *Centroid Based Data Uji*

Kode Semu 3.11 menjelaskan proses klasifikasi *patch* pada data uji menggunakan pendekatan *unsupervised*, yaitu dengan membandingkan jarak fitur *patch* terhadap *centroid* hasil *clustering* tanpa melibatkan model klasifikasi. Tujuan dari pendekatan ini adalah untuk mengevaluasi sejauh mana kualitas kluster hasil *discriminative clustering* mampu mengelompokkan data uji secara langsung berdasarkan kedekatan spasial di ruang fitur. Proses diawali dengan mendefinisikan dua kelompok kluster penting yaitu kluster dominan Jepang, yang terdiri dari Kluster 0, 2, dan 3, serta kluster dominan Cina, yang diwakili oleh Kluster 1 dan 4.

```

Input:
- Fitur uji (X_test) dan label sebenarnya (y_test)
- Centroid hasil discriminative clustering
- Indeks cluster Jepang: [0, 2, 3]
- Indeks cluster Cina: [1, 4]

1: BEGIN
2:   // === Load data test dan centroid hasil clustering ===
3:   test_data ← LOAD_FILE
4:   clustering ← LOAD_FILE
5:
6:   X_test ← test_data["X_test"]
7:   y_test ← test_data["y_test"]
8:   centroids ← clustering["centroids"]
9:
10:  // === Definisikan cluster dominan ===
11:  JEPANG_CLUSTERS ← [0, 2, 3]
12:  CINA_CLUSTERS ← [1, 4]
13:
14:  FUNCTION predict_centroid_only(feet_norm):
15:    dists ← COSINE_DISTANCES(feet_norm, centroids)
16:    jepang_score ← MEAN([1 / (dists[c] + ε) FOR c IN JEPANG_CLUSTERS])
17:    cina_score ← MEAN([1 / (dists[c] + ε) FOR c IN CINA_CLUSTERS])
18:    RETURN 1 IF jepang_score > cina_score ELSE 0
19:
20:  // === Lakukan prediksi terhadap semua data uji ===
21:  predictions ← []
22:  FOR each feat IN X_test DO:
23:    feat_norm ← NORMALIZE_VECTOR(feet)
24:    pred ← predict_centroid_only(feet_norm)
25:    APPEND pred TO predictions
26:
27:  // === Evaluasi hasil prediksi ===

```

```

28:     report ← GENERATE_CLASSIFICATION_REPORT(y_test, predictions, labels =
["Cina", "Jepang"])
29:     cm ← GENERATE_CONFUSION_MATRIX(y_test, predictions)
30:
31:     // === Visualisasi Confusion Matrix ===
32:     PLOT_HEATMAP(matrix = cm, labels = ["Cina", "Jepang"], title = "Confusion
Matrix - Centroid-Based Voting Only")
33:     DISPLAY report
34: END

```

Kode Semu 3.11 Klasifikasi Data Uji Berbasis *Centroid*

Setiap fitur *patch* dalam *x_test* diproses melalui fungsi *predict_centroid_only*, yang menghitung jarak *cosine* antara vektor fitur yang telah dinormalisasi (*feat_norm*) dengan seluruh *centroid* hasil *clustering*. Perhitungan jarak dilakukan melalui langkah *COSINE_DISTANCES*. Setelah seluruh jarak diperoleh, sistem menghitung dua skor utama. Pertama yaitu *jepang_score*, yaitu rata-rata dari kebalikan jarak *cosine distance patch* terhadap seluruh *centroid* yang mewakili klaster Jepang. Kedua yaitu *cina_score*, yaitu rata-rata dari kebalikan jarak *cosine distance patch* terhadap seluruh *centroid* yang mewakili klaster Cina. Pembalikan jarak *cosine distance* dilakukan agar nilai yang besar merepresentasikan kemiripan yang tinggi. Keputusan klasifikasi dilakukan dengan membandingkan kedua skor tersebut yaitu jika *jepang_score* lebih besar dari *cina_score*, maka *patch* diklasifikasikan sebagai lukisan Jepang (label 1). Sebaliknya, jika *cina_score* lebih besar maka diklasifikasikan sebagai lukisan Cina (label 0). Proses klasifikasi ini diterapkan terhadap seluruh fitur *patch* dalam data uji melalui iterasi *FOR each feat IN x_test*. Hasil prediksi disimpan dalam daftar *predictions*, yang kemudian digunakan dalam tahap evaluasi performa model. Evaluasi dilakukan menggunakan fungsi *GENERATE_CLASSIFICATION_REPORT* dan *GENERATE_CONFUSION_MATRIX*, yang menghasilkan metrik seperti akurasi, *precision*, *recall*, dan *F1-score*, serta visualisasi *confusion matrix*.

f) Implementasi Klasifikasi Berbasis *Centroid* pada Gambar Utuh

Kode Semu 3.12 menjelaskan proses klasifikasi gambar menggunakan pendekatan *centroid-based voting*. Pendekatan ini sepenuhnya mengandalkan informasi jarak antara fitur *patch* dengan *centroid* hasil *clustering*. Proses diawali dengan deteksi *patch* dan ekstraksi fitur. Penjelasan lengkap mengenai pengimplementasian deteksi *patch* dan ekstraksi fitur telah dijabarkan pada Subbab 3.3.2 Implementasi Deteksi *Patch* dan 3.3.3 Implementasi Ekstraksi Fitur. Perbedaan pada pengimplementasian hanya terdapat di parameter deteksi *patch* yaitu jumlah *stride*. Parameter *stride* pada implementasi klasifikasi bernilai lebih kecil daripada ukurannya agar membuat *patch* menjadi *overlap*. Setelah fitur diperoleh dari hasil ekstraksi fitur dan PCA, dihitung jarak *cosine distance* antara vektor fitur *patch* dengan seluruh *centroid* hasil *clustering*. Fungsi *classify_patch_centroid* menghitung dua skor utama yaitu *Japan Score*, yaitu rata-rata dari kebalikan jarak *cosine distance* ke Klaster 0, 2, dan 3 serta *China Score*, yaitu rata-rata dari kebalikan jarak *cosine distance* ke Klaster 1 dan 4. Jika *Japan Score* lebih besar daripada *China Score*, maka *patch* diklasifikasikan sebagai lukisan Jepang (label 1). Sebaliknya, jika *China Score* lebih besar maka diklasifikasikan sebagai lukisan Cina (label 0). Proses klasifikasi *patch* ini diterapkan terhadap seluruh *patch* valid dalam satu gambar melalui fungsi *classify_image_centroid*. Selanjutnya, hasil prediksi dari semua *patch* dalam gambar digabungkan melalui mekanisme *majority voting*. Jika mayoritas *patch* diklasifikasikan sebagai Jepang, maka seluruh gambar dianggap sebagai lukisan Jepang. Seluruh gambar dalam dataset (baik kelas positif maupun negatif) dievaluasi melalui fungsi *evaluate_folder*, yang

memproses kumpulan gambar secara sistematis dan mencatat hasil prediksi beserta label kebenarannya. Setelah semua hasil diperoleh, sistem melakukan evaluasi menggunakan *classification report* dan *confusion matrix*.

```

Input:
- Dataset gambar (positif: Jepang, negatif: Cina)
- Model CNN (ResNet50) sebagai feature extractor
- Model PCA
- Centroid hasil clustering
- Parameter patching dan validasi: ukuran patch, stride, ambang edge dan warna

Output:
- Prediksi label gambar
- Classification report dan confusion matrix

1: BEGIN
2:     // === Parameter patch dan validasi ===
3:     PATCH_SIZE ← 224
4:     STRIDE ← 112
5:     COLOR_STD_THRESHOLD ← 20
6:     EDGE_THRESHOLD ← 1200
7:     PADDING_BLACK_RATIO_THRESHOLD ← 0.2
8:     JEPANG_CLUSTERS ← [0, 2, 3]
9:     CINA_CLUSTERS ← [1, 4]

10:    // === Load komponen model ===
11:    pca_model ← LOAD_MODEL
12:    centroids ← LOAD_CENTROIDS
13:    feature_extractor ← INIT_RESNET50_GAP()

14:    FUNCTION is_valid_patch(patch):
15:        gray ← CONVERT_TO_GRAYSCALE(patch)
16:        black_ratio ← COUNT_PIXELS(gray < 10) / TOTAL_PIXELS
17:        IF black_ratio > PADDING_BLACK_RATIO_THRESHOLD THEN RETURN False
18:        std ← STD_DEV(gray)
19:        IF std < COLOR_STD_THRESHOLD THEN RETURN False
20:        edge ← COUNT_EDGES(gray, method=Canny(50, 150))
21:        IF edge < EDGE_THRESHOLD THEN RETURN False
22:        RETURN True

23:    FUNCTION extract_valid_patches(image):
24:        patches ← []
25:        FOR y FROM 0 TO image.height - PATCH_SIZE STEP STRIDE:
26:            FOR x FROM 0 TO image.width - PATCH_SIZE STEP STRIDE:
27:                patch ← CROP(image, x, y, PATCH_SIZE)
28:                IF is_valid_patch(patch) THEN ADD patch TO patches
29:        RETURN patches

30:    FUNCTION classify_patch_centroid(patch):
31:        resized ← RESIZE(patch, (224, 224))
32:        input_tensor ← PREPROCESS_FOR_RESNET(resized)
33:        features ← feature_extractor.predict(input_tensor)
34:        features_pca ← PCA_TRANSFORM(pca_model, features)
35:        features_norm ← NORMALIZE(features_pca)
36:        dists ← COSINE_DISTANCE(features_norm, centroids)
37:        japan_score ← MEAN([1 / (dists[c] + ε) FOR c IN JEPANG_CLUSTERS])
38:        cina_score ← MEAN([1 / (dists[c] + ε) FOR c IN CINA_CLUSTERS])
39:        RETURN 1 IF japan_score > cina_score ELSE 0

```

```

40:     FUNCTION classify_image_centroid(image_path):
41:         image ← READ_IMAGE(image_path)
42:         IF image IS None THEN RETURN 0
43:         rgb ← CONVERT_TO_RGB(image)
44:         patches ← extract_valid_patches(rgb)
45:         IF patches IS EMPTY THEN RETURN 0
46:         preds ← [classify_patch_centroid(p) FOR p IN patches]
47:         RETURN MAJORITY_VOTE(preds)

48:     FUNCTION evaluate_folder(folder_path, true_label, limit):
49:         files ← LIST_FIRST_N_IMAGES(folder_path, limit)
50:         labels ← [], preds ← []
51:         FOR file IN files:
52:             pred ← classify_image_centroid(file)
53:             ADD true_label TO labels
54:             ADD pred TO preds
55:         RETURN labels, preds

56:     // === Evaluasi per kelas ===
57:     true_pos, pred_pos ← evaluate_folder("dataset_positif", 1, 100)
58:     true_neg, pred_neg ← evaluate_folder("dataset_negatif", 0, 100)
59:     all_true ← CONCATENATE(true_pos, true_neg)
60:     all_pred ← CONCATENATE(pred_pos, pred_neg)

61:     // === Evaluasi klasifikasi ===
62:     report ← GENERATE_CLASSIFICATION_REPORT(all_true, all_pred, labels=["Cina",
"C Jepang"])
63:     cm ← GENERATE_CONFUSION_MATRIX(all_true, all_pred)
64:     PLOT_HEATMAP(cm, labels=["Cina", "Jepang"], title="Confusion Matrix -
Centroid-Based Voting")
65:     DISPLAY_REPORT(report)
66: END

```

Kode Semu 3.12 Implementasi Klasifikasi Berbasis *Centroid* pada *Dataset* Gambar Asli

3.3.6 Implementasi Model Klasifikasi Menggunakan Gambar Utuh

Kode Semu 3.13 menjelaskan proses pelatihan dan evaluasi model klasifikasi gambar secara global menggunakan algoritma *Support Vector Machine* (SVM). Proses diawali dengan memuat data fitur dari *file* .pkl melalui fungsi `LOAD_FEATURES_FROM_PKL`, yang menghasilkan tiga komponen utama yaitu fitur hasil ekstraksi (`x_all`), label kelas gambar (`y_all`), dan nama *file* gambar (`image_names`). Untuk menghindari ketidakseimbangan kelas yang dapat menyebabkan bias dalam proses pelatihan, dilakukan penyeimbangan jumlah data antar kelas menggunakan fungsi `BALANCE_CLASSES_BY_MINIMUM`. Fungsi ini memilih jumlah data terkecil dari masing-masing kelas (lukisan Jepang dan lukisan Cina), kemudian memfilter dan menggabungkan ulang data menggunakan `STACK()` dan `CONCAT()` agar membentuk *dataset* yang seimbang. *Dataset* yang telah diseimbangkan kemudian dibagi menjadi dua bagian yaitu data latih dan data uji, menggunakan fungsi `SPLIT()` dengan proporsi 80:20. Pembagian dilakukan secara stratifikasi agar distribusi label tetap proporsional di kedua bagian. Informasi nama file gambar (`image_names`) juga ditulis untuk mempermudah saat evaluasi.

Model *SVM* diinisialisasi melalui fungsi `INITIALIZE_SVM()` dengan kernel *Radial Basis Function* (RBF). Nilai parameter penalti regulasi *C* ditetapkan sebesar 1.0, yang berfungsi untuk mengontrol keseimbangan antara margin maksimal dan tingkat kesalahan klasifikasi. Sementara itu, parameter *gamma* ditetapkan pada skema “*scale*”, yang secara otomatis menyesuaikan nilai *gamma* berdasarkan jumlah fitur dalam data. Selain itu, opsi probabilitistik bernilai *true* agar menghasilkan skor probabilitas. Proses pelatihan dilakukan menggunakan fungsi `TRAIN()`, dan model yang telah dilatih disimpan ke dalam direktori menggunakan

SAVE_MODEL(). Data uji juga disimpan melalui SAVE_TEST_DATA() agar dapat digunakan ulang pada proses evaluasi tanpa perlu dilakukan pembagian ulang. Tahapan evaluasi dimulai dengan memprediksi label data uji menggunakan fungsi PREDICT(). Hasil prediksi kemudian dibandingkan dengan label asli (y_test) menggunakan GENERATE_CLASSIFICATION_REPORT() dan GENERATE_CONFUSION_MATRIX().

```

Input:
- Fitur gambar utuh hasil ekstraksi ResNet50 + PCA (features)
- Label kelas (labels)
- Nama file gambar (image_names)
Output:
- Model SVM terlatih
- Hasil evaluasi model pada data uji
- Classification report dan confusion matrix

1: BEGIN
2:   LOAD data fitur dari file
3:   SET X_all ← data["features"]
4:   SET y_all ← data["labels"]
5:   SET image_names ← data["image_names"]
6:
7:   // Balancing kelas (jika perlu)
8:   label_0_idx ← INDEX_WHERE(y_all == 0)
9:   label_1_idx ← INDEX_WHERE(y_all == 1)
10:  min_len ← MIN(LENGTH(label_0_idx), LENGTH(label_1_idx))
11:
12:  X_balanced ← STACK(X_all[label_0_idx[:min_len]],
X_all[label_1_idx[:min_len]])
13:  y_balanced ← CONCAT(y_all[label_0_idx[:min_len]],
y_all[label_1_idx[:min_len]])
14:  names_balanced ← CONCAT(image_names[label_0_idx[:min_len]],
image_names[label_1_idx[:min_len]])
15:
16:  // Split data latih dan uji
17:  SPLIT X_balanced, y_balanced, names_balanced INTO:
X_train, X_test, y_train, y_test, names_train, names_test
WITH test_size = 0.2, stratify = y_balanced, random_state = 42
18:
19:  // Training SVM
20:  model ← INITIALIZE_SVM(C = 1.0, kernel = "rbf", gamma = "scale", probability
= True)
21:  TRAIN model ON X_train, y_train
22:
23:  // Simpan model dan test set
24:  CREATE_DIRECTORY("svm_models_fullimage")
25:  SAVE model TO "svm_models_fullimage/svm_fullimage_balanced.pkl"
26:  SAVE {"X_test": X_test, "y_test": y_test, "image_names": names_test}
TO "svm_models_fullimage/svm_testset_fullimage.pkl"
27:
28:  // Evaluasi performa
29:  y_pred ← PREDICT model ON X_test
30:  report ← GENERATE_CLASSIFICATION_REPORT(y_test, y_pred)
31:  cm ← GENERATE_CONFUSION_MATRIX(y_test, y_pred)
32:  DISPLAY report
33:  PLOT_CONFUSION_MATRIX(cm, labels = ["Cina", "Jepang"], title = "Confusion
Matrix - SVM Full Image")
34:  END

```

Kode Semu 3.13 Implementasi Pembuatan Model Klasifikasi Gambar Utuh

3.3.7 Implementasi Model Tanpa Filtrasi *Padding*

Kode Semu 3.14 menjelaskan proses pelatihan model klasifikasi menggunakan algoritma Support Vector Machine (SVM), berdasarkan fitur *patch* yang telah dikelompokkan melalui proses *discriminative clustering*, dengan peniadaan validasi *patch* yaitu filtrasi terhadap area *padding* hitam sebelumnya. Proses diawali dengan memuat *file* hasil clustering melalui fungsi `LOAD_FILE`, yang berisi tiga komponen utama yaitu label kluster (`cluster_labels`), vektor fitur berdimensi 1024 (`features_1024`), dan label kebenaran (`labels`) yang menunjukkan apakah *patch* berasal dari lukisan Jepang atau Cina. Setelah data dimuat, dilakukan seleksi terhadap *patch* yang akan digunakan sebagai data pelatihan. *Patch* yang dianggap representatif untuk kategori lukisan Jepang dipilih seluruhnya dari kluster 0, 2, dan 4, sedangkan kluster 1 dan 3 digunakan untuk mewakili kategori lukisan Cina. Untuk menjaga keseimbangan jumlah sampel antar kelas, dilakukan proses *resampling* terhadap kelas mayoritas tanpa pengulangan (`replace=False`). Sampel dari kedua kelas kemudian disatukan dan diberi label biner (1 untuk Jepang, 0 untuk Cina), lalu digabungkan menjadi satu *set* lengkap (`x_all` dan `y_all`). Distribusi label dicek ulang menggunakan fungsi `COUNT` untuk memastikan proporsi data yang seimbang sebelum pelatihan. *Dataset* gabungan selanjutnya dibagi menjadi dua *subset*, yaitu data latih dan data uji, menggunakan fungsi `TRAIN_TEST_SPLIT` dengan rasio 80:20 dan pembagian secara *stratified* agar proporsi label tetap seimbang.

Model SVM diinisialisasi menggunakan fungsi `SVC()` dengan kernel *Radial Basis Function* (RBF), serta parameter default yaitu nilai penalti `C` sebesar 1.0 dan `gamma` sebesar "scale". Proses pelatihan dilakukan dengan fungsi `fit()`, di mana model belajar dari data latih yang telah diseleksi dan diseimbangkan. Setelah pelatihan, performa model diuji menggunakan data uji. Prediksi label dilakukan melalui fungsi `predict()`, dan hasilnya dianalisis menggunakan fungsi `classification_report()` yang mencakup metrik evaluasi seperti akurasi, precision, recall, dan F1-score.

```
Input:
- Hasil clustering (cluster_labels, features_1024, labels)
- Seleksi cluster: Jepang ← cluster 2, 4, dan sebagian cluster 0; Cina ← cluster 1 dan 3

Output:
- Model SVM terlatih
- Classification report dan confusion matrix pada data uji

1: BEGIN
2:     // === Load hasil clustering ===
3:                                     data ←
LOAD_PICKLE("discriminative_clustering_results/discriminative_clustering_nopadding.pkl")
4:     clusters ← data["cluster_labels"]
5:     features ← data["features_1024"]
6:     labels ← data["labels"]

7:     // === Seleksi cluster dominan Jepang dan Cina ===
8:     jepang_idx ← FIND_INDICES(clusters = 2 OR clusters = 4 OR (clusters = 0 AND labels = 1))
9:     cina_idx ← FIND_INDICES(clusters = 1 OR clusters = 3)

10:    X_jepang ← features[jepang_idx]
11:    y_jepang ← CREATE_ARRAY(value = 1, length = LENGTH(X_jepang))
12:    X_cina ← features[cina_idx]
13:    y_cina ← CREATE_ARRAY(value = 0, length = LENGTH(X_cina))
```

```

14:    // === Balancing dataset Jepang dan Cina ===
15:    n_sample ← MIN(LENGTH(X_jepang), LENGTH(X_cina))
16:    X_jepang_bal ← RANDOM_RESAMPLE(X_jepang, n_sample)
17:    y_jepang_bal ← CREATE_ARRAY(value = 1, length = n_sample)
18:    X_cina_bal ← RANDOM_RESAMPLE(X_cina, n_sample)
19:    y_cina_bal ← CREATE_ARRAY(value = 0, length = n_sample)

20:    X_all ← CONCATENATE(X_jepang_bal, X_cina_bal)
21:    y_all ← CONCATENATE(y_jepang_bal, y_cina_bal)

22:    // === Split data menjadi train/test (80:20 stratified) ===
23:    (X_train, X_test, y_train, y_test) ← TRAIN_TEST_SPLIT(X_all, y_all,
test_size = 0.2, stratify = True)

24:    // === Inisialisasi dan pelatihan SVM ===
25:    svm_model ← INIT_SVM(C = 1.0, kernel = "rbf", gamma = "scale", probability
= True)
26:    TRAIN(svm_model, X_train, y_train)

27:    // === Evaluasi performa di data uji ===
28:    y_pred ← PREDICT(svm_model, X_test)
29:    report ← GENERATE_CLASSIFICATION_REPORT(y_test, y_pred)
30:    cm ← GENERATE_CONFUSION_MATRIX(y_test, y_pred)

31:    // === Simpan model dan dataset uji ===
32:    CREATE_FOLDER_IF_NOT_EXISTS("svm_models_filtered")
33:    SAVE_MODEL(svm_model,
"svm_models_filtered/svm_balanced_filtered_cluster.pkl")
34:    SAVE_TO_FILE({"X_test": X_test, "y_test": y_test},
"svm_models_filtered/svm_testset_filtered_cluster.pkl")

35:    RETURN report, cm
36: END

```

Kode Semu 3.14 Implementasi Pembuatan Model Tanpa Filtrasi *Padding*

a) Implementasi Klasifikasi Model Tanpa Filtrasi *Padding* pada *Dataset Uji*

Kode Semu 3.15 ini menjelaskan tahapan evaluasi model klasifikasi menggunakan pendekatan *Hybrid Voting*, yang menggabungkan dua jenis informasi prediksi yaitu probabilitas keluaran dari model *Support Vector Machine (SVM)* dan kedekatan fitur terhadap *centroid* hasil *clustering* visual. Evaluasi dimulai dengan memuat model *SVM* yang telah dilatih menggunakan *patch* hasil *filtering padding* melalui fungsi `LOAD_MODEL`, disertai dengan pemuatan data uji (`LOAD_DATA`), dan representasi *centroid* dari proses *clustering* sebelumnya. Dataset uji terdiri atas vektor fitur (`X_test`) dan label kebenaran (`y_test`), sementara *centroid* merepresentasikan pusat kluster dalam ruang fitur berdimensi 1024. Parameter utama untuk metode *Hybrid Voting* ditentukan terlebih dahulu, yaitu bobot α sebesar 0,4 dan β sebesar 0,6.

Selain itu, ditetapkan pula daftar kluster yang mewakili gaya lukisan Jepang ([0, 2, 4]) dan Cina ([1, 3]), yang ditentukan berdasarkan hasil analisis distribusi visual pada kluster sebelumnya. Fungsi utama dalam proses ini adalah `hybrid_predict_feature`, yang menerima satu vektor fitur ter-normalisasi dan mengembalikan prediksi label. Di dalamnya, pertama-tama dihitung probabilitas kelas Jepang dari model SVM menggunakan `PREDICT_PROBA`, kemudian dihitung jarak *cosine* antara fitur dan seluruh *centroid*. Skor kedekatan ke masing-masing lukisan dihitung sebagai rata-rata *reciprocal distance* ke kluster-kluster yang relevan (`japan_score` dan `cina_score`). Nilai akhir disebut sebagai *hybrid score*, yang dihitung melalui kombinasi linear antara dua komponen yaitu rasio skor kedekatan terhadap kluster

Jepang dan Cina, serta probabilitas dari model *SVM*. Bobot α dan β digunakan dalam formulasi tersebut untuk mengatur keseimbangan kontribusi. Jika *hybrid score* melebihi nilai ambang 0,5, maka fitur diklasifikasikan sebagai lukisan Jepang, jika tidak, maka diklasifikasikan sebagai lukisan Cina.

```

Input:
- Dataset gambar Jepang dan Cina
- Model CNN (ResNet50), PCA, dan SVM hasil training pada patch-filtered
- Centroid hasil clustering (setelah filtering padding)
- Parameter voting:  $\alpha = 0.44$ ,  $\beta = 0.56$ , cluster Jepang = [0, 2, 4]

Output:
- Prediksi label gambar
- Classification report dan confusion matrix

1: BEGIN
2:   // === Muat model dan data ===
3:   svm_model ← LOAD_MODEL
4:   test_data ← LOAD_DATA
5:   clustering ← LOAD_DATA

6:   X_test ← test_data["X_test"]
7:   y_test ← test_data["y_test"]
8:   centroids ← clustering["centroids"]

9:   // === Inisialisasi parameter hybrid voting ===
10:  ALPHA ← 0.4
11:  BETA ← 0.6
12:  JEPANG_CLUSTERS ← [0, 2, 4]
13:  CINA_CLUSTERS ← [1, 3]

14:  FUNCTION hybrid_predict_feature(feet_norm):
15:    proba ← PREDICT_PROBA(svm_model, feat_norm)
16:    dists ← CALCULATE_COSINE_DISTANCES(feet_norm, centroids)
17:
18:    japan_score ← MEAN([1 / (dists[c] +  $\epsilon$ ) FOR c IN JEPANG_CLUSTERS])
19:    cina_score ← MEAN([1 / (dists[c] +  $\epsilon$ ) FOR c IN CINA_CLUSTERS])
20:
21:    hybrid_score ← ALPHA × (japan_score / (cina_score +  $\epsilon$ )) + BETA ×
proba[1]
22:    RETURN 1 IF hybrid_score > 0.5 ELSE 0

23:  // === Lakukan prediksi hybrid untuk semua data uji ===
24:  predictions ← []
25:  FOR each feat IN X_test DO
26:    feat_norm ← NORMALIZE_VECTOR(feat)
27:    pred ← hybrid_predict_feature(feat_norm)
28:    APPEND pred TO predictions

29:  // === Evaluasi hasil prediksi ===
30:  report ← COMPUTE_CLASSIFICATION_REPORT(y_test, predictions)
31:  cm ← COMPUTE_CONFUSION_MATRIX(y_test, predictions)

32:  PLOT_CONFUSION_MATRIX(cm, labels=["Cina", "Jepang"], title="Confusion Matrix
- Hybrid Voting on Test Data")
33:  DISPLAY report

34:  END

```

Kode Semu 3.15 Implementasi Klasifikasi Model dengan Filtrasi Padding pada *Dataset Uji*

b) Implementasi Klasifikasi Model Tanpa Filtrasi *Padding* pada Gambar Utuh

Kode Semu 3.16 ini mendeskripsikan tahapan evaluasi model klasifikasi gambar menggunakan pendekatan *Hybrid Majority Voting*. Proses diawali dengan deteksi *patch* dan ekstraksi fitur. Penjelasan lengkap mengenai pengimplementasian deteksi *patch* dan ekstraksi fitur telah dijabarkan pada Subbab 3.3.2 Implementasi Deteksi *Patch* dan 3.3.3 Implementasi Ekstraksi Fitur. Perbedaan pada pengimplementasian hanya terdapat di parameter deteksi *patch* yaitu tidak adanya filtrasi *padding* hitam. Selanjutnya, dihitung probabilitas prediksi terhadap kelas Jepang menggunakan model SVM, serta dihitung skor kedekatan terhadap masing-masing kelompok *centroid* (Jepang dan Cina). Kedekatan dihitung berdasarkan rata-rata kebalikan jarak *cosine* ke seluruh *centroid* dari kelas terkait. Nilai akhir disebut sebagai *hybrid score*, yang dihitung melalui formula linier sebagai gabungan dari rasio skor kedekatan dan skor probabilitas SVM. Patch diklasifikasikan sebagai lukisan Jepang apabila *hybrid score* melebihi ambang 0,5. Prediksi kelas pada tingkat gambar dilakukan melalui fungsi `classify_image_majority`, yang memproses seluruh *patch* valid dalam sebuah gambar, lalu menerapkan voting mayoritas untuk menentukan label akhir gambar. Proses evaluasi dilakukan menggunakan fungsi `evaluate_random`, yang memilih sejumlah gambar secara acak dari dua direktori data (positif dan negatif). Gambar-gambar tersebut diklasifikasikan satu per satu menggunakan *hybrid majority voting*. Hasil klasifikasi dikompilasi dan dibandingkan dengan label kebenaran, kemudian dihitung metrik evaluasi seperti akurasi, *precision*, *recall*, dan *F1-score* menggunakan fungsi `CLASSIFICATION_REPORT` dan *confusion matrix* dibentuk dan divisualisasikan menggunakan `PLOT_CONFUSION_MATRIX`.

```
Input:
- Dataset gambar Jepang dan Cina
- Model CNN (ResNet50), PCA, dan SVM hasil training pada patch-filtered
- Centroid hasil clustering (setelah filtering padding)
- Parameter voting:  $\alpha = 0.4$ ,  $\beta = 0.6$ , cluster Jepang = [0, 2, 4]

Output:
- Prediksi label gambar
- Classification report dan confusion matrix

1: BEGIN
2:   LOAD svm_model FROM
3:   LOAD pca_model
4:   LOAD clustering["centroids"]

5:   DEFINE PATCH_SIZE  $\leftarrow$  224
6:   DEFINE STRIDE  $\leftarrow$  112
7:   DEFINE COLOR_STD_THRESHOLD  $\leftarrow$  20
8:   DEFINE EDGE_THRESHOLD  $\leftarrow$  1200
9:   DEFINE ALPHA  $\leftarrow$  0.4
10:  DEFINE BETA  $\leftarrow$  0.6
11:  DEFINE JEPANG_CLUSTERS  $\leftarrow$  [0, 2, 4]
12:  DEFINE CINA_CLUSTERS  $\leftarrow$  [1, 3]

13:  FUNCTION is_valid_patch(patch):
14:    gray  $\leftarrow$  CONVERT_TO_GRAYSCALE(patch)
15:    std  $\leftarrow$  CALCULATE_STD(gray)
16:    edge  $\leftarrow$  COUNT_EDGES(gray)
17:    RETURN std  $\geq$  COLOR_STD_THRESHOLD AND edge  $\geq$  EDGE_THRESHOLD

18:  FUNCTION extract_valid_patches(image):
```

```

19:     INIT patches ← []
20:     FOR y FROM 0 TO image.height - PATCH_SIZE STEP STRIDE DO
21:         FOR x FROM 0 TO image.width - PATCH_SIZE STEP STRIDE DO
22:             patch ← CROP(image, x, y, PATCH_SIZE)
23:             IF is_valid_patch(patch) THEN
24:                 APPEND patch TO patches
25:     RETURN patches

26: FUNCTION classify_patch_hybrid(patch):
27:     resized ← RESIZE(patch, PATCH_SIZE)
28:     input ← PREPROCESS_INPUT(resized)
29:     feature ← EXTRACT_FEATURE(feature_extractor, input)
30:     feature_pca ← PCA_TRANSFORM(pca_model, feature)
31:     feature_norm ← NORMALIZE(feature_pca)

32:     proba ← PREDICT_PROBA(svm_model, feature_norm)
33:     dists ← COSINE_DISTANCES(feature_norm, centroids)

34:     japan_score ← MEAN([1 / (dists[i] + ε) FOR i IN JEPANG_CLUSTERS])
35:     cina_score ← MEAN([1 / (dists[i] + ε) FOR i IN CINA_CLUSTERS])
36:     hybrid_score ← ALPHA × (japan_score / (cina_score + ε)) + BETA ×
proba[1]
37:     RETURN 1 IF hybrid_score > 0.5 ELSE 0

38: FUNCTION classify_image_majority(path):
39:     image ← READ_IMAGE(path)
40:     IF image IS None THEN RETURN 0
41:     image_rgb ← CONVERT_TO_RGB(image)
42:     patches ← extract_valid_patches(image_rgb)
43:     IF patches IS EMPTY THEN RETURN 0
44:     preds ← [classify_patch_hybrid(patch) FOR patch IN patches]
45:     RETURN MOST_COMMON(preds)

46: FUNCTION evaluate_random(folder_path, true_label, sample_size):
47:     INIT preds ← [], labels ← []
48:     files ← RANDOM_SAMPLE(folder_path, sample_size)
49:     FOR fname IN files DO
50:         pred ← classify_image_majority(folder_path + fname)
51:         APPEND pred TO preds
52:         APPEND true_label TO labels
53:     RETURN labels, preds

54: true_pos, pred_pos ← evaluate_random("dataset_positif", 1, 100)
55: true_neg, pred_neg ← evaluate_random("dataset_negatif", 0, 100)

56: all_true ← CONCAT(true_pos, true_neg)
57: all_pred ← CONCAT(pred_pos, pred_neg)

58: report ← COMPUTE_CLASSIFICATION_REPORT(all_true, all_pred)
59: cm ← COMPUTE_CONFUSION_MATRIX(all_true, all_pred)

60: DISPLAY_CONFUSION_MATRIX(cm, ["Cina", "Jepang"])
61: DISPLAY_REPORT(report)

62: END

```

Kode Semu 3.16 Implementasi Klasifikasi Model dengan Filtrasi *padding* pada *Dataset Gambar Utuh*

3.3.8 Implementasi GUI

Antarmuka aplikasi dalam penelitian ini dikembangkan menggunakan platform *Streamlit* agar dapat interaktif kepada pengguna dalam mengklasifikasikan gambar seni lukis tradisional Jepang dan Cina, baik pada level gambar utuh maupun level potongan (*patch*). Setelah pengguna mengunggah *file* gambar berformat .jpg atau .png, sistem akan menjalankan dua tahap klasifikasi. Hasil tampilan antarmuka dapat dilihat pada Gambar 3.13.

a

Prediksi Asal Gambar: Cina vs Jepang (Hybrid Voting - Patch Level)

Unggah gambar (.jpg/.png)

Drag and drop file here
Limit 200MB per file • JPG, PNG, JPEG

Browse files

MET-54141.jpg 387.7KB

b

Gambar Lengkap

Gambar Utuh

c

Analisis Patch-Level

Patch

Patch Posisi: (672, 448)
Prediksi: Jepang
SVM Prob: 0.99
Hybrid Score: 1.19
Centroid: 0 (Jepang)
Dist ke Centroid: [0.76 1.24 0.77 0.78 1.09]

Patch

Patch Posisi: (672, 560) **d**
Prediksi: Jepang **e**
SVM Prob: 0.83 **f**
Hybrid Score: 1.05 **g**
Centroid: 0 (Jepang) **h**
Dist ke Centroid: [0.74 1.28 0.76 0.76 0.88] **i**

Patch

Patch Posisi: (672, 672)
Prediksi: Jepang
SVM Prob: 0.60
Hybrid Score: 0.88
Centroid: 0 (Jepang)
Dist ke Centroid: [0.73 1.31 0.74 0.74 0.76]

Centroid Terdekat (Dominan): 0 (Jepang) **j**

Gambar 3.13 Hasil GUI

59

Pertama, klasifikasi terhadap gambar secara utuh menggunakan model *full image classification* berbasis SVM dan kedua, klasifikasi terhadap setiap *patch* valid menggunakan model *hybrid voting* dengan filtrasi *padding* hitam. Hasil klasifikasi pada gambar utuh langsung ditampilkan dalam bentuk prediksi kelas beserta probabilitas prediksinya. Model ini memanfaatkan kombinasi probabilitas dari klasifikasi SVM dan skor kedekatan terhadap *centroid cluster*, yang dihitung berdasarkan jarak *cosine* ke *centroid cluster* Jepang dan Cina. Untuk setiap *patch*, aplikasi menampilkan informasi prediksi kelas (Jepang atau Cina), probabilitas dari model SVM, skor *hybrid*, indeks *centroid* terdekat, serta daftar jarak ke semua *centroid*. Informasi ini divisualisasikan secara rinci di bagian "Analisis Patch-Level" guna memberikan analisis mendalam terhadap kontribusi masing-masing elemen visual. Di akhir proses, sistem juga melakukan rekapitulasi *centroid* yang paling dominan dari semua *patch* yang valid, yang direpresentasikan sebagai "Centroid Terdekat (Dominan)." Dengan melihat *centroid* yang paling sering muncul dari hasil klasifikasi, dapat diketahui sebuah gambar cenderung mirip dengan karakteristik lukisan klaster Jepang atau Cina.

Antarmuka pengguna (GUI) yang dikembangkan ditampilkan pada Gambar 3.13 dan terdiri atas beberapa komponen yang mendukung proses klasifikasi dan interpretasi hasil secara visual. Pada bagian (a), terdapat komponen unggahan gambar, di mana pengguna dapat mengunggah citra yang akan diklasifikasikan. Komponen ini mendukung format .jpg dan .png dengan batasan ukuran maksimal 200MB, serta menampilkan nama file setelah berhasil diunggah. Selanjutnya, pada bagian (b) ditampilkan gambar utuh dari citra yang diunggah sebagai representasi visual keseluruhan sebelum dilakukan pemrosesan *patch*. Bagian ini juga disertai label prediksi akhir berdasarkan hasil agregasi *hybrid voting* terhadap seluruh *patch* valid yang berhasil diekstraksi dari gambar tersebut. Analisis lanjutan ditampilkan pada bagian (c), yaitu "Analisis Patch-Level", yang menyajikan informasi hasil klasifikasi untuk setiap *patch* yang lolos validasi. Setiap *patch* divisualisasikan bersamaan dengan (d) posisi koordinat *patch* dalam gambar, (e) hasil prediksi kelas (Jepang atau Cina), (f) nilai probabilitas klasifikasi dari model SVM, (g) nilai *hybrid score* yang merupakan hasil kombinasi skor SVM dan kedekatan ke *centroid*, serta (h) indeks *centroid* dominan beserta (i) daftar jarak *patch* ke seluruh *centroid* dari hasil semua *clustering*. Terakhir, pada bagian (j), sistem menampilkan *centroid* yang paling dominan atau terdekat dari semua *patch*, yang menjadi dasar penentuan label akhir pada level gambar. Dengan struktur antarmuka seperti ini, pengguna dapat menelusuri hasil prediksi secara detail dari level gambar hingga ke level *patch*, serta memahami kontribusi setiap bagian gambar terhadap keputusan klasifikasi yang dihasilkan oleh mo

BAB 4 Hasil dan Pembahasan

Bab ini menjelaskan mengenai hasil uji coba dari perancangan sistem yang telah dibuat. Uji coba dilakukan untuk mengetahui kinerja model dengan lingkungan uji coba yang telah ditentukan.

4.1 Hasil penelitian

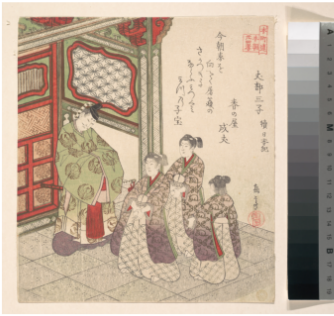
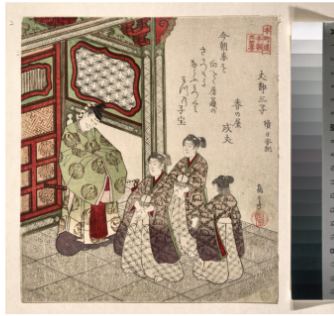
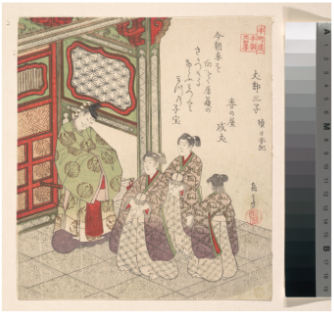
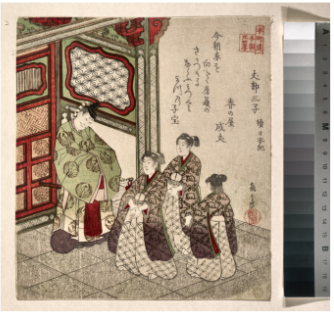
Bab ini berisikan pemaparan hasil dan pembahasan dari setiap langkah implementasi yang telah dilakukan pada Tugas Akhir ini. Hasil menampilkan luaran dari praproses data, deteksi *patch*, ekstraksi fitur, *clustering* dan hasil implementasi model *hybrid voting* yang dilakukan serta metode pembandingan dari model tersebut. Pembahasan berisi analisa lebih lanjut terkait keseluruhan hasil uji coba rancangan sistem yang telah dibuat.

4.1.1 Praproses Data

Tahap awal dalam penelitian ini adalah melakukan praproses terhadap citra pada *dataset*, baik dari lukisan Jepang maupun lukisan Cina. Proses praproses meliputi beberapa tahap, yaitu peningkatan kontras, penyesuaian ukuran (*resizing*) dan penambahan *padding*, normalisasi nilai piksel, serta penyimpanan kembali citra hasil praproses. Pertama, gambar yang dimuat ditingkatkan kontrasnya menggunakan metode *Contrast Limited Adaptive Histogram Equalization (CLAHE)* pada kanal L dari ruang warna LAB. Untuk mendapatkan parameter *CLAHE* terbaik, dilakukan beberapa percobaan dengan variasi nilai parameter. Tabel 4.1 menunjukkan hasil pengaplikasian *CLAHE* sebelum dan sesudah perubahan parameter *clipLimit*, yang merupakan batas maksimum peningkatan kontras. Nilai *clipLimit* yang terlalu tinggi dapat menyebabkan peningkatan kontras yang berlebihan dan menimbulkan *noise*. Sebaliknya, nilai yang terlalu rendah dapat menyebabkan objek penting dalam gambar tidak terlihat jelas saat tahap ekstraksi fitur. Parameter lainnya adalah *tileGridSize*, yaitu ukuran pembagian citra menjadi blok-blok kecil (*tiles*) untuk penyesuaian kontras secara lokal. Ukuran *tile* yang lebih kecil menghasilkan kontras lokal yang lebih tajam dan detail, sedangkan ukuran yang lebih besar memberikan efek kontras yang lebih menyeluruh dan halus.

Tabel 4.1 Eksperimen Parameter di CLAHE pada *Dataset* Positif

clipLimit	tileGridSize	Hasil Seni Jepang
1.5	(8, 8)	MET-54361.jpg - Before vs After CLAHE Original Enhanced (CLAHE)

1.8	(8, 8)	MET-36977.jpg - Before vs After CLAHE Original  Enhanced (CLAHE) 
2.0	(8, 8)	MET-36977.jpg - Before vs After CLAHE Original  Enhanced (CLAHE) 

Tabel 4.1 dan Tabel 4.2 menunjukkan hasil visual dari penerapan metode *CLAHE* pada lukisan Jepang dan Cina, masing-masing, dengan variasi parameter *clipLimit* sebesar 1,5; 1,8; dan 2,0. Berdasarkan pengamatan, pada nilai *clipLimit* sebesar 1,5, peningkatan kontras belum cukup menajamkan detail, terutama pada area yang memiliki tekstur atau pola kompleks. Sementara itu, nilai *clipLimit* sebesar 2,0 menghasilkan kontras yang sangat tinggi, tetapi pada beberapa lukisan Cina terlihat berlebihan dan menimbulkan warna yang terlalu mencolok, sehingga membuat *noise*. Pada konfigurasi *clipLimit* sebesar 1,8, peningkatan kontras dinilai paling seimbang karena gambar tetap mempertahankan warna awal dan fitur penting seperti goresan kuas serta pola ornamen menjadi lebih jelas tanpa mengubah karakteristik visual. Oleh karena itu, nilai *clipLimit* sebesar 1,8 dan *tileGridSize* sebesar (8, 8) ditetapkan menjadi parameter kontras yang digunakan.

Tabel 4.2 Eksperimen Parameter di CLAHE pada *Dataset* Negatif

clipLimit	tileGridSize	Hasil Seni Cina
1.5	(8, 8)	gong_bi_hua_160.jpg - Before vs After CLAHE Original  Enhanced (CLAHE) 

1.8	(8, 8)	gong_bi_hua_160.jpg - Before vs After CLAHE	
		Original	Enhanced (CLAHE)
			
2.0	(8, 8)	gong_bi_hua_160.jpg - Before vs After CLAHE	
		Original	Enhanced (CLAHE)
			

Setelah kontras ditingkatkan, gambar kemudian disesuaikan ukurannya agar sesuai dengan kebutuhan tahap analisis berikutnya. Penyesuaian ukuran dilakukan dengan dua pendekatan: jika dimensi gambar melebihi ukuran maksimum 896 piksel, maka gambar akan dikecilkan secara proporsional menggunakan metode interpolasi *cv2.INTER_AREA*. Sebaliknya, jika dimensi gambar lebih kecil dari 896 piksel, ukurannya akan disesuaikan ke kelipatan 224 terdekat. Selanjutnya, dilakukan proses penambahan *padding* agar dimensi akhir gambar menjadi kelipatan dari 224 piksel. *Padding* ditambahkan secara proporsional pada sisi kiri-kanan dan atas-bawah, sehingga objek utama tetap berada di tengah. Tabel 4.3 menunjukkan beberapa contoh gambar seni Jepang sebelum dan sesudah menjalani proses *resizing* dan *padding*.

Tabel 4.3 Hasil *Resize* pada Dataset Positif

Nama Gambar	Ukuran Sebelum (wxh)	Ukuran Sesudah (wxh)
MET-48989.jpg	1329 × 3841	448 × 896
image_3.jpg	1032 × 1024	896 × 896
MET-55714.jpg	2630 × 3863	672 × 896
MET-55865.jpg	3694 × 1888	896 × 672
image_940.jpg	1032 × 1024	896 × 896

Pada Tabel 4.4 menunjukkan beberapa contoh hasil dataset negatif (lukisan Cina) sebelum dan sesudah dilakukan proses *resizing*. Perbedaan praproses antara dataset lukisan Jepang dan Cina terletak pada strategi penyesuaian ukuran yang disesuaikan dengan kebutuhan dari masing-masing dataset. Pada dataset lukisan Jepang, ukuran gambar sangat beragam, dengan beberapa di antaranya mencapai lebih dari 4000 piksel. Oleh karena itu, gambar yang lebih kecil dari batas maksimum (896 piksel) tidak diperbesar secara proporsional, melainkan hanya disesuaikan ke kelipatan 224 terdekat. Tujuannya adalah untuk mempertahankan skala asli dan menghindari peningkatan resolusi yang dapat menyebabkan hilangnya detail visual penting.

Sebaliknya, gambar dalam dataset lukisan Cina memiliki ukuran yang relatif kecil dan seragam, yaitu 256 piksel. Seluruh gambar dalam dataset ini diperbesar secara proporsional menggunakan interpolasi *bicubic* hingga mencapai ukuran 896 piksel pada sisi terpanjang sebelum proses *padding*. Strategi ini diterapkan untuk menghindari ketimpangan jumlah *patch* antara dua kelas setelah tahap deteksi *patch*. Jika tidak diseimbangkan, perbedaan ukuran asli antar gambar dapat menyebabkan jumlah *patch* yang sangat tidak merata antara kelas Jepang dan Cina.

Tabel 4.4 Hasil *Resize* pada Dataset Negatif

Nama Gambar	Ukuran Sebelum (wxh)	Ukuran Sesudah (wxh)
shui mo hua 451.jpg	256 × 256	896 × 896
guo hua 254.jpg	256 × 256	896 × 896
wgy shui mo hua 337.jpg	256 × 256	896 × 896
wgy guo hua shan 401.jpg	256 × 256	896 × 896
guo hua 466.jpg	256 × 256	896 × 896

Setelah penyesuaian ukuran dan penambahan *padding*, gambar kemudian dinormalisasi ke dalam rentang nilai $[0, 1]$ dengan cara membagi seluruh nilai piksel dengan 255. Proses normalisasi ini dilakukan untuk menyesuaikan format input yang dibutuhkan oleh model *Convolutional Neural Network (CNN)*. Hasil akhir dari tahap praproses disimpan dalam folder *processed_dataset_positif*, dan digunakan sebagai masukan untuk tahap deteksi *patch*. Gambar 4.1 memperlihatkan contoh ukuran gambar terkecil dan terbesar dari dataset seni lukis Jepang setelah melalui seluruh tahapan praproses. Ukuran terkecil ditemukan pada gambar dengan orientasi vertikal ramping, yaitu 224×896 piksel, sedangkan ukuran terbesar adalah 896×896 piksel pada gambar berbentuk persegi.

Terkecil: 224x896



Terbesar: 896x896



Gambar 4.1 Gambar Terkecil dan Terbesar Pada *Dataset Positif*

Tabel 4.5 dan Tabel 4.6 menunjukkan perbandingan beberapa gambar di dalam dataset seni Jepang dan seni Cina sebelum dan sesudah dilakukan praproses. Terlihat bahwa ukuran awal gambar sangat bervariasi, dengan dimensi mencapai hingga 4000 piksel pada sisi panjang. Setelah dilakukan praproses, seluruh citra berhasil disesuaikan menjadi ukuran yang lebih seragam dengan tinggi tetap pada 896 piksel, dan lebar bergantung pada aspek rasio awal (tetap kelipatan 224). Selain itu, peningkatan kontras juga tampak pada hasil akhir yang terlihat lebih tajam dan terang secara visual.

Tabel 4.5 Visualisasi Gambar Setelah Praproses pada *Dataset* Positif

Visualisasi sebelum dan sesudah praproses data positif	
Sebelum: 2752x3811 	Sesudah: 672x896 
Sebelum: 2809x4000 	Sesudah: 672x896 
Sebelum: 2036x3908 	Sesudah: 672x896 

Tabel 4.6 Visualisasi Gambar Setelah Praproses pada *Dataset* Negatif

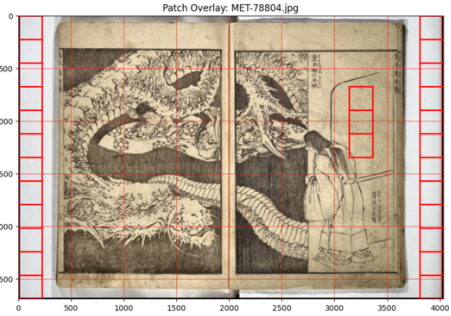
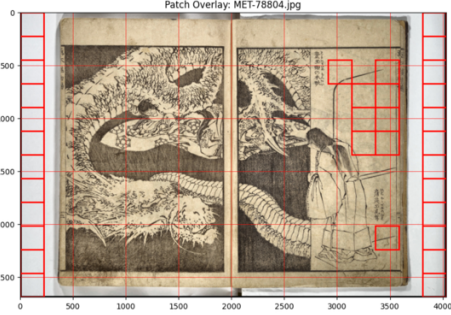
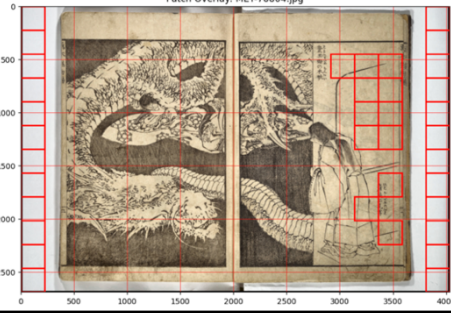
Visualisasi sebelum dan sesudah praproses data negatif	
Sebelum: 256x256 	Sesudah: 896x896 
Sebelum: 256x256 	Sesudah: 896x896 
Sebelum: 256x256 	Sesudah: 896x896 

4.1.2 Deteksi *Patch*

Setelah tahap praproses selesai dilakukan, langkah selanjutnya adalah deteksi *patch*. Deteksi *patch* dilakukan menggunakan pendekatan *sliding window* dengan ukuran tetap, yaitu 224×224 piksel. Proses ekstraksi dilakukan dengan *stride* sebesar 224 piksel, sehingga tidak terjadi *overlap* antar *patch*. Pendekatan ini dipilih agar menjaga efisiensi komputasi dari proses deteksi *patch*. Hasil dari pemotongan *patch* tidak secara otomatis digunakan. Untuk menyaring *patch* yang mengandung detail informasi yang cukup, diterapkan proses validasi menggunakan tiga metrik yaitu *color variation* (variasi warna), *edge count* (jumlah tepi) dan *padding black*

ratio (jumlah rasio padding). Hanya *patch* yang memiliki nilai minimum dari ketiga metrik tersebut yang akan diteruskan ke tahap selanjutnya.

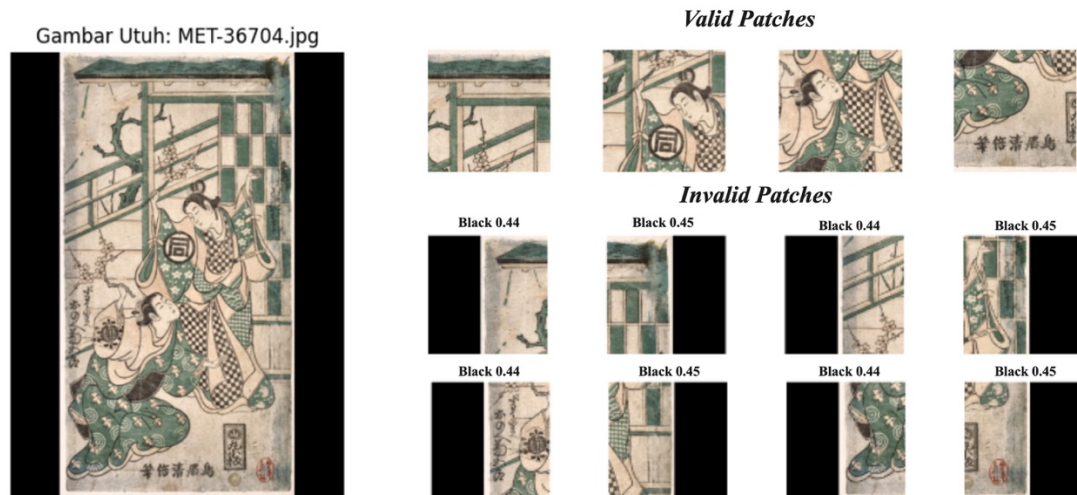
Tabel 4.7 Hasil Eksperimen Parameter Deteksi *Patch*

Eksperimen Dua Parameter Dalam Deteksi <i>Patch</i>		
COLOR_VARIATION_THRESHOLD	EDGE_THRESHOLD	Hasil
20	1200	
25	1200	
28	1200	

Untuk menentukan *threshold* parameter validasi yang paling sesuai, dilakukan eksperimen terhadap beberapa nilai *COLOR_VARIATION_THRESHOLD* dengan nilai *EDGE_THRESHOLD* tetap sebesar 1200. Seperti ditunjukkan pada Tabel 4.7, percobaan dilakukan dengan tiga nilai *threshold* dari variasi intensitas piksel, yaitu 20, 25, dan 28. Berdasarkan visualisasi hasil 1 gambar, terlihat bahwa pada nilai *threshold* 20, sistem masih mempertahankan banyak *patch* dengan tingkat variasi intensitas piksel sedang hingga tinggi, mencakup berbagai detail penting dalam citra. Pada nilai *threshold* 25, jumlah *patch* yang dipertahankan mulai menurun, dengan kecenderungan hanya mempertahankan area dengan kontras yang tinggi. Sedangkan pada nilai *threshold* 28, terlalu banyak *patch* yang dieliminasi, termasuk beberapa area yang secara visual masih mengandung informasi yang cukup. Oleh karena itu, nilai *threshold* sebesar 20 dipilih sebagai parameter yang cukup baik dalam menjaga keseimbangan antara jumlah *patch* dan kualitas informasi visual. Pemilihan nilai *EDGE_THRESHOLD* sebesar 1200 juga didasarkan pada evaluasi awal terhadap distribusi jumlah *edge* dalam berbagai *patch*. Nilai ini cukup untuk menyaring area datar seperti latar belakang atau bagian putih polos, namun tidak terlalu tinggi hingga menyebabkan *patch* yang mengandung informasi penting ikut tersaring. Kombinasi nilai

COLOR_VARIATION_THRESHOLD sebesar 20 dan *EDGE_THRESHOLD* sebesar 1200 dipilih sebagai parameter validasi akhir karena memberikan hasil paling optimal dalam mempertahankan detail penting sekaligus menyaring area yang tidak informatif.

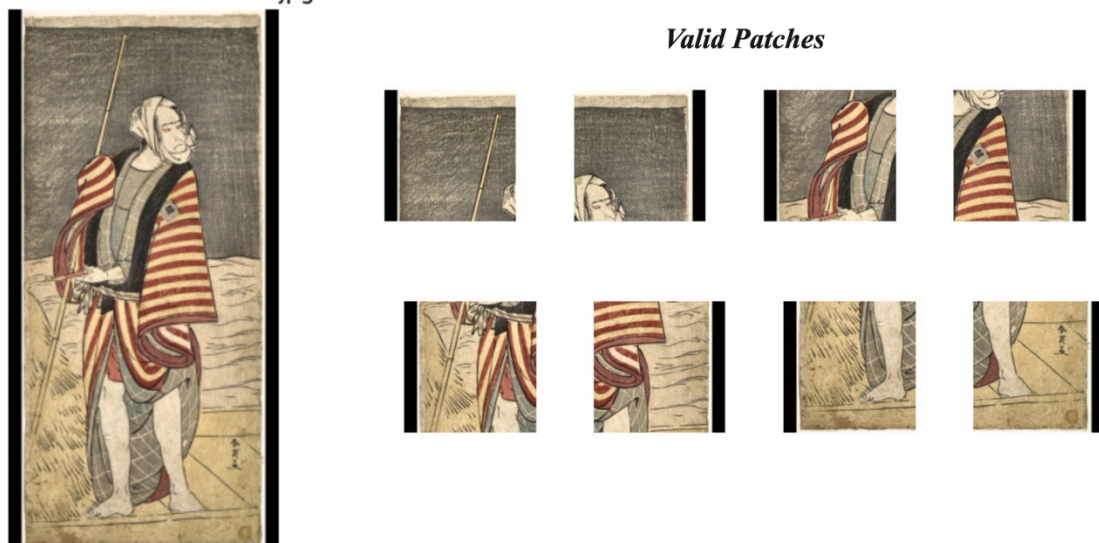
Penambahan *padding black ratio* juga menjadi parameter penting untuk menyaring *patch* yang terlalu banyak mengandung padding hitam. Threshold yang digunakan adalah 0.2 atau 20% artinya *patch* yang memiliki lebih dari 20% area berwarna hitam akan dianggap tidak valid. Hal ini ditujukan untuk mencegah masuknya area padding yang tidak mengandung informasi visual ke dalam proses pelatihan model.



Gambar 4.2 Visualisasi *Invalid Patches* Hasil Filtrasi *Padding*

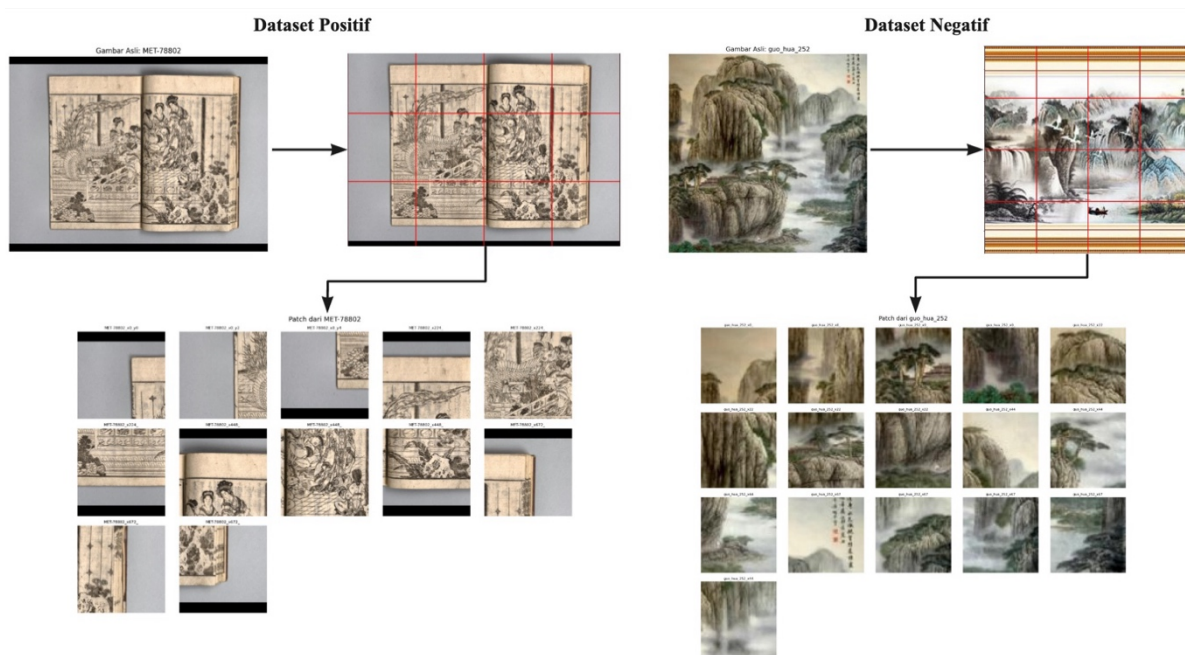
Gambar 4.2 memperlihatkan contoh visualisasi dari proses validasi *patch* pada salah satu gambar dataset positif. Di bagian atas ditampilkan *patch* yang dinyatakan valid karena memenuhi ketiga kriteria (variasi warna, *edge*, dan *padding* hitam), sedangkan bagian bawah menunjukkan *patch* yang dieliminasi karena rasio padding hitam melebihi ambang batas. Beberapa *patch* bahkan memiliki *edge* dan kontras yang tinggi, namun tetap ditolak akibat dominasi area hitam lebih dari 44–45%.

Gambar Utuh: MET-36793.jpg



Gambar 4.3 Visualisasi Hasil Lolos Validasi *Padding*

Gambar 4.3 memperlihatkan contoh gambar yang tetap menghasilkan banyak *patch* valid meskipun mengandung padding hitam. Hal ini terjadi karena rasio *padding* pada *patch* tetap berada di bawah ambang batas 20%, sehingga tidak mengganggu validasi. Perbandingan ini menunjukkan bahwa sistem validasi yang digunakan bersifat selektif namun tetap fleksibel terhadap variasi visual dalam gambar. Secara keseluruhan, penambahan metrik *padding* hitam dalam sistem validasi *patch* terbukti mampu menyaring area non-informatif yang sebelumnya masih sering lolos ke dalam *dataset*. Dengan demikian, *patch* yang diproses selanjutnya menjadi lebih representatif dan mengurangi potensi *noise* yang dapat memengaruhi hasil klasifikasi akhir



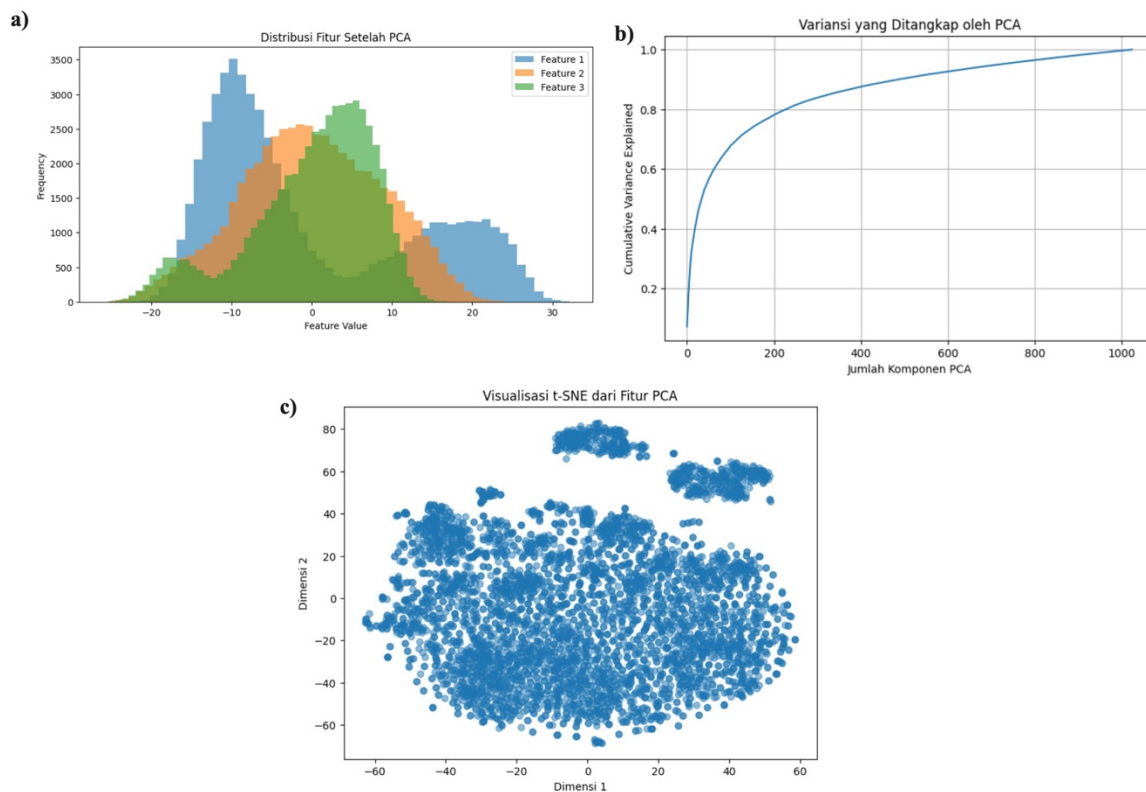
Gambar 4.4 Visualisasi Proses Deteksi *Patch*

Gambar 4.4 memperlihatkan tahapan proses deteksi *patch* menggunakan metode *sliding window* pada *dataset*. Bagian atas tabel menampilkan citra asli yang akan diproses, diikuti dengan visualisasi *overlay* grid berukuran 224×224 piksel yang diterapkan pada citra tersebut. *Grid* ini menunjukkan bagaimana posisi setiap *patch* ditentukan berdasarkan koordinat horizontal dan vertikal dengan *stride* tetap tanpa *overlap*. Sementara itu, bagian bawah tabel memperlihatkan hasil akhir dari proses pemotongan *patch*, di mana setiap potongan gambar disimpan secara terpisah dengan penamaan berdasarkan posisi, seperti *x224_y0*. Patch yang lolos validasi kemudian disimpan dalam folder *patch*, dengan format penamaan file yang mencakup posisi koordinat patch pada gambar aslinya.

4.1.3 Ekstraksi Fitur

Setelah proses deteksi *patch* selesai, setiap *patch* yang valid diekstraksi fitur visualnya menggunakan arsitektur *ResNet50*. Dalam implementasi ini, fitur diambil dari output lapisan terakhir pada blok konvolusi kelima, yaitu bagian sebelum klasifikasi akhir dari *ResNet50*. Selanjutnya, diterapkan lapisan *Global Average Pooling* (GAP) untuk mereduksi fitur spasial menjadi representasi vektor satu dimensi berdimensi 2048. Hasil ekstraksi awal berupa vektor fitur berdimensi 2048 kemudian direduksi menggunakan metode *Principal Component Analysis* (PCA) untuk menurunkan kompleksitas dan memfasilitasi proses klasifikasi

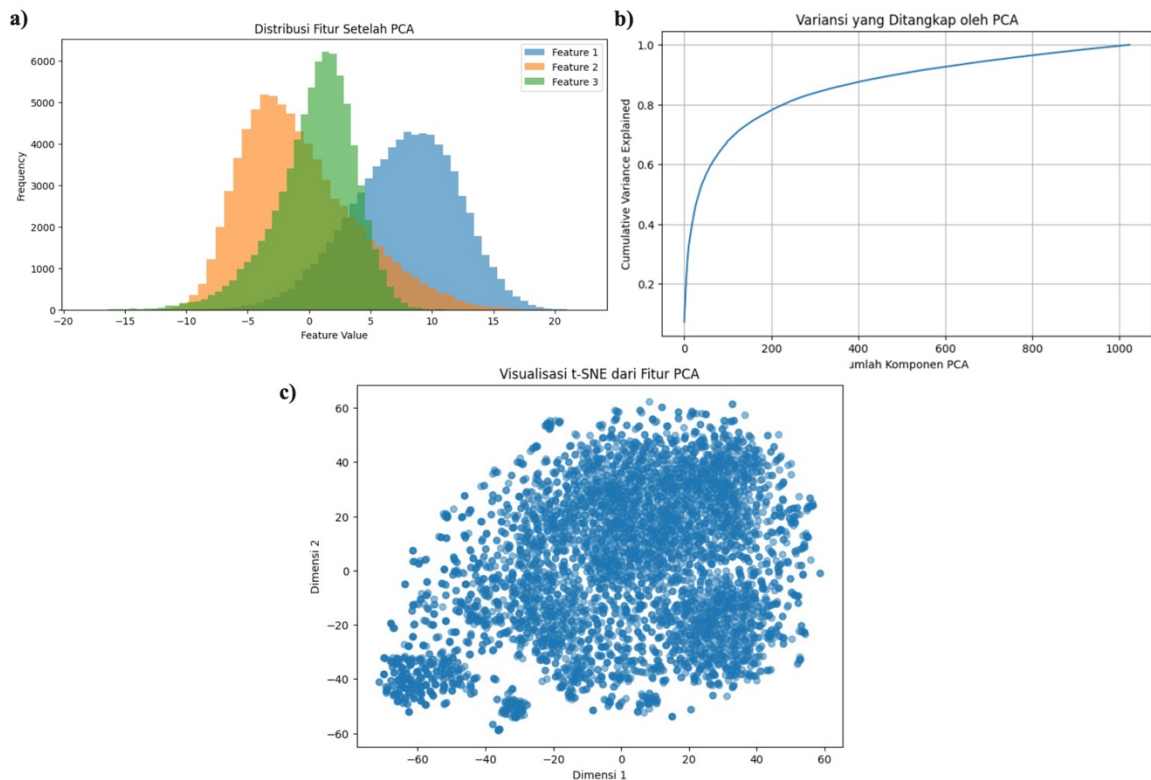
selanjutnya. *Patch* diproses dalam *batch* berukuran 1024 untuk efisiensi komputasi. Untuk mengevaluasi hasil transformasi fitur dengan *PCA*, dilakukan analisis visual terhadap distribusi komponen utama yang dihasilkan. Gambar 4.5 dan Gambar 4.6 menampilkan tiga grafik hasil ekstraksi fitur setelah dilakukannya *PCA*.



Gambar 4.5 Analisis Hasil Ekstraksi Setelah PCA pada *Dataset Positif*

Gambar 4.5(a) menunjukkan histogram distribusi dari tiga komponen utama pertama hasil *Principal Component Analysis (PCA)* pada fitur visual yang diekstraksi dari *patch* citra *dataset* positif. Ketiga komponen tersebut masing-masing memiliki sebaran nilai yang menyerupai distribusi *Gaussian* yaitu di mana sebagian besar data berkumpul di sekitar rata-rata, dan jumlah data semakin sedikit di nilai yang jauh dari rata-rata, namun pada histogram ini bentuk yang dihasilkan tidak sepenuhnya simetris. Hal ini mengindikasikan bahwa *PCA* berhasil mentransformasikan fitur berdimensi tinggi ke dalam ruang baru yang memiliki penyebaran yang stabil dan teratur. Selain itu, variasi nilai yang cukup luas pada setiap komponen menunjukkan bahwa fitur yang dihasilkan tetap mengandung informasi penting yang beragam dari *patch*. Selanjutnya, Gambar 4.5(b) memperlihatkan kurva *cumulative explained variance* dari seluruh komponen hasil *PCA*. Kurva ini menunjukkan bahwa sekitar 80% variansi total data dapat dijelaskan oleh sekitar 200 hingga 300 komponen pertama, sedangkan lebih dari 95% variansi telah tertangkap oleh 600 komponen pertama. Berdasarkan temuan ini, pemilihan jumlah komponen sebesar 1024 dinilai sangat bagus untuk mempertahankan informasi penting dalam data, sekaligus secara signifikan mengurangi kompleksitas dimensi awal dari fitur *ResNet50* yang berjumlah 2048. Sebagai validasi tambahan terhadap kualitas pemisahan fitur, dilakukan visualisasi distribusi data menggunakan metode *t-distributed Stochastic Neighbor Embedding (t-SNE)* terhadap fitur hasil *PCA*, seperti ditampilkan pada Gambar 4.5(c). Visualisasi ini menunjukkan bahwa *patch* hasil ekstraksi membentuk beberapa kelompok atau kluster secara alami, meskipun belum diterapkan algoritma *clustering* eksplisit. Hal ini menunjukkan bahwa representasi fitur hasil *PCA* memiliki potensi diskriminatif yang baik,

serta mengindikasikan adanya struktur laten dalam data yang dapat dimanfaatkan lebih lanjut pada proses klasifikasi berbasis *clustering* maupun *supervised learning*.



Gambar 4.6 Analisis Hasil Ekstraksi Setelah PCA pada *Dataset* Negatif

Gambar 4.6(a) menampilkan analisis karakteristik fitur dari dataset seni lukis Cina yang telah diekstraksi menggunakan *ResNet50* dan direduksi dimensinya menggunakan *Principal Component Analysis (PCA)*. Histogram pada bagian atas memperlihatkan distribusi dari tiga komponen utama hasil *PCA*. Ketiga komponen tersebut memiliki pola sebaran yang relatif simetris dan menyerupai distribusi *Gaussian*, di mana sebagian besar nilai berkumpul di sekitar rata-rata. Pola distribusi ini menunjukkan bahwa proses *PCA* berhasil mentransformasikan fitur ke dalam bentuk yang terpusat dan stabil secara statistik. Hal ini penting karena fitur yang terdistribusi dengan baik akan mempermudah proses klasifikasi maupun *clustering* pada tahap selanjutnya. Gambar 4.6(b) menampilkan kurva *cumulative explained variance* yang menunjukkan bahwa sebagian besar informasi dari fitur awal telah tertangkap oleh ratusan komponen utama pertama. Sekitar 90% variansi total dapat dijelaskan oleh sekitar 500 komponen, sehingga pemilihan jumlah komponen sebesar 1024 dinilai tepat untuk mempertahankan sebanyak mungkin informasi tanpa meningkatkan risiko *overfitting*. Selanjutnya, Gambar 4.6(c) menunjukkan visualisasi hasil *t-distributed Stochastic Neighbor Embedding (t-SNE)* terhadap fitur yang telah direduksi oleh *PCA*. Visualisasi ini memperlihatkan sebaran dua dimensi dari *patch* dalam ruang fitur. Titik-titik membentuk pola kepadatan yang tidak acak, menunjukkan bahwa meskipun belum diterapkan pelabelan, terdapat kecenderungan kluster alami dalam data. Hal ini mencerminkan adanya variasi gaya atau elemen visual dalam lukisan Cina yang berhasil ditangkap oleh model ekstraksi fitur dan dapat dimanfaatkan dalam proses *discriminative clustering*.

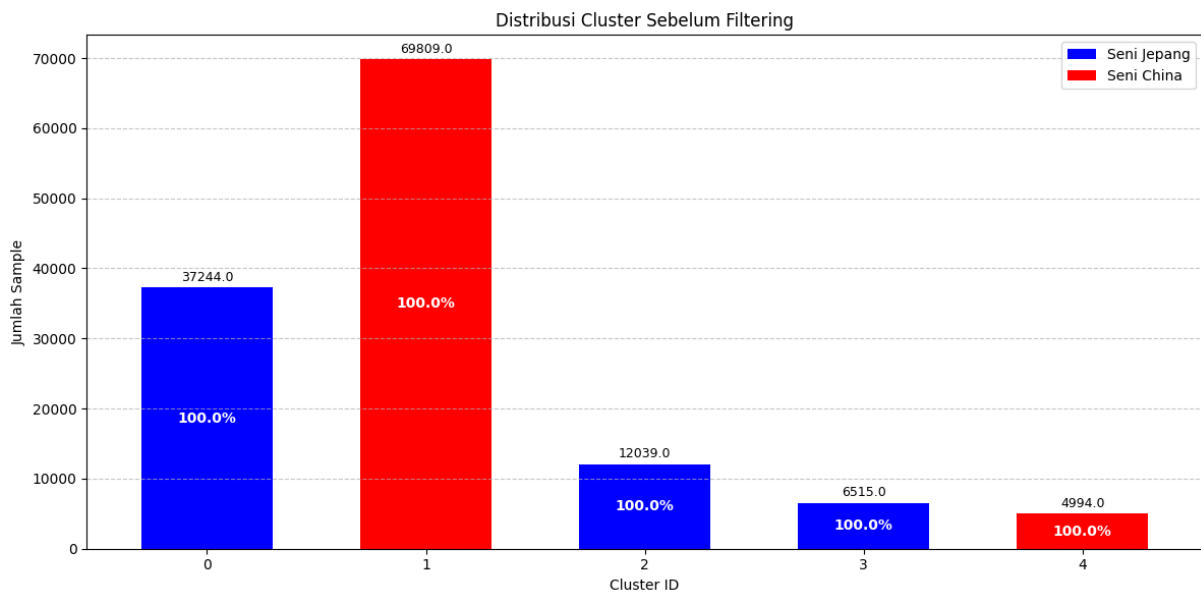
4.1.4 Discriminative Clustering

Setelah fitur dari seluruh *patch* citra diekstraksi dan direduksi menggunakan *Principal Component Analysis* (PCA), dilakukan proses *clustering* untuk mengelompokkan *patch* yang memiliki karakteristik visual serupa tanpa mempertimbangkan label awalnya. Proses *clustering* utama dilakukan menggunakan algoritma *K-Means* dengan metrik jarak *cosine distance*. Untuk menentukan jumlah kluster yang optimal, dilakukan evaluasi menggunakan metrik *silhouette score*, yang mengukur konsistensi internal suatu kluster dibandingkan dengan kluster lain.

Tabel 4.8 Top 3 Hasil Evaluasi *Silhouette Score*

Top Nilai K untuk Cosine Distance K-Means	
K	Silhouette Score
5	0.934
4	0.921
3	0.912

Pada Tabel 4.8, hasil evaluasi menunjukkan bahwa nilai *silhouette* tertinggi diperoleh pada $K = 5$, sehingga jumlah kluster yang digunakan adalah lima. Setelah proses *clustering* selesai, dihitung *centroid* dari masing-masing kluster berdasarkan representasi fitur berdimensi 1024. Seluruh hasil, termasuk label kluster, vektor fitur, koordinat *centroid*, dan nama file *patch*, disimpan untuk keperluan analisis lanjutan serta tahap klasifikasi berbasis jarak terhadap *centroid*.

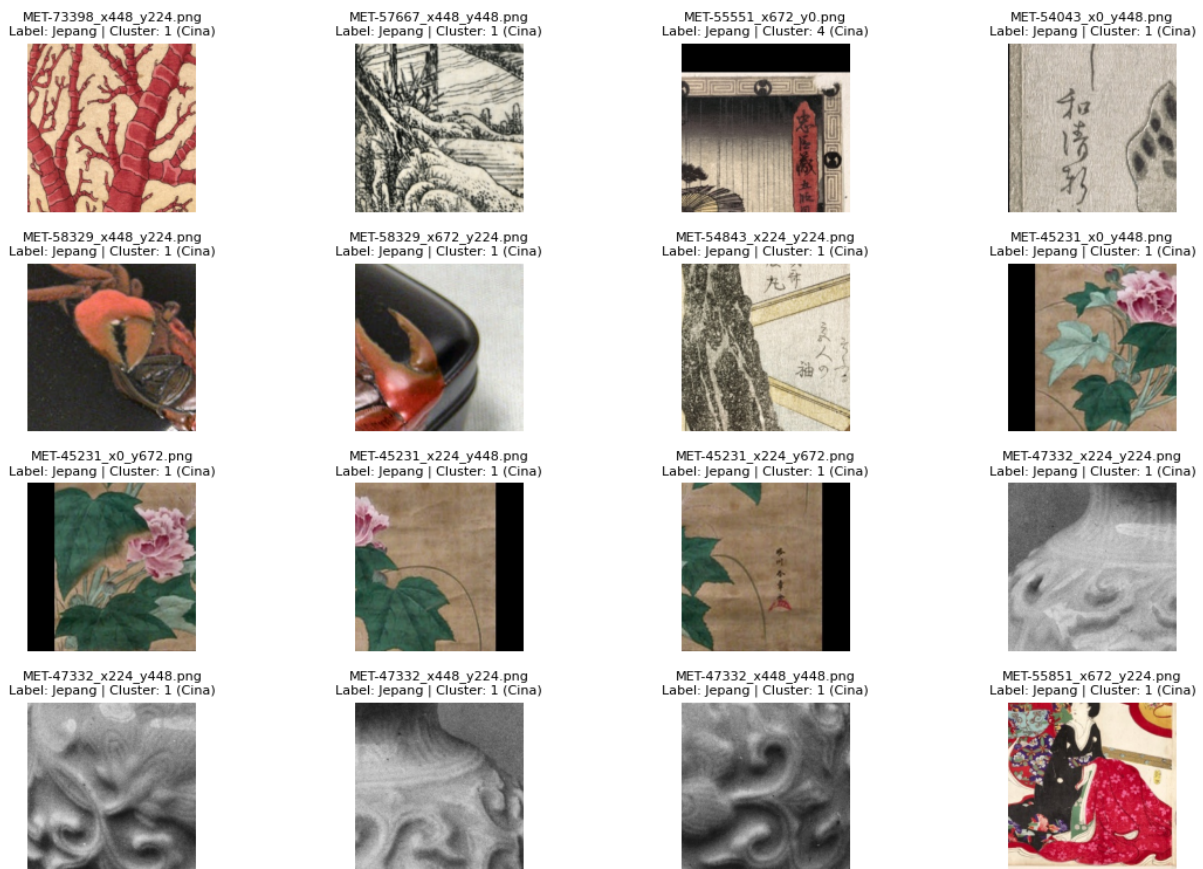


Gambar 4.7 Hasil Pengelompokkan *Discriminative Clustering*

Gambar 4.7 memperlihatkan hasil distribusi *patch* ke dalam lima kluster utama yang dibentuk menggunakan algoritma *K-Means*, berdasarkan fitur visual hasil ekstraksi ResNet50 dan reduksi dimensi menggunakan *Principal Component Analysis* (PCA). Secara umum, hasil visualisasi menunjukkan pemisahan yang sangat jelas antara dua kategori budaya, yakni seni Jepang dan seni Cina. Tiga kluster—yakni *Cluster 0*, *Cluster 2*, dan *Cluster 3*—sepenuhnya terdiri dari *patch* yang berasal dari lukisan Jepang. *Cluster 0* memuat 37.244 *patch*, sedangkan *Cluster 2* dan *Cluster 3* masing-masing memuat 12.039 dan 6.515 *patch*. Komposisi ini menunjukkan bahwa representasi fitur dari lukisan Jepang tersebar konsisten dan membentuk kelompok visual yang dapat dikenali secara terpisah dari kelas lain. Ciri visual yang dominan

dalam klaster-klaster ini meliputi pola garis halus, tekstur lembut, serta nuansa warna yang cenderung ringan dan terstruktur.

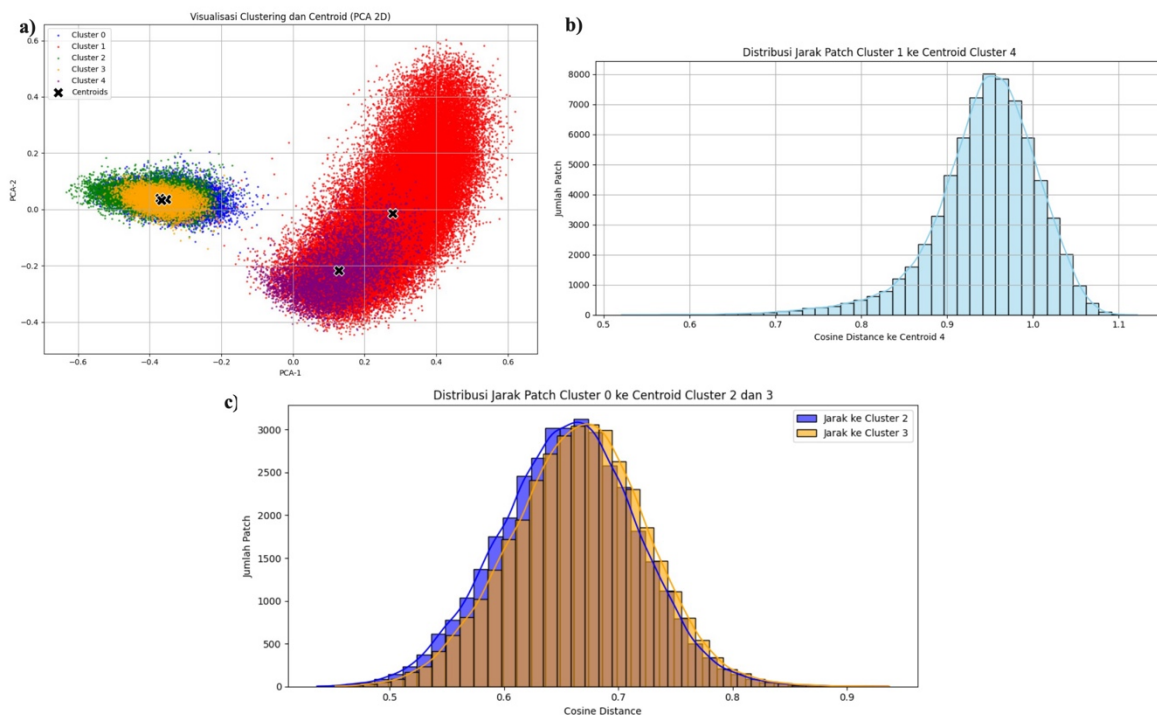
Sebaliknya, *Cluster 1* didominasi oleh lukisan Cina dengan sangat kuat, yakni 69.786 dari total 69.809 *patch* (99,97%), menyisakan hanya 23 *patch* dari lukisan Jepang. Karakteristik visual dalam klaster ini umumnya mencakup kontur pemandangan alam, penggunaan tinta tebal, dan gradasi warna gelap yang khas dalam seni lukis Cina klasik. Hal serupa juga ditemukan pada *Cluster 4*, yang memuat 4.994 *patch* dengan komposisi 99,98% berasal dari lukisan Cina. Kedua klaster ini memperlihatkan kemampuan algoritma *clustering* dalam mengelompokkan fitur yang memiliki kemiripan secara alami. Dominasi kelas yang cukup besar, bahkan hingga 100% dalam satu klaster, mengindikasikan bahwa metode ekstraksi fitur dan reduksi dimensi yang digunakan dalam penelitian ini mampu menangkap perbedaan visual mendasar antara dua budaya seni. Dengan demikian, hasil yang ditunjukkan pada Gambar 4.9 memberikan landasan kuat bahwa *patch* lukisan dari dua budaya dapat terpisah secara alami dalam ruang fitur berdimensi rendah, mencerminkan struktur dalam data.



Gambar 4.8 *Patch* yang Salah Dikelompokkan ke Kelas Lain

Gambar 4.8 menampilkan beberapa contoh *patch* dari gambar berlabel seni lukis Jepang yang secara keliru dikelompokkan ke dalam klaster yang didominasi oleh seni lukis Cina, yakni *Cluster 1* dan *Cluster 4*. Kesalahan ini terjadi meskipun label asli *patch* tersebut adalah Jepang, yang mengindikasikan adanya kemiripan karakteristik visual antara sebagian elemen dalam lukisan Jepang dengan pola dominan yang ditemukan dalam klaster Cina. Beberapa *patch* yang salah klasifikasi menampilkan elemen-elemen seperti kaligrafi, struktur garis tegas, bentuk floral, serta ornamen tekstil, yang secara tekstur atau bentuk menyerupai motif yang juga umum ditemukan dalam lukisan Cina. Temuan ini menjadi indikasi penting bahwa meskipun metode clustering mampu mengelompokkan *patch* berdasarkan kedekatan fitur visual, pendekatan ini tetap memerlukan validasi terhadap keselarasan dengan label aslinya.

Gambar 4.9(a) menyajikan visualisasi distribusi patch dalam ruang dua dimensi hasil reduksi fitur menggunakan *Principal Component Analysis* (PCA). Visualisasi ini menggambarkan hasil *clustering* menggunakan algoritma *K-Means*, dengan pewarnaan berbeda untuk masing-masing cluster. Titik-titik dalam grafik mewakili patch individu, sedangkan simbol silang hitam menunjukkan posisi centroid dari setiap cluster. Terlihat bahwa patch yang tergolong ke dalam *Cluster 0*, *Cluster 2*, dan *Cluster 3*, yang didominasi oleh lukisan Jepang, terkonsentrasi pada sisi kiri grafik dan membentuk pola yang relatif padat serta saling berdekatan. Sebaliknya, *Cluster 1* dan *Cluster 4* yang mayoritas terdiri atas *patch* dari lukisan Cina, menempati sisi kanan grafik dan memiliki pola sebaran yang lebih luas. Jarak antar *centroid* juga memberikan informasi penting terkait pemisahan antar seni. *Centroid* dari klaster-klaster Jepang berada dalam jarak yang relatif dekat satu sama lain, yang mengindikasikan kesamaan struktur di antara ketiga *cluster* tersebut. Kondisi serupa juga ditemukan pada klaster Cina, di mana *centroid* dari *Cluster 1* dan *4* berada dalam jarak yang tidak terlalu jauh. Namun, terdapat pemisahan yang cukup jelas antara dua kelompok budaya ini, ditandai oleh jarak yang relatif besar antara *centroid* klaster-klaster Jepang dan klaster-klaster Cina.

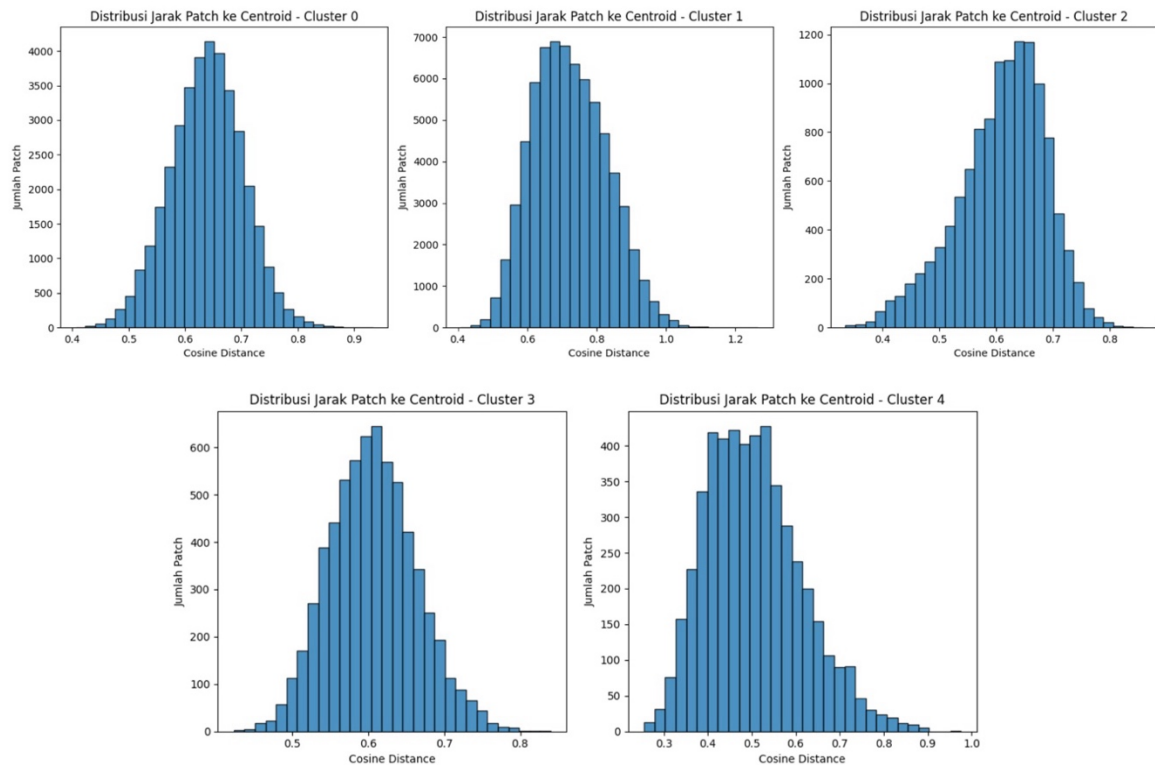


Gambar 4.9 Analisis Hasil *Discriminative Clustering*

Gambar 4.9(b) menampilkan histogram distribusi jarak *cosine* antara *patch* dari Klaster 1 yang didominasi oleh lukisan Cina terhadap *centroid* dari *Cluster 4*, yang juga merupakan dominan terhadap seni lukis Cina. Histogram menunjukkan bahwa sebagian besar *patch* dari Klaster 1 memiliki jarak *cosine* antara 0,9 hingga 1,0 terhadap *centroid* Klaster 4. Nilai ini menunjukkan bahwa meskipun kedua klaster berasal dari kategori budaya yang sama dan pada visualisasinya terlihat tumpang tindih, terdapat perbedaan visual yang cukup signifikan di antara keduanya. Interpretasi ini mengindikasikan bahwa Klaster 1 dan Klaster 4 merepresentasikan dua gaya lukis yang berbeda dalam ranah seni lukis Cina. Kemungkinan perbedaan tersebut dapat mencerminkan variasi dalam teknik sapuan kuas, komposisi ruang, penggunaan kontras tinta, atau jenis objek yang ditampilkan.

Gambar 4.9(c) menampilkan histogram distribusi jarak *cosine* antara *patch* yang berasal dari Klaster 0 (kategori Jepang) terhadap dua *centroid* lain dari klaster yang juga mewakili

kategori Jepang, yaitu Klaster 2 dan Klaster 3. Kurva berwarna biru merepresentasikan jarak terhadap *centroid* Klaster 2, sedangkan kurva berwarna kuning menunjukkan jarak terhadap *centroid* Klaster 3. Kedua distribusi menunjukkan pola yang hamper mirip, dengan rentang umum antara 0,45 hingga 0,85. Puncak distribusi dari keduanya berada di sekitar nilai 0,63. Kesamaan bentuk dan posisi puncak histogram mengindikasikan bahwa *patch* dari Cluster 0 memiliki tingkat kedekatan yang mirip terhadap dua *centroid* tersebut. Temuan ini memperkuat indikasi bahwa meskipun terbagi ke dalam tiga klaster yang berbeda, representasi fitur *patch* dalam lukisan Jepang berdekatan dan saling beririsan. Secara keseluruhan, Gambar 4.11(a–c) memberikan pemahaman yang lebih mendalam mengenai hasil pengelompokan ruang klaster.

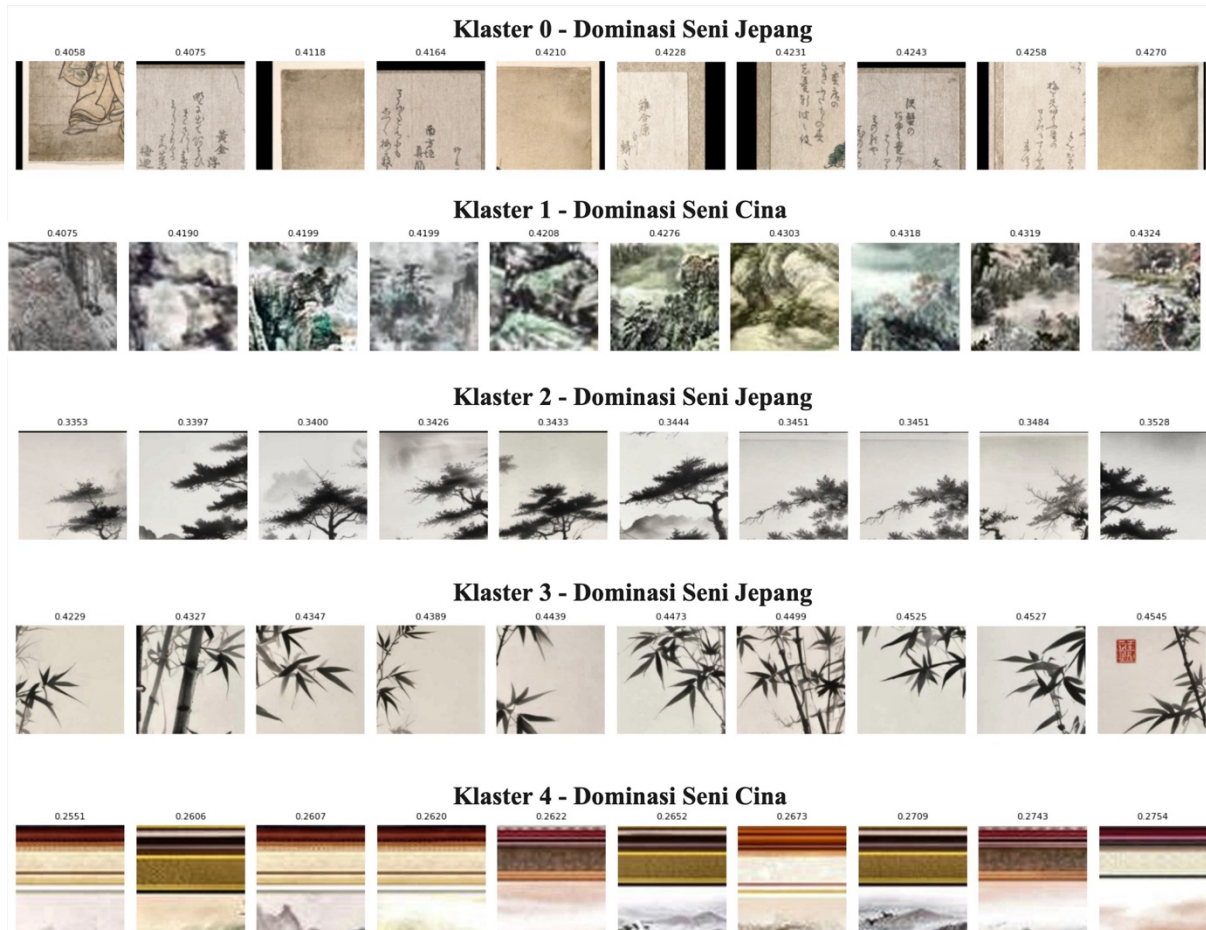


Gambar 4.10 Distribusi Jarak *Patch* ke *Centroid*

Gambar 4.10 menyajikan histogram distribusi jarak *cosine* antara masing-masing *patch* terhadap *centroid* dari *cluster*-nya sendiri. Analisis ini penting untuk memahami seberapa representatif setiap *centroid* dalam merepresentasikan fitur-fitur yang terkandung di dalam *cluster*-nya. *Cluster* 0, yang dominan terdiri dari *patch* lukisan Jepang, menunjukkan distribusi yang relatif simetris, dengan puncak distribusi di sekitar 0,65 dan rentang nilai antara 0,45 hingga 0,85. Pola ini mengindikasikan bahwa sebagian besar *patch* dalam cluster ini memiliki jarak yang seragam terhadap *centroid*, yang berarti konsistensi fitur tinggi. *Cluster* 2 dan *Cluster* 3, yang juga mendominasi kategori Jepang, memperlihatkan pola distribusi serupa namun terdapat perbedaan. *Cluster* 2 memiliki bentuk distribusi yang *right-skewed*, menunjukkan bahwa sebagian besar *patch* berjarak dekat ke *centroid*, namun terdapat kumpulan fitur yang jauh. Sementara itu, *Cluster* 3 menunjukkan sebaran yang sedikit lebih kompak dan pusat distribusinya masih berada di sekitar 0,65.

Sebaliknya, distribusi jarak *cosine* pada *Cluster* 1 yang merupakan cluster dominan dari lukisan Cina menunjukkan pola yang lebih tersebar, dengan rentang yang lebih luas dan beberapa nilai bahkan melampaui 1,0. Meskipun puncak distribusinya tetap berada di kisaran 0,65–0,70, beberapa distribusi yang memanjang ke sisi kanan menunjukkan bahwa banyak

patch dalam cluster ini berada cukup jauh dari *centroid*-nya. Pola ini mencerminkan tingkat keragaman yang tinggi dalam gaya lukisan Cina. Menariknya, *Cluster 4* yang juga berasal dari kategori lukisan Cina menunjukkan distribusi yang sangat terpusat, dengan puncak tajam pada rentang 0,45–0,55 dan rentang distribusi yang lebih sempit. Hal ini mengindikasikan tingkat kekompakan sangat besar dalam cluster, yang kemungkinan mewakili subgaya atau motif visual tertentu dalam seni lukis Cina. Temuan dari Gambar 4.10 memperkuat pemahaman bahwa tidak semua *cluster* dalam satu kategori budaya memiliki karakter distribusi yang sama.



Gambar 4.11 Top 10 Patch Terdekat pada Centroid Tiap Cluster

Gambar 4.11 menampilkan sepuluh *patch* terdekat terhadap *centroid* dari masing-masing lima *cluster* hasil *discriminative clustering*. Setiap *patch* yang ditampilkan memiliki nilai jarak *cosine* terkecil terhadap *centroid*-nya, sehingga dapat dianggap sebagai representasi visual paling “mewakili” struktur *cluster*. Pada *Cluster 0*, seluruh *patch* berasal dari lukisan Jepang dan menampilkan elemen khas seperti tekstur kertas halus, aksara kaligrafi bergaya kanji, serta *margin* atau bingkai bergaya *ukiyo-e*. Sebagian besar *patch* memiliki latar terang dengan komposisi minimalis. Nilai jarak *cosine* terhadap *centroid* berkisar antara 0,40 hingga 0,43, mengindikasikan tingkat kedekatan dan kekompakan yang tinggi serta representasi visual yang konsisten dalam *cluster* ini.

Sementara itu, *Cluster 1* sebagai *cluster* dominan untuk kategori lukisan Cina, memperlihatkan *patch landscape* alam dengan elemen khas *shan shui*, seperti gunung, pepohonan, dan kabut. Konsistensi beberapa fitur ini mengonfirmasi bahwa *cluster* ini merepresentasikan subgaya visual pemandangan alam dalam tradisi seni lukis Cina. *Cluster 2* secara konsisten memuat *patch* dari lukisan bergaya *sumi-e* yang bercirikan komposisi hitam

putih, representasi pohon pinus, dan sapuan kuas. Jarak patch ke *centroid* sangat kecil (berkisar antara 0,33–0,35), menunjukkan tingkat keseragaman fitur yang sangat tinggi. Hal ini mendukung interpretasi bahwa cluster ini berhasil mengidentifikasi subgaya visual yang representatif dalam seni lukis Jepang.

Cluster 3 menampilkan motif visual serupa, yaitu lukisan bambu hitam-putih bergaya *sumi-e*, dengan beberapa lukisan batang dan dedaunan. Meskipun gaya visualnya masih dalam ranah Jepang, variasi antar *patch* sedikit lebih besar dibandingkan *Cluster 2*, dengan jarak *cosine* terhadap centroid berada dalam kisaran 0,42 hingga 0,45. Hal ini menunjukkan bahwa meskipun masih dalam satu kategori budaya, *cluster* ini memuat subgaya yang representatif. Terakhir, *Cluster 4* memperlihatkan *patch* dengan elemen unik, seperti garis horizontal tebal berwarna merah, kuning, dan coklat. Beberapa *patch* menunjukkan kemungkinan representasi seperti bingkai atau latar dari lukisan. Menariknya, nilai jarak *cosine patch* terhadap *centroid* dalam *cluster* ini sangat kecil (antara 0,25–0,27), menjadikannya *cluster* dengan tingkat keseragaman tertinggi di antara semua. Ini menunjukkan bahwa meskipun motifnya tampak lebih beragam, fitur visual yang diekstrak sangat seragam dan sangat mudah dipetakan dalam ruang fitur berdimensi rendah.

4.1.5 Model dan Klasifikasi *Hybrid Voting*

Pada tahap ini, dilakukan pelatihan model klasifikasi berbasis *Support Vector Machine* (SVM) yang dikombinasikan dengan informasi kedekatan patch terhadap centroid hasil discriminative clustering. Model ini disebut sebagai *Hybrid Voting Model*, karena menggabungkan dua jenis informasi yaitu probabilitas prediksi dari model SVM dan skor kedekatan terhadap *centroid* kategori (Jepang atau Cina). Proses awal dimulai dengan pemilihan fitur dari *patch* yang tergolong bersih (*pure cluster*), yaitu *patch* dari seni Jepang yang berasal dari *cluster* 0, 2, dan 3, serta *patch* dari seni Cina yang berasal dari *cluster* 1 dan 4. Untuk menghindari bias distribusi, dilakukan undersampling sehingga jumlah sampel dari kedua kategori seimbang, masing-masing sebanyak 55798 fitur. Setelah proses *balancing*, data dibagi menjadi 80% untuk pelatihan dan 20% untuk pengujian. Model SVM dilatih menggunakan kernel RBF dan parameter *default*, dengan hasil evaluasi pada data uji menunjukkan performa yang sangat baik, yaitu *precision*, *recall*, dan *f1-score* sebesar 0.99 untuk kedua kelas (Tabel 4.9).

a) Klasifikasi *Hybrid Voting* pada Data Uji

Untuk pengujian model hybrid, dilakukan perhitungan skor gabungan untuk setiap *patch* berdasarkan dua komponen yaitu skor probabilitas SVM dan skor representatif *centroid* Jepang, yang dihitung dari rata-rata kebalikan jarak *cosine* ke *centroid cluster* 0, 2, dan 3. Model *hybrid voting* berdasarkan kombinasi nilai α (alpha) dan β (beta), yang merepresentasikan bobot kontribusi antara skor jarak centroid dan probabilitas dari SVM. Evaluasi dilakukan terhadap metrik utama klasifikasi, yaitu akurasi, *precision*, *recall*, dan *F1-score* untuk *dataset* uji yang sudah dipisahkan (seni Cina dan seni Jepang).

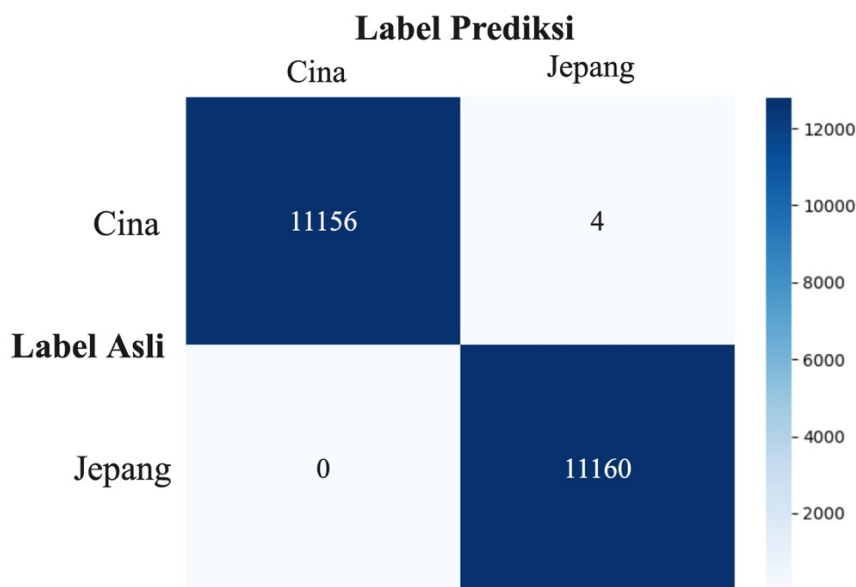
Berdasarkan Tabel 4.9, seluruh konfigurasi nilai α dan β menghasilkan akurasi klasifikasi di atas 97%, yang mencerminkan stabilitas performa model *hybrid voting* pada *dataset* uji. Meskipun demikian, pemilihan parameter optimal tidak hanya didasarkan pada nilai akurasi, melainkan juga pada keseimbangan metrik evaluasi lain seperti *precision*, *recall*, dan *F1-score* untuk masing-masing kelas. Dari seluruh variasi yang diuji, kombinasi $\alpha = 0,40$ dan $\beta = 0,60$ memberikan performa paling baik. Konfigurasi ini menghasilkan akurasi total sebesar 0,9998

serta nilai *F1-score* tertinggi dan paling seimbang untuk kedua kelas, yakni 0,9998 untuk kelas Cina dan 0,9998 untuk kelas Jepang. Hal ini menunjukkan bahwa model memiliki kemampuan deteksi yang tinggi dan tingkat kepercayaan prediksi yang sangat baik terhadap semua kelas.

Tabel 4.9 Eksperimen Konfigurasi α dan β *Hybrid Voting*

α	β	<i>Accuracy</i>	<i>Precision</i> (Cina)	<i>Recall</i> (Cina)	<i>F1-Score</i> (Cina)	<i>Precision</i> (Jepang)	<i>Recall</i> (Jepang)	<i>F1-Score</i> (Jepang)
0,20	0,80	0,9997	0,9999	0,9996	0,9998	0,9996	0,9999	0,9998
0,40	0,60	0,9998	1,0000	0,9996	0,9998	0,9996	1,0000	0,9998
0,43	0,57	0,9997	1,0000	0,9996	0,9998	0,9996	1,0000	0,9998
0,50	0,50	0,9987	1,0000	0,9975	0,9987	0,9975	1,0000	0,9987
0,60	0,40	0,9766	1,0000	0,9532	0,9761	0,9553	1,0000	0,9771

Sebaliknya, pada konfigurasi dengan nilai α lebih besar seperti pada percobaan terakhir yaitu $\alpha = 0,60$ dan $\beta = 0,40$, *recall* untuk kelas Jepang mencapai nilai sempurna (1,0000), namun nilai *precision*-nya hanya bernilai 0,9553. Penurunan ini menunjukkan adanya peningkatan jumlah *false positive* pada prediksi untuk kelas Jepang, yang berarti bahwa model terlalu banyak mengklasifikasikan *patch* sebagai Jepang secara keliru. Dengan mempertimbangkan semua metrik, konfigurasi $\alpha = 0,40$ dan $\beta = 0,60$ dipilih sebagai kombinasi terbaik untuk digunakan dalam tahap evaluasi akhir. Kombinasi ini tidak hanya memberikan akurasi tertinggi, tetapi juga menjaga keseimbangan performa antar kelas secara optimal.



Gambar 4.12 Hasil Confusion matrix *Hybrid Voting* pada Data Uji

Gambar 4.12 menampilkan hasil *confusion matrix* dari proses klasifikasi menggunakan model *hybrid voting* pada dataset uji, dengan konfigurasi parameter optimal $\alpha = 0,40$ dan $\beta = 0,60$. Dataset uji terdiri atas total 22.320 *patch*, yang terbagi merata menjadi 11.160 *patch* dari kelas Jepang dan 11.160 *patch* dari kelas Cina. Berdasarkan matriks tersebut, model berhasil mengklasifikasikan seluruh *patch* dari kelas Jepang secara benar, ditunjukkan oleh nilai *true positive* sebanyak 11.160 dan tidak terdapat *false negative* (0). Hal ini menghasilkan nilai *recall* sempurna sebesar 1,000 untuk kelas Jepang, yang berarti seluruh sampel dari kelas ini berhasil

dikenali oleh model tanpa kesalahan. Untuk kelas Cina, model menghasilkan nilai *true positive* sebesar 11.156, dengan 4 *patch* yang diklasifikasikan secara keliru sebagai Jepang (*false positive* = 4). Meskipun demikian, tingkat kesalahan ini sangat rendah, yaitu hanya sekitar 0,036% dari total *patch* kelas Cina. Ini menunjukkan bahwa model memiliki kemampuan yang sangat baik dalam membedakan gaya visual lukisan dari dua kategori budaya yang berbeda.

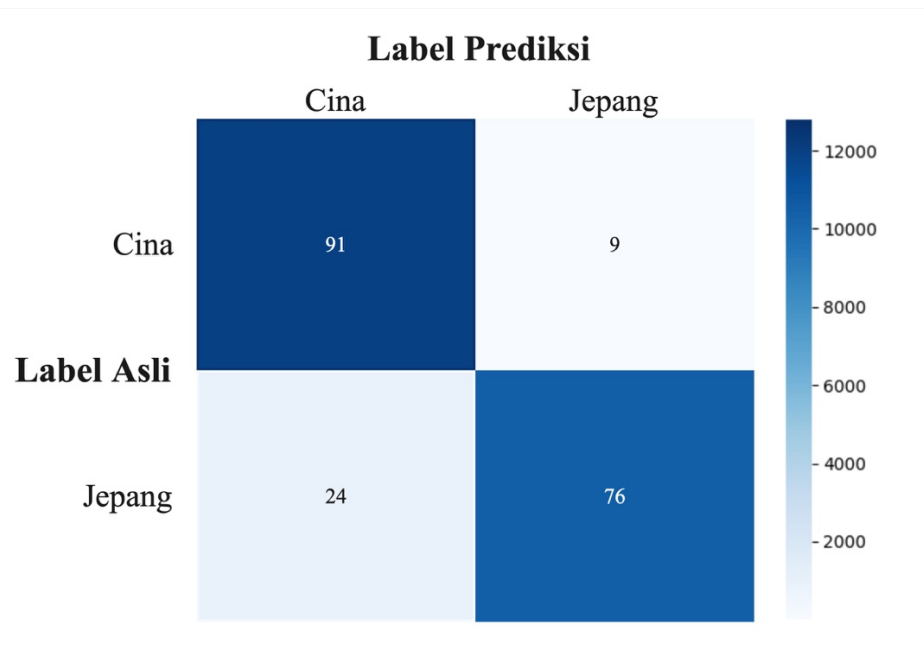
b) Evaluasi *Hybrid Voting* pada Gambar Utuh

Setelah model terbukti efektif untuk klasifikasi pada dataset uji, dilakukan pengujian lanjutan pada gambar utuh. Setiap gambar diproses menjadi *patch* dengan ukuran 224×224 piksel dan diuji satu per satu menggunakan model *hybrid voting*. Label akhir gambar ditentukan melalui mayoritas *voting* berdasarkan hasil klasifikasi semua *patch* valid dalam gambar tersebut. Evaluasi ini dilakukan terhadap 100 gambar *s* dari masing-masing kelas (seni Cina dan seni Jepang), dengan menggunakan konfigurasi parameter terbaik yaitu $\alpha = 0,40$ dan $\beta = 0,60$.

Tabel 4.10 Hasil Evaluasi *Hybrid Voting* pada Gambar Utuh

Metrik	Cina	Jepang	Rata-Rata
<i>Precision</i>	0,7913	0,8941	0,8427
<i>Recall</i>	0,9100	0,7600	0,8350
<i>F1-Score</i>	0,8465	0,8216	0,8341
Akurasi	-	-	0,8350

Berdasarkan Tabel 4.10, model mencatat nilai *precision* tertinggi pada kelas Jepang sebesar 0,8941. Hal ini menunjukkan bahwa sebagian besar gambar yang diprediksi sebagai Jepang memang benar berasal dari kelas tersebut, dengan tingkat kesalahan prediksi yang rendah. Sementara itu, nilai *recall* tertinggi dicapai oleh kelas Cina sebesar 0,9100, yang mengindikasikan bahwa model mampu mengenali hampir seluruh gambar Cina secara benar. Nilai *F1-score* yang dihasilkan adalah sebesar 0,8465 untuk kelas Cina dan 0,8216 untuk kelas Jepang, dengan rata-rata *F1-score* sebesar 0,8341. Nilai ini menunjukkan keseimbangan performa klasifikasi di antara kedua kelas. Selain itu, model berhasil mencapai akurasi keseluruhan sebesar 0,8350, yang berarti bahwa sekitar 83,5% dari total 200 gambar uji berhasil diklasifikasikan dengan benar. Hasil ini menunjukkan bahwa konfigurasi $\alpha = 0,40$ dan $\beta = 0,60$ tidak hanya efektif dalam klasifikasi berbasis *patch*, tetapi juga cukup baik saat diterapkan pada klasifikasi gambar secara menyeluruh (*image-level classification*). Meskipun hasil evaluasi model *hybrid voting* pada gambar utuh menunjukkan performa yang cukup tinggi, terdapat sedikit penurunan jika dibandingkan dengan evaluasi pada tingkat *patch* individu. Penurunan ini diduga disebabkan oleh kemungkinan adanya fitur visual yang berasal dari gambar yang sama dan terbagi antara data pelatihan dan pengujian. Pada saat pembuatan model, pembagian *dataset* dilakukan menggunakan metode *stratified split*, yang hanya mempertimbangkan distribusi label secara merata tanpa memperhatikan keterkaitan antar *patch* dalam satu gambar utuh. Dengan demikian, tidak dapat dipastikan bahwa seluruh *patch* dari satu gambar berada secara konsisten dalam subset *training* atau *testing*. Kondisi ini memungkinkan terjadinya *data leakage* tidak langsung, yaitu ketika model telah mempelajari sebagian ciri visual dari gambar yang sama selama pelatihan, sehingga berpotensi mempengaruhi hasil prediksi pada saat pengujian.



Gambar 4.13 Hasil *Confusion matrix Hybrid Voting* pada Gambar Utuh

Gambar 4.13 menyajikan matriks kebingungan (*confusion matrix*) dari hasil evaluasi model klasifikasi menggunakan pendekatan *hybrid voting* terhadap gambar utuh, yang terpilih *random* 100 gambar dari masing-masing kelas, yaitu lukisan Cina dan lukisan Jepang. Model yang digunakan merupakan hasil pelatihan awal, dengan konfigurasi parameter *hybrid voting* tetap mengacu pada hasil eksperimen sebelumnya, yakni dengan nilai α sebesar 0,40 dan β sebesar 0,60. Berdasarkan hasil klasifikasi, model berhasil mengidentifikasi sebanyak 91 dari 100 citra kelas Cina secara benar (*true positive*), sementara 9 citra lainnya salah diklasifikasikan sebagai lukisan Jepang (*false negative*). Sebaliknya, pada kelas Jepang, model hanya mampu mengklasifikasikan 76 citra secara tepat (*true positive*), sedangkan 24 citra lainnya salah diklasifikasikan sebagai lukisan Cina (*false positive*).

Hasil evaluasi menunjukkan bahwa model memiliki *recall* sebesar 0,9100 pada kelas Cina, yang menunjukkan kemampuannya dalam mengenali sebagian besar citra Cina secara akurat. Namun demikian, pada kelas Jepang, nilai *recall* hanya mencapai 0,7600, yang menunjukkan bahwa sekitar 24% citra Jepang tidak berhasil dikenali dengan benar. Penurunan ini diduga disebabkan oleh kemungkinan adanya fitur visual yang berasal dari gambar yang sama dan terbagi antara data pelatihan dan pengujian. Sebelumnya, model dilatih dan diuji menggunakan fitur *patch* hasil *clustering*, yang kemudian dibagi secara merata untuk *training* dan *testing* (*stratified split*). Artinya, distribusi data saat pelatihan dan pengujian memiliki proporsi seimbang, dan *patch* diuji secara independen. Sebaliknya, dalam skenario klasifikasi gambar utuh, prediksi bergantung pada kumpulan *patch* dari satu gambar yang bisa sangat bervariasi dalam jumlah dan kualitas informasi visual. *Patch* yang berasal dari satu gambar bisa saja sebagian besar pernah dilihat oleh model selama *training*, atau justru tidak sama sekali. Ini menjadikan hasil klasifikasi per gambar lebih rentan terhadap ketidakseimbangan distribusi *patch*, serta kualitas informasi yang terkandung di tiap *patch*, sehingga menyebabkan akurasi lebih rendah dibandingkan evaluasi berbasis *patch* individu. Dengan demikian, meskipun model menunjukkan performa tinggi saat diuji pada *level patch*, evaluasi berbasis gambar utuh menunjukkan adanya tantangan tambahan yang perlu dipertimbangkan.

4.1.6 Metode Pembanding

Untuk mengevaluasi efektivitas model *hybrid voting* yang telah dikembangkan, diperlukan metode pembanding yang dapat memberikan tolok ukur performa klasifikasi dalam kondisi dasar (*baseline*). Subbab ini menjelaskan beberapa metode pembanding yang digunakan untuk mengukur efektivitas model *hybrid voting* yang telah dikembangkan. Metode-metode ini mencakup pendekatan klasifikasi dengan strategi dan asumsi yang berbeda. Pada tabel 4.11 dijelaskan keempat metode ini dipilih sebagai pembanding untuk memberikan tolok ukur yang seimbang terhadap model *hybrid voting*. Dengan mencakup pendekatan *supervised*, *unsupervised*, *full-image*, dan tanpa filtrasi *patch*, evaluasi performa model utama dapat dilakukan secara lebih adil, menyeluruh, dan berbasis variasi pendekatan representatif.

Tabel 4.11 Metode Pembanding

Metode Pembanding	Deskripsi	Alasan Perbandingan
SVM Only (tanpa centroid)	Klasifikasi dilakukan menggunakan model SVM berbasis fitur <i>patch</i> hasil <i>clustering</i> , tanpa memperhitungkan jarak ke <i>centroid</i> .	Digunakan sebagai <i>baseline</i> model terawasi (<i>supervised</i>). Mengukur efektivitas model tanpa bantuan informasi kedekatan <i>centroid</i> .
<i>Centroid Only</i> (tanpa SVM)	Prediksi dilakukan berdasarkan jarak <i>patch</i> ke <i>centroid</i> terdekat, tanpa melibatkan prediksi probabilitas dari SVM.	Sebagai <i>baseline</i> model tidak terawasi (<i>unsupervised</i>). Digunakan untuk menilai apakah informasi dari cluster cukup kuat tanpa dukungan klasifikasi <i>supervised</i> .
Klasifikasi Gambar Utuh	Prediksi label dilakukan langsung pada gambar utuh tanpa dipotong menjadi <i>patch</i> , menggunakan model SVM berbasis fitur <i>full image</i> .	Untuk mengetahui sejauh mana model dapat bekerja tanpa proses pemotongan <i>patch</i> , dan apakah pendekatan <i>patch-based</i> benar-benar memberikan keunggulan.
Tanpa Filtrasi <i>Padding</i> (All Patch)	Menggunakan semua <i>patch</i> hasil <i>sliding window</i> , termasuk <i>patch</i> dengan konten <i>padding</i> lebih dari 20%. Klasifikasi model memakai metode <i>hybrid voting</i> .	Digunakan untuk menguji apakah filtrasi <i>patch</i> berdasarkan rasio <i>padding</i> benar-benar memberikan kontribusi signifikan terhadap akurasi.

a) Klasifikasi SVM

Evaluasi terhadap performa model klasifikasi juga dilakukan menggunakan pendekatan *Support Vector Machine* (SVM), tanpa melibatkan mekanisme *hybrid voting* maupun pemanfaatan informasi jarak terhadap pusat kelompok (*centroid*). Pendekatan ini berfungsi sebagai pembanding dasar (*baseline*) untuk menilai sejauh mana kontribusi dari integrasi skor *centroid* dan teknik *voting* dalam meningkatkan akurasi klasifikasi. Pada metode ini, model SVM dilatih sepenuhnya dengan memanfaatkan fitur *patch* yang telah direduksi menggunakan metode *Principal Component Analysis* (PCA), serta telah disesuaikan agar seimbang antara jumlah data dari kelas Jepang dan kelas Cina. Evaluasi dilakukan dalam dua skenario utama, yaitu pada *subset* data uji, serta pada tingkat gambar utuh melalui *voting* mayoritas hasil

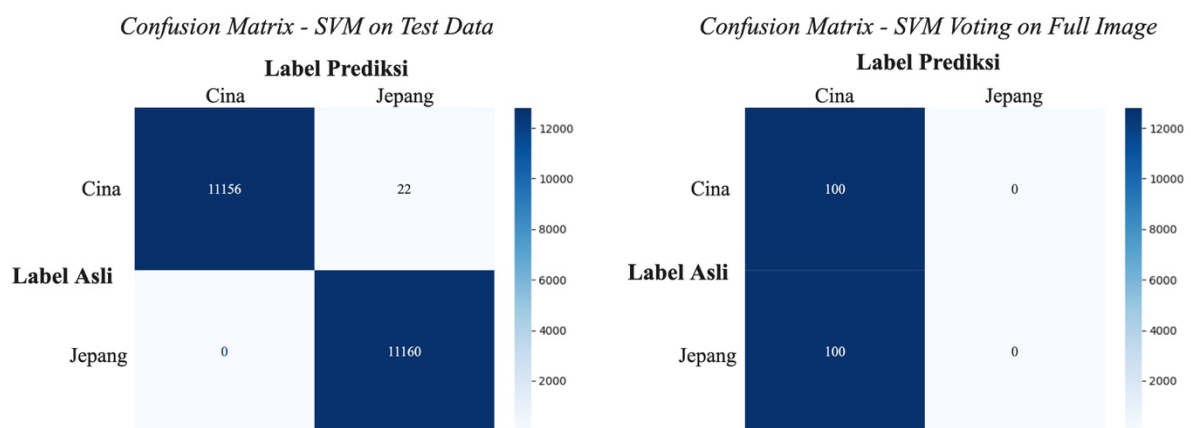
prediksi *patch* dalam satu gambar. Perbedaan antara kedua pendekatan ini menjadi penting untuk menilai sejauh mana kemampuan generalisasi model.

Tabel 4.12 menyajikan ringkasan metrik evaluasi dari kedua skenario tersebut. Berdasarkan hasil evaluasi pada data uji, model SVM menunjukkan performa yang sangat tinggi dengan akurasi sebesar 0,9998. Seluruh metrik utama seperti *precision*, *recall*, dan *F1-score* mencapai nilai hampir sempurna ($\geq 0,9996$) untuk kedua kelas, baik Cina maupun Jepang. Hal ini mengindikasikan bahwa model mampu membedakan ciri visual antar kelas secara konsisten dan akurat ketika diuji pada fitur data uji.

Tabel 4.12 Hasil Evaluasi Klasifikasi *SVM-Only*

Metode	<i>Accuracy</i>	<i>Precision</i> (Cina)	<i>Recall</i> (Cina)	<i>F1-Score</i> (Cina)	<i>Precision</i> (Jepang)	<i>Recall</i> (Jepang)	<i>F1-Score</i> (Jepang)
Data Uji	0,9998	1,0000	0,9996	0,9998	0,9996	1,0000	0,9998
Gambar utuh	0,5000	0,5000	1,0000	0,6667	0,0000	0,0000	0,0000

Namun demikian, ketika evaluasi diperluas ke tingkat gambar utuh, performa model menurun secara drastis. Akurasi keseluruhan hanya mencapai 0,5000, dengan ketimpangan yang signifikan antar kelas. Seluruh gambar dari kelas Jepang gagal diklasifikasikan dengan benar, ditunjukkan oleh nilai *precision*, *recall*, dan *F1-score* sebesar 0,0000. Sementara itu, kelas Cina mencatat nilai *recall* sempurna (1,0000), namun *precision* hanya sebesar 0,5000, karena seluruh prediksi model cenderung condong ke satu kelas. Penurunan ini menandakan bahwa tanpa pendekatan *voting* yang mempertimbangkan distribusi cluster, klasifikasi berbasis SVM saja belum cukup andal untuk skenario gambar utuh.



Gambar 4.14 Hasil *Confusion Matrix* Klasifikasi *SVM-Only*

Gambar 4.14 memperkuat perbedaan performa antara kedua skenario evaluasi model SVM. Pada gambar sebelah kiri, yang menunjukkan *confusion matrix* untuk data uji *patch*, terlihat bahwa model berhasil mengklasifikasikan sebagian besar *patch* dengan benar. Dari total 11.160 *patch* kelas Cina, sebanyak 11.156 berhasil diklasifikasikan secara akurat, dengan hanya 22 *patch* yang salah diklasifikasikan sebagai Jepang. Sementara itu, seluruh *patch* dari kelas Jepang (11.160 *patch*) diklasifikasikan dengan benar tanpa kesalahan. Hal ini menunjukkan bahwa pada level *patch* individual, model SVM menunjukkan kinerja yang sangat baik dalam mengenali fitur visual yang membedakan antara dua kelas. Sebaliknya, gambar sebelah kanan

menunjukkan *confusion matrix* untuk klasifikasi pada tingkat gambar utuh, di mana prediksi dilakukan berdasarkan *voting* mayoritas dari hasil klasifikasi seluruh patch dalam satu gambar. Pada skenario ini, terlihat bahwa seluruh gambar, baik dari kelas Cina maupun Jepang, diklasifikasikan sebagai Cina oleh model. Hal ini menghasilkan nilai *recall* sebesar 1.000 pada kelas Cina, namun 0.000 pada kelas Jepang, serta tidak adanya gambar Jepang yang berhasil dikenali. Kecenderungan model untuk mengklasifikasikan seluruh gambar ke satu kelas menunjukkan kelemahan pendekatan ini dalam menangani representasi gambar secara menyeluruh, terutama karena tidak ada pertimbangan terhadap sebaran atau struktur patch dalam satu gambar. Temuan ini menegaskan bahwa meskipun model SVM mampu bekerja dengan sangat baik pada data patch individu, namun model tidak dapat diandalkan untuk klasifikasi pada tingkat gambar utuh jika tidak didukung oleh mekanisme tambahan seperti *hybrid voting* atau *filtering patch*.

b) Klasifikasi Berbasis *Centroid*

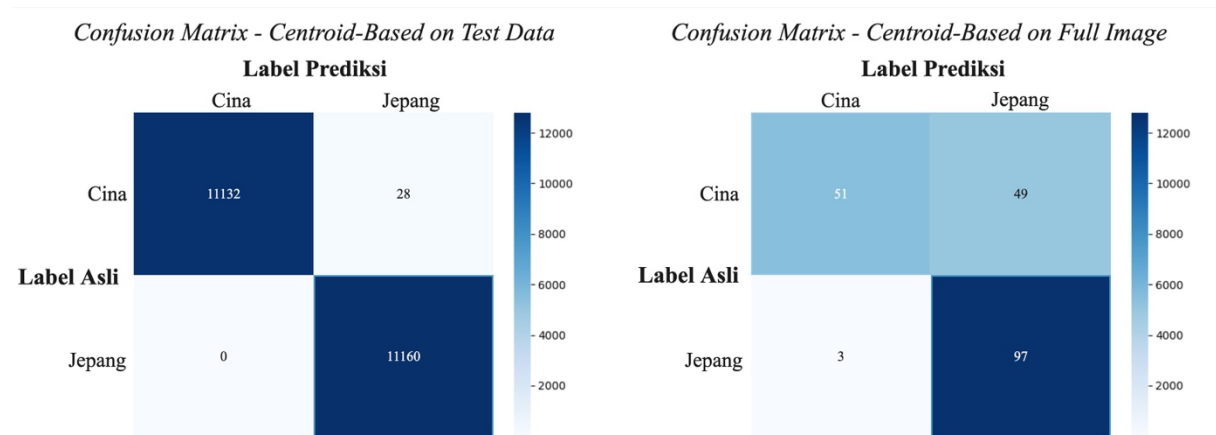
Pada pendekatan ini, proses klasifikasi dilakukan berdasarkan jarak fitur terhadap *centroid* hasil *discriminative clustering*. Prediksi dilakukan dengan memanfaatkan pemetaan posisi fitur terhadap pusat kluster yang telah terbentuk sebelumnya, sehingga proses klasifikasi sepenuhnya bergantung pada kedekatan antara fitur dan *centroid* yang mewakili kategori Jepang maupun Cina. Evaluasi dilakukan pada dua skenario, yaitu klasifikasi pada tingkat *patch* (*dataset uji*) dan klasifikasi pada tingkat gambar utuh. Pada skenario pertama, model diuji menggunakan *subset* data uji. Setiap *patch* dibandingkan terhadap lima *centroid* hasil *clustering*, dengan cara menghitung jarak *cosine* dan menentukan label berdasarkan skor rerata terhadap *centroid* Jepang (*cluster* 0, 2, dan 3) dan Cina (*cluster* 1 dan 4). Jika skor untuk Jepang lebih besar, maka *patch* diprediksi sebagai Jepang, dan sebaliknya. Berdasarkan evaluasi menggunakan *confusion matrix* dan *classification report*, model berhasil mencapai akurasi sebesar 0,9987, dengan *precision* dan *recall* di atas 0,9975 untuk kedua kelas. Nilai *F1-score* pun sangat tinggi, yaitu sebesar 0,9987 untuk kelas Cina dan 0,9987 untuk kelas Jepang, sebagaimana ditunjukkan pada Tabel 4.13 baris pertama.

Tabel 4.13 Hasil Evaluasi Klasifikasi Berbasis *Centroid*

Metode	<i>Accuracy</i>	<i>Precision</i> (Cina)	<i>Recall</i> (Cina)	<i>F1-Score</i> (Cina)	<i>Precision</i> (Jepang)	<i>Recall</i> (Jepang)	<i>F1-Score</i> (Jepang)
Data Uji	0,9987	1,0000	0,9975	0,9987	0,9975	1,0000	0,9987
Gambar utuh	0,7400	0,9444	0,5100	0,6623	0,6644	0,9700	0,7886

Namun, ketika model diuji pada skenario klasifikasi gambar utuh, performanya menurun cukup drastis. Proses pengujian dilakukan dengan mengekstrak patch dari 100 gambar Cina dan 100 gambar Jepang. *Patch* yang lolos proses validasi diekstraksi fiturnya, direduksi menggunakan PCA, dan diklasifikasikan. Label akhir dari setiap gambar kemudian ditentukan melalui *voting* mayoritas dari hasil klasifikasi *patch*. Berdasarkan evaluasi, akurasi hanya mencapai 0,7400. Meskipun *recall* untuk kelas Jepang sangat tinggi (0,9700), nilai *precision*-nya lebih rendah yaitu 0,6644. Sebaliknya, kelas Cina menunjukkan *recall* rendah sebesar 0,5100, meskipun *precision*-nya cukup tinggi (0,9444). *F1-score* masing-masing adalah 0,6623 untuk Cina dan 0,7886 untuk Jepang. Hasil ini menunjukkan bahwa pendekatan berbasis

centroid masih memiliki keterbatasan dalam menangkap representasi visual secara menyeluruh pada skala gambar utuh, terutama dalam konteks distribusi dan dominansi fitur antar *patch* dalam satu citra.



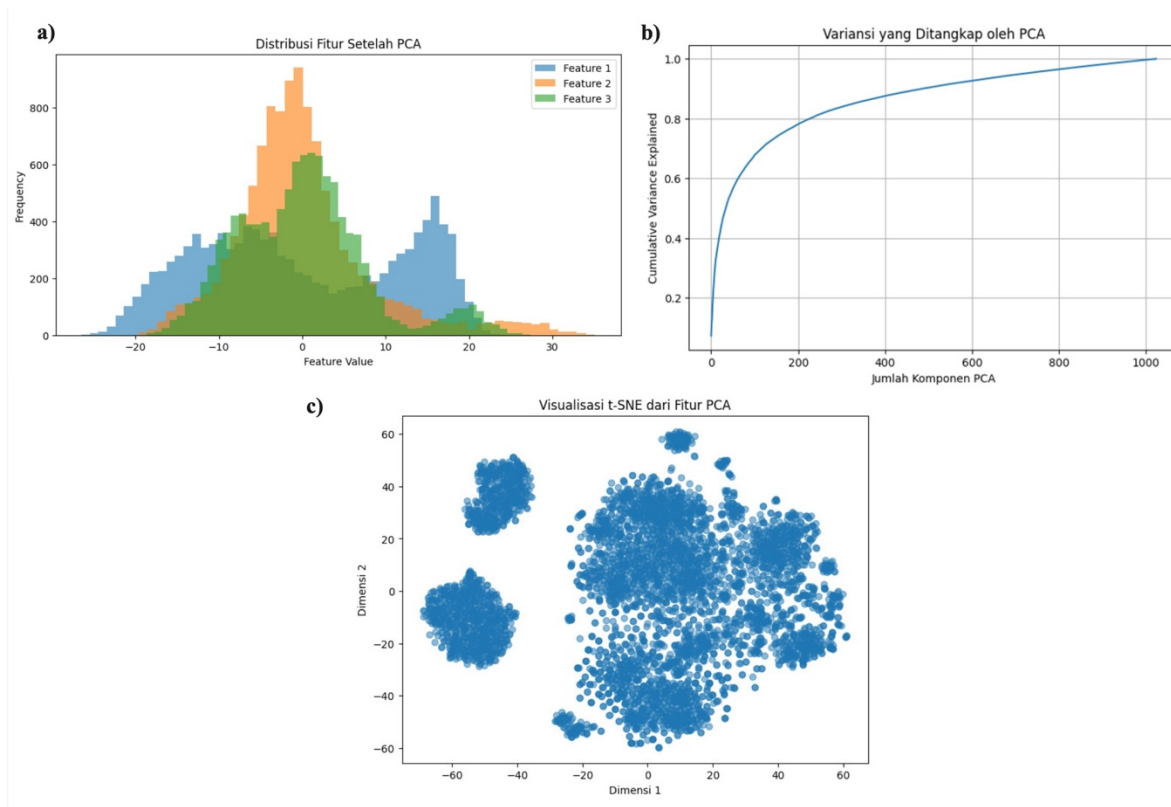
Gambar 4.15 Hasil *Confusion Matrix* Klasifikasi Berbasis *Centroid*

Gambar 4.15 menampilkan matriks *confusion matrix* dari hasil evaluasi model klasifikasi berbasis *centroid* dalam dua skenario berbeda, yaitu pada data uji *patch* dan pada gambar utuh. Pada bagian kiri (data uji *patch*), model menunjukkan performa klasifikasi yang sangat tinggi dan seimbang antar kelas. Dari total 11.160 *patch* kelas Cina, sebanyak 11.132 berhasil diklasifikasikan dengan benar, sedangkan hanya 28 yang salah diklasifikasikan sebagai Jepang. Untuk kelas Jepang, model berhasil mengklasifikasikan seluruh *patch* dengan benar (11.160 *patch*), tanpa satu pun kesalahan. Hal ini menghasilkan nilai *recall* sempurna untuk kelas Jepang dan nilai *precision* yang sangat tinggi untuk kedua kelas. Evaluasi ini menunjukkan bahwa pendekatan berbasis *centroid* mampu mengenali fitur visual dari *patch* individu secara baik ketika menggunakan representasi cluster yang tepat. Sebaliknya, bagian kanan (data gambar utuh) menunjukkan hasil klasifikasi terhadap 100 gambar Cina dan 100 gambar Jepang menggunakan pendekatan mayoritas *voting* dari *patch* yang valid. Dari hasil tersebut, model berhasil mengklasifikasikan 51 gambar Cina dengan benar dan 49 sisanya salah diklasifikasikan sebagai Jepang. Pada kelas Jepang, 97 gambar berhasil diklasifikasikan dengan benar, sementara 3 sisanya salah diklasifikasikan sebagai Cina. Hasil ini menunjukkan bahwa performa model menurun ketika berpindah dari klasifikasi pada tingkat *patch* ke tingkat gambar utuh. Meskipun akurasi pada kelas Jepang tetap tinggi, performa model pada kelas Cina menurun cukup tajam, sebagaimana terlihat dari hanya 51 gambar yang berhasil dikenali secara benar. Hal ini mengindikasikan bahwa pendekatan berbasis *centroid* memiliki keterbatasan dalam menangkap representasi gambar, terutama untuk kelas dengan sebaran fitur yang lebih beragam seperti lukisan Cina.

c) Model dan Klasifikasi Gambar Utuh

Pendekatan klasifikasi pada tingkat gambar utuh dilakukan tanpa melibatkan tahap klasifikasi per *patch* seperti pada metode sebelumnya. Meskipun demikian, proses awal yang dilakukan tetap melalui tahapan praproses, ekstraksi fitur menggunakan model konvolusional, dan reduksi dimensi menggunakan *Principal Component Analysis* (PCA). Pendekatan ini bertujuan untuk memperoleh representasi global dari setiap gambar melalui fitur utuh yang dihasilkan dari proses ekstraksi dan reduksi dimensi. Analisis terhadap distribusi dan kualitas

fitur hasil reduksi PCA disajikan pada Gambar 4.16. Gambar 4.16 menyajikan hasil analisis ekstraksi fitur dari *dataset* gambar positif dan negatif setelah dilakukan reduksi dimensi menggunakan PCA. Subgambar (a) menunjukkan histogram distribusi nilai dari tiga komponen fitur pertama. Terlihat bahwa ketiga komponen memiliki sebaran nilai yang relatif simetris di sekitar nol, menunjukkan bahwa data telah melalui proses normalisasi dan mengandung keragaman nilai yang cukup baik untuk mendukung klasifikasi. Subgambar (b) memperlihatkan kurva akumulatif varian yang dijelaskan oleh PCA terhadap jumlah komponen utama yang digunakan. Berdasarkan grafik tersebut, terlihat bahwa sekitar 90% informasi dalam data dapat direpresentasikan dengan kurang dari 200 komponen utama, yang menandakan efisiensi tinggi dalam proses reduksi dimensi tanpa kehilangan informasi penting secara signifikan. Selanjutnya, subplot (c) menampilkan visualisasi distribusi fitur menggunakan metode *t-distributed Stochastic Neighbor Embedding* (t-SNE) ke dalam ruang dua dimensi. Dari hasil visualisasi tersebut tampak bahwa terdapat pola-pola pengelompokan yang terbentuk secara alami, yang mengindikasikan bahwa fitur hasil ekstraksi memiliki struktur semantik dan potensi diskriminatif yang baik. Hal ini menunjukkan bahwa fitur yang diperoleh dapat dimanfaatkan untuk mendukung proses klasifikasi gambar utuh secara lebih efektif.



Gambar 4.16 Hasil Ekstraksi Fitur Setelah PCA

Fitur hasil ekstraksi kemudian digunakan sebagai input untuk pelatihan model klasifikasi SVM. Untuk menjaga keseimbangan antara kelas, dilakukan undersampling sehingga jumlah gambar Jepang dan Cina yang digunakan dalam pelatihan seimbang. Data kemudian dibagi menjadi data latih dan data uji dengan rasio 80:20 secara stratified. Model SVM dilatih menggunakan kernel RBF dengan konfigurasi default ($C=1.0$, $\gamma=\text{'scale'}$). Model SVM dilatih pada fitur gambar utuh yang telah direduksi, dengan data yang telah diseimbangkan

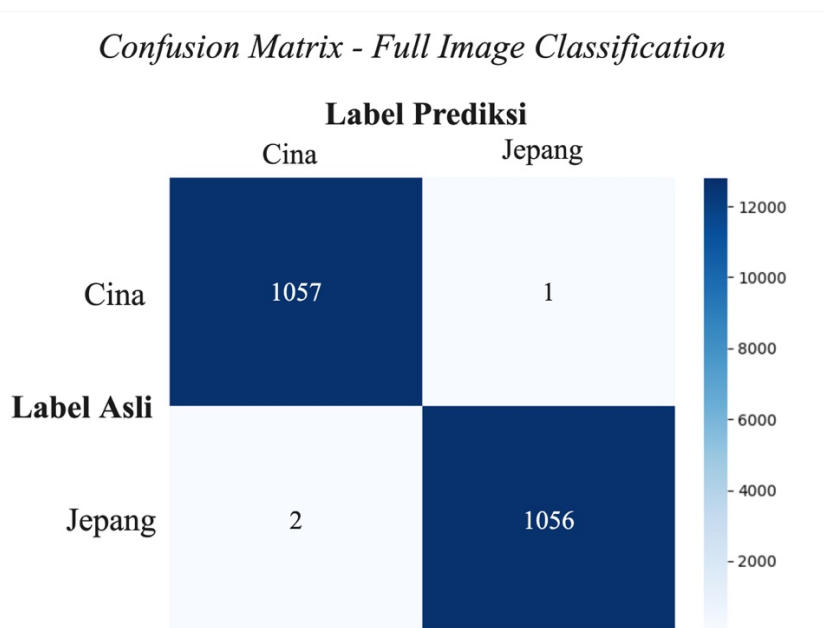
antara kedua kelas. Evaluasi dilakukan pada 2.116 gambar uji yang terdiri dari 1.058 gambar Cina dan 1.058 gambar Jepang. Hasil evaluasi disajikan pada Tabel 4.13 dan Gambar 4.18.

Berdasarkan Tabel 4.14, model menunjukkan performa klasifikasi yang sangat tinggi dan seimbang antara kedua kelas. Nilai *precision* untuk kelas Cina mencapai 0,9981 dan *recall* sebesar 0,9991, sedangkan kelas Jepang memiliki *precision* sebesar 0,9991 dan *recall* 0,9981. Kedua kelas memiliki nilai F1-score yang identik, yaitu 0,9986. Akurasi keseluruhan model juga sangat tinggi, yaitu 0,9986. Hasil ini menunjukkan bahwa model mampu mengklasifikasikan gambar utuh secara akurat, tanpa mengalami bias signifikan terhadap salah satu kelas.

Tabel 4.14 Hasil Evaluasi Klasifikasi Gambar Utuh

Metrik	Cina	Jepang	Rata-Rata
<i>Precision</i>	0,9981	0,9991	0,9986
<i>Recall</i>	0,9991	0,9981	0,9986
<i>F1-Score</i>	0,9986	0,9986	0,9986
Akurasi	-	-	0,9986

Gambar 4.17 menunjukkan hasil *confusion matrix* dari proses evaluasi model klasifikasi terhadap gambar utuh menggunakan pendekatan SVM berbasis fitur global. Dari total 1.058 gambar Cina, model berhasil mengklasifikasikan 1.057 gambar secara benar, dengan hanya 1 gambar yang salah diklasifikasikan sebagai Jepang. Sementara itu, dari 1.058 gambar Jepang, sebanyak 1.056 berhasil diklasifikasikan dengan benar, dan hanya 2 gambar yang salah diklasifikasikan sebagai Cina. Distribusi prediksi yang ditampilkan menunjukkan keseimbangan yang sangat baik antara kedua kelas. Tingkat kesalahan yang sangat kecil (masing-masing hanya 1 atau 2 gambar salah klasifikasi) mengindikasikan bahwa model memiliki performa tinggi dan mampu membedakan representasi visual antara lukisan Cina dan Jepang secara konsisten pada tingkat gambar utuh. Hal ini juga sesuai dengan hasil metrik evaluasi pada Tabel 4.13, di mana *precision*, *recall*, dan *F1-score* untuk kedua kelas mencapai angka yang hampir sempurna (0,9986), serta akurasi keseluruhan sebesar 0,9986. Dengan demikian, model SVM menunjukkan kinerja yang baik dalam skenario klasifikasi akhir terhadap gambar utuh.



Gambar 4.17 Hasil Evaluasi Klasifikasi Gambar Utuh



Gambar 4.18 Visualisasi Contoh Klasifikasi yang Akurat

Gambar 4.18 dan Gambar 4.19 menyajikan contoh visualisasi hasil klasifikasi gambar utuh oleh model, baik yang berhasil diklasifikasikan dengan benar maupun yang mengalami kesalahan. Gambar 4.18 memperlihatkan empat contoh klasifikasi yang akurat, yaitu dua gambar Jepang dan dua gambar Cina yang berhasil dikenali sesuai label aslinya. Gambar Jepang pada bagian kiri menunjukkan gaya khas *ukiyo-e*, seperti penggambaran manusia dan tekstur kertas serta tulisan kanji. Sementara itu, gambar Cina menampilkan gambar ikan koi dan lanskap bunga dengan gaya pewarnaan dan komposisi yang lebih beragam, mencerminkan estetika lukisan *shan shui*. Keempat gambar ini menunjukkan bahwa model mampu mengidentifikasi elemen visual dominan yang mencirikan masing-masing kategori dengan tepat.



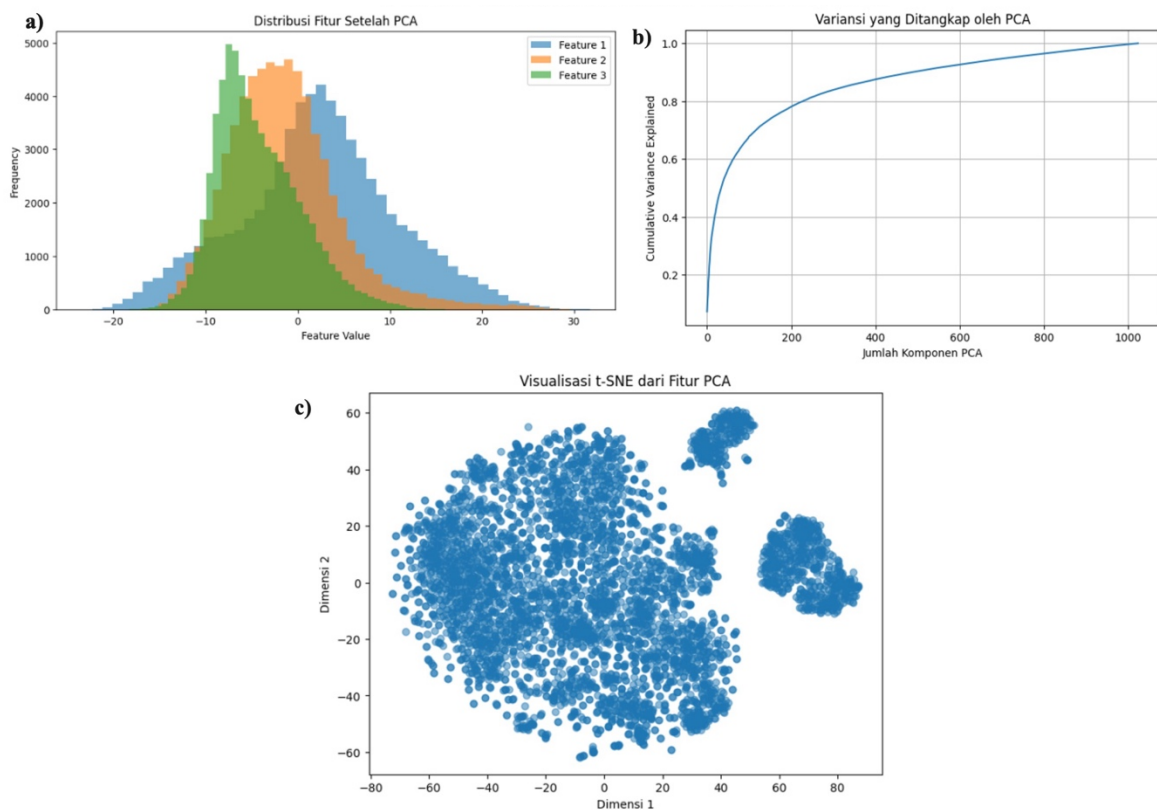
Gambar 4.19 Visualisasi Contoh Klasifikasi yang Tidak Benar

Sebaliknya, Gambar 4.19 menampilkan contoh kasus kesalahan klasifikasi oleh model. Pada sisi kiri dan tengah, ditunjukkan dua contoh gambar lukisan Jepang yang salah diklasifikasikan sebagai lukisan Cina. Meskipun kedua gambar tersebut berasal dari kategori Jepang, bentuk visual seperti garis batang bambu yang dominan serta penggunaan tinta hitam dengan kontras tinggi menyerupai karakteristik umum dalam lukisan Cina, khususnya gaya *shan shui* atau tinta bambu. Hal ini menyebabkan model mengasosiasikannya lebih dekat dengan distribusi visual lukisan Cina dalam ruang fitur. Sebaliknya, pada sisi kanan diperlihatkan contoh lukisan Cina yang justru diklasifikasikan sebagai lukisan Jepang. Motif bambu dan teknik kuas dalam lukisan tersebut menampilkan kemiripan visual yang tinggi

dengan gaya sumi-e Jepang, yang juga menggunakan sapuan tinta monokromatik dan komposisi sederhana. Ketiga contoh ini menunjukkan bahwa model mengalami kesulitan dalam membedakan elemen visual yang bersifat lintas budaya, terutama ketika representasi visual kedua kelas memiliki gaya yang tumpang tindih atau serupa. Hal ini menjadi indikasi perlunya pendekatan tambahan yang mempertimbangkan konteks global atau metadata pendukung untuk meningkatkan akurasi klasifikasi.

d) Model dan Klasifikasi Tanpa Filtrasi *Patch*

Subbab ini membahas metode pembandingan klasifikasi gambar berbasis *patch* yang dilakukan tanpa menerapkan tahap filtrasi. Berbeda dengan pendekatan utama sebelumnya yang menyaring *patch* berdasarkan parameter valid (seperti rasio padding hitam, standar deviasi warna, dan jumlah tepi), pada metode ini seluruh *patch* yang dihasilkan dari proses *sliding window* digunakan secara langsung tanpa pengecualian. Tujuan dari eksperimen ini adalah untuk mengevaluasi sejauh mana pengaruh dari *patch* tidak valid terhadap hasil klasifikasi akhir. Sebelum dilakukan pelatihan model klasifikasi, seluruh *patch* dari *dataset* positif (lukisan Jepang) dan negatif (lukisan Cina) diekstraksi fitur visualnya menggunakan model konvolusional dan selanjutnya direduksi dimensinya menggunakan *Principal Component Analysis (PCA)*. Hasil analisis distribusi dan kualitas fitur setelah *PCA* ditampilkan pada Gambar 4.20 dan Gambar 4.21.

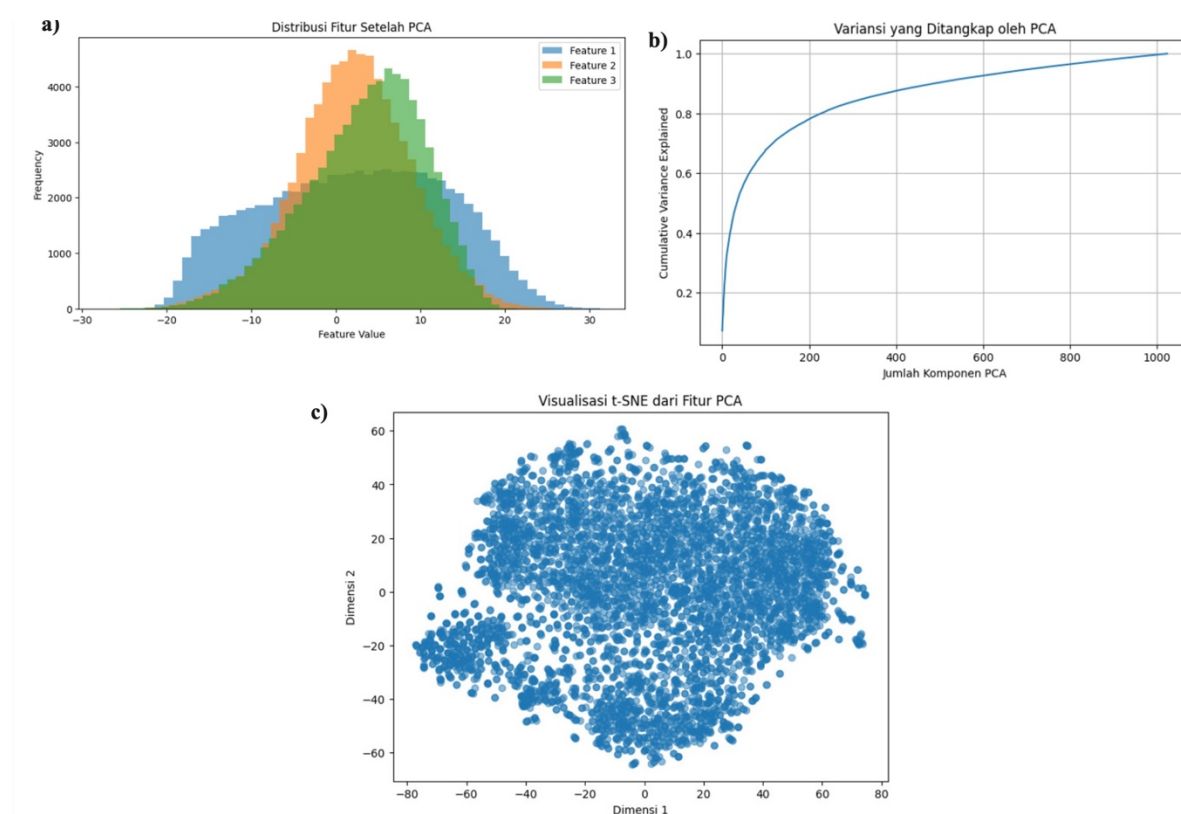


Gambar 4.20 Analisis Ekstraksi Fitur *Dataset* Positif Setelah PCA

Gambar 4.20 menampilkan analisis hasil ekstraksi fitur dari *dataset* positif (lukisan Jepang) setelah melalui proses *PCA*. Subgambar (a) menunjukkan distribusi dari tiga komponen hasil reduksi dimensi, yang tampak tersebar simetris dengan rentang yang lebar dan sebaran yang tidak terlalu saling tumpang tindih. Hal ini menunjukkan bahwa fitur dari *patch* lukisan Jepang

memiliki keragaman nilai yang cukup baik dan saling berbeda, yang dapat meningkatkan kemampuan pemisahan antar kelas saat proses klasifikasi. Subgambar (b) menampilkan kurva kumulatif varians yang dijelaskan oleh *PCA* terhadap jumlah komponen. Terlihat bahwa sekitar 100% informasi dalam data dapat ditangkap dengan memakai 1000 komponen, yang menunjukkan bahwa proses reduksi dimensi tetap efektif dalam mempertahankan informasi penting. Visualisasi pada subplot (c) menunjukkan pemetaan fitur ke dalam dua dimensi menggunakan teknik *t-distributed Stochastic Neighbor Embedding (t-SNE)*. Dari visualisasi tersebut tampak bahwa terdapat beberapa kelompok atau kluster yang terbentuk secara alami, yang mengindikasikan bahwa fitur dari *patch* positif masih membentuk struktur yang cukup terlihat.

Sementara itu, Gambar 4.21 menyajikan hasil analisis terhadap *dataset* negatif (lukisan Cina). Subgambar (a) menunjukkan distribusi nilai fitur yang tampak lebih tumpang tindih antar komponen dibandingkan dengan *dataset* positif, yang mengindikasikan distribusi fitur yang lebih *homogen*. Subgambar (b) menunjukkan bahwa sebagian besar informasi juga dapat ditangkap oleh kurang dari 200 komponen utama dan informasi dapat ditangkap semua jika jumlah komponen 1000, serupa dengan *dataset* positif. Namun, visualisasi *t-SNE* pada subplot (c) menunjukkan bahwa penyebaran fitur dari *patch* negatif tanpa filtrasi tidak berbeda secara signifikan dibandingkan dengan hasil setelah dilakukan filtrasi.

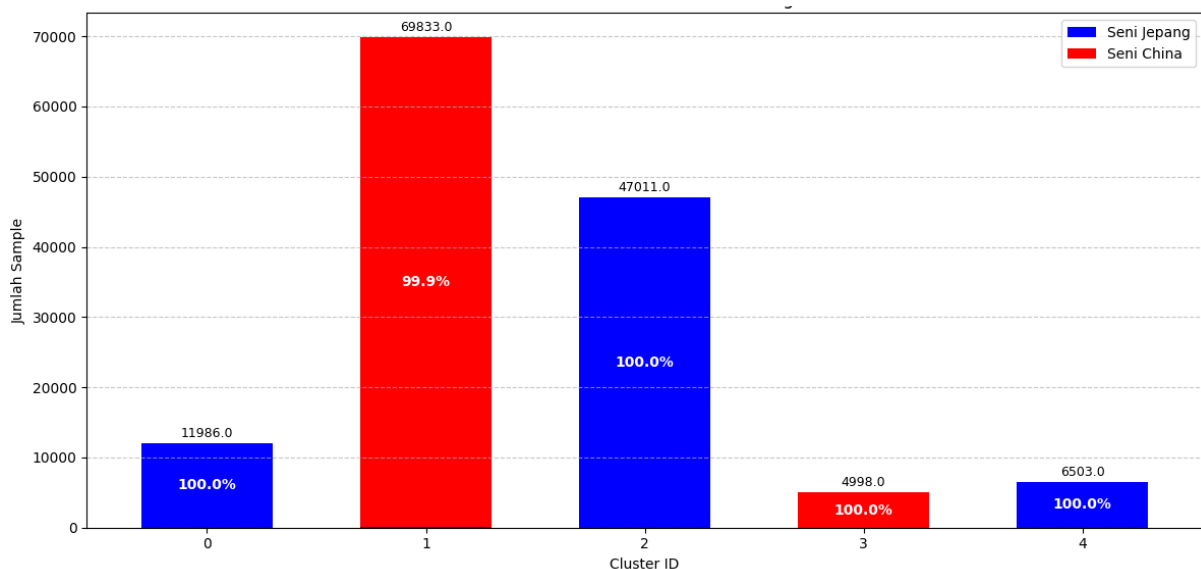


Gambar 4.21 Analisis Ekstraksi Fitur *Dataset* Negatif Setelah PCA

Pola distribusi titik tetap menunjukkan persebaran yang merata tanpa pembentukan kluster dominan, mengindikasikan bahwa proses ekstraksi fitur oleh model konvolusional dan reduksi dimensi melalui *PCA* masih dapat mempertahankan struktur fitur meskipun tanpa tahap penyaringan *patch*. Hal ini menunjukkan bahwa keberadaan *patch* dengan area padding yang

besar tidak secara drastis memengaruhi karakteristik distribusi fitur secara keseluruhan. Meski demikian, filtrasi tetap dilakukan untuk menjamin bahwa hanya *patch* dengan visual yang representatif dapat digunakan dalam proses pelatihan dan pengujian model, terutama untuk mencegah gangguan dari informasi yang tidak bermakna.

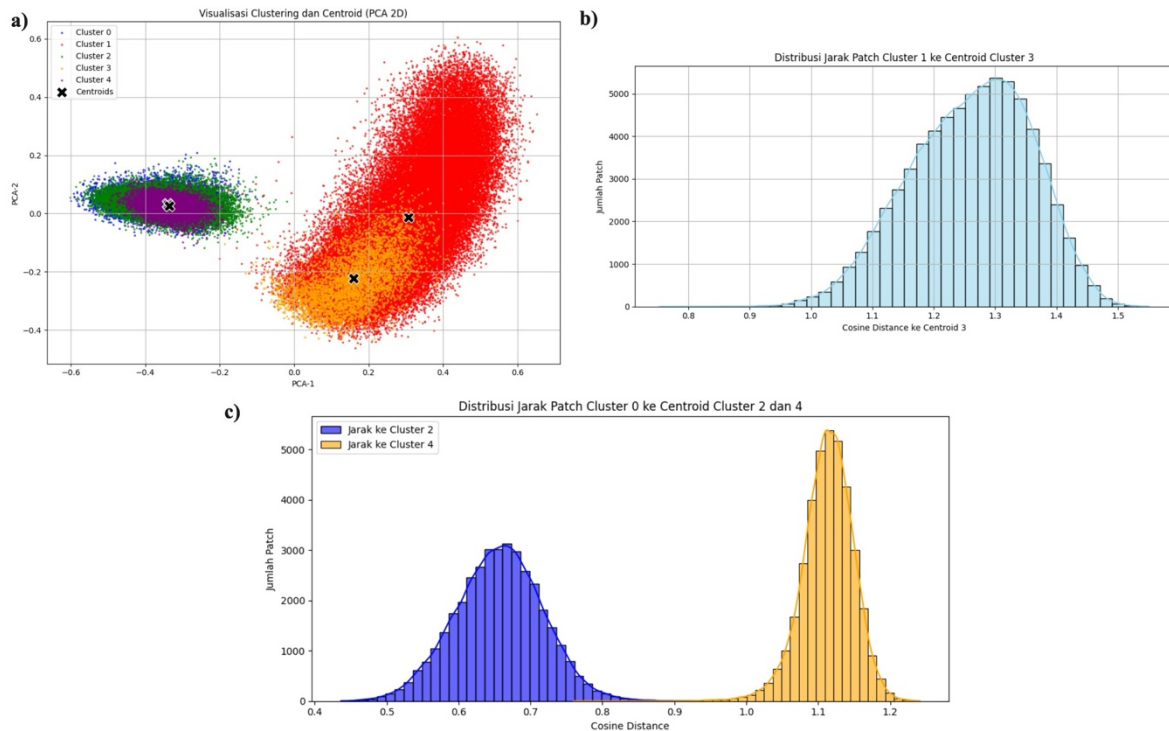
Gambar 4.22 menunjukkan hasil *discriminative clustering* yang dilakukan terhadap keseluruhan data *patch* dari lukisan Jepang dan Cina tanpa penerapan tahap filtrasi. Proses ini menghasilkan lima *cluster* utama dengan total 140.331 fitur, terdiri atas 65.552 fitur dari seni Jepang (label positif) dan 74.779 fitur dari seni Cina (label negatif). Hasil distribusi menunjukkan bahwa sebagian besar *cluster* memiliki dominasi yang sangat tinggi terhadap salah satu kelas, dengan tingkat kemurnian (*purity*) mendekati atau mencapai 100%. *Cluster* 0, 2, dan 4 sepenuhnya terdiri dari fitur-fitur milik lukisan Jepang, masing-masing berjumlah 11.986, 47.011, dan 6.503 fitur, dengan proporsi 100% seni Jepang dan 0% seni Cina. Sementara itu, *cluster* 1 dan 3 didominasi oleh fitur dari lukisan Cina, dengan jumlah 69.833 dan 4.998 fitur. *Cluster* 1 hanya mengandung 50 fitur dari lukisan Jepang (0,07%) dan sisanya dari lukisan Cina (99,93%), sedangkan *cluster* 3 hanya terdiri dari 2 fitur Jepang (0,04%) dan 4.996 fitur Cina (99,96%). Tingginya kemurnian pada setiap *cluster* menunjukkan bahwa metode *discriminative clustering* mampu mengelompokkan fitur berdasarkan pola visual yang sangat khas dan konsisten antara kedua kategori lukisan. Hal ini mengindikasikan bahwa fitur hasil ekstraksi mampu memisahkan domain visual antara seni Jepang dan Cina secara signifikan, bahkan tanpa bantuan label selama proses *clustering*. Namun demikian, dominasi *cluster* yang terlalu absolut juga mengisyaratkan potensi ketidakseimbangan distribusi yang perlu diantisipasi dalam tahap pelatihan model klasifikasi selanjutnya, terutama jika ada *cluster* minoritas dengan jumlah fitur yang jauh lebih sedikit.



Gambar 4.22 Hasil *Discriminative Clustering* dengan Tidak Memakai Filter

Gambar 4.23 menyajikan hasil analisis *discriminative clustering*. Visualisasi pada subgambar (a) menunjukkan hasil pemetaan dua dimensi menggunakan PCA terhadap fitur *patch*, yang dikelompokkan ke dalam lima *cluster* hasil *clustering*. Pada gambar tersebut, *cluster* yang berwarna merah merupakan *cluster* 1, yang hampir seluruhnya didominasi oleh lukisan Cina, sementara *cluster* berwarna kuning merupakan *cluster* 3, yang juga berasal dari data Cina. Kedua *cluster* ini tampak terletak berjauhan dari tiga *cluster* lainnya,

mengindikasikan bahwa fitur-fitur dari lukisan Cina membentuk ruang tersendiri yang berbeda dari fitur lukisan Jepang. Sementara itu, tiga *cluster* sisanya (berwarna ungu, hijau, dan biru) masing-masing mewakili *cluster* 4, 2, dan 0, yang berasal dari lukisan Jepang. Ketiganya membentuk satu wilayah yang cukup padat dan saling berdekatan di ruang PCA dua dimensi. Keberdekatan ini menunjukkan bahwa meskipun fitur dari lukisan Jepang tersebar ke dalam beberapa *cluster*, distribusinya tetap berada dalam satu tempat, menandakan kemiripan representasi visual antar gaya seni Jepang.

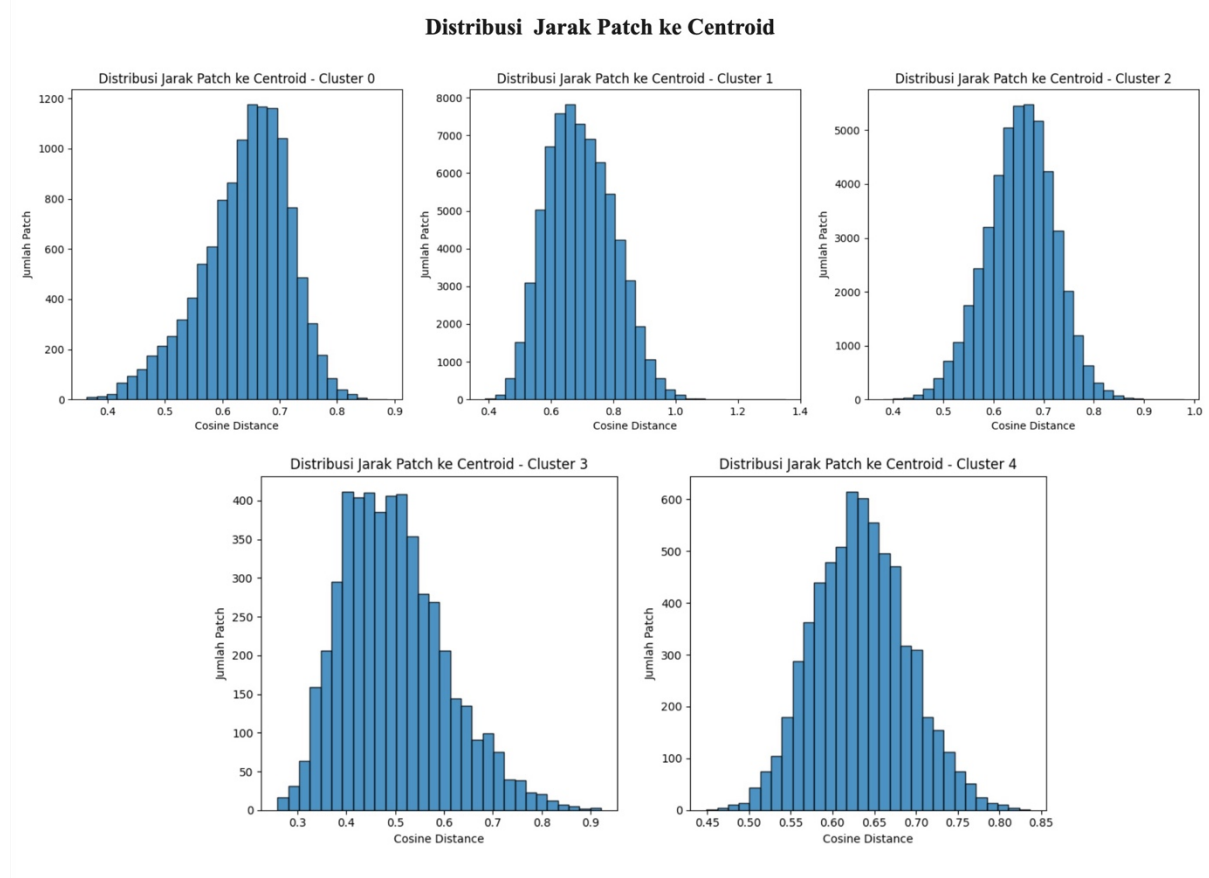


Gambar 4.23 Analisis Hasil *Discriminative Clustering* Tanpa Menggunakan *Padding*

Subgambar (b) pada Gambar 4.25 menunjukkan distribusi jarak *cosine* antara *patch* yang tergolong dalam *cluster* 1 (mayoritas seni lukis Cina) terhadap *centroid* dari *cluster* 3 (juga representatif seni lukis Cina). Berdasarkan grafik histogram, terlihat bahwa sebagian besar jarak berada pada rentang antara 1,1 hingga 1,4, dengan puncak distribusi berkisar di nilai 1,3. Nilai ini menandakan bahwa meskipun keduanya merepresentasikan kelas yang sama, yaitu seni lukis Cina, terdapat perbedaan yang cukup signifikan secara spasial dalam ruang fitur setelah reduksi dimensi. Hal ini memperkuat bahwa representasi visual antar *cluster* 1 dan *cluster* 3 tidak identik, meskipun keduanya didominasi oleh kelas yang sama.

Subgambar (c) pada menyajikan distribusi jarak *cosine* antara *patch* yang tergolong dalam *cluster* 0 (dominan lukisan Jepang) terhadap dua *centroid* lain yang juga merepresentasikan lukisan Jepang, yaitu *cluster* 2 dan *cluster* 4. Hasil distribusi menunjukkan dua kurva yang terpisah secara jelas, dengan jarak rata-rata ke *centroid cluster* 2 berada di kisaran 0,6 hingga 0,8, sedangkan jarak ke *centroid cluster* 4 berada pada rentang yang lebih tinggi, sekitar 1,0 hingga 1,2. Jarak yang relatif jauh dan distribusi yang tidak tumpang tindih ini menunjukkan bahwa *cluster* 0 lebih dekat secara semantik ke *cluster* 2 daripada ke *cluster* 4. Temuan ini menegaskan bahwa meskipun ketiganya merupakan representasi dari kelas yang sama (lukisan Jepang), terdapat perbedaan yang direpresentasikan oleh masing-masing *cluster*. Hal ini

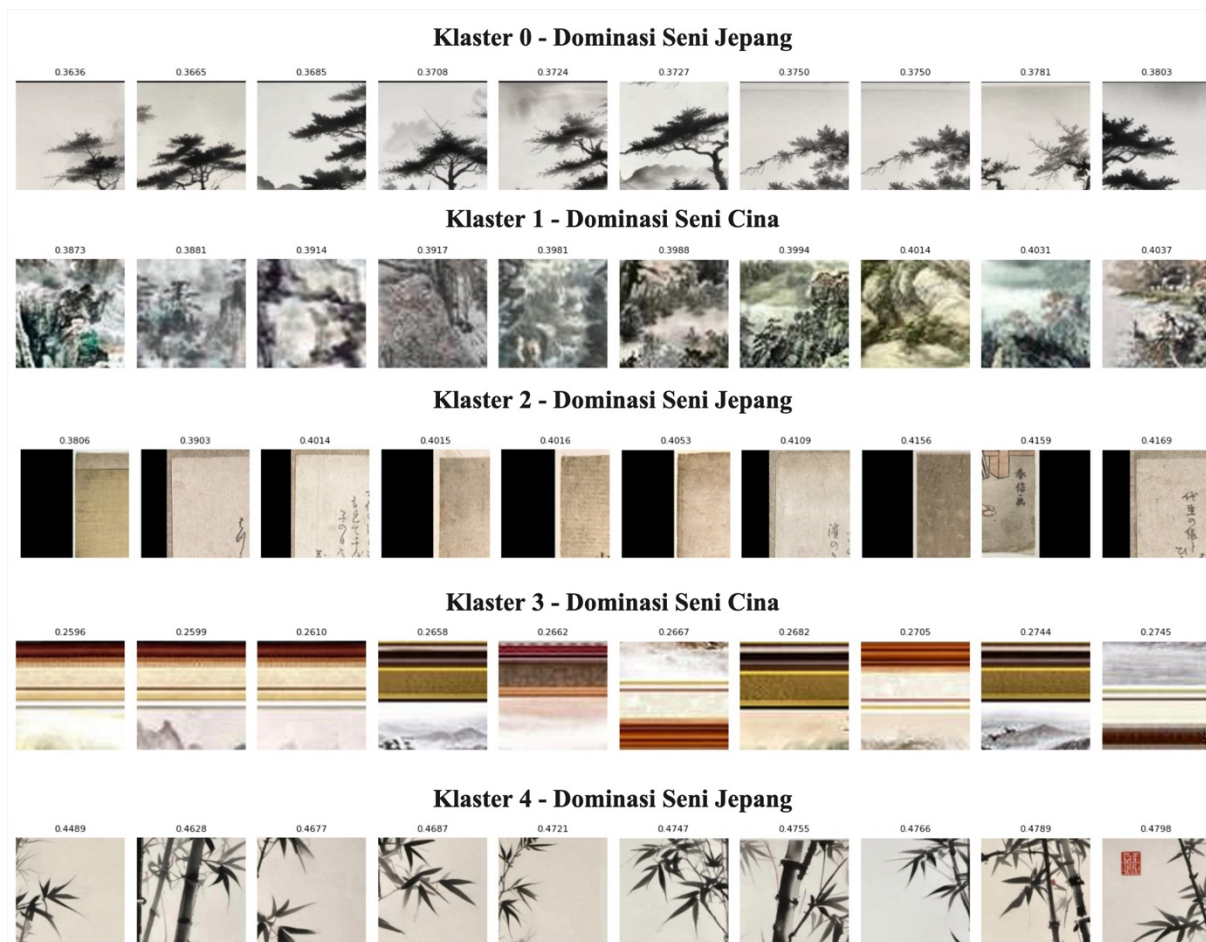
mengindikasikan adanya subkategori atau variasi gaya visual dalam lukisan Jepang yang berhasil diidentifikasi oleh metode *clustering*.



Gambar 4.24 Distribusi Jarak *Patch* Ke *Centroid*

Gambar 4.24 menyajikan histogram distribusi jarak *cosine* antara setiap *patch* dalam *dataset* terhadap *centroid* masing-masing *cluster* hasil proses *discriminative clustering*. Tujuan dari visualisasi ini adalah untuk mengamati pola penyebaran internal dari setiap *cluster*, serta mengidentifikasi tingkat atau keragaman (*cohesiveness*) yang dimiliki oleh tiap-tiap *cluster*. Distribusi jarak pada *cluster* 0, 2, dan 4 — yang ketiganya didominasi oleh lukisan Jepang cenderung memiliki pola yang simetris dengan rentang jarak yang sempit. Hal ini menunjukkan bahwa *patch* dalam ketiga *cluster* tersebut terdistribusi secara konsisten di sekitar *centroid*-nya masing-masing, yang mengindikasikan keragaman yang baik. Terutama pada *cluster* 2 dan 4, puncak distribusi berada di kisaran jarak *cosine* 0.65 hingga 0.7, memperkuat bahwa keduanya merepresentasikan subgaya visual yang cukup stabil dalam kategori lukisan Jepang. Sementara itu, distribusi jarak pada *cluster* 1 dan 3 yang masing-masing didominasi oleh lukisan Cina juga menunjukkan pola simetris, meskipun dengan rentang distribusi yang sedikit lebih lebar. Hal ini menunjukkan bahwa fitur dari *patch* dalam *cluster* lukisan Cina lebih beragam, namun tetap terpusat di sekitar representasi utama yang ditentukan oleh *centroid*. Rentang *cosine distance* untuk *cluster* 1 bahkan mencapai lebih dari 1.0, yang berarti bahwa variasi dalam kluster lukisan Cina cukup tinggi dibandingkan dengan lukisan Jepang. Secara keseluruhan, visualisasi distribusi jarak terhadap *centroid* dalam Gambar 4.24 memberikan gambaran yang penting mengenai seberapa representatif suatu *cluster*. Informasi ini menjadi krusial dalam tahap klasifikasi selanjutnya, karena *cluster* dengan penyebaran yang sempit dan puncak distribusi yang jelas cenderung menghasilkan keputusan klasifikasi yang lebih baik.

Gambar 4.25 menyajikan sepuluh *patch* yang memiliki jarak cosine terdekat terhadap masing-masing centroid dari lima cluster hasil *discriminative clustering*. Visualisasi ini bertujuan untuk memberikan gambaran terhadap karakteristik paling mewakili setiap *cluster*. Dengan mengamati *patch* yang terdekat ke *centroid*, dapat dianalisis karakteristik dalam setiap klaster. Klaster 0, yang didominasi oleh seni Jepang, menampilkan *patch* dengan elemen khas seperti latar bertekstur kertas, tinta halus, dan tulisan kaligrafi yang merupakan ciri dari lukisan *sumi-e*. Klaster 1, yang didominasi oleh seni Cina, memuat *patch* lanskap pemandangan menunjukkan bentuk pegunungan, pepohonan, dan elemen lanskap lainnya yang umum dijumpai dalam lukisan *shan shui* (pemandangan alam khas Cina). Klaster 2 juga mewakili seni Jepang, dengan *patch* bertulisan kanji dan tekstur kertas. Gaya ini identik dengan *ukiyo-e* yang memiliki gaya tulisan kanji dan tekstur kertas yang unik. Keberadaan elemen tulisan juga memperkuat kemiripan dengan seni tradisional Jepang. Klaster 3 didominasi oleh seni Cina, dengan *patch* yang menampilkan pola horizontal berupa garis-garis seperti bagian bingkai lukisan. Terakhir, Klaster 4 kembali menampilkan dominasi seni Jepang, khususnya dengan gambar bambu berwarna hitam-putih. Gaya ini merujuk pada aliran *sumi-e*, yaitu seni lukis tinta khas Jepang yang mengedepankan spontanitas goresan kuas.



Gambar 4.25 Top 10 Patch Terdekat ke Centroid

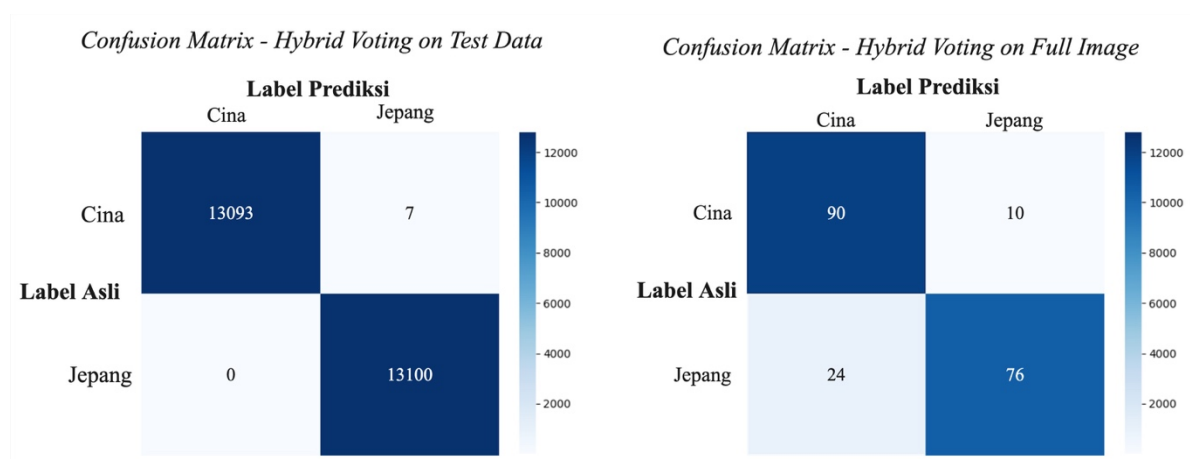
Model klasifikasi yang diuji dalam penelitian ini adalah pendekatan *hybrid voting* tanpa penerapan tahap filtrasi padding pada *patch*. Tujuan dari pengujian ini adalah untuk mengevaluasi sejauh mana keberadaan *patch* dengan area hitam (*padding*) memengaruhi kualitas representasi fitur dan performa klasifikasi secara keseluruhan. Pengujian dilakukan

dengan tetap mempertahankan parameter *alpha* sebesar 0,4 dan *beta* sebesar 0,6 sebagaimana konfigurasi pada model utama.

Hasil evaluasi untuk model *hybrid voting* pada data uji patch ditunjukkan dalam Tabel 4.15 dan Gambar 4.28 (kiri). Model menunjukkan performa sangat baik dengan akurasi mencapai 0,9997 serta nilai *precision* dan *recall* yang hampir sempurna untuk kedua kelas. Hal ini menunjukkan bahwa model mampu mengenali karakteristik visual seni Jepang dan seni Cina secara konsisten pada level *patch*. Namun, saat pendekatan diperluas ke klasifikasi gambar utuh dengan mekanisme *hybrid voting*, terjadi penurunan performa. Pada Gambar 4.28 (kanan) dan ringkasan metrik pada Tabel 4.14, akurasi menurun menjadi 83%, dengan F1-Score kelas Cina sebesar 0,8411 dan Jepang sebesar 0,8172. Meskipun hasil ini tetap baik, penurunan performa ini wajar. Penurunan ini tidak disebabkan oleh penggunaan padding secara langsung, melainkan karena perbedaan pendekatan evaluasi antara *patch* dan gambar utuh. Pada evaluasi berbasis patch, model dilatih dan diuji dengan *patch* yang telah dibagi secara *stratified*, sehingga distribusi fitur lebih merata dan independen. Sebaliknya, dalam klasifikasi gambar utuh, prediksi sangat tergantung pada jumlah dan kualitas *patch* dalam satu gambar yang bisa sangat bervariasi. Beberapa gambar mungkin memiliki banyak *patch* yang informatif, sementara yang lain tidak. Selain itu, beberapa *patch* dalam gambar uji bisa saja telah dilihat model selama proses pelatihan, sementara lainnya tidak, sehingga dapat menyebabkan akurasi lebih rendah dibandingkan evaluasi berbasis *patch* individu.

Tabel 4.15 Hasil Evaluasi Model Hybrid Voting dengan Padding

Metode	Accuracy	Precision (Cina)	Recall (Cina)	F1-Score (Cina)	Precision (Jepang)	Recall (Jepang)	F1-Score (Jepang)
Data Uji	0,9997	1,0000	0,9995	0,9997	0,9995	1,0000	0,9997
Gambar utuh	0,8300	0,7895	0,9000	0,8411	0,8837	0,7600	0,8172



Gambar 4.26 Hasil *Confusion Matrix Hybrid Voting Model* dengan *Padding*

Perbandingan dengan metode utama memakai filtrasi padding yang ditampilkan pada Tabel 4.10 dan Gambar 4.12 menunjukkan bahwa hasil keduanya relatif mirip. Model memakai filtrasi menghasilkan akurasi sebesar 83,00%, sedikit lebih rendah dibanding model dengan filtrasi. Hal ini menunjukkan bahwa filtrasi *padding* berkontribusi terhadap kestabilan fitur yang digunakan untuk klasifikasi, namun bukan satu-satunya faktor yang memengaruhi

penurunan akurasi pada level gambar. Dengan demikian, tantangan utama dalam klasifikasi gambar utuh terletak pada penggabungan informasi antar *patch* yang tidak selalu konsisten, bukan karena keberadaan *padding*.

4.1.7 Output Program

Output program penelitian ini bertujuan untuk menjawab rumusan masalah yang telah dirumuskan pada Bab 1, khususnya dalam mengidentifikasi karakteristik visual yang membedakan lukisan *Jepang* dan *Cina*, serta memahami elemen-elemen visual yang berkontribusi dalam proses klasifikasi. Untuk mencapai tujuan tersebut, penelitian ini memakai pendekatan klasifikasi berbasis *patch* menggunakan metode *hybrid voting*. Model ini memanfaatkan prediksi SVM dan skor jarak ke *centroid* dari hasil *discriminative clustering* untuk melakukan klasifikasi setiap *patch* dan *majority voting* pada klasifikasi gambar utuh, secara lebih detail. Hasil prediksi dari setiap *patch* kemudian dikombinasikan menggunakan metode *voting* berbobot untuk menghasilkan prediksi akhir pada level gambar. Pendekatan ini baik dipakai dalam tingkat klasifikasi *patch* dan dapat menjelaskan elemen visual apa yang menjadi dasar prediksi, karena dapat menampilkan informasi setiap, termasuk klasifikasi, probabilitas SVM, skor *hybrid*, dan jarak ke masing-masing *centroid*.

Gambar 4.27 menampilkan keluaran dari sistem klasifikasi yang sepenuhnya berbasis pendekatan *patch-level hybrid voting*, yang dirancang untuk menjawab rumusan masalah utama dalam penelitian ini, yaitu mengidentifikasi karakteristik visual khas lukisan Jepang dan Cina. Tidak seperti pendekatan klasifikasi global berbasis gambar utuh, pendekatan ini mengelompokkan dan mengevaluasi setiap bagian (*patch*) dari gambar secara individu, lalu menggabungkan prediksinya melalui proses *voting* untuk menghasilkan keputusan akhir.

Pada bagian kiri gambar, ditampilkan hasil prediksi label akhir dari keseluruhan gambar berdasarkan rata-rata skor *hybrid* seluruh *patch* yang valid. Pada contoh ini, gambar diprediksi sebagai Jepang dengan rata-rata skor *hybrid* sebesar 1.06. Skor *hybrid* ini diperoleh dari gabungan dua komponen utama: (1) probabilitas hasil klasifikasi dari model *Support Vector Machine* (SVM), dan (2) rasio skor kedekatan ke *centroid* kluster Jepang dibandingkan Cina berdasarkan hasil *discriminative clustering*. Pendekatan ini mengintegrasikan pembelajaran *supervised* (SVM) dengan *unsupervised* (*centroid clustering*), sehingga mampu mengidentifikasi dan menjelaskan karakteristik visual secara lebih komprehensif. Di bagian kanan gambar, diperlihatkan visualisasi dari beberapa *patch* yang dipilih secara representatif. Setiap *patch* ditampilkan bersama informasi prediksi kelas, skor probabilitas dari SVM, skor *hybrid* akhir, indeks *centroid* terdekat, serta jarak *cosine* ke seluruh *centroid*.

Dari visualisasi ini, terlihat bahwa sebagian besar *patch* memiliki skor *hybrid* tinggi dan cenderung menuju kluster Jepang, yang mencerminkan konsistensi elemen visual khas seperti tekstur, kaligrafi, atau motif lukisan. Selain itu, sistem juga menampilkan informasi *centroid* dominan, yaitu *centroid* yang paling sering menjadi titik terdekat bagi seluruh *patch* dalam gambar. Pada contoh ini, *centroid* dominan adalah kluster 0, yang sebelumnya telah diidentifikasi sebagai representasi kuat dari karakteristik seni lukis Jepang, khususnya *ukiyo-e* dan *sumi-e*. Penanda seperti ini memperkuat interpretasi bahwa gambar yang diuji memiliki kesesuaian visual tinggi dengan kluster Jepang, dan bukan hanya berdasarkan label prediksi semata. Dengan demikian, pendekatan *patch-based hybrid voting* ini tidak hanya memberikan hasil klasifikasi yang akurat, tetapi juga transparansi dalam proses pengambilan keputusan model. Penelusuran visual terhadap kontribusi tiap *patch* memungkinkan analisis lanjutan terhadap elemen visual kunci yang membedakan lukisan Jepang dan Cina, menjadikan metode ini sangat relevan untuk aplikasi dalam studi representasi seni visual tradisional.

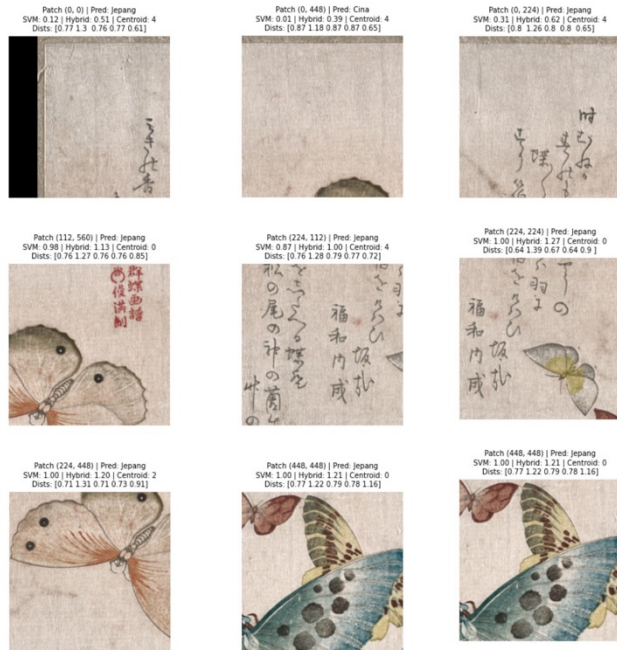
Prediksi Gambar yang Dihasilkan

Prediksi Gambar: Jepang (Skor Hybrid Rata-rata: 1.06)



📌 Centroid Dominan: 0 (Jepang)

Contoh Prediksi *Patch* yang Dihasilkan



Gambar 4.27 Output Program

4.2 Pembahasan/Diskusi

Penelitian ini mengimplementasikan pendekatan klasifikasi berbasis *patch* untuk membedakan antara seni lukis tradisional Jepang (terdiri dari gaya *ukiyo-e* dan *sumi-e*) dan seni lukis Cina. Dataset yang digunakan terdiri atas dua kategori utama: dataset positif yang mencakup karya seni Jepang, serta dataset negatif yang terdiri dari seni lukis Cina. *Dataset* positif berasal dari dua sumber, yaitu *The Metropolitan Museum of Art Ukiyo-e Dataset* yang berisi 4.043 gambar, dan *Sumi-e no Kurozuappu* dari Hugging Face yang mencakup sekitar 1.250 gambar. *Dataset* negatif diambil dari *Chinese Painting Dataset* (G. Wang et al., 2018) yang berisi 5.798 gambar lukisan Cina. Untuk memahami efektivitas pendekatan klasifikasi yang diusulkan, dilakukan perbandingan performa antara beberapa metode klasifikasi yang berbeda. Setiap metode dirancang untuk merepresentasikan pendekatan tertentu, mulai dari model klasifikasi berbasis *patch* secara terawasi (*SVM only*), pendekatan tak terawasi (berbasis *centroid*), hingga pendekatan *hybrid* yang menggabungkan keduanya. Selain itu, juga dilakukan evaluasi terhadap model klasifikasi berbasis gambar utuh tanpa pemotongan *patch* untuk mengetahui sejauh mana metode *patch-based* memberikan keunggulan. Tabel 4.16 merangkum hasil evaluasi dari masing-masing metode, baik pada data uji tingkat *patch* maupun tingkat gambar utuh, beserta analisis terhadap kekuatan dan kelemahan masing-masing pendekatan.

Berdasarkan hasil evaluasi yang disajikan pada Tabel 4.16, dapat disimpulkan bahwa metode *Hybrid Voting* dengan filtrasi padding merupakan pendekatan terbaik dan paling stabil dalam penelitian ini. Metode ini tidak hanya menghasilkan akurasi tinggi baik pada data uji tingkat *patch* (0,9998) maupun pada klasifikasi gambar utuh (0,8350), tetapi juga memiliki keunggulan dalam menangkap elemen-elemen visual yang benar-benar informatif. Penerapan

filter patch dengan padding lebih dari 20% terbukti mampu mengurangi noise yang berasal dari area kosong atau kurang relevan, sehingga menghasilkan prediksi yang lebih baik dan konsisten. Selain itu, model ini memanfaatkan kekuatan kombinasi antara probabilitas SVM dan skor kedekatan terhadap *centroid*, menjadikannya sebagai metode yang paling komprehensif dalam mengidentifikasi karakteristik visual khas dari gaya lukisan Jepang dan Cina. Oleh karena itu, pendekatan ini dijadikan sebagai model utama dalam implementasi dan analisis penelitian

Tabel 4.16 Perbandingan Semua Metode Klasifikasi

No	Metode	Akurasi Data Uji (Patch)	Akurasi Pada Gambar Utuh	Keterangan
1	SVM <i>Only</i>	0,9998	0,5000	Akurasi tinggi di level <i>patch</i> , tetapi performa drastis turun di gambar utuh, <i>recall</i> Jepang sangat rendah.
2	<i>Centroid Only</i>	0,9987	0,7400	Stabil dan konsisten, namun <i>recall</i> Cina rendah (0.5100); kurang menangani variasi antar- <i>patch</i> .
3	<i>Hybrid Voting</i> (tanpa <i>padding filter</i>)	0,9997	0,8300	Performa baik namun sedikit menurun karena <i>noise</i> dari <i>patch-padding</i>
4	Hybrid Voting (dengan padding filter)	0,9998	0,8350	Metode utama dan terbaik, paling stabil, mampu menangkap elemen visual informatif dan mengurangi noise
5	Klasifikasi Gambar Utuh (Tanpa <i>Patch</i>)	-	0,9986	Performa sangat tinggi, tetapi tidak dapat menjelaskan kontribusi elemen dan juga karakteristik dari masing-masing seni

(Halaman ini sengaja dikosongkan)

BAB 5 Kesimpulan dan Saran

5.1 Kesimpulan

Dalam pengerjaan penelitian ini, setelah melalui tahapan perancangan sistem, implementasi metode, dan serangkaian uji coba yang meliputi pengumpulan dataset lukisan tradisional Jepang dan Cina, praproses data, deteksi *patch* menggunakan *sliding window*, ekstraksi fitur menggunakan ResNet50, reduksi dimensi dengan PCA, *discriminative clustering* untuk pemetaan data, pelatihan model *Support Vector Machine* (SVM), hingga pengujian klasifikasi menggunakan pendekatan *hybrid voting* serta *full image classification*, diperoleh kesimpulan sebagai berikut:

1. Karakteristik visual yang membedakan lukisan Jepang dan lukisan Cina berhasil diidentifikasi melalui analisis hasil *discriminative clustering*. Tiga klaster yang didominasi oleh *patch* lukisan Jepang memperlihatkan pola visual khas seperti tekstur kertas dan kaligrafi kanji yang mempresentasikan lukisan *ukiyo-e* serta motif hitam-putih dari gaya *sumi-e* seperti pohon pinus dan bambu. Sebaliknya, *cluster* yang merepresentasikan lukisan Cina menunjukkan motif pemandangan alam bergaya *shan shui*, seperti gunung dan pepohonan, serta latar dengan garis horizontal berwarna tegas. Hasil visualisasi terhadap *patch* terdekat *centroid* membuktikan bahwa elemen-elemen visual tersebut cukup konsisten, sehingga dapat diidentifikasi sebagai karakteristik pembeda utama antara dua kategori seni tersebut.
2. Elemen-elemen visual yang berkontribusi untuk seni Jepang mencakup tulisan kaligrafi Jepang, serta garis kuas tegas dan halus pada objek seperti manusia, gunung, pohon pinus, dan bambu. Pola-pola ini banyak ditemukan pada *patch* di klaster 0, 2, dan 3 yang secara dominan berasal dari seni Jepang. Sebaliknya, pada seni lukis Cina, elemen yang dominan meliputi lanskap, tekstur batuan atau kabut, serta pola-pola repetitif dari dedaunan bambu dan pegunungan yang divisualisasikan dengan gradasi tinta dan detail tekstur lebih kompleks. Karakteristik ini paling banyak muncul pada *patch* yang tergolong dalam klaster 1 dan 4 yang didominasi oleh seni Cina. Selama proses klasifikasi, elemen-elemen ini tercermin dalam nilai prediksi dari model SVM, skor *hybrid voting*, dan kedekatan fitur *patch* terhadap *centroid cluster*. *Patch* yang memiliki elemen visual khas akan menghasilkan skor prediksi yang tinggi dan menunjukkan jarak terdekat ke *centroid* dari *cluster* dominan. Dengan demikian, proses klasifikasi tidak hanya dilakukan berdasarkan label akhir, tetapi juga memperlihatkan kontribusi visual nyata dari elemen-elemen khas yang membentuk identitas masing-masing gaya lukisan.
3. Evaluasi kinerja model *hybrid voting* berbasis *patch* menunjukkan performa yang baik dalam mengenali karakteristik visual lukisan. Dengan menerapkan mekanisme filtrasi *padding* dan klasifikasi berbasis kombinasi probabilitas SVM dan skor kedekatan ke *centroid*, model ini mampu mengklasifikasikan gambar dengan akurasi sebesar 0,9998 pada data uji *patch* dan 0,8350 pada pengujian gambar utuh berbasis *majority voting*. Model ini tidak hanya efektif secara performa klasifikasi, tetapi juga memberikan fleksibilitas dalam mengidentifikasi elemen-elemen visual penting dari setiap bagian gambar. Dengan pendekatan ini, analisis mendalam terhadap kontribusi *patch* terhadap keputusan akhir klasifikasi dapat dilakukan, sehingga sistem menjadi lebih transparan dan interpretatif. Pendekatan ini tidak hanya efektif secara performa klasifikasi, tetapi juga memungkinkan analisis lebih dalam terhadap kontribusi visual setiap bagian gambar terhadap keputusan akhir klasifikasi.

Dengan demikian, dapat disimpulkan bahwa pendekatan gabungan antara *hybrid voting* berbasis *patch* dan klasifikasi gambar utuh merupakan dua metode yang paling efektif dalam membedakan lukisan tradisional Jepang dan Cina. Model yang dibangun tidak hanya memberikan akurasi yang baik, tetapi juga mampu mengidentifikasi elemen karakteristik yang mencerminkan gaya dari masing-masing budaya.

5.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa saran yang dapat dijadikan pertimbangan untuk pengembangan penelitian selanjutnya. Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa saran yang dapat dijadikan pertimbangan untuk pengembangan penelitian selanjutnya. Pertama, penelitian ini dapat disempurnakan dengan menambahkan mekanisme *threshold voting* adaptif pada tahap klasifikasi gambar utuh, sehingga keputusan akhir tidak hanya bergantung pada mayoritas sederhana, tetapi juga mempertimbangkan jumlah patch valid dan tingkat kepercayaan prediksi. Kedua, penanganan kasus *edge-case* seperti gambar dengan jumlah patch valid yang sangat sedikit juga perlu ditingkatkan, misalnya dengan menetapkan batas minimal patch valid atau menggunakan strategi fallback berbasis klasifikasi gambar utuh. Ketiga, untuk meningkatkan akurasi dan generalisasi model, eksperimen dengan model CNN yang berbeda atau kombinasi model ensemble dapat dilakukan.

DAFTAR PUSTAKA

- Adewopo, V. A., & Elsayed, N. (2024). Smart City Transportation: Deep Learning Ensemble Approach for Traffic Accident Detection. *IEEE Access*, 12, 59134–59147. <https://doi.org/10.1109/ACCESS.2024.3387972>
- Agresti, A. (1992). A Survey of Exact Inference for Contingency Tables. *Statistical Science*, 7(1). <https://doi.org/10.1214/ss/1177011454>
- Altman, N., & Krzywinski, M. (2017). Points of Significance: Clustering. *Nature Methods*, 14(6), 545–546. <https://doi.org/10.1038/nmeth.4299>
- Arulprakash, E., & Aruldoss, M. (2022). A study on generic object detection with emphasis on future research directions. *Journal of King Saud University - Computer and Information Sciences*, 34(9), 7347–7365. <https://doi.org/10.1016/j.jksuci.2021.08.001>
- Brailean, J. C., Kleihorst, R. P., Efstratiadis, S., Katsaggelos, A. K., & Lagendijk, R. L. (1995). Noise reduction filters for dynamic image sequences: A review. *Proceedings of the IEEE*, 83(9), 1272–1292. <https://doi.org/10.1109/5.406412>
- Chaib, S., Liu, H., Gu, Y., & Yao, H. (2017). Deep Feature Fusion for VHR Remote Sensing Scene Classification. *IEEE Transactions on Geoscience and Remote Sensing*, 55(8), 4775–4784. <https://doi.org/10.1109/TGRS.2017.2700322>
- Chandra, M. A., & Bedi, S. S. (2021). Survey on SVM and their application in imageclassification. *International Journal of Information Technology*, 13(5), 1–11. <https://doi.org/10.1007/s41870-017-0080-1>
- Chang, Y., Jung, C., Ke, P., Song, H., & Hwang, J. (2018). Automatic Contrast-Limited Adaptive Histogram Equalization With Dual Gamma Correction. *IEEE Access*, 6, 11782–11792. <https://doi.org/10.1109/ACCESS.2018.2797872>
- Chen, F., & Tsou, J. Y. (2022). Assessing the effects of convolutional neural network architectural factors on model performance for remote sensing image classification: An in-depth investigation. *International Journal of Applied Earth Observation and Geoinformation*, 112, 102865. <https://doi.org/10.1016/j.jag.2022.102865>
- Doersch, C., Singh, S., Gupta, A., Sivic, J., & Efros, A. A. (2015). What makes Paris look like Paris? *Commun. ACM*, 58(12), 103–110. <https://doi.org/10.1145/2830541>
- Dong, W., Zhou, N., Paul, J.-C., & Zhang, X. (2009). Optimized image resizing using seam carving and scaling. *ACM Trans. Graph.*, 28(5), 1–10. <https://doi.org/10.1145/1618452.1618471>
- Du, K.-L. (2010). Clustering: A neural network approach. *Neural Networks*, 23(1), 89–107. <https://doi.org/10.1016/j.neunet.2009.08.007>
- Duchenne, O., Laptev, I., Sivic, J., Bach, F., & Ponce, J. (2009). Automatic annotation of human actions in video. *2009 IEEE 12th International Conference on Computer Vision*, 1491–1498. <https://doi.org/10.1109/ICCV.2009.5459279>
- Dolezel, P., Skrabanek, P., Stursa, D., Baroque Zanon, B., Cogollos Adrian, H., & Kryda, P. (2022). Centroid based person detection using pixelwise prediction of the position. *Journal of Computational Science*, 63, 101760. <https://doi.org/10.1016/j.jocs.2022.101760>
- Gao, S., & Gruev, V. (2011). Bilinear and bicubic interpolation methods for division of focal plane polarimeters. *Optics Express*, 19(27), 26161–26173. <https://doi.org/10.1364/OE.19.026161>
- Grandini, M., Bagli, E., & Visani, G. (2020). *Metrics for Multi-Class Classification: An Overview* (No. arXiv:2008.05756). arXiv. <https://doi.org/10.48550/arXiv.2008.05756>

- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- Heisler, M., Karst, S., Lo, J., Mammo, Z., Yu, T., Warner, S., Maberley, D., Beg, M. F., Navajas, E. V., & Sarunic, M. V. (2020). Ensemble Deep Learning for Diabetic Retinopathy Detection Using Optical Coherence Tomography Angiography. *Translational Vision Science & Technology*, 9(2), 20. <https://doi.org/10.1167/tvst.9.2.20>
- Henderi, H., Wahyuningsih, T., & Rahwanto, E. (2021). Comparison of Min-Max normalization and Z-Score Normalization in the K-nearest neighbor (kNN) Algorithm to Test the Accuracy of Types of Breast Cancer. *International Journal of Informatics and Information Systems*, 4(1), Article 1. <https://doi.org/10.47738/ijiis.v4i1.73>
- Hidayati, S. C., Rahardja, A. A. I., & Suciati, N. (2024). Stacking-based ensemble learning for identifying artist signatures on paintings. *Indonesian Journal of Electrical Engineering and Computer Science*, 36(3), Article 3. <https://doi.org/10.11591/ijeecs.v36.i3.pp1683-1693>
- Hou, Q., & Qian, C. (2023). Analysis of Ukiyo-e's Painting Style and Characteristics. *Journal of Humanities, Arts and Social Science*, 7(8). <https://doi.org/10.26855/jhass.2023.08.005>
- Huang, X., Ye, Y., & Zhang, H. (2014). Extensions of Kmeans-Type Algorithms: A New Clustering Framework by Integrating Intracluster Compactness and Intercluster Separation. *IEEE Transactions on Neural Networks and Learning Systems*, 25(8), 1433–1446. *IEEE Transactions on Neural Networks and Learning Systems*. <https://doi.org/10.1109/TNNLS.2013.2293795>
- Imocha Singh, O., Manglem Singh, K., Rupachandra Singh, T., Romen Singh, T., Imocha Singh, O., Manglem Singh, K., Rupachandra Singh, T., & Sinam, T. (2009). Image Magnification based on directed linear interpolation. *International Journal of Computer Sciences and Engineering Systems (IJCSSES)*, 3. <https://www.researchgate.net/publication/281900457>
- Joulin, A., Bach, F., & Ponce, J. (2010). Discriminative clustering for image co-segmentation. *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 1943–1950. <https://doi.org/10.1109/CVPR.2010.5539868>
- Khan, A., Sohail, A., Zahoor, U., & Qureshi, A. S. (2020). A survey of the recent architectures of deep convolutional neural networks. *Artificial Intelligence Review*, 53(8), 5455–5516. <https://doi.org/10.1007/s10462-020-09825-6>
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2017). ImageNet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6), 84–90. <https://doi.org/10.1145/3065386>
- Lee, S. E. (1962). Contrasts in Chinese and Japanese Art. *The Journal of Aesthetics and Art Criticism*, 21(1), 3–12. <https://doi.org/10.2307/427632>
- Lucas, A. (2024). *Sumi-e: The Mindful Art of Japanese Ink Painting by Akemi Lucas* | Lowell Books. Herbert Press. <https://doi.org/10.5040/9781789941647>
- Madusanka, N., Jayalath, P., Fernando, D., Yasakethu, L., & Lee, B.-I. (2023). Impact of H&E Stain Normalization on Deep Learning Models in Cancer Image Classification: Performance, Complexity, and Trade-Offs. *Cancers*, 15(16), Article 16. <https://doi.org/10.3390/cancers15164144>
- Mascarenhas, S., & Agarwal, M. (2021). A comparison between VGG16, VGG19 and ResNet50 architecture frameworks for Image Classification. *2021 International Conference on Disruptive Technologies for Multi-Disciplinary Research and Applications (CENTCON)*, 96–99. <https://doi.org/10.1109/CENTCON52345.2021.9687944>

- McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction.
- Menis-Mastromichalakis, O., Sofou, N., & Stamou, G. (2020). Deep Ensemble Art Style Recognition. *2020 International Joint Conference on Neural Networks (IJCNN)*, 1–8. <https://doi.org/10.1109/IJCNN48605.2020.9207645>
- Ming Zhang, & Gunturk, B. K. (2008). Multiresolution Bilateral Filtering for Image Denoising. *IEEE Transactions on Image Processing*, 17(12), 2324–2333. <https://doi.org/10.1109/TIP.2008.2006658>
- Müller, J., Fregin, A., & Dietmayer, K. (2018). *Disparity Sliding Window: Object Proposals From Disparity Images* (No. arXiv:1805.06830). arXiv. <https://doi.org/10.48550/arXiv.1805.06830>
- Mutlag, W. K., Ali, S. K., Aydam, Z. M., & Taher, B. H. (2020). Feature Extraction Methods: A Review. *Journal of Physics: Conference Series*, 1591(1), 012028. <https://doi.org/10.1088/1742-6596/1591/1/012028>
- Nguyen, M. H., Torresani, L., de la Torre, F., & Rother, C. (2009). Weakly supervised discriminative localization and classification: a joint learning process. *2009 IEEE 12th International Conference on Computer Vision*, 1925–1932. <https://doi.org/10.1109/ICCV.2009.5459426>
- Oğuz, A., & Ertuğrul, Ö. F. (2023). Introduction to deep learning and diagnosis in medicine. In *Diagnostic Biomedical Signal and Image Processing Applications with Deep Learning Methods* (pp. 1–40). Elsevier. <https://doi.org/10.1016/B978-0-323-96129-5.00003-2>
- Okamoto, N. (1996). *Japanese Ink Painting: The Art of Sumi-E*. Sterling Publishing Company, Inc.
- Pearce, N., Best, B., & Hu, M. (2018). Art aesthetics in China: Some observations. *The Senses and Society*, 13(3), 282–298. <https://doi.org/10.1080/17458927.2018.1522168>
- Qi, C., & Tang, X. (2018). A hybrid ensemble method for improved prediction of slope stability. *International Journal for Numerical and Analytical Methods in Geomechanics*, 42(15), 1823–1839. <https://doi.org/10.1002/nag.2834>
- Raitoharju, J. (2022). Convolutional neural networks. In *Deep Learning for Robot Perception and Cognition* (pp. 35–69). Elsevier. <https://doi.org/10.1016/B978-0-32-385787-1.00008-7>
- Raschka, S. (2020). *Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning* (No. arXiv:1811.12808). arXiv. <https://doi.org/10.48550/arXiv.1811.12808>
- Roy, R., Pal, M., & Gulati, T. (2013). Zooming Digital Images Using Interpolation Techniques. *International Journal of Applied or Innovation in Engineering and Management*, 2(4).
- Sajana, T., Syamala, M., Phaneendra Maguluri, L., & Usha Kumari, Ch. (2021). WITHDRAWN: A hybrid approach for classification of infectious diseases. *Materials Today: Proceedings*. <https://doi.org/10.1016/j.matpr.2020.11.727>
- Salem, N., & Hussein, S. (2019). Data dimensional reduction and principal components analysis. *Procedia Computer Science*, 163, 292–299. <https://doi.org/10.1016/j.procs.2019.12.111>
- Saxena, A., Prasad, M., Gupta, A., Bharill, N., Patel, O. P., Tiwari, A., Er, M. J., Ding, W., & Lin, C.-T. (2017). A review of clustering techniques and developments. *Neurocomputing*, 267, 664–681. <https://doi.org/10.1016/j.neucom.2017.06.053>
- Setiawan, B., Djanali, S., & Ahmad, T. (2017). A Study on Intrusion Detection Using Centroid-Based Classification. *Procedia Computer Science*, 124, 672–681. <https://doi.org/10.1016/j.procs.2017.12.204>
- Shahapure, K. R., & Nicholas, C. (2020). Cluster Quality Analysis Using Silhouette Score. *2020 IEEE 7th International Conference on Data Science and Advanced Analytics (DSAA)*, 747–748. <https://doi.org/10.1109/DSAA49011.2020.00096>

- Shinichi, H., & Matsui, A. (2024). *A Deep Learning Approach to Quantitative Analysis of Japanese Artworks: Exploring the Latent Features of Ukiyo-e Artists* (SSRN Scholarly Paper No. 4706101). Social Science Research Network. <https://doi.org/10.2139/ssrn.4706101>
- Simonyan, K., & Zisserman, A. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition*.
- Singh, R. V., & Bhatia, M. P. S. (2011). Data clustering with modified K-means algorithm. *2011 International Conference on Recent Trends in Information Technology (ICRTIT)*, 717–721. <https://doi.org/10.1109/ICRTIT.2011.5972376>
- Sinkkonen, J., & Kaski, S. (2002). Clustering Based on Conditional Distributions in an Auxiliary Space. *Neural Computation*, 14(1), 217–239. <https://doi.org/10.1162/089976602753284509>
- Sonali, Sahu, S., Singh, A. K., Ghrera, S. P., & Elhoseny, M. (2019). An approach for denoising and contrast enhancement of retinal fundus image using CLAHE. *Optics & Laser Technology*, 110, 87–98. <https://doi.org/10.1016/j.optlastec.2018.06.061>
- Suban, I. (2019). *Magnifikasi Perbaikan Citra Digital Dengan Metode Interpolasi Bicubic Basis Spline Berbasis Pemrograman Paralel* [Tugas Akhir]. Universitas Atma Jaya.
- Tversky, A. (1997). Psychology Review. In *Features of similarity* (Vol. 84). American Psychological Association. <http://www.ai.mit.edu/projects/dm/Tversky-features.pdf>
- Usino, W., Prabuwo, A. S., Allehaibi, K. H. S., Bramantoro, A., A, H., & Amaldi, W. (2019). Document Similarity Detection using K-Means and Cosine Distance. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 10(2), Article 2. <https://doi.org/10.14569/IJACSA.2019.0100222>
- Wakita, W., Hara, J., & Tanaka, H. T. (2014). A Real-Object-Oriented Exhibition System of Ukiyo-e of Anisotropic Reflection Properties. *ITE Transactions on Media Technology and Applications*, 2(3), 209–216. <https://doi.org/10.3169/mta.2.209>
- Wang, G., Chen, Y., & Chen, Y. (2018). *Chinese Painting Generation Using Generative Adversarial Networks*. <https://www.semanticscholar.org/paper/Chinese-Painting-Generation-Using-Generative-Wang-Chen/a3b5256c879bd8be9816715a07569124037460d1>
- Wang, Q., & Yuan, Y. (2014). Learning to resize image. *Neurocomputing*, 131, 357–367. <https://doi.org/10.1016/j.neucom.2013.10.007>
- Wen, L., Li, X., & Gao, L. (2020). A transfer convolutional neural network for fault diagnosis based on ResNet-50. *Neural Computing and Applications*, 32(10), 6111–6124. <https://doi.org/10.1007/s00521-019-04097-w>
- Yue, S., Li, P., & Hao, P. (2003). SVM classification: Its contents and challenges. *Applied Mathematics-A Journal of Chinese Universities*, 18(3), 332–342. <https://doi.org/10.1007/s11766-003-0059-5>
- Zhang, T., Kasichainula, K., Zhuo, Y., Li, B., Seo, J.-S., & Cao, Y. (2023). *Patch-based Selection and Refinement for Early Object Detection* (No. arXiv:2311.02274). arXiv. <https://doi.org/10.48550/arXiv.2311.02274>
- Zhang, J., Marszałek, M., Lazebnik, S., & Schmid, C. (2007). Local Features and Kernels for Classification of Texture and Object Categories: A Comprehensive Study. *International Journal of Computer Vision*, 73(2), 213–238. <https://doi.org/10.1007/s11263-006-9794-4>
- Zheng, X., Wang, M., & Ordieres-Meré, J. (2018). Comparison of Data Preprocessing Approaches for Applying Deep Learning to Human Activity Recognition in the Context of Industry 4.0. *Sensors*, 18(7), Article 7. <https://doi.org/10.3390/s18072146>

Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H., & He, Q. (2021). A Comprehensive Survey on Transfer Learning. *Proceedings of the IEEE*, 109(1), 43–76. Proceedings of the IEEE. <https://doi.org/10.1109/JPROC.2020.3004555>

(Halaman ini sengaja dikosongkan)

LAMPIRAN

(Halaman ini sengaja dikosongkan)

BIODATA PENULIS



Penulis dilahirkan di Bojonegoro, 18 April 2003, merupakan anak pertama dari 3 bersaudara. Penulis telah menempuh pendidikan formal yaitu di TK Al-Azhar Sentra Primer, SD Al-Azhar 19 Sentra Primer, SMP Labschool Jakarta dan SMAN 81 Jakarta Timur. Setelah lulus dari SMAN tahun 2021, Penulis mengikuti SNMPTN dan diterima di Departemen Teknik Informatika FTEIC-ITS pada tahun 2021 dan terdaftar dengan NRP 5025211028.

Di Departemen Teknik Informatika Penulis sempat aktif di beberapa kegiatan Seminar yang diselenggarakan oleh Departemen, Himpunan Mahasiswa Teknik Computer-Informatika (HMTTC) dan aktif sebagai Asisten Dosen.